IES Accelerator Implementation Guide

India Energy Stack

2026-06-29

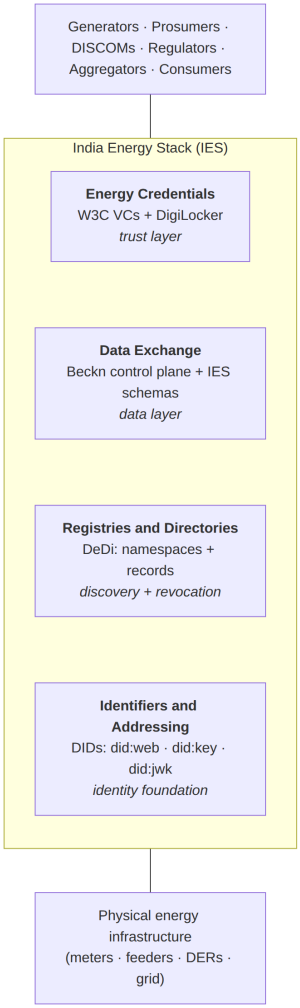
Contents

Chapter 1

Home

India Energy Stack (IES) is an open digital infrastructure layer for India's power sector — protocols and shared registries that let DISCOMs, AMISPs, regulators, and consumers exchange data and trust each other's claims, without anyone owning the network.

Printable version: download this entire guide as a single PDF — **ies-report.pdf**. It is a printable version of this GitBook, regenerated automatically whenever the docs change.



This **Accelerator** is the developer hub for building on it. Five core sections:

Section	What it does
IES Identifiers and Addressing	DIDs and addressing grammar for DISCOMs, regulators, consumers, assets, meters, credentials, and datasets
IES Registries and Directories	DeDi-based public registries — namespaces, the IES reference registries, Beckn subscriber registries, revocation, public-keys
IES Energy Credentials	Issue, hold, and verify cryptographically signed digital credentials for energy assets and consumers
IES Data Exchange	Discover and contract dataset exchanges over Beckn — telemetry, regulatory filings, tariff policies. Payload rides **inline** for small / simple datasets, or Beckn delivers an **access method** (signed URL / SFTP / Kafka / MQTT / OpenADR endpoint) when an established channel already moves the bytes

Section	What it does
IES Pathways	Structured, step-by-step onboarding roadmaps for utilities, regulators, and other sector participants to adopt capabilities
IES Schemas	Mirror of canonical schema specifications, JSON-LD contexts, and vocabularies (Credentials, Telemetry, Regulatory filings, and requests)

1.1 Where to Start

- **New to IES?** Read Getting Started for a five-minute orientation.
- **Onboarding as a DISCOM, regulator, or NP?** Go to the Registries checklist.
- **Integrating Energy Credentials?** Go to the Energy Credentials onboarding guide.
- **Building a data exchange application?** Go to the Data Exchange quick start.
- **Need a term defined?** Check the always-visible Glossary.

1.2 Why IES?

The Indian power sector runs on data that is siloed, bespoke, and hard to trust — regulatory filings arrive as PDFs, telemetry lives inside proprietary MDMS systems, subsidy eligibility is verified manually, and credentials of ownership or identity have no standard form. IES provides a **shared digital infrastructure layer** — open standards, verifiable data, and interoperable protocols — so that every actor in the ecosystem can transact on common ground.

1.3 Key Standards and Protocols

Standard	Role in IES
W3C Verifiable Credentials	Data model for signed, machine-verifiable credentials
W3C Decentralized Identifiers (DIDs)	Cryptographic identity for issuers, holders, and verifiers
Beckn Protocol v2.0	Peer-to-peer protocol for dataset discovery, contracting, consent, and audit. Carries payload inline for small / simple cases, or hands off an access method for established channels (signed URL, Kafka, MQTT, SFTP, OpenADR)
DLMS-COSEM / IS 15959	Smart-meter wire protocol used in RDSS AMI deployments
IEC 61968 / CIM / MultiSpeak	HES<->MDMS interoperability standards named in CEA AMI interoperability guidance
OpenADR 3.1.0	DR / DER event and flexibility-reporting protocol (not generic interval reads)
DigiLocker	India's national digital document wallet — consumer-facing credential store

1.4 Related Repositories

- ies-docs — IES architecture primitives and implementation guides
- Digital Energy Grid (DEG) — upstream open protocol specification
- DDM — Dataset Descriptor Model (`DatasetItem` schema)

Chapter 2

Getting Started

New here? Unfamiliar terms (DID, DeDi, BAP/BPP, ERA, ARR…) link to the **Glossary** on first use. The Glossary is always available from the left nav.

This page gives you a five-minute orientation to IES before you dive into either capability.

2.1 What is the India Energy Stack?

IES is an **open protocol and governance layer** for India's energy sector. It does not operate data — it defines the standards and interaction patterns that let diverse organisations share data and trust each other's claims.

IES has four developer-facing sections. The bottom two are the **foundations** every participant needs in place; the top two are the **capabilities** built on top of them.

2.1.1 Identifiers and Addressing

The DID grammar IES uses to name every actor, asset, document, and dataset on the network — `did:web` for institutions and their assets, `did:key` / `did:jwk` for consumer wallets. All three are standard W3C DID methods; DeDi acts as a key-discovery layer for `did:web`, not as a separate method. Internal numbering (consumer numbers, asset SAP codes) is preserved and wrapped, never replaced.

Who should use this: Every participant — this is the addressing layer below everything else.

2.1.2 Registries and Directories

DeDi-based public registries that resolve identifiers to records and act as the trust layer for credentials and Beckn. Covers the IES reference registries (DISCOMs, regulators, network), the per-participant registries you need to operate (Beckn subscriber, revocation, public-keys), and the step-by-step process for creating them.

Who should use this: Every participant onboarding to IES. Start here with the Registries checklist.

2.1.3 Energy Credentials

A framework for issuing, holding, and verifying **digital attestations** about energy assets and consumers, built on W3C Verifiable Credentials and DigiLocker.

v1 scope. DISCOMs are the sole **issuer**; consumers (or authorized actors on their behalf) are the **holder**; third parties **verify** or **receive** credentials depending on the use case. Other issuer roles (SERCs, DER OEMs, government bodies) are architecturally supported but out of scope for v1.

In v1, energy credentials enable:

- Consumers to carry a verified digital proof of their electricity connection
- DISCOMs to issue tamper-proof credentials without manual paper processes
- Third-party service providers to verify consumer identity and energy status instantly

Who should use this: DISCOMs, service providers doing KYC/address verification, fintech apps integrating with energy consumers.

2.1.4 Data Exchange

A federated, policy-governed mechanism for discovering and exchanging structured energy datasets. Beckn always carries the **control plane** — discovery, offer, consent, contract, audit. The **payload** has two equally valid modes: it can ride **inline in the Beckn callback** when the dataset is small and a single signed message is the simplest workflow, or — for bulky / streaming / already-channelised data — Beckn hands off an **access method** (signed URL / SFTP / REST / MQTT / Kafka / OpenADR / XBRL artifact / Akoma Ntoso document) that the consumer uses to pull or subscribe over the established channel. Covers:

- Smart meter telemetry — typically DLMS-COSEM / IS 15959 at the field layer, IEC 61968 / CIM / MultiSpeak between HES and MDM
- Regulatory filings — ARR submissions, tariff orders
- Tariff and policy data — rate structures, time-of-day surcharges

Who should use this: DISCOMs, SERCs, AMISPs, VAS providers, researchers, grid operators.

2.2 Choosing Your Path

- **A DISCOM, regulator, or NP onboarding to the IES network for the first time?** -> Go to Registries -> Checklist.
- **Issuing or verifying credentials about energy consumers or assets?** -> Go to Energy Credentials -> Onboarding Guide. You will also need the revocation + public-keys registries from the Registries chapter.
- **Exchanging structured energy datasets (telemetry, filings, tariffs)?** -> Go to Data Exchange -> Quick Start. You will also need a Beckn subscriber registry from the Registries chapter.
- **Building a new application that needs both?** -> Start with Registries (foundation) -> then Energy Credentials (simpler integration surface) -> then Data Exchange.

2.3 Key Terminology

A complete, plain-language glossary lives at **Glossary** (also in the left nav). It covers identity (DID, DeDi, VC), Beckn (BAP, BPP, ONIX, ERA), India energy sector (DISCOM, AMISP, SERC, ARR, MYT, HT/LT, ToD, RDSS, HES, MDMS), and standards (CIM, IEC 61968, DLMS-COSEM, MultiSpeak, OpenADR, XBRL, Akoma Ntoso, DCAT 3, MQTT).

2.4 Prerequisites by Capability

2.4.1 Energy Credentials

- An organisation registered with API Setu as a DigiLocker issuer (for DISCOMs issuing consumer credentials)
- Access to the IES Credential Service API (contact the IES team)
- A CIS / billing database to look up consumer records

2.4.2 Data Exchange

- Docker and Docker Compose (for local development)
- Access to the IES testnet (`nfh.global/testnet-deg`)
- A Beckn-compatible adapter (ONIX is provided; see Quick Start)

2.5 Getting Help

Raise an issue or start a discussion in the ies-accelerator repository. For testnet access and API keys, contact the IES team directly.

Chapter 3

Glossary

Plain-language definitions for every acronym and term used across this book. Each term is anchor-linked from its first use in a chapter — click any link to jump directly to the definition below.

This page is always visible in the left nav. If you hit a term you don't recognise, come back here.

3.1 Identity and credentials

3.1.1 DID

Decentralized Identifier — a globally unique, cryptographically verifiable identifier of the form `did:method:identifier` (e.g. `did:web:ies.tpddl.in`). Defined by the W3C DID Core standard. Resolves to a **DID Document** containing public keys and service endpoints. IES uses DIDs to name every participant, asset, and credential issuer. See Identifiers chapter.

3.1.2 DID methods used in IES

IES uses three standard W3C DID methods. There is no separate `did:dedi` method — DeDi-anchored identifiers are `did:web` DIDs whose document is hosted by a DeDi runtime instead of (or alongside) the issuer's own domain.

- **`did:web`** — the DID resolves to a DID document fetched over HTTPS. Two forms in IES:
  - **Self-hosted by the institution.** `did:web:ies.tpddl.in` -> `https://ies.tpddl.in/.well-known/did.json`. Used for DISCOMs, regulators, and other entities with a public web presence.
  - **Hosted by a DeDi runtime.** `did:web:<dedi-host>:<namespace>:<class>:<id>` -> `https://<dedi-host>/<namespace>/<class>/<id>/did.json`. Used to give consumers, meters, assets, connections, and credential subjects a network-resolvable DID without requiring each one to operate a web server. The method is still `did:web`; DeDi just hosts the document. Example: `did:web:dedi.global:tpddl:consumers:TPDDL-2025-001234567`.
- **`did:key`** — the public key is encoded directly into the DID string. No domain or certificate required. Verifies fully offline; cannot rotate keys. Used for consumer wallets and first-deploy software keys.
- **`did:jwk`** — same idea as `did:key`, using JSON Web Key encoding. Wallet-friendly.

3.1.3 DeDi

Decentralised Directory — distinguish the protocol from any runtime that implements it:

- **DeDi the protocol** — an open specification developed under the Linux Foundation Decentralized Trust labs. Defines record shapes, namespace ownership, lookup/query semantics, revocation, and trust-anchor records. **IES picks DeDi-the-protocol** as the registry primitive.
- **DeDi runtimes** — any service that implements the DeDi protocol. `dedi.global` is one such hosted runtime; a DISCOM (or a network operator) can self-host a DeDi-compatible runtime — for a private internal mirror, for a regulated jurisdiction, or for redundancy. IES does not mandate a specific runtime; the same record shapes work across any conformant implementation.

A DeDi record has a `subscriber_id` and `record_id` and is looked up via a public HTTPS URL (e.g. `https://<runtime-host>/dedi/lookup/...`). IES uses DeDi for namespaces, network membership, public keys, revocation, and Beckn subscriber records. See Registries chapter.

3.1.4 Verifiable Credential (VC)

A tamper-evident JSON-LD document containing claims about a subject (the **holder**), signed by an **issuer**, that any party (the **verifier**) can validate offline using the issuer's published public key. Follows the W3C VC Data Model. IES uses VCs for electricity credentials, consumer energy passports, and meter digests.

3.1.5 Issuer / Holder / Verifier

The three roles in any credential exchange. **Issuer** signs and emits the credential. **Holder** keeps it (in DigiLocker, a wallet, or a DID-controlled store) and presents it on demand. **Verifier** receives a presentation and checks the signature against the issuer's published public key. In IES v1 the DISCOM is the issuer of electricity credentials; the consumer (or an authorized actor on their behalf) is the holder.

3.1.6 OpenCred

The open-source credential service IES DISCOMs deploy to sign, verify, and revoke credentials. Holds your private signing key locally (or in a KMS) and exposes an HTTP API. **W3C-compliant** issuer: produces credentials that any standards-conformant verifier can validate. **DeDi-integrated** out of the box: credential revocation entries flow into a DeDi revocation registry automatically, and the container can optionally back up your `did.json` to DeDi so your DID stays resolvable even when your own domain is unreachable. Swap it for any other W3C-compliant signing pipeline if you prefer — the published `did.json` and credential format are unchanged.

- Image: `ghcr.io/nfh-trust-labs/opencred/opencred-server` (releases · docs)
- Bootcamp / first deploy: opencred.gitbook.io/docs/bootcamp/local-docker
- See the Energy Credentials chapter for the IES-specific issuance walkthrough.

3.1.7 DigiLocker

India's national digital document wallet. Consumer-facing store for government-issued documents. IES DISCOMs can deliver issued credentials to a consumer's DigiLocker via a Pull URI.

3.1.8 API Setu

The Government of India API exchange where DISCOMs register as DigiLocker issuers.

3.2 Data exchange and Beckn

3.2.1 Beckn

The open, asynchronous, peer-to-peer protocol IES Data Exchange uses for the **control plane** — discovery, offer, consent, contract, and audit. Beckn can also carry the **payload inline** in the same flow when the

dataset is small and a single signed message is the simplest workflow. For bulky datasets, telemetry streams, or anything already moved over an established channel (signed URL, REST, SFTP, MQTT, Kafka, OpenADR, etc.), Beckn instead delivers the **access method** via the `accessMethod` field on the DatasetItem. See Data Exchange chapter.

3.2.2 BAP

Beckn Application Platform — the consumer / buyer / data-requesting side in a Beckn exchange. In IES, a DISCOM consuming telemetry from an AMISP is the BAP.

3.2.3 BPP

Beckn Provider Platform — the provider / seller / data-publishing side in a Beckn exchange. In IES, an AMISP publishing meter telemetry, or a SERC publishing a tariff order, is the BPP.

3.2.4 ONIX

The reference Beckn adapter used in IES. Handles message signing, signature verification, DeDi lookup, network-membership checks, schema validation, and async callback routing. Ships as a container; your application code does not handle signing or key management directly.

3.2.5 NFO

Network Facilitator Organisation — the organisation that operates a Beckn network: claims a verified DeDi namespace under its own domain, publishes one or more **network registries** under that namespace (one per environment / sector), curates membership by writing **subscriber reference records** that point at participant-owned subscriber records, and publishes the network's policy manifest. The network's identifier (`network_id`) is `<nfo-domain>/<registry-name>`. For IES, the network operator (REC / IES Secretariat) acts as the NFO under the `indiaenergystack.in` namespace. See NFH docs — Setting up the network environment.

3.2.6 DEG

Digital Energy Grid — the broader architecture from which IES Data Exchange inherits primitives (Energy Resource, Energy Catalogue, Energy Contract, Energy Credentials). See Beckn DEG repo.

3.2.7 DDM

Decentralized Data Marketplace — the Beckn-protocol-based open network for buyers and providers to discover, offer, contract, and exchange datasets without a central middleman. The schema family describing the exchanged datasets is published at beckn/DDM; IES extends it with the `IES_*` schemas.

3.2.8 ERA

Energy Resource Address — a unique digital address for any energy asset (meter, generator, storage, EV). DEG primitive; surfaces in IES data exchanges via the `participants[]` and `provider` fields.

3.3 India energy sector

3.3.1 DISCOM

Distribution Company — the regulated entity that distributes electricity to consumers in a state or licence area (e.g. TPDDL, BSES, MSEDCL). In IES v1, DISCOMs are the sole issuer of electricity credentials about consumers, meters, and assets.

3.3.2 AMISP

Advanced Metering Infrastructure Service Provider — the entity (often a contracted vendor under RDSS) that deploys, owns, and operates smart meters and the associated Head-End System on behalf of a DISCOM. AMISPs typically deliver meter data to the DISCOM's MDMS under an SLA.

3.3.3 SERC

State Electricity Regulatory Commission — the state-level regulator that approves tariffs, reviews DISCOM filings, and issues regulatory orders. SERCs are the typical consumer of `IES_ARR_Filing` and publisher of `IES_Policy`/`IES_Program` (tariff) datasets.

3.3.4 ARR

Aggregate Revenue Requirement — the annual filing in which a DISCOM tells its SERC what total revenue it needs to recover for the coming year, broken down by cost head. The basis on which tariff is set.

3.3.5 APR

Annual Performance Review — the mid-year reconciliation filing in which a DISCOM reports actual costs and revenues against the ARR projections.

3.3.6 MYT

Multi-Year Tariff — the regulatory framework where tariffs are set for a multi-year control period (typically 3–5 years) with annual true-ups.

3.3.7 True-up

The end-of-period adjustment by which actual costs are reconciled against the approved ARR, with the difference recovered or refunded through future tariffs.

3.3.8 ACS-ARR gap

Average Cost of Supply minus Average Revenue from sale — the per-unit revenue shortfall a DISCOM carries. A standard KPI in regulatory and policy conversations.

3.3.9 ATC loss

AT&C loss — Aggregate Technical & Commercial loss — the percentage of units injected into the network that are not realised as billed revenue, combining technical losses (line losses) and commercial losses (theft, under-billing, uncollected dues).

3.3.10 HT-LT

High Tension / Low Tension — voltage-class classification of consumers. HT typically >=11 kV (industrial, commercial); LT typically <= 440 V (residential, small commercial).

3.3.11 DT

Distribution Transformer — the transformer that steps voltage down to LT for delivery to residential and small consumers. The unit of granularity for many distribution-network operations and analytics.

3.3.12 Feeder

A medium-voltage line emerging from a substation that supplies a group of DTs and HT consumers.

3.3.13 ToD

Time of Day tariff — a tariff structure in which the per-unit rate varies by time block (peak, off-peak, etc.).

3.3.14 Regulatory asset

A deferred cost that the regulator allows the DISCOM to recover through future tariffs rather than the current year — typically used when actual costs significantly exceed approved ARR.

3.3.15 NP

Network Participant — any entity (DISCOM, regulator, AMISP, aggregator) joined to an IES network with a published Beckn subscriber record and a verified DeDi namespace.

3.3.16 RDSS

Revamped Distribution Sector Scheme — the Government of India scheme (announced 2021) under which DISCOMs receive central funding for smart metering, feeder segregation, and loss-reduction, in exchange for performance-linked obligations. RDSS is the policy context behind most of India's AMI deployment.

3.3.17 CIS

Consumer Information System — the DISCOM's billing and consumer-master database. The system of record for customer numbers, addresses, tariff categories, and billing history.

3.3.18 NMS

Network Management System — the DISCOM's operational system for monitoring the distribution network (substations, feeders, DTs).

3.3.19 HES

Head-End System — the AMI software layer that talks to smart meters over the field network, collects readings, and forwards them to the MDM. Owned by the AMISP in an RDSS deployment.

3.3.20 MDMS

Meter Data Management System (also **MDM**) — the utility-side platform that validates, stores, and analyses meter data received from the HES. The DISCOM's system of record for meter readings.

3.3.21 DER

Distributed Energy Resource — any generation, storage, or controllable load on the consumer side of the meter (rooftop solar, battery, EV charger, behind-the-meter genset). Increasingly the subject of credentials and data exchanges in IES.

3.4 Standards and protocols

3.4.1 CIM

Common Information Model — the IEC 61970 / 61968 data model for power system information exchange. The lingua franca for HES<->MDM and enterprise integrations.

3.4.2 IEC 61968

The series of standards for **information exchange between distribution management systems**. Part 9 (`IEC 61968-9`) specifically covers meter reading and control interfaces. The CEA's RDSS AMI interoperability guidance names IEC 61968 / IEC 61968-100 alongside MultiSpeak and Web Services for HES<->MDMS integration.

3.4.3 DLMS-COSEM

The wire protocol between a **smart meter** and the AMI head-end (also referred to as **IS 15959**, the Bureau of Indian Standards profile aligned with the international `IEC 62056` family). The default for Indian RDSS smart-meter deployments.

3.4.4 MultiSpeak

A utility software interoperability standard widely used in North America, named alongside CIM and IEC 61968 in India's AMI interoperability guidance. Version 3.0 is the common reference in Indian RDSS specifications.

3.4.5 OpenADR

Open Automated Demand Response — the protocol for sending demand-response and flexibility signals to and from distributed resources. Version 3.1.0 is current. Used in IES for DR/DER events and flexibility reporting — **not** for generic interval meter reads.

3.4.6 XBRL

eXtensible Business Reporting Language — the international standard for tagged, comparable structured financial and regulatory filings. The target structured format for `IES_ARR_Filing`. iXBRL embeds XBRL tags in human-readable XHTML.

3.4.7 Akoma Ntoso

The OASIS LegalDocML XML standard for structured legal documents. Recommended for the order text of tariff and regulatory documents in IES (`IES_Policy`).

3.4.8 DCAT 3

Data Catalog Vocabulary version 3 — the W3C standard for describing datasets and data services. Recommended metadata format for tariff schedule discovery in IES.

3.4.9 MQTT

A lightweight publish/subscribe messaging protocol commonly used for IoT and event-driven data flows. MQTT 5.0 is the current version. In IES, MQTT is an optional adapter for event or near-real-time meter telemetry — not the default for batch reads.

3.4.10 W3C VC

Shorthand for W3C Verifiable Credentials Data Model. See Verifiable Credential.

Chapter 4

Pathways

India Energy Stack (IES) Pathways provide structured, step-by-step onboarding and integration roadmaps designed for specific energy sector participants — including **utilities**, **regulators**, **AMISPs**, and **third-party service providers** — to systematically adopt IES capabilities.

4.1 Why Pathways?

The India Energy Stack introduces a rich collection of standards (DIDs, DeDi Registries, W3C Verifiable Credentials, Beckn Protocols). Adopting these capabilities involves coordinates across multiple internal IT, commercial, and operational divisions.

Rather than diving into raw technical standards, a **Pathway** organizes these tasks into a logical sequence, aligning high-level business goals with engineering requirements.

4.2 Available Pathways

Currently, the following pathways are available:

Pathway	Who is it for?	Capabilities Integrated	Description
Utility Pathway	Electricity Distribution Utilities	Identifiers · Registries · Energy Credentials · Data Exchange · DER Visibility	Comprehensive roadmap to issue Energy Passports, execute Smart Meter Data Exchange, issue verifiable bills, and achieve DER visibility.
Secretariat Pathway	IES Secretariat & Operators	Identifiers · Registries · Network Governance · Schemas & Protocols	Detailed roadmap for registry setup, verifying and whitelisting participants, governing the Beckn network, and managing canonical schemas.

4.3 How to Use the Pathways

Each pathway is divided into operational phases (e.g., *Preparation*, *Getting Started*, *Energy Passport*, etc.).

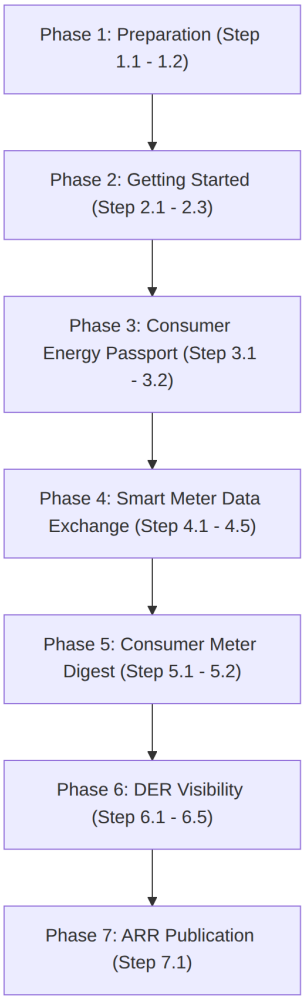
- **Collapsible Sections**: Technical and operational details are housed inside collapsible panels (`<details>`) so you can scan the high-level roadmap or dive deep into implementation specifics as needed.
- **Checks and Prerequisites**: Use the inline checklists to assign tasks to your IT, Commercial, and Legal teams.
- **Documentation Links**: Under each step, direct links to the relevant reference specifications, registries, and deployment guides are provided.

4.4 Utility Pathway

Welcome to the **Utility Pathway**. This guide provides an actionable and structured roadmap for an electricity distribution utility to easily adopt the capabilities of the India Energy Stack (IES).

To keep this guide extremely clean and focus on utility progress, technical specifications are referenced via hyperlinks rather than repeated. Expand any step to find actionable guidelines, cross-team advice, and prework checkpoints.

4.4.1 Roadmap Overview



4.4.2 Prework & Pre-Alignment Matrix

Before commencing the integration pathway, we recommend aligning the following internal teams and core systems. Setting up these channels early ensures a seamless deployment experience:

Department / Role	System / Resource Involved	Purpose in Pathway
DNS & Web Master	Utility domain controller (`tpddl.co.in`)	Exposing public `did:web` and verifying DeDi namespaces
IT & Security Admin	Cloud KMS / HSM, Docker Environments	Securing signing keypairs and deploying OpenCred/ONIX services
Legal / Signatory	Corporate credentials, API Setu	Submitting whitelists to IES Secretariat and registering on DigiLocker
Billing & CRM Teams	CIS, MDMS, Customer Databases	Mapping telemetry, tariff profiles, and triggering billing credentials

---

4.4.3 Phase 1: Preparation (Identity & Addressing)

In this phase, you will establish your institutional cryptographic identity and define clear naming grammars for your grid resources and consumers.

Step 1.1: Establish Your Institutional Identity (did:web)

[tip] Phase Advice

> Set up a quick call with your DNS administrator as your very first step. Securing a dedicated subdomain takes only a few minutes but forms the foundation of your secure cryptographic brand.

checklist Prework Required

- Confirm that your web admin has write-access to your target domain (e.g., `ies.tpddl.co.in`) to host the verification path.

Execution Guidance

A `did:web` identifier leverages your existing DNS and SSL infrastructure to publish your public keys. 1. **Assign a Dedicated Domain**: Allocate an institutional subdomain, e.g., `ies.tpddl.co.in`. 2. **Expose the DID Document (First Step)**: Host your verification keys in a standard `did.json` file served over HTTPS under the path: `https://ies.tpddl.co.in/.well-known/did.json` Construct the `did.json` document according to the W3C DID Core specification, ensuring your public verification key is correctly mapped and referenced. Refer to Step-by-step: publish your did:web for the structural checklist.

References & Anchors

- Identifiers & Addressing Overview
- Resolution & Routing Specification (Detailed resolution rules for `did:web` endpoints)
- Step-by-step: publish your did:web (Document parameters)
- Identifiers Checklist

Step 1.2: Define Your Naming Grammars (Identifiers and Addressing)

[tip] Phase Advice

> Aligning your IES identifiers with your existing SAP, GIS, or CIS master codes avoids double-mapping. Wrap your existing internal serials rather than replacing them!

Execution Guidance

Map your internal customer numbers, SAP codes, and meter serial numbers to the standard `did:web` path-based naming grammars. Before constructing your identifiers, internalise the rule that **an identifier is just a name** — it carries no inherent business meaning, and every identifier must resolve to a DID Document. See Identifiers and Addressing -> Appendix D.

The reference patterns (all `did:web` under your DISCOM's own domain): 1. **Consumers (holder DID)**: `did:key:<wallet-generated>` — generated by the wallet, not the DISCOM. The consumer's existing CIS number stays as the literal `customerNumber` string inside the credential. 2. **Grid Assets**: `did:web:<your-domain>:assets:<asset-class>:<internal id>` *(e.g., `did:web:ies.tpddl.in:assets:transformer:DT-11KV-F02-452`)* 3. **Smart Meters**: `did:web:<your-domain>:assets:meter:<manufacturer code>_<serial number>` *(e.g., `did:web:ies.tpddl.in:assets:meter:GEN_12345678`)* 4. **Service Connections**: `did:web:<your-domain>:connections:<connection id>` *(e.g., `did:web:ies.tpddl.in:connections:CON-90234`)*

> [!NOTE] All identifiers here use standard W3C `did:web`. There is no separate `did:dedi` method; when DeDi is used as the discovery layer for keys or DID documents, the method on the wire is still `did:web`. See Identifiers and Addressing -> Appendix A.

References & Anchors

- Identifiers and Addressing — How DIDs work, and the three methods IES uses
- Identifier vs. record
- Identifying assets, meters, connections, datasets

4.4.4 Phase 2: Getting Started (Registry Setup)

Establish your administrative namespaces on the public Decentralized Directory (DeDi) and register your active endpoints with the network.

Step 2.1: Setup DeDi Namespace

[tip] Phase Advice

> Using an institutional role-mailbox (e.g., `registry-admin@tpddl.co.in`) instead of a personal account ensures that namespace controls remain securely managed by your IT department across staff rotations.

[warn] Caution

> **Namespace Domain Verification**: Domain ownership verification leverages your existing DNS TXT record or setup rather than requiring you to request a brand new record class from your network administrators, easing compliance boundaries.

Execution Guidance

1. Register a secure role-mailbox on the DeDi Portal.
2. Establish a namespace matching your utility short-code (`<utility>`).
3. Expose the verification token in your DNS TXT records. Refer to Registries — Step 3 (verify your domain) for formatting.

References & Anchors

- Why DeDi (and the three questions it answers)
- Registries — Step-by-step (claim namespace + create registries)
- Registries Checklist

Step 2.2: Create Operational Registries

[tip] Phase Advice

> Maintain strict separation of duties! Use separate registries for your keys, revocation list, and Beckn profiles to keep access privileges isolated.

Execution Guidance

Create and initialize the following registries under your namespace (OpenCred auto-creates several of these on first boot — see Idempotency note for OpenCred users): * `opencred-key-registry` (tag `public_key`): Versioned signing keys. **Auto-created by OpenCred.** * `vc-revocation-registry` (tag `revocation`): Real-time verifiable credential revocation. **Auto-created by OpenCred.** * `subscribers-test` & `subscribers-prod` (tag `beckn_subscriber`): Beckn participant identity records. **You create these manually** — see As a Beckn Network Participant. * **Optional fallback: publish DID doc to DeDi.** Set `OPENCRED_DEDI_HOST_DID_DOC=true` (or call `/v1/keys/publish`) so the DID document is reachable via DeDi when your HTTPS endpoint is temporarily unreachable.

References & Anchors

- The registries you'll touch in IES (by role)
- As a DISCOM / issuer running OpenCred
- As a Beckn Network Participant
- OpenCred Key & DID Publishing Configuration

Step 2.3: Register with India Energy Stack

[tip] Phase Advice

> Pre-verify your onboarding package! Double-checking that your URLs, service areas, and JWK keys are formatted correctly before emailing ensures immediate approval.

checklist Prework Required

- Stand up your subscriber registries (from Step 2.2) and prepare your endpoint URL payloads.

Execution Guidance

Email your registration package (utility short-code, legal name, service area list, endpoints, and `did:web` JWK) to the IES Secretariat: * **Primary Email**: IES.Secretariat@fsrglobal.org * **Alternate Email**: ies@recindia.com

The Secretariat will verify your credentials and register your endpoints inside the authoritative `ies-discoms-reference-registry` path.

References & Anchors

- How to apply for an IES listing
- Registries Checklist

4.4.5 Phase 3: Consumer Energy Passport (Verifiable Credentials)

Provide citizens with a secure, tamper-evident digital passport of their utility connection parameters, load limits, and registered assets.

Step 3.1: Setup Credential Issuance (Consumer Energy Passport)

[tip] Phase Advice

Leverage your existing Customer Relationship Management (CRM) tables. Since the Passport is a composite of standard customer master data, you only need to expose your existing billing/CIS databases to OpenCred.

Execution Guidance

1. **Connect CRM Systems**: Map customer master data (sanctioned load, tariff slabs, connection status) from your CRM (e.g., SAP IS-U) to OpenCred.
2. **Configure Authentication**: Setup OTP or Aadhaar reference authentication gateways on your client portal.
3. **Deploy OpenCred**: Deploy the containerized OpenCred service using your secured P-256 signing keys.
4. **CRM Status Logging & Issuance**: Store active credential UUIDs in your database to coordinate revocations. Format and trigger credential issuance via OpenCred's `/v1/credentials/issue` API using the `ElectricityCredential` JSON shape.
5. **Verify Revocation Handling**: Ensure the verifier's revocation check logic is active. Be mindful of the silent-skip behavior if DeDi namespaces are misconfigured (refer to the OpenCred Silent-Skip Warning).
6. **Validate the Issued Credential**: Call the OpenCred verification endpoint (`/v1/credentials/verify`) using the issued VC to test the full verification cycle. This confirms that the verifier can successfully resolve the issuer's DID and validate the cryptographic signature. Note that if DeDi is disabled or the namespaces are not fully synchronized, the validation engine might exhibit specific resolve behaviors (refer to the OpenCred Resolve Verification Check).

References & Anchors

- Energy Credentials Overview
- Energy Credentials Deployment Guide
- OpenCred Resolve Verification Check & Env Options
- OpenCred Silent-Skip Revocation Warning
- Batch Issuance at Scale (Queue & Workers)
- Multi-Replica Deployment Guidance
- Consumer Energy Passport Use Case
- Consumer Energy Passport Checklist
- Consumer Energy Passport Schema (ElectricityCredential) Reference

Step 3.2: Register with DigiLocker

[tip] Phase Advice

API Setu compliance checks can take time. Start your legal and institutional registrations on API Setu early in Phase 3 so that production gateways are cleared by the time your code is ready.

checklist Prework Required

- Secure authorization letters and corporate certificates from your legal department for API Setu access.

References & Anchors

- DigiLocker Integration Guide
- Issuing Credentials Checklist

4.4.6 Phase 4: Smart Meter Data Exchange (Beckn Data Pipes)

(See Data Exchange — Core Concepts for the underlying Beckn primitives.)

Enable federated, policy-governed data sharing of smart meter telemetry and master data with authorized third parties.

Step 4.1: Setup BPP Nodes (BECKN Network)

[tip] Phase Advice

Deploying the ONIX adapter as a Docker service takes less than 10 minutes. Run the test sandbox local container first to easily verify your configurations before going to production.

Execution Guidance

1. Deploy the standard Docker-based Beckn ONIX container.
2. Generate your Ed25519 node keypair.
3. Register your BPP credentials in DeDi `subscribers-test`.

References & Anchors

- Data Exchange Concepts
- Data Exchange Quick Start
- ONIX Registry Setup
- Data Exchange Checklist

Step 4.2: Establish MDM Integration for Telemetry

[tip] Phase Advice

Protect your MDM database performance! Configure your MDMS adapter to write query results to a fast read-only caching replica or cache intermediate intervals to avoid overloading production transactional databases.

[warn] Caution

Batch Telemetry Latency: MDM database queries can be slow and can violate Beckn's transaction timeouts (usually 5 seconds). Ensure your telemetry API is highly optimized.

Execution Guidance

Map HES DLMS-COSEM or IEC 61968-9 interval profiles to standard **IntervalProfile**, **DailyProfile**, **InstantaneousProfile**, and **EventProfile** formats.

References & Anchors

- Smart Meter Data Exchange Use Case
  - How It Works
- IES Meter Data Model
- MeterData Schema Specification

Step 4.3: Integrate Customer Master Data

[tip] Phase Advice

Billing and customer profile files map directly to the **CustomerProfile** schema. Ensure your CRM export script correctly aligns the tariff category, connection status, and sanctioned load.

References & Anchors

- MeterData Attributes & Customer Schema

Step 4.4: Establish Data Exchange Authorisation

[tip] Phase Advice

> Leverage the **MeterDataRequestCredential** schema to formalise incoming B2B third-party data authorisations. The scope of the actual request for sharing can be a subset of the broader data authorised.

Execution Guidance

While the technical authorisation logic is ultimately left to the utility, we suggest executing scoped access control by requiring the BAP to present a **MeterDataRequestCredential** (see credential example) OR a **Consumer Energy Passport with consent**. Your BPP ONIX adapter should verify these presented credentials at runtime before dispensing any interval profiles.

[warn] Caution

> **Scoped Access Violations**: Never expose granular consumer interval data without verifying that the presented credential permits that specific access window and profile.
>
> [!WARNING] **Clarity Gap**: Standardized B2B automated token validation policies on BPP ONIX are currently under-specified in the core guidelines, requiring custom token validation structures for consumer-consented exchanges.

References & Anchors

- Data Exchange Security & Auth
- MeterDataRequestCredential Schema
- MeterDataRequestCredential Example

Step 4.5: Enable Meter Data Exchange Go-Live

[tip] Phase Advice

> Run a parallel pilot! Trade smart meter telemetry with a friendly test consumer or partner BAP to confirm that signing, encryption, and logging flows operate perfectly before live production exchange.

References & Anchors

- Data Exchange Checklist

4.4.7 Phase 5: Consumer Meter Digest (Electricity Bill)

(See Use cases -> Consumer Meter Digest.)

Move beyond static PDFs to compile and issue verifiable, machine-readable monthly electricity bills.

Step 5.1: Create the Consumer Meter Digest Verifiable Credential

[tip] Phase Advice

We suggest utilizing a credential that directly includes the **MeterData** schema. When compiling the Digest for a billing cycle, the `CustomerProfile` and `BillingProfile` **must** be included. Additionally, an `IntervalProfile` containing intervals that span the entire billing period is strongly recommended to provide complete consumption transparency.

Execution Guidance

1. **Request the Data**: Programmatically query your Data Exchange nodes with a `MeterDataRequest` spanning the billing duration and specifically targeting the required profiles.

   Example `MeterDataRequest` for compiling a Digest:

```
{
    "@context": "https://india-energy-stack.github.io/ies-accelerator/schemas/MeterDataRequest/v0.6/context.j
    "@type": "MeterDataRequest",
    "resources": [
        "did:dedi:ies:meter:IN-MH-MTR-89721"
    ],
    "scope": "ResourceOnly",
    "from": "2026-04-01T00:00:00Z",
    "duration": "P1M",
    "capabilitiesRequested": {
        "profiles": [
            { "profileType": "CustomerProfile" },
            { "profileType": "IntervalProfile" }
        ]
    }
}
```

2. **Structure the Credential**: Package the resulting `MeterData` payload inside the data subject of your verifiable credential envelope.
3. **Sign and Issue**: Sign and issue the verifiable credential utilizing your OpenCred service.

References & Anchors

- Electricity Bills and Digest Use Case
  - How It Works
- Consumer Meter Digest Checklist

Step 5.2: Link Bills to DigiLocker

[tip] Phase Advice

Re-use your DigiLocker gateway! Since you cleared API Setu audits in Step 3.2, adding the verifiable bill credential is a simple path extension on your existing issuer record.

References & Anchors

- DigiLocker Issuer Setup

4.4.8 Phase 6: DER Visibility (Distributed Energy Resources)

Acquire real-time visibility into solar generation, battery storage, and feeder loading to balance the grid.

Step 6.1: Establish DER Sources

[tip] Phase Advice

Onboard consumer generation assets dynamically. Capture DER parameters (solar inverter rating, battery capacities) and map them to standard DIDs (e.g. `did:dedi:<utility>:assets:inverter:<inverter-serial-no>`).

Step 6.2: Start Receiving Data via Daily Profiles

[tip] Phase Advice

Prioritize your own grid telemetry first! Start by ingesting and mapping the utility's own smart meter consumption and generation profiles. Once established, expand your integration as a next step to connect with independent generators and public EV charging stations.

checklist Prework Required

- Confirm that smart meter generation register reads are active and mapped to standard profiles.

Step 6.3: Grid Topology & Data Exchange

[tip] Phase Advice

Publish hierarchical relationships. By linking substations, feeders, distribution transformers, smart meters, and DER assets, grid operators can programmatically map downstream loading. Leveraging **Data Exchange** networks enables you to share these grid datasets securely.

References & Anchors

- Grid Identifiers & Hierarchy Guide

Step 6.4: Build a Feeder-Level Aggregator

[tip] Phase Advice

Keep fine-grained customer PII out of grid planning! By aggregating meter telemetry at the distribution transformer or feeder level, you can share real-time loading profiles without exposing individual customer details. These aggregated feeds are published securely via your **Data Exchange** nodes.

Leveraging Associations for Aggregation: Use the optional `parentResources` property within the `CustomerProfile`'s `Association` block to trace every smart meter to its upstream parents (like feeder and Distribution Transformer). This deterministic linkage allows you to programmatically sum individual telemetry feeds into a single `AggregatedFeeder` payload!

[warn] Caution

Imputation for Zero Readings: Ensure your aggregator engine handles missing or zero readings securely (e.g., forward-fill or mean-imputation) to prevent aggregated peaks from showing artificial drops.

Since the **MeterData** schema is used for telemetry exchange, we strongly suggest utilizing the `AccumulationBehaviour` property for annotating aggregation datasets. Ensure that aggregated datasets explicitly annotate `SUMMATION` as the `accumulationBehaviour`. Refer to this Aggregated Feeder Example showing a feeder-related aggregated `IntervalProfile` for a day.

Step 6.5: Build a DER Visualiser Dashboard

[tip] Phase Advice

Keep it visual! Query feeder aggregations over your Beckn BAP node and plot live timeseries graphs showing feeder loading, battery state-of-charge, and reverse solar back-feeding.

4.4.9 Phase 7: ARR Publication (Annual Revenue Requirement)

Publish your Annual Revenue Requirement data in a standardized, machine-readable format to enable programmatic tariff analysis and regulatory transparency.

Step 7.1: Map Existing Data to ARR Schema

[tip] Phase Advice

Rather than building new reporting pipelines from scratch, we suggest using your existing regulatory data sets and mapping them directly to the provided ARR schema.

Execution Guidance

1. Extract historical, current, and forecasted monetary data from your existing tariff orders and regulatory filings.
2. Focus strictly on providing **only machine-readable data related to monetary data**. Avoid embedding unstructured text or PDF blobs.
3. Map this extracted monetary data directly to the canonical `ArrFiling` schema structure.
4. **Note your experiences**: Treat this as an iterative mapping exercise. Document your experiences, friction points, or any missing schema fields encountered during the data mapping process to help inform future specification refinements.

References & Anchors

- ARR Filing Schema Reference
- ARR Filing Machine-Readable Example

Chapter 5

Identifiers and Addressing

In a hurry? Jump straight to the Checklist — every step with checkboxes.

This page is the single home for everything IES has to say about identifiers. Read the first three sections and you will have enough to claim your DISCOM's identity on the network and issue your first credential. The appendices are there when you want more depth, want to issue at scale, or need to identify assets, meters, and datasets too.

> **About the walkthroughs.** The concrete commands below use **OpenCred** — see the glossary for what it is, its W3C compliance, its DeDi integration, and release links. Any W3C-compliant signing pipeline that publishes the same `did.json` and VC-2.0 proofs is a drop-in replacement.

5.1 Why this matters

When your DISCOM hands a consumer a digital electricity credential, or shares meter data with a regulator, the recipient needs to answer one question on their own: *"Is this really from TPDDL?"* If they have to call you, email your IT team, or trust a screenshot, the system does not scale and it is not really verifiable.

The way digital-public-infrastructure stacks solve this is with **Decentralized Identifiers (DIDs)**. You publish a small JSON file on a web address you control. That file lists the public key you sign things with. Anyone — a wallet, another DISCOM, a regulator — can fetch that file over plain HTTPS and check signatures on their own. No central authority, no API key, no special middleware.

You need two things: **a domain you own** and **a key your IT team can generate**. The rest of this page shows what to put where.

5.1.1 Pick your role

The same page covers four roles. Skim the row that matches you to jump to the right section.

If you are…	Read	Then
A DISCOM / issuer (you sign and emit ElectricityCredentials)	§(a) Org identity -> Energy Credentials — Set up OpenCred and publish your did:web	Energy Credentials — Issue your first credential, Appendix C (asset IDs)
A Beckn participant (BAP / BPP, aggregator, AMISP, trading platform)	§(b) Beckn network identity -> Appendix E	Registries — by role for the registry mechanics

If you are…	Read	Then
A regulator (you license DISCOMs and may sign credentials yourself)	§(a) Org identity — same `did:web` flow as a DISCOM	Note that your `did:web` is what DISCOMs cite as `issuer.idRef.issuedBy`; see Where each ID goes in a credential
A verifier or wallet (you receive and check credentials)	Appendix A (DID methods) -> Energy Credentials — Verify -> Appendix F (holder binding at presentation time)	Registries — Verifying a credential for the end-to-end resolution walk

---

5.2 Two identities you'll set up (and why)

A DISCOM on IES needs **two** identifier setups, and they exist for different reasons. They use **different keys**, generated by different procedures (the credential-issuance key is an EC P-256 key generated as part of your OpenCred / signing-service setup; the Beckn network key is an Ed25519 keypair generated per the NFH onboarding procedure), and they live in different registries. They share only the organisational root — your domain — and serve different verifiers.

5.2.1 (a) Org identity — for credentials and data-exchange payloads

This is your DISCOM's `did:web`. It is the issuer string on every ElectricityCredential you sign, and the root that **all the IDs you reference inside payloads** — meter DIDs, transformer DIDs, connection DIDs — extend by adding path segments. Verifiers of a credential resolve only this one DID to check your signature; the asset DIDs ride inside the signed payload as stable references.

Who	Identifier method	What it looks like
Your DISCOM (the issuer)	`did:web` on a domain you own	`did:web:ies.tpddl.in`
A regulator	Same — `did:web` on their own domain	`did:web:ies.derc.gov.in`
A consumer holding a credential *(optional)*	A holder identifier — `did:key` (wallet) or `tel:+91...` (RFC 3966) — set when you want presentation-time proof of subject. See Appendix F.	`did:key:z6Mkj...` or `tel:+919876543210`
Your meter / transformer / feeder	`did:web` under your domain	`did:web:ies.tpddl.in:assets:meter:MET-001`

Two steps get you here:

1. **Pick a domain or subdomain you control.** Either works. Most DISCOMs use a dedicated subdomain (e.g. `ies.tpddl.in`) so the credential-issuing identity is cleanly separated from the marketing site, but a bare apex domain (`tpddl.in`) is equally valid. The host you pick becomes the host portion of your `did:web`, and you will publish one small JSON file under it — `did.json` — that declares your public key.

   > **About path segments.** `did:web` lets you encode a sub-path with colons. If you don't want to host at `.well-known/`, host the document at any path and reflect it in the DID via the colon hierarchy:

DID string	DID document URL
`did:web:ies.tpddl.in`	`https://ies.tpddl.in/.well-known/did.json`
`did:web:tpddl.in:ies`	`https://tpddl.in/ies/did.json`
`did:web:tpddl.in:ies:issuer`	`https://tpddl.in/ies/issuer/did.json`
`did:web:tpddl.in%3A8443`	`https://tpddl.in:8443/.well-known/did.json` (port encoded as `%3A`)

Same identifier system covers your DISCOM's own identity *and* every asset ID you reference inside payloads (meters, transformers, datasets) — see Appendix C.

2. **Generate a key pair and publish `did.json`.** The concrete commands — install OpenCred, generate the signing key, assemble `did.json` from the container, publish to your web host, and verify — live in **Energy Credentials -> Set up OpenCred and publish your did:web**. Keeping them with the rest of the OpenCred operations means you don't context-switch between chapters during a single setup.

That is everything credential issuance requires. **You do not need to be listed in any IES-side DISCOM registry to issue credentials.** Verifiers fetch your `did.json` for the key and check the signature; that is the only mandatory leg of the trust chain. If you also have a regulator (DERC / KERC / etc.) who can vouch for your licence, set `issuer.idRef` to point at them — verifiers will resolve the regulator and treat the credential as licence-anchored. `issuer.idRef` is optional in both the v1.2 schema and the W3C VC Data Model 2.0; only `issuer.id` and `issuer.name` are required. The IES DISCOMs reference registry is a separate, **Beckn-side** concern — see (b) below.

Internal consumer numbers, meter SLNOs, and asset codes do **not** need to change — they ride inside the credentials you sign with this DID. The simplest first credential carries no holder identifier (bearer style); when the verifier needs presentation-time proof of subject, bind to a wallet DID or `tel:` URI — full guidance in Appendix F.

5.2.2 (b) Beckn network identity — for participating on a Beckn network

To send and receive Beckn messages (search, select, init, confirm, on_status…), your `did:web` is not enough on its own. Beckn is a **trust-bounded network**: the Network Facilitator Organisation (NFO) curates who is on the network, and counterparties verify each message against that membership boundary. Two registries enforce this boundary:

- **Your own Beckn subscriber registry** under your verified DeDi namespace — declares your callback URL, your role (BAP / BPP), and your Ed25519 signing public key. Other nodes look this up to verify your message signatures and route to you.
- **The NFO's network reference registry** — a curated allow-list of which subscribers belong to the network. For IES, this is also where a "DISCOM" is recognised as a network participant. The NFO writes a reference entry pointing at your subscriber record; counterparties then know your record is in-network.

Different key (Ed25519, not P-256), different registries (a Beckn subscriber registry under your DeDi namespace, plus an NFO-side reference; not `.well-known/did.json`), different consumer (other Beckn nodes, not credential verifiers). The setup is independent from the credential-issuance flow.

The end-to-end practical flow is in Appendix E — Joining a Beckn network.

5.3 Publish your `did:web`

The end-to-end walkthrough — install OpenCred, generate the signing key, assemble `did.json` from the container, publish it on your web host, and verify it resolves from the outside — lives in **Energy Credentials**

-> Set up OpenCred and publish your did:web. Pick up there once you have a domain or subdomain decided per (a) above.

5.4 ID patterns you'll use day one

You only need two or three identifier shapes to start issuing credentials. They're collected here in one table.

Subject	Identifier	Example
Your DISCOM (issuer)	`did:web:<your-domain>`	`did:web:ies.tpddl.in`
A regulator	`did:web:<their-domain>`	`did:web:ies.derc.gov.in`
A consumer (holder identifier, optional)	`did:key:...` (wallet) **or** `tel:+91XXXXXXXXXX` (RFC 3966)	`did:key:z6MkjVQ8r4f3rPuY...`
The consumer's existing CIS account number	Plain string, kept as-is	`TPDDL-2025-00987654`

A few things worth knowing before you start:

- **Your CIS consumer numbers stay exactly as they are.** They go into `customerProfile.customerNumber` in the credential, in the same human-readable form your call centre and billing letters use today.
- **The holder identifier is optional and you can defer it.** See Appendix F for the bearer-vs-bound trade-off and the binding patterns.
- **You do not assign wallet DIDs.** If you opt for the wallet-DID pattern, the consumer's wallet (or DigiLocker) generates the `did:key` and sends it to you. You verify the consumer controls it (Appendix F explains how), then include it verbatim in `credentialSubject.id`. You never store the consumer's private key.

For identifiers covering assets (meters, transformers, feeders), service connections, and datasets, see Appendix C — these are not needed for your first credential.

5.5 Where each ID goes in a credential

Here is one filled-in ElectricityCredential v1.2 showing every identifier in one place. Everything in **bold** is something you control.

```
{
  "@context": [
    "https://www.w3.org/ns/credentials/v2",
    "https://schema.beckn.io/ElectricityCredential/v1.2/context.jsonld"
  ],
  "id": "urn:uuid:b2c3d4e5-0000-0000-0000-aabbccdd0001",
  "type": ["VerifiableCredential", "ElectricityCredential"],

  "issuer": {
    "id":   "did:web:ies.tpddl.in",
    "name": "Tata Power Delhi Distribution Limited",
    "idRef": {
      "_comment":  "optional — include only when citing a regulator",
      "issuedBy":  "did:web:ies.derc.gov.in",
      "subjectId": "derc.delhi.gov.in:TPDDL-REG-0042"
```

```
    }
  },

  "validFrom":  "2025-03-01T00:00:00+05:30",
  "validUntil": "2026-03-01T00:00:00+05:30",

  "credentialSubject": {
    "customerProfile": {
      "customerNumber": "TPDDL-2025-00987654",
      "energyResources": [{
        "id":   "did:web:ies.tpddl.in:assets:meter:MET-IMPORT-001",
        "type": "METER",
        "attributes": {"meterCapability": "AMI", "energyDirection": "Forward"}
      }]
    },
    "customerDetails": {
      "fullName": "Arjun Mehra"
    }
  },

  "proof": {
    "type": "Ed25519Signature2020",
    "verificationMethod": "did:web:ies.tpddl.in#key-1",
    "proofPurpose": "assertionMethod",
    "proofValue": "z58DAdFfa9SkqZMVPxAQpic7ndTaXoT..."
  }
}
```

Field	What it carries	Set by
`issuer.id`	Your `did:web`	You
`issuer.idRef` *(optional)*	A pointer the regulator gave you that confirms you are a licensed DISCOM in their service area. Include when you have a regulator to cite; omit otherwise — `issuer.idRef` is optional per both the v1.2 schema and W3C VC 2.0.	Regulator + you
`customerProfile.customerNumber`	Your existing CIS number, unchanged	You
`energyResources[].id`	A `did:web` for each meter / asset, built from your domain plus a path segment	You
`proof.verificationMethod`	A pointer back into your `did.json` saying which key did the signing	OpenCred / your signing pipeline
`credentialSubject.id` *(optional)*	A holder identifier — a wallet `did:key` or a `tel:` URI — set when you want presentation-time proof that the presenter is the legitimate subject	Set by you, after verifying the consumer controls it. See Appendix F.

Notice what is *not* required there: no central registry of consumers, no national ID, no separate identifier system that competes with your CIS. Everything cross-references your DID document plus the regulator's,

both of which are just static files on websites you control.

5.6 Checklist

A DISCOM (or any organisation) joining IES, working top-to-bottom. Each item maps to a section above.

5.6.1 1. Decide your organisation's public name on the network

Pick a domain or subdomain you control (e.g. `ies.discom.org`). It anchors your `did:web` and hosts your `did.json`. For a sub-path, reflect it in the DID (`did:web:discom.org:ies`) — see What you'll need.

- [ ] Domain/subdomain selected and available
- [ ] Decided whether `did.json` lives at `/.well-known/did.json` or a sub-path

5.6.2 2. Set up your digital identity

Run OpenCred, generate an EC P-256 key (private key in KMS), and publish the DID document on your domain.

- [ ] OpenCred running with `OPENCRED_ISSUER_DID_METHOD=web` and your subdomain
- [ ] `did.json` published at `https://<subdomain>/.well-known/did.json`
- [ ] `curl https://<subdomain>/.well-known/did.json` returns the expected DID

5.6.3 3. Agree how you will identify consumers and assets

Wrap existing internal numbers into network identifiers — no renumbering; deterministic and reversible (see Appendix C). DISCOM: `did:web:<subdomain>`; consumer holder: `did:key:<wallet>`; consumer CIS number kept verbatim as `customerNumber`; asset: `did:web:<subdomain>:assets:<class>:<internal-id>`.

- [ ] Mapping rule agreed for consumers, meters, assets, documents
- [ ] Confirmed no internal renumbering needed

5.6.4 4. Decide what stays private and what becomes public

For each identifier type, decide internal-only vs. network-visible.

- [ ] Privacy decision recorded per identifier type

5.6.5 5. (Beckn only) Get referenced in the IES DISCOM registry

Not needed for credential issuance (your `did:web` + `issuer.idRef` carry that trust). Required only to join the **inter-DISCOM data exchange network**, where the network operator references you into its curated registry.

- [ ] Referenced in the network registry — only if joining the data exchange network

5.6.6 6. (Beckn only) Publish your Beckn subscriber record

For data exchange over Beckn, publish a subscriber record under your verified DeDi namespace so nodes find your callback URL and Ed25519 key — see Appendix E.

- [ ] DeDi namespace verified (TXT-record domain ownership) and Ed25519 keypair generated
- [ ] Subscriber record published (subscriber ID, callback URL, role, signing public key)
- [ ] Lookup resolves on the DeDi fabric; the NFO has referenced your record

5.6.7 7. Nominate your team

- [ ] IT SPOC (OpenCred / keys / ONIX) — name, email, phone
- [ ] Governance / Compliance SPOC (approves public publishing) — name, email, phone

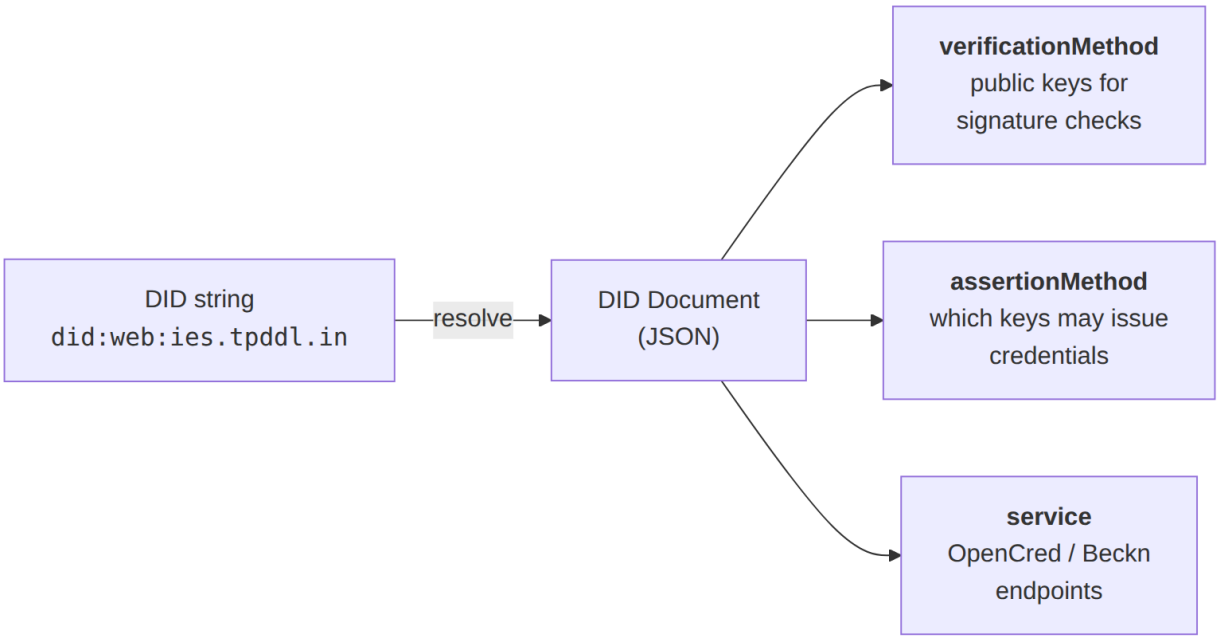
Once these are in place you can issue ElectricityCredential v1.2; for issuance/revocation commands see Energy Credentials -> Issue your first credential.

5.7 Appendix A — How DIDs work, and the three methods IES uses

This appendix exists so that if a colleague asks "but how does this actually work?" you can answer. Skip it if you are mid-deployment.

5.7.1 What's in a DID document

A DID string by itself is just a name. The useful part is the **DID document** it resolves to — the small JSON object you published in the step above. That document carries the public keys a verifier needs, and optionally the service endpoints where the network can reach you.



Two rules cover most confusion in DID systems:

- **An identifier is just a name.** Never parse it to extract meaning ("if the string contains `tpddl`, treat it as a TPDDL credential" is a bug waiting to happen). Resolve it first; read the document; decide from that.
- **The identifier and the record it resolves to are different things.** The DID is what travels in a credential or a Beckn message. The record is what a verifier fetches to make a trust decision.

5.7.2 The three DID methods IES uses

IES uses three standard W3C DID methods. All three are listed in the W3C DID Spec Registries, so any standards-compliant verifier already knows how to resolve them. **There is no separate `did:dedi` method.**

When you see DeDi mentioned, it is acting as a key-discovery layer for `did:web` (or `did:key`), not as a new DID method.

For the underlying theory — when each method is appropriate, what they buy you, what they cost — OpenCred -> DIDs is a good reference. This section is the IES-flavoured summary.

`did:web` — the one your DISCOM will use

The DID is a URL in disguise. `did:web:ies.tpddl.in` resolves to `https://ies.tpddl.in/.well-known/did.json`. Path segments after the host become URL path segments:

```
did:web:example.com:students
->  https://example.com/students/did.json
```

A port becomes `%3A` (so `did:web:localhost%3A8443` is `https://localhost:8443/.well-known/did.json`).

There are two ways `did:web` shows up in IES:

- **Self-hosted by the institution.** You serve `did.json` on your own domain, as Step 3 above shows. Any verifier in the world can resolve it. This is the default and the right starting point for every DISCOM, regulator, and aggregator with a public web presence.
- **Anchored on DeDi.** The identifier is still your DISCOM's own `did:web:<your-domain>`. The difference is that, in addition to (or instead of) hosting `did.json` yourself, you publish your public key — and optionally a frozen `did.json` snapshot — into DeDi's key registry, under a namespace tied to your verified identity. DeDi-aware verifiers can then fetch your key from DeDi via its API. This is useful as a fallback when your domain is unreachable, or where the network wants a curated allow-list of registered DISCOMs. The method is still `did:web`; DeDi is just a second discovery path for the same key. Note that a DeDi-only document is not served at `.well-known/did.json`, so a plain W3C `did:web` resolver will not see it — only DeDi-aware tooling will.

The strengths of `did:web` for an institutional issuer:

- Trust roots in your existing TLS certificate, which public CAs already validate.
- Key rotation is one file replace.
- No new infrastructure beyond a static-file host.

The main limitation is that domain hijack would mean identity hijack of the issuer key. If you cite a regulator in `issuer.idRef`, credential-level mitigation comes from that **licensing assertion**: a verifier resolves the regulator's DID and confirms the regulator vouches for this DISCOM. Even if a domain is hijacked, the attacker cannot forge the regulator's vouching record without also compromising the regulator's key. When `issuer.idRef` is omitted (e.g. for a pilot or non-regulated issuer), this leg of the mitigation isn't available — verifiers fall back to whatever out-of-band recognition they have of your `did:web`. For the inter-DISCOM data exchange network, the NFO's curated network reference registry adds a separate mitigation by gating which subscribers are on the network.

`did:key` — what wallets give consumers

For `did:key`, the public key *is* the identifier. The verifier decodes the string directly — no network call, no website needed. That is exactly what you want for consumers, who do not own domains:

```
did:key:z6MkjVQ8r4f3rPuY7CG2D6Lf8WJxJBs5sjkR8d3v2Bv4nP4Z
```

IES uses `did:key` for:

- Consumer holder DIDs, generated client-side by a wallet or DigiLocker.
- DISCOM dev/test deployments before a real subdomain is provisioned.
- Demos and field deployments where there is no domain.

The price you pay is that you cannot rotate the key — rotating means a new DID. So `did:key` is fine for short-lived or device-bound identities, but inappropriate for long-lived institutional issuers (which is why DISCOMs use `did:web`).

`did:jwk` — same shape as `did:key`, JWK-encoded

`did:jwk` encodes the public key as a JSON Web Key inside the DID string. Functionally similar to `did:key` (offline-resolvable, no rotation, no service endpoints), but interoperates with JWK-based tooling — useful for RSA keys, for example, where `did:key` is awkward. IES accepts `did:jwk` for holders. If you don't have a specific reason to use it, default to `did:key`.

5.7.3 Quick reference

Subject	Method	Why
DISCOM (issuer)	`did:web`	Domain anchors the key, regulator's registry anchors trust
Regulator	`did:web`	Same
Consumer (holder)	`did:key` or `did:jwk`	Wallet-generated, no domain needed, verifies offline

---

5.8 Appendix B — Issuing credentials (moved)

The end-to-end walkthrough for issuing, verifying, and revoking an ElectricityCredential v1.2 now lives in **Energy Credentials**. Pick up there once your `did:web` is published (Step-by-step above).

5.9 Appendix C — Identifying assets, meters, connections, datasets

You will eventually want stable identifiers for the things you operate (meters, transformers, feeders, service connections) and the data you publish (telemetry, tariff schedules). Use the same `did:web` you already own and add path segments. No new DID method needed.

> **Asset DIDs are stable identifiers, not necessarily resolvable documents.** Most asset DIDs ride inside credentials your DISCOM signs; trust comes from the issuer's signature, not asset-level resolution. See Appendix D — Asset-DID resolution patterns for when to promote an asset DID to a resolvable document.

5.9.1 Conventions

Symbol	Meaning
`<discom-domain>`	The domain hosting your `did.json` — e.g. `ies.tpddl.in`
`<internal-id>`	Your existing internal ID, used verbatim as the last path segment
URL-safe	Replace any characters outside `[A-Za-z0-9._-]` with `%xx` percent-encoding

5.9.2 Meter

```
did:web:<discom-domain>:assets:meter:<meter-slno>

-> did:web:ies.tpddl.in:assets:meter:MET-IMPORT-001
```

This is the identifier that goes in `energyResources[].id` for METER entries on an ElectricityCredential v1.2.

5.9.3 Other assets — transformer, feeder, substation, solar, BESS, EV charger

```
did:web:<discom-domain>:assets:<class>:<internal-id>
```

```
-> did:web:ies.tpddl.in:assets:transformer:DT-NDL-11KV-04572
-> did:web:ies.tpddl.in:assets:feeder:FDR-11KV-NDL-072
-> did:web:ies.tpddl.in:assets:substation:SS-22OK-NJP-001
-> did:web:ies.tpddl.in:assets:solar-plant:ROOFTOP-SOLAR-001
```

`<class>` values used in IES today: `feeder`, `transformer`, `substation`, `solar-plant`, `wind-farm`, `bess`, `ev-charger`, `meter`. Use kebab-case.

5.9.4 Service connection

The connection binds a consumer, a meter, and a feeder:

```
did:web:<discom-domain>:connections:<connection-id>
```

```
-> did:web:ies.tpddl.in:connections:CONN-TPDDL-2025-001234567
```

`<connection-id>` is whatever your CIS already uses — often the same string as the consumer number.

5.9.5 Dataset (Beckn `DatasetItem`)

For data-exchange resources (meter telemetry, ARR filings, tariff schedules):

```
did:web:<discom-domain>:datasets:<class>:<id>
```

```
-> did:web:ies.tpddl.in:datasets:meter-telemetry:2026-01
-> did:web:ies.derc.gov.in:datasets:tariff-orders:2025-26-domestic
```

The DID resolves to a `DatasetItem` record (DDM schema) carrying the Beckn BPP endpoint that serves the actual data.

5.9.6 Summary

Subject	Pattern	Internal ID becomes
DISCOM	`did:web:<domain>`	The domain itself
Regulator	`did:web:<domain>`	Same
Consumer (holder)	`did:key:…` (wallet-generated)	Not derived from anything internal
Consumer (CIS reference)	Kept as the literal `customerNumber` string	CIS number, verbatim
Meter	`did:web:<domain>:assets:meter:<slno>`	Meter SLNO
Other asset	`did:web:<domain>:assets:<class>:<id>`	SAP / GIS code
Connection	`did:web:<domain>:connections:<id>`	SC number
Dataset	`did:web:<domain>:datasets:<class>:<id>`	Dataset key

5.10 Appendix D — Identifier vs. record

Think of a DID like a **vehicle's licence plate**. The plate (`KA-01-MN-1234`) stays the same for the life of the registration. The RTO record it resolves to — owner, insurance status, address — can change. When a cop

reads your plate at a checkpoint, they query the RTO for the *current* record; they don't try to guess your address from the digits.

A DID works the same way:

Identifier (stable, travels in payloads)	Record (current, can change over time)
`did:web:ies.tpddl.in`	The `did.json` at `https://ies.tpddl.in/.well-known/did.json` — TPDDL's current public keys and endpoints
`did:web:ies.tpddl.in:assets:meter:MET-IMPORT-001`	The asset record under that path — current make, model, geo, commissioning date

Two rules follow:

- **Don't parse the identifier for business logic.** Resolve it and read the record's fields. A path that looks predictable today (`...:consumers:TPDDL-2025-...`) may need percent-encoding or restructuring tomorrow; structured fields in the record won't.
- **Records can update without re-issuing identifiers.** Key rotation, meter replacement, address correction — change the record, the identifier stays.

5.10.1 Five places this matters in IES

1. **DISCOM signing-key rotation.** TPDDL rotates its credential-issuing key. `did:web:ies.tpddl.in` doesn't change; the new key is added to `did.json`. Every ElectricityCredential ever issued by TPDDL keeps verifying because verifiers fetch the current document.
2. **Meter replacement.** A meter is swapped for a newer model. `did:web:ies.tpddl.in:assets:meter:MET-001` stays the same; the record gets a new `make`, `model`, `commissioningDate`. No credential rewrites.
3. **Beckn subscriber key rotation.** A BPP rotates its Ed25519 signing key. The DeDi subscriber record is updated. The `subscriber_id` in every Beckn header keeps working — ONIX re-resolves for each message.
4. **Audit / historical lookup.** `GET .../<record>?as_on=2025-01-01` returns the record version that was live on that date — same identifier, point-in-time record. Useful for "what was the public key when this credential was signed?".
5. **Anti-pattern: parsing the DID string.** Code that reads `did:web:ies.tpddl.in:consumers:<n>` to route a request will break the day a special character needs percent-encoding or a DISCOM restructures its asset hierarchy. Always resolve, then read.

5.10.2 Asset-DID resolution patterns (pragmatic / programmatic / per-asset)

Strictly per the W3C `did:web` method, every DID should resolve to a DID document at the corresponding URL. In IES practice, the trust on an asset DID inside a credential comes from the **issuer's** signature on that credential — the issuer's `did:web` is what verifiers must resolve. Asset DIDs ride inside the signed payload as stable references.

Three patterns are in active use; pick the one that matches your operational constraints:

Pattern	What you host	When to use
Pragmatic — no per-asset document	Only your DISCOM's top-level `did.json`. Asset DIDs are stable identifiers carried inside signed credentials and Beckn payloads; verification is via the issuer's signature, not asset-level resolution.	Most internal asset IDs (meters, feeders) that only appear inside credentials your DISCOM signs.
Programmatic — one endpoint, many DIDs	A small service under your domain that synthesises a DID document for any `assets/<class>/<id>` path on demand.	When you want strict `did:web` compliance without operating a static file per asset.
Per-asset documents	A separate `did.json` for each asset (e.g. `https://ies.tpddl.in/assets/meter/MET-001/did.json`).	High-value, public-facing assets (substations, large DERs) that other networks may resolve independently.

Start pragmatic; promote individual assets to programmatic or per-asset documents only when an external verifier actually needs to resolve them.

5.11 Appendix E — Joining a Beckn network (subscriber registry on the Beckn fabric)

Your DISCOM's `did:web` proves who issued a credential. It does **not** by itself put you on a Beckn network. To send and receive Beckn messages (search, select, init, confirm, on_status…), other nodes need to discover your callback URL and your signing key, and they do that through a **subscriber registry** published on the Beckn fabric.

Upstream documentation for this flow lives at the NFH docs: Onboarding Network Participants. The steps below are the practical summary aligned to IES.

5.11.1 Step 1 — Set up a DeDi account and verify your namespace

Follow Registries -> Step-by-step: claim your DeDi namespace for the account, namespace, and DNS-TXT verification steps. Use your DISCOM short code or FQDN as the namespace name (e.g. `tpddl`, `np.example.com`). Once verified, the namespace is your root of trust on the Beckn fabric.

5.11.2 Step 2 — Generate your Beckn signing keypair

Beckn signing uses **Ed25519** keys. This is a different key from your OpenCred P-256 issuer key; the two identities (credential issuance vs. Beckn participation) intentionally use separate keys.

Prerequisite: Go 1.24 or higher — see the official Go install guide. Beckn ONIX ships its keypair tool as a small Go program.

Clone the repo and run `tools/sign`:

```
git clone https://github.com/beckn/beckn-onix.git
cd beckn-onix/tools/sign
go run . gen-key --priv beckn_private.key --pub beckn_public_key.pem
# keypair generated
```

```
#   public key  -> beckn_public_key.pem  (publish in the subscriber record)
#   private key -> beckn_private.key     (store in your CI secret manager, never commit)
```

The tool writes:

- **Private key** (`beckn_private.key`): raw 32-byte Ed25519 seed, Base64-encoded — this is what ONIX loads to sign outgoing messages.
- **Public key** (`beckn_public_key.pem`): PKIX-encoded PEM. Extract the raw 32-byte Ed25519 public key from this PEM and Base64-encode it (no header/footer) for the `signing_public_key` field in Step 4.

Any other Ed25519 keypair generator works as long as it produces those two artefacts in the same encoding.

5.11.3 Step 3 — Create a Beckn subscriber registry under your namespace

Under your verified namespace, create a registry with the built-in `beckn_subscriber` tag — see Registries -> built-in tags. This registry holds one record per role you take on each Beckn network (separate `subscribers-test` and `subscribers-prod` are the conventional split).

5.11.4 Step 4 — Publish your subscriber record

Add a record with the following fields:

Field	What to fill	Example
`subscriber_id`	Your unique identifier, typically your domain	`ies.tpddl.in`
`subscriber_url`	Your Beckn ONIX receiver endpoint	`https://ies.tpddl.in/bpp/beckn`
`type`	Your role on the network	`BAP` or `BPP`
`signing_public_key`	Your Ed25519 public key, Base64-encoded, no header/footer	`eyAeqGFtAuks...`
`encryption_public_key`	(Optional) encryption public key	`lCI84I0Q0U0w...`
`countries`	Countries where you operate	`["IND"]`

If you operate in multiple roles (e.g. both BAP and BPP on the same network), publish a separate record per role.

Once published, note the **record ID** — this is the key ID you will configure in your ONIX instance.

5.11.5 Step 5 — Verify your key lookup

Other nodes resolve your subscriber details through the Beckn fabric lookup URL:

`https://fabric.nfh.global/registry/dedi/lookup/<your_subscriber_id>/subscribers.beckn.one/<your_record_id>`

Allow 5–10 minutes for the cache to update, then `curl` the URL and confirm it returns the record you just published.

5.11.6 Step 6 — Get added to the NFO's network registry

Apply via the IES Secretariat per Registries -> How to apply for an IES listing — that page lists every field the NFO needs and which IES networks are available. The NFO does **not** copy your record; it writes a small reference entry pointing at your DeDi-published subscriber record (or your whole subscriber registry, if every record under it belongs to the network), so your identity stays self-owned.

What the NFO checks before approving — useful to know so you don't get bounced:

- You have published subscriber details under your own verified namespace.
- The published callback URL is correct and reachable.
- The published public key is valid for signing and verification.
- The role and participant details match the network's expectations.
- You are being added to the correct environment-specific registry (test, prod, etc.).

Once the NFO writes the reference entry, your DISCOM is part of the curated network and other nodes can route to you.

5.11.7 How other Beckn nodes consume your identity

When a counterparty BAP or BPP receives a Beckn message claiming to come from `ies.tpddl.in`:

1. It calls `https://fabric.nfh.global/registry/dedi/lookup/ies.tpddl.in/subscribers.beckn.one/<record_id>` (or the equivalent NFO-side lookup) to retrieve your subscriber record.
2. It uses the `signing_public_key` from that record to verify the Ed25519 signature on the incoming Beckn header.
3. It uses the `subscriber_url` to route its callback.

You do the symmetric thing for messages you receive: look up the counterparty's subscriber record, verify their signature, and route to their callback. ONIX handles all of this once it is configured with your subscriber ID, record ID, and Ed25519 private key.

5.11.8 Step 7 — Configure your ONIX to accept the right networks (`allowedNetworkIDs`)

A subscriber record on DeDi carries a `network_memberships[]` field listing every network the subscriber has been referenced into (e.g. `["indiaenergystack.in/ies-data-sharing-network"]`). Each entry is a `network_id` — the canonical `<nfo-domain>/<registry-name>` string. Membership is written by the NFO when it adds the subscriber-reference record (Step 6 above) and read by every receiving ONIX instance during signature validation.

To enforce a network boundary at your receiver — i.e. only accept signed messages from subscribers who belong to the IES networks you've joined — configure the `dediregistry` plugin in your ONIX module. The relevant key is **`allowedNetworkIDs`**:

```yaml
modules:
  - name: bppTxnReceiver
    handler:
      plugins:
        registry:
          id: dediregistry
          config:
            url: "https://fabric.nfh.global/registry/dedi"
            allowedNetworkIDs: "indiaenergystack.in/ies-data-sharing-network,indiaenergystack.in/test-ies-data-sha
            timeout: 30
            retry_max: 3
      steps:
        - validateSign
        - addRoute
```

`allowedNetworkIDs` is a comma-separated list of full network IDs. When the plugin looks up an incoming subscriber on DeDi, it intersects the response's `data.network_memberships` with this list — if the subscriber doesn't belong to any configured network, ONIX rejects the message with `registry entry with subscriber_id '...' does not belong to any configured networks (registry.config.allowedNetworkIDs)`.

Practical rules of thumb:

- **One ONIX instance per environment.** Configure the prod instance with prod-network IDs and the test instance with `test-` IDs. Mixing creates audit trail confusion and accidentally lets test traffic into prod.
- **Use full network IDs**, not bare registry names — `indiaenergystack.in/ies-data-sharing-network`, not `ies-data-sharing-network`.
- **Leaving `allowedNetworkIDs` unset accepts everything.** Useful for a discovery / catalogue node; never the right setting for a transactional BAP / BPP.
- The older config key `allowedParentNamespaces` is **deprecated**; the plugin errors until you migrate to `allowedNetworkIDs` with full network membership IDs.

5.11.9 Why the two-identity model

Your `did:web` identity proves *"this credential was issued by TPDDL"* to anyone who fetches your `did.json`. Your subscriber-registry identity proves *"this Beckn message was sent by TPDDL"* to other nodes on the network. The first is content-level trust; the second is transport-level trust. They share an organisational root (your domain) but live on different rails and use different keys, so a compromise of one does not automatically compromise the other.

5.12 Appendix F — Binding the credential to a holder identity

A credential signed by you proves *who issued it*. It does **not** by itself prove *who is allowed to present it*. Without an explicit holder binding, the credential is a **bearer token** — whoever has the JSON is treated as the subject. That's fine for paper-style issuance, demos, and counter-issued attestations, but for consumer-facing flows (Consumer Energy Passport, Consumer Meter Digest, anything a bank or marketplace will read) you usually want presentation-time proof that the presenter is the same person the credential was issued to.

The W3C VC Data Model 2.0 § Presentations handles this through `credentialSubject.id`: set it to an identifier the subject controls, and require a proof-of-control at presentation time. The v1.2 schema makes `credentialSubject.id` optional, so you can pick the pattern that fits the consumer's situation.

Two patterns are practical in IES today, plus the no-binding default.

5.12.1 Before you bind anything: identity-proofing at issuance

This is the easy-to-miss step. The holder identifier (a DID, a phone number, anything) lands inside `credentialSubject.id` because **you put it there**. If you copy it verbatim from an unauthenticated request, an attacker can submit their own DID or their own phone and you will happily bind a Passport to it. Holder binding only works if you verify, **at issue time**, that the consumer controls the identifier you're about to embed.

So before any `POST /v1/credentials/issue` call that sets `credentialSubject.id`:

Holder identifier	What to verify at issue time
`did:key:...` (wallet)	The wallet signs a fresh nonce you send it; you verify the signature against the public key in the DID. This proves the requester holds the wallet's private key.
`tel:+91XXXXXXXXXX`	You confirm the number is the one already on file for that `customerNumber` in your CIS, **and** you send a fresh OTP to that number and confirm the requester reads it back.

Holder identifier	What to verify at issue time
DigiLocker-mediated	DigiLocker has already done the Aadhaar-backed identity check before issuing the access grant; you can rely on that for the identity binding, and just record DigiLocker as the binding channel.

Without that check, holder binding adds zero security; with it, the wallet/phone/DigiLocker identifier becomes a real second factor at presentation time.

5.12.2 Pattern 1 — Wallet DID (cryptographic, recommended where a wallet exists)

Set `credentialSubject.id` to a DID the consumer's wallet controls — typically `did:key`, sometimes `did:jwk`. Both are offline-resolvable (the public key is encoded in the DID string), so the verifier needs no network call to fetch the holder's key.

```
"credentialSubject": {
  "id": "did:key:z6MkjVQ8r4f3rPuY7CG2D6Lf8WJxJBs5sjkR8d3v2Bv4nP4Z",
  "customerProfile": { "customerNumber": "TPDDL-2025-00987654", ... },
  "customerDetails": { ... }
}
```

The presentation-time flow. When the consumer presents the credential to a verifier (a bank, a marketplace, a housing society), here is the step-by-step:

1. The verifier asks the wallet for the credential.
2. The wallet wraps the credential in a **Verifiable Presentation (VP)** — a small JSON envelope that can carry one or more credentials and add a fresh signature from the holder.
3. The verifier sends a **challenge** (a random nonce, often a UUID) and a **domain** (an identifier for the verifier itself, e.g. `bank.example.com`) — these go into `proof.challenge` and `proof.domain` on the VP.
4. The wallet signs the VP with the private key matching `credentialSubject.id`, embedding the challenge and domain inside the signature.
5. The verifier:
   - Resolves `credentialSubject.id`. For `did:key` this means decoding the multibase-encoded public key out of the DID string itself — no network call needed.
   - Verifies the VP's `proof.signature` against that public key.
   - Confirms `proof.challenge` matches the nonce the verifier just sent (rules out replays).
   - Confirms `proof.domain` matches the verifier's own identifier (rules out a presentation captured from another verifier).
   - Separately verifies the embedded credential's `proof` against the issuer DISCOM's `did:web` key (this is the standard issuer-side check).
6. If all four checks pass, the presenter has demonstrated control of the wallet's private key — they are the subject the credential was issued to.

A stolen credential JSON does not pass step 5: the attacker doesn't have the wallet's private key, so they cannot produce a VP signature that matches the public key embedded in `credentialSubject.id`.

This is the model OpenID for Verifiable Presentations (OID4VP), DIDComm, and most W3C wallet ecosystems implement.

Wallet rotation. A wallet `did:key` cannot rotate (rotating means a new DID). If a consumer's wallet is lost or compromised, they generate a new `did:key` and you re-issue the credential bound to it. The old credential is revoked through your DeDi revocation registry (see Energy Credentials — Revoke).

5.12.3 Pattern 2 — Phone-number URI (out-of-band, when there is no wallet)

If the consumer doesn't have a wallet — and many Indian consumers won't, at least initially — you can still bind the credential to a channel they control: a phone number. Use the RFC 3966 `tel:` URI scheme so the identifier is a proper URI rather than a bare string:

```
"credentialSubject": {
  "id": "tel:+919876543210",
  "customerProfile": { "customerNumber": "TPDDL-2025-00987654", ... },
  "customerDetails": { ... }
}
```

The URI form is strict: **`tel:` + full E.164** (country code + national number, no spaces, hyphens, parentheses, or leading zeros). For India that is `tel:+91` followed by the ten-digit mobile number. This is the only form a verifier can compare unambiguously across systems.

The presentation-time flow.

1. The consumer presents the credential to a verifier (often via QR scan, email attachment, or a DigiLocker share).
2. The verifier reads `credentialSubject.id` and extracts the phone number.
3. The verifier sends a fresh OTP to that number through SMS, IVR, or a push notification on a known app (DigiLocker, the verifier's own app, an aggregator).
4. The presenter reads the OTP back to the verifier.
5. If the OTP matches and is unexpired, the presenter has demonstrated control of the phone number.
6. The verifier independently verifies the credential's `proof` against the issuer's `did:web` key.

Compared to Pattern 1, this is weaker security: phone numbers can be ported, SIM-swapped, or temporarily shared, and the OTP channel itself can be phished. It is also slower (out-of-band OTP delivery). It is the right pragmatic choice when the consumer has no wallet and the verifier already runs phone-OTP plumbing.

Phone-number changes. Phone numbers change far more often than wallet DIDs. Either keep `validUntil` short (e.g. annual re-issue) so a stale number doesn't outlive its accuracy, or re-issue the credential whenever the consumer updates their number in your CIS.

5.12.4 Pattern 3 — DigiLocker-mediated (Indian-context shortcut)

For consumers who already use DigiLocker, the identity binding is largely solved before the credential reaches the consumer: DigiLocker performs an Aadhaar-backed identity check before granting access, and the credential is delivered into the consumer's DigiLocker vault. A verifier consuming a credential out of DigiLocker can rely on DigiLocker's own access-control rather than re-running a presentation-time challenge.

In this case you may issue the credential without a `credentialSubject.id` (bearer style) and document DigiLocker as the binding channel, **or** set `credentialSubject.id` to the consumer's DigiLocker-resident `did:key` if their wallet exposes one. The first is simpler; the second future-proofs the credential for re-presentation outside DigiLocker.

5.12.5 Where does the contact identifier live in the schema?

The `customerDetails` block in ElectricityCredential v1.2 (which references the shared `CustomerDetails/v1.0` shape) currently carries **`fullName`, `installationAddress`, and `serviceConnectionDate`** — there is **no telephone field**. So if you choose Pattern 2 today, the `tel:` URI lives in `credentialSubject.id`, not in `customerDetails`. If a future schema revision adds a `telephone` field, the canonical URI form there should be `tel:+<E.164>` so that the two locations agree.

5.12.6 Picking a pattern

Consumer situation	Pattern	`credentialSubject.id` value
Has a digital wallet (DID-capable)	**Pattern 1** — wallet DID	`did:key:z6Mkj...`
Has only a phone number on record	**Pattern 2** — `tel:` URI	`tel:+919876543210`
Uses DigiLocker, no separate wallet	**Pattern 3** — DigiLocker-mediated	*Omit, or use DigiLocker's* `did:key` *if exposed*
Paper / counter issuance, presented in person	None — bearer	*Omit*

Patterns are not exclusive. You can issue two credentials carrying the same `customerProfile.customerNumber` — one bound to a wallet DID for digital presentation, one bound to a phone URI for SMS-based flows. They share the same revocation handle, so revoking one does the right thing operationally.

5.12.7 Quick consistency checklist for adopters

- [ ] Issuer-side identity proof in place before any `credentialSubject.id` is set (challenge-sign for wallet DIDs, OTP for phones, DigiLocker grant otherwise).
- [ ] `tel:` URIs use full E.164 — `tel:+91` + ten digits — no spaces, hyphens, parentheses, or leading zeros.
- [ ] Verifier flow documented: how the verifier will challenge the holder at presentation time (VP-with-challenge for Pattern 1, OTP for Pattern 2, DigiLocker grant for Pattern 3).
- [ ] Revocation tested: revoking a credential invalidates **all** presentations of it, regardless of which binding pattern is in use.
- [ ] `validUntil` set with the binding's churn rate in mind (years for wallet DIDs, months-to-a-year for phone bindings).

Chapter 6

Registries and Directories

In a hurry? Jump straight to the Checklist — every step with checkboxes.

This page is the single home for everything IES has to say about registries. Read the first three sections and you will know **why your organisation should claim a DeDi namespace, how to do it, and which registries you actually need**. The appendices cover deeper DeDi mechanics, a worked verification workflow, and troubleshooting.

> Upstream documentation for DeDi (the open protocol) lives at DeDi developer documentation and Setting up a Beckn network. This page focuses on the **IES-specific** wiring on top of DeDi: which registries exist under the `india-energy-stack` namespace today, which ones OpenCred auto-creates for you, and how to get listed in the curated IES allow-lists.

6.1 Why DeDi (and the three questions it answers)

When two organisations exchange data without trusting a central middleman, the receiver needs to answer three questions about the data — and DeDi is designed to answer all three through cryptography rather than calls to the publisher.

Question	What DeDi gives you
Integrity — has this data changed since it was published?	Every record carries a BLAKE2 H256 digest anchored on the CORD blockchain. Recompute the digest, compare with the on-chain proof, done. Tampering is detectable.
Validity — is this data still current?	Version history per record, optional `valid_till`, a `state` field (`live` / `inactive` / `draft`), and dedicated revocation registries with `as_on=<date>` time-travel.
Authenticity — is the publisher who they claim to be?	A namespace is bound to a domain via a DNS TXT-record proof. Only the namespace controller's DID key can write under that namespace, and that DID resolves to the same domain.

A verifier never has to call the publisher; they call DeDi. That is the whole reason IES routes trust through public registries — revocation, public-key lookup, network membership — instead of phoning the issuer every time.

DeDi itself is open-source (Linux Foundation Decentralized Trust labs); the default hosted runtime is `dedi.global`.

6.2 Step-by-step: claim your DeDi namespace and create registries

This section is enough for any IES participant to stand up the registries they need. The detailed UI flow and API reference live in the NFH docs (Quickstart, namespace and registry creation); the commands below mirror them in the IES-specific framing.

6.2.1 What you'll need

- A domain you control, with **DNS access** (to add a TXT record). The same domain you use for your `did:web` is fine.
- An email address for an organisational account (use a role mailbox, not a personal one).
- 15 minutes to several hours — DNS TXT propagation is usually 15 min but can take up to 48 h.

6.2.2 1. Sign up at dedi.global

Create an account at `dedi.global`. Use a role mailbox so the account survives staff changes, and turn on MFA.

6.2.3 2. Create a namespace

In the console, create a namespace under a short name that maps to your organisation:

- DISCOMs typically use their short code (`tpddl`, `bescom`, `kseb`).
- Aggregators, AMISPs, trading platforms use their FQDN-style name (`np.example.com`).
- The IES network operator uses `india-energy-stack` under the domain `indiaenergystack.in`.

A namespace is a **trust container**. Only the controller's DID key can write registries or records under it.

6.2.4 3. Verify your domain (DNS TXT record)

In the console, request whitelisting for your domain. DeDi issues a TXT record. Add it to your DNS zone, click **Verify**, and wait for propagation. Once verified, the namespace is anchored to that domain — anyone resolving it can see it is genuinely yours.

If you cannot add a TXT record for some operational reason (delegated DNS, dev environments), contact the IES Secretariat — there are evaluation paths that don't require domain verification up-front.

6.2.5 4. Create the registries you need

Inside your verified namespace, create one registry per kind of record you publish. Each registry is typed by either a **built-in schema tag** (DeDi's templates for common shapes) or a **custom JSON Schema** you provide.

The IES-relevant tags:

Built-in tag	What it's for	Who creates it
`beckn_subscriber`	A Beckn participant's identity + signing key	You (any NP joining a Beckn network)

Built-in tag	What it's for	Who creates it
`beckn_subscriber_reference`	A pointer to another subscriber record or registry — the membership primitive NFOs use	The NFO
`public_key`	Versioned issuer public keys	OpenCred (or you, manually)
`revocation`	Per-credential revocation hashes	OpenCred (or you, manually)

A practical rule of thumb: **create one registry per environment** (`subscribers-test`, `subscribers-prod`) so a misconfigured test run can't pollute a production lookup.

> **If you run OpenCred, four of these registries are managed for you on first boot** — `vc-revocation-registry`, `opencred-key-registry`, `schema_registry`, `context_registry`. The OpenCred container auto-creates them if missing and reuses them if they exist. Details: Appendix A — OpenCred auto-creates four registries.

To add records once a registry exists, the simplest start is the **DeDi console** — pick the registry, click *Add record*, fill the fields. For the API equivalent (signed `POST` for writes, plain `GET` for lookups) see **Appendix A — API at a glance**.

6.3 The registries you'll touch in IES, by role

Pick the row that describes you. Each row tells you the **minimum** registries to operate yourself and which IES-side registries to ask to be **referenced into**.

6.3.1 As a DISCOM / issuer running OpenCred

Registry	Who creates	Purpose
`<discom>/vc-revocation-registry`	OpenCred (auto)	Per-credential revocation status
`<discom>/opencred-key-registry`	OpenCred (auto)	Your versioned signing keys
`<discom>/schema_registry`	OpenCred (auto)	JSON Schemas you use
`<discom>/context_registry`	OpenCred (auto)	JSON-LD contexts

To **issue credentials**, that's all you need on the registry side. The credential's trust chain is your `did:web` plus, optionally, the regulator's `issuer.idRef` — no IES-curated registry entry required. See Identifiers and Addressing.

To also **exchange data over Beckn**, add the Beckn-participant rows below.

6.3.2 As a Beckn Network Participant (BAP / BPP, aggregator, AMISP, trading platform)

Registry	Who creates	Purpose
`<your-namespace>/subscribers-test`	You	Your BAP/BPP subscriber record (test environment)
`<your-namespace>/subscribers-prod`	You	Same, production
IES network registry	NFO writes a reference pointing at your record	Marks you as in-network for a specific IES Beckn network

The IES network registries — `ies-data-sharing-network`, `ies-p2p-trading-network`, `ies-der-integration-network` (plus their `test-` variants) — are operated by the IES network operator (acting as NFO). You don't write to them directly; the NFO references your record after a short onboarding review. See IES networks and registries today below for the names and how to apply.

The end-to-end "publish your subscriber details + get added to a network" walkthrough is in Identifiers and Addressing — Appendix E.

6.3.3 As an NFO

(Network Facilitator Organisation — the organisation that curates a Beckn network.)

Registry	Who creates	Purpose
`<nfo-namespace>/<network-name>` (tag `beckn_subscriber_reference`)	You (the NFO)	The curated network — references to NP-owned subscriber records or whole subscriber registries

The `network_id` is `<nfo-domain>/<network-name>`. One registry per network you facilitate (typically separate `test-` and `prod` variants per sector). Membership is curated by **writing reference records that point at NP-owned subscriber records on DeDi** — you do not copy NP data into the network registry.

For the NFO operational details (manifest, network policies, key rotation), refer to the NFH docs on setting up the network environment and configuring network policies.

6.3.4 As a verifier or wallet

You don't write to anything; you only read. Two registries matter:

When you want to …	Read
Verify a credential's signature	The issuer's own `did:web` (`.well-known/did.json`) — and, if you cache, the issuer's `opencred-key-registry` for key history
Check revocation status	The issuer's `vc-revocation-registry`

A worked verification walkthrough is in Appendix B.

6.4 IES networks and registries today

The IES network operator publishes its registries under the namespace `india-energy-stack`, anchored to the domain `indiaenergystack.in`, with namespace DID:

```
did:web:did.cord.network:76EU9AJNL25X4LAxgb92rA8op4co7n892oeySAuEk9gAay2N28ctma
```

You can list everything in the namespace at any time:

```
curl https://api.dedi.global/dedi/query/indiaenergystack.in | jq '.data.registries[] | {registry_name, description
```

At time of writing the namespace holds **14 registries**, grouped below:

6.4.1 Beckn networks (NFO-operated reference registries)

Registry	Env	Purpose
`ies-data-sharing-network`	prod	Network for energy data sharing (DISCOM <-> DISCOM, regulators, aggregators)
`test-ies-data-sharing-network`	test	Same, for evaluation and conformance
`ies-p2p-trading-network`	prod	P2P energy trading between consumers, prosumers, and aggregators
`test-ies-p2p-trading-network`	test	Same, test
`ies-der-integration-network`	prod	DER (rooftop solar, BESS, EV) integration with DISCOMs
`test-ies-der-integration-network`	test	Same, test

All carry the `beckn_subscriber_reference` tag and are curated by the NFO (the IES network operator). To get a participant listed in any of them, see How to apply below.

6.4.2 Reference allow-lists (industry coordination)

Registry	Tag	Purpose
`ies-discoms-reference-registry`	`beckn_subscriber_reference`	Curated allow-list of recognised DISCOMs
`ies-regulators-reference-registry`	`beckn_subscriber_reference`	Curated allow-list of recognised regulators (SERCs, CERC)
`ies-service-providers-reference-registry`	`beckn_subscriber_reference`	Curated allow-list of service providers (consultancies, integrators)
`test-ies-discoms-reference-registry`	`beckn_subscriber_reference`	Test variant of the DISCOMs list

These are the curated industry-coordination lists that NFOs (including the IES NFO) reference into network registries to enforce a trust boundary. Credential issuance does **not** require an entry here — see Identifiers and Addressing — Two identities.

6.4.3 How to apply for an IES listing

Send an email with the fields below to the IES Secretariat. They write the reference record once they've verified you:

Channel	Address
Email (primary)	IES.Secretariat@fsrglobal.org
Email (alternate)	ies@recindia.com
Web	ies.fsrglobal.org

What to send (fields the Secretariat needs to write your reference record):

- Your short identifier (`<discom-short-code>` for DISCOMs, FQDN for NPs).

- Your verified DeDi namespace (e.g. `tpddl`, `np.example.com`).
- The DeDi lookup URL of either your **whole subscriber registry** (typically `subscribers-prod`) or a **single record** inside it — your choice depends on whether all your subscribers belong to the network or only some.
- Whether the URL is a `Registry` or a `Record` (one of those two values).
- Which IES network(s) you want to join (`ies-data-sharing-network` and/or `ies-p2p-trading-network` and/or `ies-der-integration-network`, plus the `test-` variant if you want sandbox first).
- Role: `BAP`, `BPP`, or both.
- Service areas / countries (`IND` for India).
- A point of contact (name, email, phone).

Before approving, the Secretariat validates: your namespace is domain-verified, your callback URL is reachable, your signing public key is present and valid, and the role you've declared matches your network entitlement.

6.5 Checklist

Standing up your registries trust layer. Each item maps to a section above.

6.5.1 1. Claim your DeDi namespace

Create an account on `dedi.global`, choose a namespace (your DISCOM short code), and complete domain verification — see Step-by-step.

- [ ] Account created on `dedi.global`
- [ ] Namespace claimed and **domain-verified** (TXT record)

6.5.2 2. Publish your trust anchor

Publish the record that names your organisation, its public signing key, and its domain — what verifiers look up before trusting anything you sign.

- [ ] Trust-anchor record published and resolvable via public lookup

6.5.3 3. Create the registries your use case needs

Add only the registries you'll actually use (e.g. revocation list, allowed-subscriber list, asset registry). OpenCred auto-creates its four credential registries — see The registries you'll touch in IES.

- [ ] Required registries created under your namespace
- [ ] Owner named for each registry

6.5.4 4. Set writers per registry

For each registry, list who may add/change records (keep it short — this is a security boundary); everyone else is read-only.

- [ ] Writer role(s) assigned and documented per registry
- [ ] Write credentials issued only to those roles

6.5.5 5. Agree an update cadence

Decide who updates each registry when a key rotates, a credential is revoked, a meter is replaced, or an asset is decommissioned.

- [ ] Update process documented per registry

- [ ] Namespace-key backup / recovery plan agreed

6.5.6 6. Nominate your team

- [ ] IT SPOC (operates the registries) — name, email, phone
- [ ] Authorised Signatory (approves what's published) — name, email, phone

For end-to-end network onboarding that builds on this layer, see the Energy Credentials -> Checklist and Data Exchange -> Checklist.

6.6 Appendix A — DeDi primer (just enough to navigate)

This appendix is a condensed orientation. For the full developer reference, go to the NFH DeDi docs.

6.6.1 The three-coordinate model

```
api.dedi.global
└── <namespace>              <- owned by one verified domain; only the controller's DID key can write
   └── <registry>         <- a typed collection of records (schema = built-in tag or custom JSON Schema)
       └── <record-id>       <- a specific record — the resolvable end-point
           └── { record JSON }
```

Three things uniquely address any record: **namespace + registry + record-id**. The namespace is the unit of governance; the registry is the unit of schema; the record is the unit of data.

6.6.2 Built-in schema tags used in IES

Tag	Required fields	Used for
`beckn_subscriber`	`subscriber_id`, `url`, `type` (BAP/BPP), `countries`, `signing_public_key`	A participant's identity on a Beckn network
`beckn_subscriber_reference`	`subscriber_id`, `url`, `type` (Registry/Record)	A pointer that includes another subscriber record or registry into a network
`public_key`	Key id, JWK, validity window	Versioned signing keys for an issuer
`revocation`	Credential hash, status	Per-credential revocation entries

6.6.3 OpenCred auto-creates four registries

If you run OpenCred, it manages four of these registries for you on first boot:

Registry	Tag	OpenCred behaviour
`vc-revocation-registry`	`revocation`	Auto-created if missing; reused if it already exists
`opencred-key-registry`	`public_key`	Auto-created if missing; reused if it already exists
`schema_registry`	custom	Auto-created if missing; reused if it already exists
`context_registry`	custom	Auto-created if missing; reused if it already exists

OpenCred's startup hook calls `ensureRegistries()` on first boot; idempotent restarts log "already published" and continue. Required env vars: `OPENCRED_DEDI_BASE_URL`, `OPENCRED_DEDI_AUTH_TYPE` (`api-key` or `bearer`), `OPENCRED_DEDI_NAMESPACE`, plus matching credentials. See the OpenCred deployment docs.

A caveat from OpenCred's docs: if those registries pre-exist with **schemas that don't match OpenCred's expectations** (e.g. DeDi's built-in `public_key` tag with a different shape), `/v1/keys/publish` will fail validation. So either let OpenCred create them, or pre-create them with OpenCred's exact schemas — don't mix.

6.6.4 API at a glance

Operation	Endpoint shape
Resolve a record	`GET /dedi/lookup/<namespace>/<registry>/<record-id>`
List records in a registry	`GET /dedi/query/<namespace>/<registry>?filter=...`
List registries in a namespace	`GET /dedi/query/<namespace>`
Time-travel	append `?as_on=YYYY-MM-DD`
Pin a version	append `?version_id=<id>`
Create / update a record	signed write by the namespace controller

6.6.5 Deployment options

Two practical options today:

Option	What it is	When to choose
Hosted on `dedi.global`, embedded in your site	You publish on `dedi.global`; your own site surfaces the data via the public API or a widget. Zero infra cost, you keep human-facing branding.	Default for institutional publishers (DISCOMs, regulators) who already operate a public website.
Hosted on `dedi.global` only	Same publish flow; consumers query `api.dedi.global` directly, no embedding.	Simplest path — appropriate when there is no public site to embed in, or the registry is purely machine-to-machine.

A self-hosted DDP node is technically possible (the protocol is open-source) but no IES participant operates one today; revisit only if you have a hard requirement.

6.6.6 State, versioning, time-travel

A record never disappears silently — DeDi tracks the full history.

Concept	Behaviour
Versions	Every update creates a new version. `version_count` and `version` returned on lookups.
Historical lookup	`?as_on=YYYY-MM-DD` returns the version current on that date.
Version pin	`?version_id=<id>` returns one specific version.
Registry state	`live` (queryable) or `inactive` (disabled, recoverable).

Concept	Behaviour
Record state	`draft` (saved, unpublished) or `live` (published, queryable).
TTL	Optional time-to-live in seconds; default 600.

6.7 Appendix B — Verifying a credential, end-to-end

This is the one worked workflow worth carrying inline; everything else hangs off it.

A wallet hands a verifier a signed ElectricityCredential. The verifier:

1. **Parse** the credential JSON. Read `issuer.id` (e.g. `did:web:ies.tpddl.in`).
2. **Resolve `issuer.id`** over HTTPS — fetch `https://ies.tpddl.in/.well-known/did.json` and extract the `verificationMethod` public key.
3. **Verify the credential's `proof`** signature against that key. If it fails, stop — the credential is forged or corrupted.
4. **(Optional) Check `issuer.idRef`** if present. Resolve `issuer.idRef.issuedBy` (the regulator's `did:web`) and confirm the regulator vouches for this DISCOM under the cited `subjectId`. This is the licensing leg of the trust chain; skip when `idRef` is absent.
5. **Check revocation.** GET the URL in `credentialStatus.id` — typically `https://api.dedi.global/dedi/lookup/<discom>/revocation-registry/<credential-id>`. A `404` (or `status: not_revoked`) means valid; a record with `status: revoked` means rejected.
6. **Check validity window.** Confirm `validFrom <= now <= validUntil`.

A verifier that caches the issuer's `did.json` and the regulator's `did.json` (e.g. refresh every 10 minutes) can complete steps 2–4 entirely offline; only step 5 needs a live DeDi lookup.

6.8 Appendix C — Troubleshooting

Symptom	Likely cause	Action
`404` on `GET /dedi/lookup/<namespace>/<registry>/<record-id>`	Wrong record-id (case-sensitive), or the record was never published (left as `draft`)	Verify in the DeDi console; re-issue with the correct id
`403 / signature invalid` on write	Namespace controller key changed, or you're signing with the wrong key	Confirm you are using the key currently bound to the namespace
DNS TXT verification stuck on "pending" for >1 hour	Propagation delay (normal up to 48 h) or wrong record in zone	Check with `dig +short TXT <your-domain>`; ensure the record matches what DeDi issued
OpenCred `/v1/keys/publish` returns a validation error	A pre-existing registry has a schema OpenCred does not recognise	Either delete and let OpenCred recreate, or align the registry's schema to OpenCred's expectations — see OpenCred deployment
Beckn counterparty cannot find your subscriber	Network reference not yet written, or you're looking at the wrong env	Verify the lookup URL the NFO wrote; check that `test-` vs prod matches the network you're targeting

Chapter 7

Energy Credentials

In a hurry? Jump straight to the Checklist — every step with checkboxes.

This page is the single home for everything IES has to say about issuing, verifying, and revoking energy credentials. The first three sections take you from a fresh deployment to your first signed credential. The appendices cover trust theory, batch / production patterns, and core concepts.

> **About the walkthrough.** The concrete commands below use **OpenCred** — see the glossary for what it is, its W3C compliance, its DeDi integration, and release links. Any W3C-compliant signing pipeline that publishes the same `did.json` and VC-2.0 proofs is a drop-in replacement.

7.1 Why credentials

When a DISCOM hands a consumer a digital electricity attestation, or shares meter readings with a regulator or a marketplace, the receiver needs to answer one question on their own: *"Is this really from TPDDL, intact, and still valid?"* If they have to call you, the system does not scale and is not really verifiable.

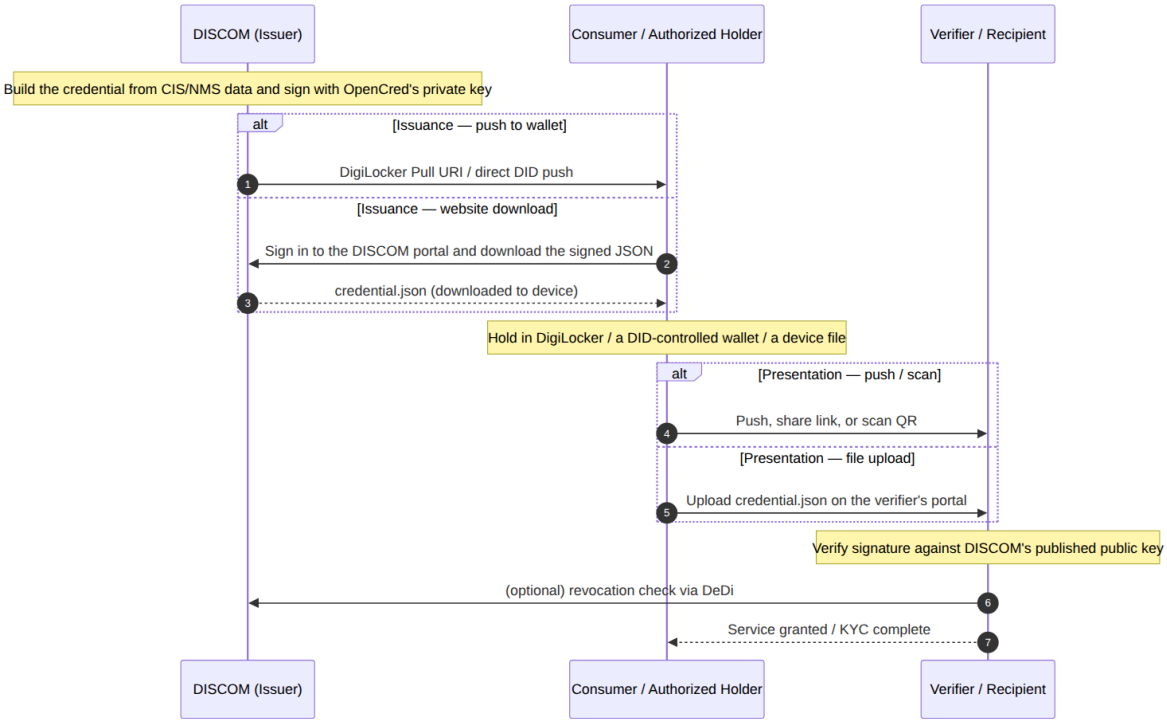
A **Verifiable Credential** is a small JSON object you sign with the private key behind your `did:web`. Anyone — a wallet, another DISCOM, a bank, a regulator — can fetch your `did.json` over HTTPS, check the signature, and consult a public revocation list. No callback to you required.

Three credentials cover almost everything IES does:

Credential	What it attests	Who signs	Typical receiver
ElectricityCredential v1.2	A service connection — customer number, sanctioned load, tariff, meter info, energy resources (rooftop solar, BESS, EV chargers)	DISCOM	The consumer's wallet, or a verifier (bank, marketplace, regulator) the consumer shares it with
MeterDataCredential v0.6	A signed meter-reading payload (raw `MeterData` profiles or derived summaries) for a specified period	AMISP, MDM, or DISCOM	DISCOM (B2B telemetry) or the consumer (their own readings)

Credential	What it attests	Who signs	Typical receiver
MeterDataRequestCredential v0.1	A signed request for meter data — proves the requester has the right to ask	Seeker (typically a DISCOM)	Provider (typically an AMISP) at Beckn `confirm` time

7.1.1 Lifecycle at a glance



7.2 Pick your role

If you are…	Read	Then
A DISCOM / issuer (you sign and emit credentials)	Prerequisites -> Issue your first credential -> Checklist	Credential variants, Appendix B — Operational notes
An AMISP / MDM / aggregator (you sign telemetry)	Prerequisites -> Issue your first credential using `MeterDataCredential`	Smart Meter Data Exchange use case
A holder / wallet (you hold credentials on behalf of a consumer)	Holder binding -> DigiLocker delivery	Identifiers — Appendix F for binding patterns
A verifier (you receive and check credentials)	Verify, Revocation check -> Appendix A — Trust model	Registries — Verifying a credential for the resolution walk

7.3 Prerequisites

Before you can issue, get these in place:

1. **A domain or subdomain you control**, with the ability to host one small static file under it. This becomes the host portion of your `did:web`. See Identifiers — (a) Org identity for naming + path-segment guidance.
2. **A DeDi namespace** under your verified domain. See Registries — Step-by-step. OpenCred will auto-create the four registries it needs (`vc-revocation-registry`, `opencred-key-registry`, `schema_registry`, `context_registry`) on first boot.
3. **Docker 24+**, plus `curl`, `jq`, `openssl`, and ~2 GB free disk. The OpenCred container ships ready to issue.
4. *(Optional, recommended for licensed utilities)* **A regulator's licensing pointer** to quote in `issuer.idRef` — the regulator's `did:web` and the regulator-issued licence identifier for your DISCOM. Omit for pilots / non-regulated issuers.
5. **A signed payload schema in mind.** Default: ElectricityCredential v1.2. For telemetry, MeterDataCredential v0.6.

   No IES-side DISCOM-registry entry is required to issue credentials. That registry is the inter-DISCOM data exchange network's trust boundary, not a credential prerequisite. See Registries — IES networks today for when you'd need it.

7.4 Set up OpenCred and publish your `did:web`

The practical setup is one JSON file on a web server you already run, plus the OpenCred container that signs credentials with the matching private key. Six steps; ~15 minutes end-to-end.

7.4.1 1. Pull the OpenCred image

```
docker pull ghcr.io/nfh-trust-labs/opencred/opencred-server:latest
docker tag  ghcr.io/nfh-trust-labs/opencred/opencred-server:latest opencred:bootcamp
```

7.4.2 2. Generate a signing key and API token

The same EC P-256 key works for both `did:web` and `did:key`; the difference is just which DID method OpenCred presents it as.

```
mkdir -p ~/opencred/keys
cd ~/opencred

openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:P-256 \
  -out keys/issuer-key.pem
chmod 600 keys/issuer-key.pem

export OPENCRED_API_KEY="$(openssl rand -base64 32)"
echo "Save this: $OPENCRED_API_KEY"
```

Keep `issuer-key.pem` in your KMS in production. Treat it like a TLS private key.

7.4.3 3. Run OpenCred in `did:web` mode

```
docker run -d \
  --name opencred \
  -p 3100:3100 \
```

```
  -e OPENCRED_API_KEY="$OPENCRED_API_KEY" \
  -e OPENCRED_KEY_PATH=/secrets/issuer-key.pem \
  -e OPENCRED_ISSUER_DID_METHOD=web \
  -e OPENCRED_ISSUER_DOMAIN=ies.tpddl.in \
  -v "$HOME/opencred/keys/issuer-key.pem:/secrets/issuer-key.pem:ro" \
  --read-only --cap-drop ALL \
  opencred:bootcamp

curl -s http://localhost:3100/v1/health | jq
# expect "signingKeyLoaded": true
```

> **Want offline-verifiable identity instead?** Drop `OPENCRED_ISSUER_DID_METHOD` and `OPENCRED_ISSUER_DOMAIN` and OpenCred runs in `did:key` mode by default. The same key, the same API — only the `did:` string the container reports changes. This is the right choice for first-deploy testing, demos, and consumer wallets. See Identifiers — `did:key`.

7.4.4 4. Assemble your `did.json` from the container

You need the **public JWK** of your signing key for the DID document. The keys endpoint reports key metadata:

```
curl -s http://localhost:3100/v1/keys \
  -H "Authorization: Bearer $OPENCRED_API_KEY" | jq '.keys[0]'
```

Current OpenCred builds return key *metadata* here (`id`, `algorithm`, `fingerprint`) — not the `x`/`y` coordinates. Derive the JWK straight from your key; it is the public half of the exact key OpenCred signs with:

```
python3 - <<'PY'
import subprocess, base64, json
der = subprocess.run(["openssl", "pkey", "-in", "keys/issuer-key.pem", "-pubout", "-outform", "DER"],
                     capture_output=True, check=True).stdout
pt = der[-65:]                          # uncompressed EC point: 0x04 || X(32) || Y(32)
b64u = lambda b: base64.urlsafe_b64encode(b).rstrip(b"=").decode()
print(json.dumps({"kty": "EC", "crv": "P-256", "x": b64u(pt[1:33]), "y": b64u(pt[33:65])}))
PY
```

Drop that JWK into the standard DID document template:

```
{
  "@context": [
    "https://www.w3.org/ns/did/v1",
    "https://w3id.org/security/suites/jws-2020/v1"
  ],
  "id": "did:web:ies.tpddl.in",
  "verificationMethod": [{
    "id": "did:web:ies.tpddl.in#key-0",
    "type": "JsonWebKey",
    "controller": "did:web:ies.tpddl.in",
    "publicKeyJwk": { "kty": "EC", "crv": "P-256", "x": "...", "y": "..." }
  }],
  "authentication":  ["did:web:ies.tpddl.in#key-0"],
  "assertionMethod": ["did:web:ies.tpddl.in#key-0"]
}
```

Three things matter, and you can ignore the rest until later:

- **`verificationMethod`** is the public key. Verifiers use it to check your signatures.
- **`assertionMethod`** says which key is allowed to issue credentials.

- **`authentication`** says which key can sign requests on behalf of the DID.

You can add a `service` array later when your Beckn BPP and OpenCred endpoints are publicly addressable; the DID is valid without it.

7.4.5 5. Publish the file

Upload it so this URL returns the JSON:

`https://ies.tpddl.in/.well-known/did.json`

The `.well-known/` path is a standard convention; verifiers know to look there. A normal TLS cert is enough — the same one that already terminates your subdomain — and there must be no redirect.

7.4.6 6. Verify it from the outside

```
curl -s https://ies.tpddl.in/.well-known/did.json | jq .id
# "did:web:ies.tpddl.in"
```

If that command prints your DID, you're done — any participant on the network can now resolve `did:web:ies.tpddl.in` to your public key and verify any credential you sign. Move on to Issue your first credential.

7.5 Issue your first credential

This is the bootcamp-aligned step-by-step. Five steps from a running OpenCred to a signed, revocable ElectricityCredential v1.2.

7.5.1 1. Confirm the issuer DID OpenCred reports

```
export OPENCRED_API_KEY="…"   # from your secret manager
export ISSUER_DID="$(curl -s http://localhost:3100/v1/keys \
  -H "Authorization: Bearer $OPENCRED_API_KEY" | jq -r '.keys[0].id | split("#")[0]')"
echo "$ISSUER_DID"
# did:web:ies.tpddl.in
```

If you ran the container in `did:key` mode (for early dev), this prints `did:key:z…` instead — the rest of the flow is identical, only the issuer string changes.

7.5.2 2. Issue

Default: **bearer-style** (no `credentialSubject.id`). This mirrors the OpenCred bootcamp. For consumer-facing flows where the presenter must be the legitimate subject, bind to a holder identifier — see Holder binding.

```
curl -s http://localhost:3100/v1/credentials/issue \
  -H "Authorization: Bearer $OPENCRED_API_KEY" \
  -H "Content-Type: application/json" \
  -d "{
    \"schemaId\":    \"ies/electricity-credential/v1.2\",
    \"issuerDid\":   \"$ISSUER_DID\",
    \"proofFormat\": \"vc-jwt\",
    \"validFrom\":   \"2026-04-01T00:00:00+05:30\",
    \"validUntil\":  \"2031-04-01T00:00:00+05:30\",
    \"credentialSubject\": {
```

```
      \"customerProfile\": {
        \"customerNumber\": \"TPDDL-2025-00987654\",
        \"energyResources\": [{
          \"id\":   \"did:web:ies.tpddl.in:assets:meter:MET-IMPORT-001\",
          \"type\": \"METER\",
          \"attributes\": {\"meterCapability\": \"AMI\", \"energyDirection\": \"Forward\"}
        }],
        \"consumptionProfiles\": [{
          \"meterId\":            \"did:web:ies.tpddl.in:assets:meter:MET-IMPORT-001\",
          \"sanctionedLoad\":     {\"value\": 10, \"unit\": \"kW\"},
          \"tariffCategoryCode\": \"DS-I\",
          \"premisesType\":       \"Residential\",
          \"connectionType\":     \"Single-phase\"
        }]
      },
      \"customerDetails\": {
        \"fullName\":              \"Arjun Mehra\",
        \"installationAddress\": {
          \"geo\": {
            \"type\": \"Point\",
            \"coordinates\": [-122.4194, 37.7749]
          },
          \"address\": {
            \"streetAddress\": \"123 Energy Street, Unit 4\",
            \"addressLocality\": \"Metro City\",
            \"addressRegion\": \"Example State\",
            \"postalCode\": \"12345\",
            \"addressCountry\": \"US\"
          }
        },
        \"serviceConnectionDate\": \"2018-07-15T00:00:00+05:30\"
      }
    }
  }" | tee credential.json | jq .credential
```

Three things worth noting:

- **Asset IDs are `did:web` under your own domain**, with colon-path segments (`did:web:ies.tpddl.in:assets:meter:<sl` Same pattern for transformers, feeders, substations — see Identifiers — Asset patterns. No per-asset `did.json` hosting required for the pragmatic case.
- **`issuer.idRef` is optional.** OpenCred fills `issuer` with the DID string only. Your integration service appends `name` and (if you have a regulator to cite) `idRef` on egress, then re-signs if your flow requires a single signed artefact.
- **`credentialSubject.id` is absent here.** That's the bearer-style default. Set it to a wallet `did:key` or `tel:+91...` URI for holder-bound issuance — full guidance in Identifiers — Holder binding.

7.5.3 3. Verify

```
jq -n --arg c "$(jq -r '.credential.proof.jwt' credential.json)" '{credential: $c}' | \
  curl -s http://localhost:3100/v1/credentials/verify \
    -H "Authorization: Bearer $OPENCRED_API_KEY" \
    -H "Content-Type: application/json" \
    -d @- | jq
# expect "valid": true
```

For `vc-jwt`, the verify endpoint takes the compact JWS string (`.credential.proof.jwt`), not the JSON

envelope. For `data-integrity` proofs, send the full credential JSON. The full verification flow — checking the issuer's signature, the regulator's licensing assertion (if cited), and revocation status — is detailed in Appendix A.

7.5.4 4. Revoke

OpenCred publishes revocation as a hash entry into your DeDi revocation registry; the verifier reads `credentialStatus` and looks up the hash. DeDi stores **only revoked** hashes — the per-credential lookup returns `200` when revoked and `404` when not (record existence is the signal). For the credential to carry that `credentialStatus`, pass `revocationRegistryUrl` at issue (as the MeterDataCredential and MeterDataRequestCredential examples do); the default issue above omits it.

```
# Compute the revocation hash
HASH=$(jq '{credential: .credential}' credential.json | \
  curl -s http://localhost:3100/v1/credentials/revocation-hash \
    -H "Authorization: Bearer $OPENCRED_API_KEY" \
    -H "Content-Type: application/json" -d @- | jq -r .revocationHash)

# Revoke
curl -s http://localhost:3100/v1/credentials/revoke \
  -H "Authorization: Bearer $OPENCRED_API_KEY" \
  -H "Content-Type: application/json" \
  -d "{\"hash\": \"$HASH\", \"reason\": \"connection-terminated\"}" | jq

# Check status
curl -s http://localhost:3100/v1/credentials/revocation-status \
  -H "Authorization: Bearer $OPENCRED_API_KEY" \
  -H "Content-Type: application/json" \
  -d "{\"hash\": \"$HASH\"}" | jq
```

Revocation requires the relevant `OPENCRED_DEDI_*` env vars on the container (`OPENCRED_DEDI_BASE_URL`, `OPENCRED_DEDI_AUTH_TYPE`, `OPENCRED_DEDI_API_KEY`, `OPENCRED_DEDI_NAMESPACE`). See OpenCred Revocation for the conceptual model.

7.5.5 5. Smoke test

A passing integration test should:

1. Issue a credential.
2. Resolve `issuer.id` (`did.json` over HTTPS) and confirm the public key matches the one that signed `proof`.
3. *(If `issuer.idRef` is present)* Resolve `issuer.idRef.issuedBy` and confirm the regulator vouches for your DISCOM.
4. `POST /v1/credentials/verify` — expect `valid: true`.
5. Check `revocation-status` — expect not revoked.
6. Revoke.
7. Re-check `revocation-status` — expect revoked.

Run on every release. It exercises every leg of the trust chain.

7.6 Credential variants

The same schemas cover several use cases. **No new VC `type` values are introduced** — the variants are issuance configurations over the existing schemas.

7.6.1 ElectricityCredential v1.2 — the default

A DISCOM-signed attestation about a service connection. Carries `customerProfile` (non-PII: customer number, energy resources, consumption profile), `customerDetails` (optional PII: name, address, service-connection date), and the issuer block.

Two common shapes:

Bearer / counter-issued (no `credentialSubject.id`). Anyone holding the JSON is treated as the subject. Used for paper-style attestations, demos, or in-person verification. This is what the bootcamp walkthrough above produces.

Holder-bound, consumer-presentable — the **Consumer Energy Passport** pattern. Same schema, but: - `credentialSubject.id` = the consumer's wallet `did:key` (or `did:jwk`). - `customerProfile.idRef` carries a verifiable government-ID reference (Aadhaar offline KYC, DigiLocker pull, etc.) — **the reference**, never the raw number. - At presentation time, the verifier issues a challenge, the wallet signs a Verifiable Presentation, and the verifier confirms the presenter holds the matching private key. See Identifiers — Pattern 1.

The Consumer Energy Passport use case (use-cases/consumer-energy-passport/) is about *who*, *when*, and *why* the holder-bound shape is issued; the credential itself is an ElectricityCredential v1.2.

7.6.2 MeterDataCredential v0.6 — telemetry signing

A signed VC wrapping a `MeterData` v0.6 payload (raw `INTERVAL`/`DAILY`/`MONTHLY` profiles or derived summaries) for a specified period. Issued by the AMISP or MDM, typically B2B to a DISCOM, and delivered over Beckn at `on_status`. Wraps MeterData v0.6; schema MeterDataCredential v0.6.

Same `POST /v1/credentials/issue` flow as the walkthrough above — the `schemaId` is the OpenCred registry id **`ies/meter-data-credential/v0.6`**, and `credentialSubject.meterData` carries the `MeterData` payload (a profile object or array — see the v0.6 examples). Pass `revocationRegistryUrl` so the credential carries a `credentialStatus` verifiers can check (see Revoke); its value is your DeDi revocation registry, addressed by namespace DID **or** verified domain:

```
curl -s http://localhost:3100/v1/credentials/issue \
  -H "Authorization: Bearer $OPENCRED_API_KEY" \
  -H "Content-Type: application/json" \
  -d "{
    \"schemaId\":     \"ies/meter-data-credential/v0.6\",
    \"issuerDid\":    \"$ISSUER_DID\",
    \"proofFormat\":  \"vc-jwt\",
    \"validFrom\":    \"2026-06-28T00:00:00+05:30\",
    \"validUntil\":   \"2026-07-05T00:00:00+05:30\",
    \"revocationRegistryUrl\": \"https://api.dedi.global/dedi/query/<your-namespace>/vc-revocation-registry\",
    \"credentialSubject\": {
      \"meterData\": { \"@type\": \"DailyProfile\", \"profileType\": \"DAILY\", \"…\": \"a MeterData v0.6 profile
    }
  }" | tee credential.json | jq .credential
```

Two common shapes:

B2B, typically without `credentialSubject.id`. The AMISP signs; the DISCOM consumes the payload at Beckn `on_status`.

Holder-bound, consumer-presentable — the **Consumer Meter Digest** pattern. `credentialSubject.id` = the consumer's wallet DID, `validUntil` is short (hours to days), and the readings or summary cover a period the consumer asked for. The credential is delivered into the consumer's wallet / DigiLocker; verifiers (banks, marketplaces) check it without phoning the DISCOM. Use case page: use-cases/consumer-meter-digest/.

7.6.3 MeterDataRequestCredential v0.1 — proof of right-to-ask

A signed VC carried at Beckn `confirm` time by a seeker (typically a DISCOM) when an AMISP's offer policy requires it. Proves the seeker has been authorised to request the data they're confirming. Schema: MeterDataRequestCredential v0.1.

This schema is **not** in OpenCred's built-in registry, so issue it with an **`inlineSchema`** rather than a `schemaId`: pass the JSON Schema in the request and OpenCred validates `credentialSubject` against it, writes the schema `$id`, and signs. Here we reuse the published MeterDataRequest `$defs` so the inline schema stays canonical:

```
# pull the MeterDataRequest v0.6 $defs and wrap them as the credentialSubject schema
REQ_DEFS=$(curl -s https://india-energy-stack.github.io/ies-accelerator/schemas/MeterDataRequest/v0.6/schema.json

jq -n --argjson defs "$REQ_DEFS" --arg iss "$ISSUER_DID" '{
  inlineSchema: {
    "$id": "https://india-energy-stack.github.io/ies-accelerator/schemas/MeterDataRequestCredential/v0.1/schema.js
    type: "object", required: ["meterDataRequest"],
    properties: {
      id: { type: "string", format: "uri" },
      meterDataRequest: { "$ref": "#/$defs/MeterDataRequestObject" }
    },
    "$defs": $defs
  },
  issuerDid: $iss,
  proofFormat: "vc-jwt",
  validFrom: "2026-06-28T00:00:00+05:30",
  validUntil: "2026-12-28T00:00:00+05:30",
  revocationRegistryUrl: "https://api.dedi.global/dedi/query/<your-namespace>/vc-revocation-registry",
  credentialSubject: {
    meterDataRequest: {
      "@context": "https://india-energy-stack.github.io/ies-accelerator/schemas/MeterDataRequest/v0.6/context.json
      "@type": "MeterDataRequest",
      scope: "ResourceOnly",
      from: "2026-06-04T00:00:00Z",
      duration: "P6M",
      maxRecordsShared: 10000,
      capabilitiesRequested: { profiles: [ { profileType: "CustomerProfile" }, { profileType: "IntervalProfile" }
    }
  }
}' | curl -s http://localhost:3100/v1/credentials/issue \
  -H "Authorization: Bearer $OPENCRED_API_KEY" -H "Content-Type: application/json" -d @- | jq .credential
```

`scope` must be one of the `ScopeType` enum (`ResourceOnly`, `ResourceAndChildren`, `ChildrenOnly`). The seeker embeds the resulting credential in the `confirm` message's receiver participant; the provider's adapter verifies it before fulfilling. A full did:web walkthrough (issue -> publish `did.json` -> verify -> revoke) plus ready-to-send `confirm` payloads live in the DEG data-exchange devkit.

7.6.4 Summary

Pattern	Schema	`credentialSubject.id`	`validUntil`	Issued by
Bearer Electricity-Credential	`ElectricityCredential v1.2`	*(user)*	years	DISCOM

Pattern	Schema	credentialSubject.id	validUntil	Issued by
Consumer Energy Passport	ElectricityCredential/v1.2	wallet did:key (+ customerProfile.idRef)	years	DISCOM
B2B MeterData-Credential	MeterDataCredential/v0.6	consent	hours to days	AMISP / MDM
Consumer Meter Digest	MeterDataCredential/v0.1	wallet did:key	hours to days	DISCOM (on consumer demand)
Meter-data request	MeterDataRequestCredential/v0.1	consent	minutes (per Beckn message)	Seeker (typically DISCOM)

7.7 Holder binding

Holder binding turns a credential from a bearer token into something only the consumer's wallet can present. Choose a pattern (wallet DID, `tel:+91...` URI, or DigiLocker-mediated) based on the consumer's situation. **Identity-proofing at issuance is mandatory** — you must verify the consumer controls the identifier before embedding it.

Full guidance: Identifiers — Appendix F.

7.8 DigiLocker delivery

DigiLocker is the dominant consumer wallet in India. Once issued, an ElectricityCredential or MeterDataCredential can be delivered into a consumer's DigiLocker via a Pull URI, and any verifier reading from DigiLocker inherits DigiLocker's Aadhaar-mediated identity binding.

Walkthrough (Pull URI shape, callback flow, signature pinning, common failure modes): digilocker.md.

7.9 Checklist

Zero to issuing your first signed credential. Each item links to the section that walks it.

7.9.1 Phase 1 — Foundations

- [ ] Domain selected; apex-vs-subdomain and path layout decided -> Identifiers — What you'll need
- [ ] Signing key generated; private key in KMS / HSM (software PEM for dev only) -> Signing-key sources
- [ ] OpenCred running; `/v1/health` returns `signingKeyLoaded: true` -> Run OpenCred
- [ ] `did.json` published; `curl` returns the expected DID -> Publish the file
- [ ] DeDi namespace claimed and **domain-verified** -> Registries — Step-by-step
- [ ] OpenCred's four registries auto-created; `/v1/health` reports `dediConfigured: true` -> Prerequisites

7.9.2 Phase 2 — Issue and revoke

- [ ] Issuer DID confirmed -> Step 1
- [ ] First ElectricityCredential v1.2 issued (bearer) -> Step 2
- [ ] Verify returns `valid: true` -> Step 3

- [ ] Revocation publishes; status flips -> Step 4
- [ ] Smoke test wired into CI -> Step 5
- [ ] Integration service appends `issuer.name` (and `issuer.idRef` if citing a regulator) on egress -> Step 2 notes

7.9.3 Phase 3 — Decide variant(s) you'll issue

- [ ] Variant(s) chosen -> Credential variants: bearer ElectricityCredential v1.2; Consumer Energy Passport (+ holder binding + `customerProfile.idRef`); B2B MeterDataCredential v0.6; Consumer Meter Digest (+ holder binding, short `validUntil`); MeterDataRequestCredential v0.1
- [ ] For holder-bound variants: binding pattern chosen -> Identifiers — Appendix F
- [ ] Identity-proofing-at-issuance documented (challenge-sign for wallet DIDs, OTP for phone, DigiLocker grant otherwise)

7.9.4 Phase 4 — Production hardening

- [ ] Signing key in HSM / Cloud KMS -> Signing-key sources
- [ ] Schema validation before `POST /v1/credentials/issue` -> Schema validation
- [ ] Key-rotation runbook in place -> Key rotation
- [ ] Reverse proxy + TLS in front of OpenCred (never expose `:3100`) -> Reverse proxy + TLS
- [ ] Backups of `did.json`, KMS key handle, DeDi credentials
- [ ] DigiLocker delivery configured if applicable -> DigiLocker delivery

7.9.5 Phase 5 — Data exchange (optional, only if joining Beckn)

- [ ] Beckn subscriber record published -> Identifiers — Appendix E
- [ ] IES Secretariat has referenced your subscriber into the network registry -> Registries — How to apply
- [ ] ONIX `allowedNetworkIDs` set for the right environment -> Identifiers — Step 7

7.9.6 Team

- [ ] IT SPOC — operates OpenCred, owns key rotation (name, email, phone)
- [ ] Governance / Compliance SPOC — approves publishing, owns regulator-pointer setup

---

7.10 Appendix A — Trust model

A credential's trust chain has at most two legs:

1. **Mandatory** — the issuer's `did:web` signature. A verifier resolves `issuer.id` over HTTPS to `did.json`, extracts the public key, and verifies `proof`. If this fails, stop — the credential is forged or corrupted.
2. **Optional** — the regulator's licensing assertion in `issuer.idRef`. When present, the verifier resolves `issuer.idRef.issuedBy` (the regulator's `did:web`) and confirms the regulator vouches for the DISCOM under the cited `subjectId`. When absent (pilots, non-regulated issuers), the verifier falls back to whatever out-of-band recognition they have of your `did:web`.

Plus a freshness check:

3. **Revocation status.** GET the URL in `credentialStatus.id` (typically `https://api.dedi.global/dedi/lookup/<discom>/revocation-registry/<credential-id>`). Not-found / `not_revoked` valid; `revoked` reject.

And a validity-window check:

4. **`validFrom <= now <= validUntil`.**

The Consumer Energy Passport and Consumer Meter Digest variants add a fifth check at presentation time:

5. **Holder-binding proof.** The wallet signs a Verifiable Presentation with the private key matching `credentialSubject.id`, embedding a fresh `challenge` and `domain`. The verifier verifies the VP signature against the public key in `credentialSubject.id`. See Identifiers — Pattern 1.

No IES-curated registry sits between the credential and the verifier. The IES DISCOMs Reference Registry is the **inter-DISCOM data exchange network**'s trust boundary (Beckn-side); it is not a credential prerequisite — see Identifiers — Two identities.

7.10.1 Signing-key sources

OpenCred loads exactly one signing key from one of:

Source	Env var	When to use
Software file (PEM, JWK, PKCS#8, PFX)	`OPENCRED_KEY_PATH`	Dev, small DISCOMs
AWS KMS	`OPENCRED_KMS_PROVIDER=aws`, `OPENCRED_KMS_KEY_ARN`	Production on AWS
Azure Key Vault	`OPENCRED_KMS_PROVIDER=azure`, `OPENCRED_AZURE_*`	Production on Azure
GCP Cloud KMS	`OPENCRED_KMS_PROVIDER=gcp`, `OPENCRED_GCP_KMS_KEY_NAME`	Production on GCP

The private key never leaves the container; in KMS modes it never leaves the HSM at all. There is no shared signing service and no key escrow.

7.10.2 Proof formats

Format	When to choose	Where it shines
`vc-jwt` (default)	Most flows	Compact wire form, easy to embed in headers, fast to verify
`data-integrity`	Custom-registered clean-context schemas	Linked-data-friendly, supports selective disclosure variants
`sd-jwt-vc`	Selective disclosure	The holder presents only chosen fields to each verifier

---

7.11 Appendix B — Operational notes

The bare minimum to run OpenCred in production.

7.11.1 Key rotation

1. Generate a new signing key in your KMS.
2. Publish the new key in `did.json` (keep the old key listed for a transition window).
3. Restart OpenCred pointing at the new key.
4. After the transition window, remove the old key from `did.json`.

Existing credentials signed by the old key keep verifying as long as the old key remains in `did.json`. Once you drop it, those credentials stop validating — schedule re-issuance before dropping.

7.11.2 Schema validation

Validate the body of every `POST /v1/credentials/issue` against the schema **before** sending it. OpenCred validates server-side too, but a client-side check catches integration bugs earlier and avoids logging PII into OpenCred's error trail. Use the JSON Schema at `schemas/ElectricityCredential/v1.2/schema.json` (or the matching version).

7.11.3 Batch issuance

For high-volume flows (annual re-issue, bulk Passport rollout):

- Process in batches of 100–1000; respect `Retry-After` if OpenCred returns 429.
- Sign in-process; do not externalise to a queue that buffers credentials in the clear.
- Persist `(credentialId, customerNumber, status)` after each successful issue so retries are idempotent.
- Run integration tests against `test-` networks before flipping the prod flag.

OpenCred's API reference covers concurrency, rate limits, and error shapes for production scale.

7.11.4 Reverse proxy + TLS

Never expose OpenCred's `:3100` directly. Terminate TLS at nginx / Envoy / your existing edge; forward to OpenCred over an internal network. The `OPENCRED_API_KEY` is the only auth — leaking it gives the bearer full issuance powers.

7.11.5 Troubleshooting

Symptom	Likely cause	Action
`/v1/health` returns `signingKeyLoaded: false`	Key path wrong, key file permission denied, or KMS creds missing	Check the container logs; verify the mount and the env vars
`POST /v1/credentials/issue` returns `400 schema_validation_error`	Body doesn't match the schema for the declared `schemaId`	Diff against `schemas/<Credential>/<version>/schema.json`
`POST /v1/credentials/verify` returns `valid: false` for a freshly issued credential	You sent the JSON envelope instead of the compact JWS for a `vc-jwt` proof	Send `.credential.proof.jwt`, not the whole envelope
`/v1/keys/publish` fails with a validation error	A pre-existing DeDi registry has an incompatible schema	Either let OpenCred recreate, or align the registry schema; see OpenCred Deployment
`revoke` succeeds but verifiers still see `not_revoked`	DeDi cache; OpenCred's local cache	Wait 5–10 minutes; verify the registry directly via `curl https://api.dedi.global/dedi/lookup/<ns>/vc-revocation-registry/<hash>`

7.12 Appendix C — Core concepts

For readers new to verifiable credentials. Skip if you're mid-deployment.

7.12.1 What's a Verifiable Credential

A **Verifiable Credential (VC)** is a JSON object with three properties:

- Who made the statement (`issuer`).

- The statement itself (`credentialSubject`).
- A cryptographic proof (`proof`) so anyone can verify the issuer signed it.

The IES profile uses the W3C VC Data Model 2.0. Required top-level fields: `@context`, `id`, `type`, `issuer`, `validFrom`, `credentialSubject`. Optional: `validUntil`, `credentialStatus`, `evidence`, `name`, `description`. The `proof` is added by the signing step.

7.12.2 What's a DID

A **Decentralized Identifier (DID)** is a globally unique string that resolves to a DID document — a small JSON object listing the subject's current public keys. IES uses three standard W3C methods:

- `did:web` — backed by an HTTPS-hosted `did.json` on the issuer's domain (used for DISCOMs, regulators, AMISPs).
- `did:key` — the public key is encoded in the DID string itself (offline-resolvable; used for consumer wallets).
- `did:jwk` — same idea as `did:key`, JWK-encoded.

There is no `did:dedi` method; DeDi is a key-discovery and registry layer over `did:web`. Full treatment: Identifiers — Appendix A.

7.12.3 Identifier vs. record

A DID is a stable identifier that resolves to a record (the DID document, or in DeDi's case any registry record). Records change — keys rotate, addresses update — without the identifier changing. See Identifiers — Appendix D for the licence-plate analogy and concrete IES scenarios.

7.12.4 Credential lifecycle

`Issued → Held / presented → Verified → (eventually) Revoked or expired`

- **Issued**: the issuer signs and emits the credential JSON.
- **Held**: a wallet or DigiLocker stores it.
- **Presented**: the holder shares it (raw or in a Verifiable Presentation) with a verifier.
- **Verified**: the verifier checks the issuer's signature, the regulator's `idRef` if present, revocation status, and the validity window.
- **Revoked**: the issuer publishes a hash in the DeDi revocation registry. Verifiers reject revoked credentials.
- **Expired**: `validUntil` passes. Verifiers reject expired credentials. Issue a fresh one on material change (rate revision, meter swap, ownership transfer) rather than relying on long expiry windows.

7.13 References

- Identifiers and Addressing — `did:web` setup, asset IDs, holder binding
- Registries and Directories — DeDi namespace, revocation registry, IES networks
- Schemas — canonical schema mirrors for ElectricityCredential, MeterData(Credential), MeterDataRequest(Credential)
- DigiLocker delivery — Pull URI, callback, signature pinning
- Use cases — Consumer Energy Passport
- Use cases — Consumer Meter Digest
- Use cases — Smart Meter Data Exchange
- OpenCred upstream documentation

7.14 DigiLocker delivery

This guide covers everything your DISCOM needs to build the DigiLocker Pull URI endpoint — the single piece of custom code required to make IES energy credentials available to consumers in DigiLocker.

Two document types flow into DigiLocker through the **same Pull URI mechanism**:

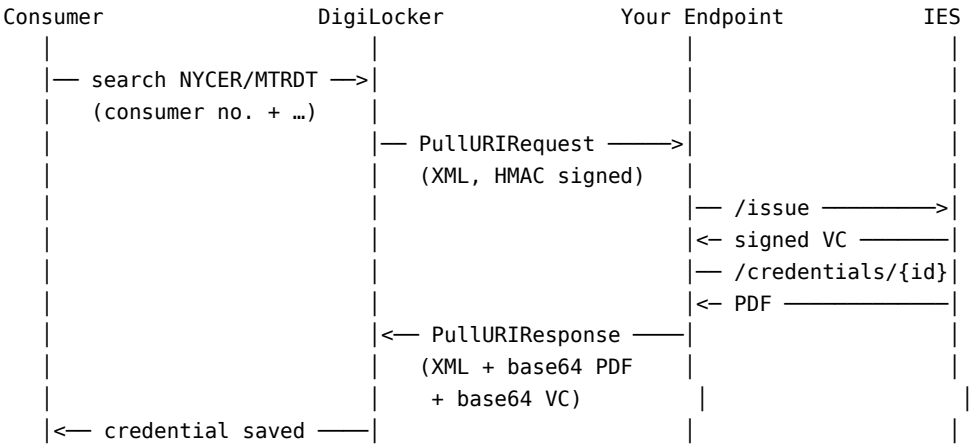
DocType	Credential	What it carries	Keyed on
`NYCER`	**Electricity Credential v1.2**	Slow-changing facts — who holds the connection and every typed asset behind the meter (meter, solar, battery, EV, inverter, load, network), each power/capacity value a `{value, unit}` pair	Consumer number
`MTRDT`	**Consumer Meter Digest** (`MeterDataCredential` v0.6)	The consumer's own meter readings for a chosen period — a signed energy *statement*, not a bill	Consumer number + **period**

> **Temporary DocType.** `MTRDT` is used here as a working DocType for the Consumer Meter Digest while the formal registration with NeGD is finalised. Treat it as a placeholder — the final DocType string will be confirmed in the written ask to DigiLocker and may change.

The Digest reuses the connection-credential flow almost unchanged. The one structural difference is the document key: the Digest puts the **period into the URI** so every period coexists instead of overwriting — exactly the difference between a statement and a bill.

7.14.1 What DigiLocker Does

DigiLocker is India's national digital document wallet. When a consumer searches for one of these credentials in DigiLocker, DigiLocker calls your endpoint, your endpoint calls IES (OpenCred) to issue a signed credential, and DigiLocker stores it — all in real time.



DigiLocker matters because the credential only has value when it reaches a use case. A consumer does not wake up wanting to download meter data; they download it because an analytics app will break down their

consumption, a lender will assess them, or a subsidy portal will verify eligibility. DigiLocker is the bridge between the signed credential and that ecosystem — it carries the consent and sharing rails (QR scan in person, OAuth consent pull for third-party apps) that make a credential useful beyond the wallet.

7.14.2 Flow Overview

The integration has three phases:

- **Phase 0 — One-time setup:** Register on API Setu, create issuer DID, register the DocTypes
- **Phase 1 — Pull URI endpoint:** Build and host the HTTPS endpoint DigiLocker calls (one handler, both DocTypes)
- **Phase 2 — Consumer sharing:** Consumers share the credential with verifiers via DigiLocker OAuth (no DISCOM involvement)

7.14.3 Phase 0 — One-Time Setup

Step 1 — Register on API Setu

Go to https://apisetu.gov.in and register as a document issuer. You receive: - `issuer_id` (e.g. `in.gov.tpddl`) - `api_key` — store this securely; used for HMAC verification

> If your DISCOM already issues electricity bills (ELBIL) on DigiLocker, contact NeGD to add the `NYCER` and `MTRDT` document types to your existing account. Do not re-register.

Step 2 — Stand Up OpenCred and Your Issuer DID

Follow Deployment end-to-end. By the time you reach this page you should have:

- OpenCred running with `signingKeyLoaded: true`
- An issuer DID — `did:web:ies.tpddl.in` (recommended) or `did:key:…` (from an imported DSC)
- (Optional, recommended) DeDi configured for revocation

Save your issuer DID — it goes into every `POST /v1/credentials/issue` call.

Step 3 — Register the Pull URI Endpoints on API Setu

Register both DocTypes against the same Pull URI endpoint. They differ only in the UDF fields and the URI key.

NYCER — Electricity Credential v1.2

Field	Value
Endpoint URL	`https://{your-domain}/digilocker/pulluri`
HTTP Method	`POST`
Content-Type	`application/xml`
DocType	`NYCER`
UDF1 Label	`Consumer Number`
UDF2 Label	`Registered Mobile Number`

MTRDT — Consumer Meter Digest (temporary DocType)

Field	Value
Endpoint URL	`https://{your-domain}/digilocker/pulluri`
HTTP Method	`POST`
Content-Type	`application/xml`
DocType	`MTRDT`
UDF1 Label	`Consumer Number`
UDF2 Label	`Statement Period` (e.g. `2026-05` or `last-12-months`)
UDF3 Label	`Registered Mobile Number`

> **Two items need DigiLocker to confirm behaviour for MTRDT** — see The Consumer Meter Digest: (1) **period-in-URI** keying so multiple periods coexist, and (2) **configurable rendering** of the statement card. Everything else reuses the NYCER flow unchanged.

7.14.4 Phase 1 — The Pull URI Endpoint

This is the **only endpoint your DISCOM builds**. DigiLocker calls it; you respond. A single handler serves both DocTypes — branch on `DocType` (see Routing by DocType).

Endpoint Specification

```
POST https://{your-domain}/digilocker/pulluri
Content-Type: application/xml
x-digilocker-hmac: <Base64(HMAC-SHA256(api_key, request_body))>
```

Step 1 — Verify the Inbound HMAC

Always verify the HMAC before doing anything else. Reject with HTTP 401 if invalid.

```python
import hmac, hashlib, base64

def verify_digilocker_hmac(request_body: bytes, api_key: str, received_hmac: str) -> bool:
    expected = base64.b64encode(
        hmac.new(api_key.encode(), request_body, hashlib.sha256).digest()
    ).decode()
    return hmac.compare_digest(expected, received_hmac)  # constant-time comparison
```

Step 2 — Parse the Inbound Request

DigiLocker sends a `PullURIRequest` XML v3.0. For NYCER:

```xml
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<PullURIRequest ver="3.0"
                ts="2026-04-01T10:30:00+05:30"
                txn="TXN-20260401-001"
                orgId="tpddl">
  <DocDetails>
    <DocType>NYCER</DocType>
    <DigiLockerId>7a3f9b2c-1e4d-4f8a-b6c2-0d5e8f1a2b3c</DigiLockerId>
    <UDF1>TPDDL-2025-001234567</UDF1>   <!-- consumer number -->
    <UDF2>9876543210</UDF2>             <!-- registered mobile number -->
    <FullName>Priya Sharma</FullName>    <!-- optional, from DigiLocker profile -->
  </DocDetails>
</PullURIRequest>
```

For MTRDT the period travels as a UDF, so the consumer can pull a chosen window:

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<PullURIRequest ver="3.0"
                ts="2026-05-15T10:00:00+05:30"
                txn="TXN-20260515-042"
                orgId="tpddl">
  <DocDetails>
    <DocType>MTRDT</DocType>
    <DigiLockerId>7a3f9b2c-1e4d-4f8a-b6c2-0d5e8f1a2b3c</DigiLockerId>
    <UDF1>TPDDL-2025-001234567</UDF1>   <!-- consumer number -->
    <UDF2>last-12-months</UDF2>          <!-- statement period: YYYY-MM, a range, or a keyword -->
    <UDF3>9876543210</UDF3>             <!-- registered mobile number -->
    <FullName>Priya Sharma</FullName>
  </DocDetails>
</PullURIRequest>
```

Field	Description
`ts`	Request timestamp — echo in response
`txn`	Transaction ID — echo in response
`UDF1`	Consumer number — primary CIS lookup key
`UDF2` (NYCER)	Registered mobile number — secondary verification
`UDF2` (MTRDT)	Statement period — `YYYY-MM`, a range, or a keyword like `last-12-months`. A recurring pull with the current month fetches the latest statement.
`UDF3` (MTRDT)	Registered mobile number — secondary verification
`FullName`	DigiLocker profile name — DigiLocker validates name match against your response

Step 3 — Look Up Consumer in CIS

```
consumer = cis.lookup(consumer_number=udf1, mobile=mobile)
if not consumer:
    return pulluri_error_response(ts, txn, status=0, message="Consumer not found")
```

Step 4 — Call OpenCred to Issue the Credential

This step differs by DocType — see Issuing NYCER (v1.2) and Issuing the Digest (MTRDT) below. Both return a signed VC; both can ask OpenCred to render the PDF via `packageFormats: ["pdf"]`.

Step 5 — Package the PDF and VC for the response

OpenCred can render the PDF for you when you pass `packageFormats: ["pdf"]`. The PDF carries the signed credential payload in its info dictionary and embeds a scannable QR — DigiLocker stores the PDF, and verifiers can later re-extract the credential from the PDF itself.

```
import base64, json

# pdf_bytes came back from the OpenCred response in Step 4
pdf_b64 = base64.b64encode(pdf_bytes).decode()
vc_b64  = base64.b64encode(json.dumps(vc_json).encode()).decode()
```

If your DISCOM wants a custom PDF design instead (branded letterhead, regional language, a consumption-ladder or time-of-day card for the Digest), render server-side with your own template engine and embed the QR + the credential JSON as you prefer. The `VcContent` field in the PullURIResponse is what verifiers actually parse — the PDF is mostly for human display.

In every mode the signed JSON moves as-is. DigiLocker may render a summary card on screen, but what it shares with a verifier is the original signed credential in `VcContent`, unaltered — because the third party needs a verifiable document, not a PDF. A PDF cannot be checked against the issuer's signature; the signed JSON can.

Step 6 — Return the PullURIResponse

```
def build_pulluri_response(ts, txn, doc_type, uri, issued_to, valid_from, valid_to, pdf_b64, vc_b64):
    return f"""<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<PullURIResponse ver="3.0" ts="{ts}" txn="{txn}" orgId="tpddl">
  <ResponseStatus Status="1" ts="{ts}" txn="{txn}"/>
  <DocDetails>
    <DocType>{doc_type}</DocType>
    <URI>{uri}</URI>
    <IssuedTo>{issued_to}</IssuedTo>
    <ValidFrom>{valid_from}</ValidFrom>
    <ValidTo>{valid_to}</ValidTo>
    <DocContent format="pdf">{pdf_b64}</DocContent>
    <VcContent format="json-ld">{vc_b64}</VcContent>
  </DocDetails>
</PullURIResponse>"""
```

Field	Description
`Status="1"`	Success. Use `Status="0"` for errors.
`URI`	Unique identifier for this document in DigiLocker. NYCER: `{issuer_id}-NYCER-{consumerNo}`. MTRDT: `{issuer_id}-MTRDT-{consumerNo}-{period}` — **the period makes each statement a distinct document** (see The document key).
`IssuedTo`	Consumer's full name — DigiLocker validates this against the user's profile
`DocContent`	Base64-encoded PDF (the rendered statement / certificate card)
`VcContent`	Base64-encoded W3C VC JSON-LD — the signed credential, proof intact

Handler Logic Summary

```
1. Read raw request body (keep bytes for HMAC)
2. Verify x-digilocker-hmac -> HTTP 401 if invalid
3. Parse XML -> extract DocType, UDF1..n, ts, txn
4. Lookup consumer in CIS -> error response if not found
5. Resolve / generate the consumer's holder DID
6. Branch on DocType:
     NYCER -> build v1.2 credentialSubject (energyResources[])
     MTRDT -> build MeterData v0.6 payload for the requested period
7. POST /v1/credentials/issue with packageFormats=["pdf"] -> signed VC + PDF
8. Base64-encode PDF and VC
9. Return PullURIResponse XML with Status=1 and the DocType-appropriate URI
```

Routing by DocType

```
doc_type = request_xml.find(".//DocType").text
if doc_type == "ELBIL":
    return handle_elbil(request_xml)   # existing electricity bill, if you serve it
elif doc_type == "NYCER":
    return handle_nycer(request_xml)   # Electricity Credential v1.2
elif doc_type == "MTRDT":
    return handle_mtrdt(request_xml)   # Consumer Meter Digest
else:
    return pulluri_error_response(ts, txn, status=0, message="Unsupported DocType")
```

7.14.5 Issuing NYCER — the Electricity Credential v1.2

The NYCER credential DISCOMs issue today carried a flat set of customer-and-connection fields. The IES Electricity Credential is now finalised at **v1.2**, and it carries far more: every physical asset behind the connection — the meter itself, plus solar, battery, EV, inverter, and controllable-load assets — as a single typed resource model with unit-bearing quantities.

What changed from the flat shape

	Today (flat)	v1.2 Electricity Credential
Customer & connection	Flat fields on `credentialSubject`	Non-PII `customerProfile` + PII `customerDetails`
Meter	`meterNumber` / `meterType` fields	`energyResources` entry, `type` `METER`, `attributes.meterCapability` + `energyDirection`
Generation, storage, EV, …	Not carried	Typed resources — `SOLAR_PV`, `BESS`, `EV_CHARGER`, `INVERTER`, …
Power & capacity values	n/a	`QuantitativeValue {value, unit}` — e.g. `maxExport {10, kW}`
Asset topology	Not carried	`parentResources` / `subResources` — submetering, parallel metering
Consumption / tariff	Not carried	`consumptionProfiles[]`, `sanctionedLoad` as `{value, unit}`

credentialSubject shape

```
credentialSubject
├─ id                       (optional)  customer DID
├─ customerProfile          (required)  non-PII
│  ├─ customerNumber        (required)  utility CA number
│  ├─ idRef                 (optional)  external identity reference (masked)
│  ├─ energyResources[]     (required)  typed assets, min 1 (at least one METER)
│  └─ consumptionProfiles[] (optional)  tariff / load per meter (QV units)
└─ customerDetails          (optional)  PII: fullName, address, connection date
```

Seven typed `EnergyResource` kinds, each aligned to CIM classes (IEC 61968 / 61970):

Kind	`type` values	CIM alignment
Meter	`METER`	`cim:Meter` / `EndDevice`
Generator	`SOLAR_PV`, `WIND`, `HYDRO`, `BIOGAS`, `CHP`, `FUEL_CELL`	`cim:GeneratingUnit`
Storage	`BESS`	`cim:BatteryUnit`
EV Charger	`EV_CHARGER`, `EV_V2G`	`cim:EVChargingStation`
Inverter	`INVERTER`	`cim:PowerElectronicsConnection`
Load	`SMART_HVAC`, `SMART_WATER_HEATER`, `CONTROLLABLE_LOAD`	`cim:EnergyConsumer`
Network	`DT`, `BUS`, `FEEDER`, `MICROGRID`	`cim:PowerTransformer` / `Feeder`

Common attributes (all kinds): `make`, `model`, `maxExport`, `maxImport`, `commissioningDate`, `location`. Per-kind adds, e.g. Storage -> `storageCapacity`, `storageType`, `stateOfHealthPct`; Meter -> `meterCapability`, `energyDirection`, `functions[]`. Every power or capacity figure is a `QuantitativeValue` — a `{value, unit}` pair with short unit aliases (`kW`, `kWh`, `kVA`, `kVAR`, `kV`) mapped to QUDT IRIs in the JSON-LD context.

> Only the meter is mandatory — a consumer with no DERs has one `METER` entry. PII (the name) lives only in `customerDetails`, so a verifier needing asset facts but not identity can be given `customerProfile` alone. The v1.1 type aliases `SOLAR` and `BATTERY` remain valid for backward compatibility, but `SOLAR_PV` / `BESS` are preferred.

The OpenCred issue call

```python
import requests
from datetime import datetime, timezone, timedelta

IST = timezone(timedelta(hours=5, minutes=30))

credential_response = requests.post(
    "http://opencred:3100/v1/credentials/issue",
    headers={"Authorization": f"Bearer {OPENCRED_API_KEY}"},
    json={
        "issuerDid":  ISSUER_DID,                # e.g. "did:web:ies.tpddl.in"
        "schemaId":   "electricity/v1.2",
        "additionalTypes": ["ElectricityCredential"],
        "proofFormat": "vc-jwt",
        "validFrom":  datetime.now(IST).isoformat(),
        "validUntil": (datetime.now(IST) + timedelta(days=365 * 5)).isoformat(),
        "subjectDid": holder_did,
        "credentialSubject": {
            "id": holder_did,
            "customerProfile": {
                "customerNumber": consumer.consumer_number,
                "idRef": {
                    "issuedBy":  "did:web:uidai.gov.in",
                    "subjectId": f"uidai.gov.in:{consumer.masked_aadhaar}",
                },
                "energyResources": [
                    {
                        "id":   consumer.meter_id,
                        "type": "METER",
                        "attributes": {
                            "meterCapability":   consumer.meter_capability,   # e.g. "AMI"
```

```
                            "energyDirection":   consumer.energy_direction,    # e.g. "Bidirectional"
                            "functions":         consumer.meter_functions,     # ["NetMetering", "ToU"]
                            "applicationProtocol": "DLMS_COSEM",
                        },
                    },
                    # … append SOLAR_PV / BESS / EV_CHARGER / INVERTER entries as present,
                    #    each power/capacity field a {"value": …, "unit": "kW"|"kWh"|…} pair,
                    #    each linked to the meter via "parentResources": [meter_id]
                ],
                "consumptionProfiles": [
                    {
                        "meterId":           consumer.meter_id,
                        "sanctionedLoad":    {"value": consumer.sanctioned_load_kw, "unit": "kW"},
                        "tariffCategoryCode": consumer.tariff_code,
                        "premisesType":      consumer.premises_type,
                        "connectionType":    consumer.connection_type,
                        "paymentMode":       consumer.payment_mode,
                    }
                ],
            },
            "customerDetails": {
                "fullName": consumer.full_name,
                "installationAddress": {
                    "address":   consumer.address_line,
                    "city":      {"name": consumer.city,  "code": consumer.city_code},
                    "state":     {"name": consumer.state, "code": consumer.state_code},
                    "country":   {"name": "India",        "code": "IN"},
                    "area_code": consumer.pincode,
                },
                "serviceConnectionDate": consumer.connection_date.isoformat(),
            },
        },
        "revocationRegistryUrl": DEDI_REVOCATION_URL,
        "packageFormats": ["pdf"],
    }
).json()

vc_json = credential_response["credential"]

# Inject the schema-required issuer object before delivery — OpenCred returns
# issuer as the DID string, but the ElectricityCredential schema requires the
# full object with name and (optional) idRef. This invalidates the original
# proof; re-sign downstream if you need a single signed-and-conformant artifact.
vc_json["issuer"] = {
    "id":   ISSUER_DID,
    "name": ISSUER_NAME,
    "idRef": {
        "issuedBy":  IES_REGISTRY_DID,         # namespace DID of india-energy-stack on dedi.global
        "subjectId": IES_REGISTRY_SUBJECT_ID,  # e.g. india-energy-stack:tpddl
    },
}

pdf_bytes = base64.b64decode(
    next(p for p in credential_response.get("packagedOutputs", [])
```

```
        if p["format"] == "pdf")["contentBase64"]
)
```

The URI for NYCER stays keyed on the consumer number — a refresh overwrites the last, which is correct for a slow-changing connection credential:

```
uri = f"in.gov.tpddl-NYCER-{consumer.consumer_number}"
```

What DigiLocker needs to accept for v1.2

1. **Accept the v1.2 schema for NYCER.** Validate against the published v1.2 context and JSON Schema — a `customerProfile` with `energyResources[]` (min one `METER`), optional `consumptionProfiles[]`, and `customerDetails` for PII. Power/capacity fields are `{value, unit}` objects, not bare numbers.
2. **Pass `VcContent` through unchanged.** The signed JSON-LD is the v1.2 credential as issued, proof intact — no reshaping. The proof is over the canonical form, and the QUDT unit context is part of it.
3. **Carry resource fields into the Certificate XML.** Extend the `DataContent` block so meter and DER resources (`type`, key attributes with value+unit) appear for DigiLocker-native parsers, alongside the existing connection and address fields.
4. **Render whatever resources are present.** Drive the card from `energyResources[]` as a list, showing each value with its unit. A meter-only consumer shows one row; a prosumer shows meter, solar, battery, EV. New kinds and unit types need no card change.

7.14.6 The Consumer Meter Digest (DocType `MTRDT`)

The **Consumer Meter Digest** is a DISCOM-issued, digitally signed credential carrying a consumer's own meter readings for a chosen window of time. Where NYCER attests to slow-changing facts — who holds the connection, what assets are behind the meter — the Digest is the consumer's **energy statement**. Like a bank statement records transactions, the meter records a reading roughly every fifteen minutes — around 96 blocks a day — and the Digest packages those readings for a requested period into a signed document the consumer can hold and present.

The core idea — a statement, not a bill

Treat the meter as a statement, not a bill. A **bill** is the latest snapshot that overwrites the last. A **statement** is a series the consumer can pull by period — "April, May, last twelve months" — and each one is kept, not replaced. This single shift is what the `MTRDT` integration is about, and it is why the document key carries the period.

Schema compliance — `MeterDataCredential` v0.6

The Digest is **not a new schema**. It is a consumer-facing issuance of the standard `MeterDataCredential` v0.6 — a W3C VC 2.0 that subclasses `EnergyCredential` v2.0 and wraps a `MeterData` v0.6 payload. The same credential AMISPs and MDMs issue provider-to-DISCOM in the Beckn data-exchange flow is what the consumer pulls into DigiLocker. Only the trigger differs: the consumer requests it for a period.

The envelope — `issuer` (with regulatory `licenseNumber`), `validFrom` / `validUntil`, DeDi `credentialStatus`, `proof` — is inherited from `EnergyCredential` v2.0. What the Digest defines is `credentialSubject.meterData`: a `MeterData` v0.6 payload of one or more typed profiles.

```
MeterDataCredential  (W3C VC 2.0  ·  subclass of EnergyCredential v2.0)
├─ @context        W3C credentials/v2 + EnergyCredential v2.0 + MeterDataCredential v0.6
├─ type            ["VerifiableCredential", "MeterDataCredential"]
├─ issuer          id, name, licenseNumber  (DISCOM / AMISP / MDM DID)
├─ validFrom / validUntil                  (inherited envelope)
├─ credentialStatus  DeDi revocation registry  (inherited)
```

```
├─ credentialSubject
│  ├─ id           consumer / asset DID
│  └─ meterData    MeterData v0.6 payload — single profile OR array of profiles
│                  profileType  in  CUSTOMER | INTERVAL | DAILY | MONTHLY |
│                  BILL_DETAILS | INSTANTANEOUS | EVENT | ALARM | DESCRIPTOR
└─ proof           Ed25519Signature2020
```

The **period and shape** the consumer chose are expressed by the profiles themselves:

- A **twelve-month statement** is a `MONTHLY` profile (a `P1M` interval-block array).
- **Raw analytics data** is an `INTERVAL` profile (`PT15M` / `PT30M` blocks, ~96 readings a day), optionally preceded by a `DESCRIPTOR` profile that the interval blocks reference via `payloadDescriptorSetRef`.

A monthly-bill view and a 15-minute analytics feed are the **same credential type** with different `meterData` profiles. The consumer's requested window simply bounds which profiles the DISCOM returns — within the scope authorised by the originating request.

Example — a twelve-month statement (`MONTHLY` profile)

This is the exact JSON-LD that travels in `VcContent`.


```json
{
  "@context": [
    "https://www.w3.org/ns/credentials/v2",
    "https://schema.beckn.io/EnergyCredential/v2.0/context.jsonld",
    "https://india-energy-stack.github.io/ies-accelerator/schemas/MeterDataCredential/v0.6/context.jsonld"
  ],
  "id": "urn:uuid:9b3c1f0a-2d4e-4ba8-9f1c-1e7a90c8a5d2",
  "type": ["VerifiableCredential", "MeterDataCredential"],
  "issuer": {
    "id": "did:web:ies.tpddl.in",
    "name": "Tata Power Delhi Distribution Limited",
    "licenseNumber": "DERC-DISCOM-2025-007"
  },
  "validFrom": "2026-05-15T10:00:00+05:30",
  "validUntil": "2027-05-15T10:00:00+05:30",
  "credentialStatus": {
    "id": "https://dedi.global/dedi/lookup/tpddl/vc-revocation-registry/9b3c1f0a",
    "type": "dedi",
    "statusPurpose": "revocation"
  },
  "credentialSubject": {
    "id": "did:key:z6MkjVQ8r4f3rPuY7CG2D6Lf8WJxJBs5sjkR8d3v2Bv4nP4Z",
    "meterData": {
      "profileType": "MONTHLY",
      "meterId": "MTR-98765432",
      "servicePointId": "TPDDL-2025-001234567",
      "intervalBlocks": [
        { "start": "2025-05-01T00:00:00+05:30", "duration": "P1M", "readings": [{ "type": "ACTIVE_ENERGY_IMPORT",
        { "start": "2025-06-01T00:00:00+05:30", "duration": "P1M", "readings": [{ "type": "ACTIVE_ENERGY_IMPORT",
        { "start": "2025-07-01T00:00:00+05:30", "duration": "P1M", "readings": [{ "type": "ACTIVE_ENERGY_IMPORT",
        { "start": "2025-08-01T00:00:00+05:30", "duration": "P1M", "readings": [{ "type": "ACTIVE_ENERGY_IMPORT",
        { "start": "2025-09-01T00:00:00+05:30", "duration": "P1M", "readings": [{ "type": "ACTIVE_ENERGY_IMPORT",
        { "start": "2025-10-01T00:00:00+05:30", "duration": "P1M", "readings": [{ "type": "ACTIVE_ENERGY_IMPORT",
        { "start": "2025-11-01T00:00:00+05:30", "duration": "P1M", "readings": [{ "type": "ACTIVE_ENERGY_IMPORT",
        { "start": "2025-12-01T00:00:00+05:30", "duration": "P1M", "readings": [{ "type": "ACTIVE_ENERGY_IMPORT",
```

```
        { "start": "2026-01-01T00:00:00+05:30", "duration": "P1M", "readings": [{ "type": "ACTIVE_ENERGY_IMPORT",
        { "start": "2026-02-01T00:00:00+05:30", "duration": "P1M", "readings": [{ "type": "ACTIVE_ENERGY_IMPORT",
        { "start": "2026-03-01T00:00:00+05:30", "duration": "P1M", "readings": [{ "type": "ACTIVE_ENERGY_IMPORT",
        { "start": "2026-04-01T00:00:00+05:30", "duration": "P1M", "readings": [{ "type": "ACTIVE_ENERGY_IMPORT",
      ]
    }
  },
  "proof": { "type": "Ed25519Signature2020", "proofValue": "z5wQ…" }
}
```

Example — raw analytics (`INTERVAL` profile + `DESCRIPTOR`)

For raw analytics data the same wrapper carries an `INTERVAL` profile — `PT15M` blocks, around 96 readings a day — typically preceded by a `DESCRIPTOR` profile that the interval blocks reference via `payloadDescriptorSetRef`. The `meterData` value becomes an **array of profiles**:

```
{
  "credentialSubject": {
    "id": "did:key:z6MkjVQ8r4f3rPuY7CG2D6Lf8WJxJBs5sjkR8d3v2Bv4nP4Z",
    "meterData": [
      {
        "profileType": "DESCRIPTOR",
        "id": "desc-usage-15m",
        "payloadDescriptors": [
          { "type": "ACTIVE_ENERGY_IMPORT", "unit": "kWh", "readingType": "DIRECT_READ" }
        ]
      },
      {
        "profileType": "INTERVAL",
        "meterId": "MTR-98765432",
        "servicePointId": "TPDDL-2025-001234567",
        "payloadDescriptorSetRef": "desc-usage-15m",
        "intervalBlocks": [
          { "start": "2026-05-01T00:00:00+05:30", "duration": "PT15M", "readings": [{ "value": 0.18 }] },
          { "start": "2026-05-01T00:15:00+05:30", "duration": "PT15M", "readings": [{ "value": 0.16 }] }
        ]
      }
    ]
  }
}
```

Mapping to DigiLocker as it works today

DigiLocker already does everything the Digest needs — the **same Pull URI mechanism** the electricity-connection credential uses. The Digest is a new document type returned through the same response, with the signed `MeterDataCredential` in `VcContent` and a rendered statement in `DocContent` / `DataContent`. The one change that matters is the document key.

DigiLocker mechanism (today)	How the Digest uses it
Pull URI endpoint + DocType	Register a new DocType (`MTRDT`) for the Digest; DigiLocker calls the **same Pull URI** the connection credential uses.

DigiLocker mechanism (today)	How the Digest uses it
URI — the document key	Put the period into the URI: `in.gov.{discom}-MTRDT-{consumerNo}-{YYYY-MM}`. This is the composite key that lets every period coexist instead of overwriting.
UDF request fields	Consumer number as `UDF1`; the period (or month) as a UDF — so the consumer pulls a chosen window, and a recurring pull fetches the current one.
`VcContent` (JSON-LD)	The signed `MeterDataCredential` as-is — proof intact — for verifiers to parse and check.
`DocContent` (PDF) / `DataContent` (XML)	A rendered statement card — consumption ladder or time-of-day — built from a configurable template, with a summary the consumer reads on screen.
OAuth consent pull (`/xml/{uri}`)	A third-party app pulls a chosen period directly with consumer consent — no app-switching.

The one real gap — the document key

Today the electricity bill (and the NYCER connection credential) is keyed on the **consumer number alone**, so each refresh overwrites the last. The Digest must be keyed on **consumer number plus period**. With the period in the URI, April's statement and May's statement are distinct documents the consumer holds side by side — exactly the difference between a statement and a bill.

```
# NYCER — overwrite on refresh (correct for a connection credential)
uri = f"in.gov.tpddl-NYCER-{consumer.consumer_number}"

# MTRDT — period in the key, so every statement coexists
uri = f"in.gov.tpddl-MTRDT-{consumer.consumer_number}-{period}"   # period e.g. "2026-05" or "2025-05_2026-04"
```

Issuing the Digest (`MeterDataCredential` v0.6)

Inside `handle_mtrdt`, resolve the requested window from `UDF2`, query MDMS/MDM for the matching readings, build the `MeterData` v0.6 payload, and call OpenCred. For a consumer-pulled Digest, `validUntil` may be set **shorter than the v0.6 default** where a verifier wants freshness (loan applications often want fresh; subsidy portals tolerate a week).

```
import requests
from datetime import datetime, timezone, timedelta

IST = timezone(timedelta(hours=5, minutes=30))

# 1. Resolve the requested window from UDF2 -> (start, end, profile_type, uri_period)
window = resolve_period(udf2)            # e.g. "last-12-months" -> MONTHLY, 2025-05..2026-04
readings = mdm.query(consumer.meter_id, window.start, window.end, window.granularity)

# 2. Build the MeterData v0.6 payload (single profile here; use an array for INTERVAL + DESCRIPTOR)
meter_data = {
    "profileType": window.profile_type,          # "MONTHLY" | "INTERVAL" | "DAILY" | …
    "meterId": consumer.meter_id,
    "servicePointId": consumer.consumer_number,
    "intervalBlocks": readings.to_interval_blocks(),
}

# 3. Issue via OpenCred — same instance, MeterDataCredential schema, short validUntil
```

```
credential_response = requests.post(
    "http://opencred:3100/v1/credentials/issue",
    headers={"Authorization": f"Bearer {OPENCRED_API_KEY}"},
    json={
        "issuerDid":  ISSUER_DID,
        "schemaId":   "meterdata/v0.6",
        "additionalTypes": ["MeterDataCredential"],
        "proofFormat": "vc-jwt",
        "validFrom":  datetime.now(IST).isoformat(),
        "validUntil": (datetime.now(IST) + timedelta(days=7)).isoformat(),   # freshness window
        "subjectDid": holder_did,
        "credentialSubject": {
            "id": holder_did,
            "meterData": meter_data,
        },
        "revocationRegistryUrl": DEDI_REVOCATION_URL,
        "packageFormats": ["pdf"],
    }
).json()

vc_json = credential_response["credential"]

# Inject the schema-required issuer object (with licenseNumber) — same pattern as NYCER.
vc_json["issuer"] = {
    "id":   ISSUER_DID,
    "name": ISSUER_NAME,
    "licenseNumber": DISCOM_LICENSE_NUMBER,      # e.g. "DERC-DISCOM-2025-007"
}

pdf_bytes = base64.b64decode(
    next(p for p in credential_response.get("packagedOutputs", [])
         if p["format"] == "pdf")["contentBase64"]
)

# 4. Period-keyed URI — this is the one structural difference from NYCER
uri = f"in.gov.tpddl-MTRDT-{consumer.consumer_number}-{window.uri_period}"
```

Revocations are rare in practice — Digests are usually short-lived enough to expire before any revoke would matter — but when a revoke is needed, pass an optional `reason` such as `"data-correction"` or `"holder-request"` on `POST /v1/credentials/revoke`; see Issuance -> revoking.

What the consumer can do with it

Three actions, all on DigiLocker's existing rails:

1. **View** a readable statement card (rendered from `DocContent` / `DataContent`) — a consumption ladder or time-of-day profile, with a summary the consumer reads on screen.
2. **Download** the signed document locally.
3. **Share** it — by **QR scan** in person, or by granting **OAuth consent** so a third-party app pulls the chosen period directly, with no app-switching.

In every mode the signed JSON moves as-is. What DigiLocker shares with a verifier is the original signed credential in `VcContent`, unaltered — because the third party needs a verifiable document, not a PDF.

The ask to DigiLocker (MTRDT)

Capability	What it means
Introduce the Digest document type	Register the Consumer Meter Digest (a `MeterDataCredential`) as a new DocType (`MTRDT`, temporary) with its schema and tags, alongside the existing electricity credential.
Key the URI on consumer + period	Put the period into the URI so multiple periods coexist rather than overwriting — the single change from today's connection credential.
Accept period as a pull parameter	Let the consumer pull a chosen historical window via a UDF, and let the current statement refresh on a regular cadence.
Render from a configurable template	Drive the `DocContent` / `DataContent` card from an externalised, per-DocType renderer — not hardcoded fields — so the card need not change when a parameter does.
Share `VcContent` unmodified	Pass the signed JSON-LD as-is over both QR-scan and OAuth consent pull, with no manipulation in between.

> The **period-in-URI** and **configurable-rendering** items are the two that need DigiLocker to confirm behaviour; everything else reuses the connection-credential flow unchanged. A formal, written ask should capture the initial DocType list (including the final DocType string that replaces the temporary `MTRDT`) and a committed timeline.

7.14.7 Error Response Format

```xml
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<PullURIResponse ver="3.0" ts="{ts}" txn="{txn}" orgId="tpddl">
  <ResponseStatus Status="0" ts="{ts}" txn="{txn}">Consumer not found</ResponseStatus>
</PullURIResponse>
```

7.14.8 Phase 2 — Consumer Shares Credential with a Verifier

Phase 2 requires no DISCOM involvement. Consumers share their NYCER credential or Meter Digest from DigiLocker using the standard DigiLocker OAuth flow. See Verifying Credentials for the verifier-side implementation.

7.14.9 Security Checklist

- [ ] HMAC verified on every inbound request before any processing
- [ ] `hmac.compare_digest` used (not `==`) to prevent timing attacks
- [ ] Raw request bytes used for HMAC computation (not re-serialised XML)
- [ ] `customerProfile.idRef.subjectId` uses a **masked** identifier — full Aadhaar / ID never stored or logged
- [ ] Digest `meterData` carries readings only — no PII beyond the consumer/meter identifiers needed to bind the statement
- [ ] Period UDF validated and bounded to the scope authorised by the originating request (no arbitrary historical ranges)
- [ ] API key stored in environment variable / secrets manager, not in code

- [ ] Endpoint accessible only over HTTPS (TLS 1.2+)
- [ ] Rate limiting applied (DigiLocker can retry aggressively on timeout)

7.14.10 Reference

- Consumer Meter Digest — credential schema
- Electricity Credential — schema reference
- OpenCred Documentation — credential service API reference
- API Setu DigiLocker Partner Portal
- DigiLocker Technical Specification v1.0.5

Chapter 8

Data Exchange

In a hurry? Jump straight to the Checklist — every step with checkboxes.

This page is the single home for everything IES has to say about exchanging structured datasets over the network. If you're a **TSP, DISCOM, or AMISP** looking to share data as fast as possible — meter telemetry, ARR filings, tariff orders, anything else — start with Quick start and you'll have a signed `confirm -> on_confirm` round-trip running locally in under 10 minutes. The appendices cover the protocol theory, ONIX architecture, and validation mechanics when you need them.

> **About the walkthrough.** The commands below use the DEG Data Exchange devkit — a ready-to-run Docker stack that bundles the **ONIX** Beckn protocol adapter (signing, verification, registry lookup), a sandbox BAP/BPP pair, and a Caddy router. ONIX is the recommended adapter for IES; if you already run one, swap it in — the wire format and registry contracts are the same.

8.1 Why this page

Energy data in India moves through bespoke bilateral channels today — PDF filings, vendor-locked MDMS exports, opaque tariff interpretations. IES Data Exchange replaces those with one **coordination layer**: Beckn Protocol v2.0 for discovery / terms / consent / contract / audit, and the same flow either delivers the payload **inline** (for small datasets — typical IES default) or carries an **access method** (signed URL, MQTT topic, Kafka credential, etc.) for established bulk channels. Every exchange — inline or handoff — gets the same signed-audit story.

8.2 Pick your role

If you are…	You run	You do
BAP — data consumer (DISCOM, regulator, VAS provider, researcher)	BAP-side ONIX + your application	Send `confirm` (and optional discovery / negotiation / `status`), receive `on_confirm` / `on_status` callbacks
BPP — data provider (AMISP, DISCOM, SERC, data custodian)	BPP-side ONIX + your provider implementation	Receive `confirm`, emit `on_confirm` (and `on_status` for async / paged delivery) with the dataset

If you are…	You run	You do
Both	Both stacks	Both flows in parallel — common for DISCOMs that consume telemetry *and* publish their own datasets

The walkthrough uses the **BAP** path by default. The minimal flow — `confirm` -> `on_confirm` — is all you need to start; optional Beckn actions live in Optional Beckn actions.

8.3 Prerequisites

- **Docker 24+**, **Docker Compose**, **Git**, **Python 3**, and **Postman** *(or `curl`)*. ~2 GB free disk.
- *(For real-network exchange, not the local sandbox)* A **published DeDi subscriber identity** under a verified namespace — see Registries — Step-by-step and Identifiers — Joining a Beckn network.
- *(For real-network exchange)* **IES NFO has referenced you into a network registry** — see Registries — How to apply for an IES listing.

Domains to whitelist — if your organisation restricts outbound traffic, allow these before starting:

Purpose	Domain
Devkit and related repos	`https://github.com/beckn`
Discovery service (DS) — where ONIX routes `discover`	`https://34.93.165.42.sslip.io`
Catalog service (CS) + DeDi registry lookups	`https://fabric.nfh.global`
DeDi publishing	`https://publish.dedi.global`
Schema contexts	`https://schema.beckn.io`, `https://schema.nfh.global`, `https://raw.githubusercontent.com`
IES docs and hosted schemas	`https://india-energy-stack.gitbook.io/docs`, `https://india-energy-stack.github.io`

The devkit ships pre-configured sandbox identities (`bap.example.com` / `bpp.example.com`) — no real credentials needed for the local walkthrough.

8.4 Quick start — run a local exchange in 10 minutes

Five steps from a fresh clone to a verified `confirm` -> `on_confirm` round-trip.

8.4.1 1. Clone and start the stack

```
git clone https://github.com/beckn/DEG.git
cd DEG/devkits/data-exchange/install
docker compose up -d
```

Verify the services:

```
docker compose ps                        # all 7 containers should be running (redis/sandbox: healthy)
curl -s http://localhost:8081/health     # onix-bap -> {"status":"ok"}
```

```
curl -s http://localhost:8082/health       # onix-bpp -> {"status":"ok"}
curl -s http://localhost:9000/             # beckn-router -> "beckn-router ok"
```

If checks fail, give ONIX ~15s after `docker compose up` to initialise, then re-curl. Persistent `connection refused` usually means a container isn't running; `docker compose logs onix-bap` / `onix-bpp` shows why.

8.4.2 2. Import the Postman collection for your use case

Each use case ships matching BAP and BPP collections. Import the **BAP** collection for your use case:

```
devkits/data-exchange/uc1-meter-data/postman/data-exchange-uc1-meter-data.BAP-DEG.postman_collection.json
devkits/data-exchange/uc2-regulatory-data/postman/data-exchange-uc2-regulatory-data.BAP-DEG.postman_collection.jso
devkits/data-exchange/uc3-tariff-policy/postman/data-exchange-uc3-tariff-policy.BAP-DEG.postman_collection.json
```

Defaults are correct for local runs — don't change `beckn_adapter_url` (`http://localhost:8081/bap/caller`) or the `bap_host_root`/`bpp_host_root` docker hostnames. The two layers of URL are explained in Appendix D — Hostnames sandbox vs production.

8.4.3 3. Send `confirm`

Open the `confirm` request in Postman and click **Send**.

The synchronous response is the **ACK envelope**: `{"message":{"status":"ACK","messageId":<id>}}` — *"accepted, async callback pending"*. The ACK originates at the provider's webhook and cascades back through the adapters to Postman.

8.4.4 4. Watch `on_confirm` land

Tail the BAP sandbox logs:

```
docker logs sandbox-bap 2>&1 | grep on_confirm | tail -5
```

The `on_confirm` body carries the dataset (inline) or a handle pointing to a later `on_status`. Either way, seeing `on_confirm` in `sandbox-bap` is your end-to-end signal.

If nothing arrives: (a) check `bap_host_root` / `bpp_host_root` are still the docker hostnames, (b) `docker logs onix-bap | grep -i error` + `onix-bpp`, (c) host-alias DNS — see beckn/DEG#319.

8.4.5 5. Stop the stack when done

```
docker compose down
```

8.4.6 BPP path

Same sanity check in reverse — import the **BPP** collection, trigger `on_confirm`, and tail `docker logs sandbox-bpp 2>&1 | grep -E 'confirm|status' | tail -5`. To replace the sandbox with your own provider, see Make the sandbox your own below.

8.5 Go over the public internet (ngrok)

Once the local exchange is green, prove the same stack interoperates through a real public tunnel — typically with another participant's laptop or a cloud host — without DeDi identity yet:

1. Create a free ngrok account; copy the devkit's tunnel config and add your authtoken:

```
cd DEG/devkits/data-exchange/install
cp ngrok.yml.example ngrok.yml   # edit: paste your authtoken
ngrok start --all --config ngrok.yml
```

2. In Postman, set `bap_host_root` / `bpp_host_root` to the HTTPS URL ngrok prints (e.g. `https://abc123.ngrok-free.dev`) — docker-only dummy hostnames aren't reachable from outside, so the payload URIs must carry your public tunnel URL.
3. Fire `confirm` again; watch the hops in the ngrok inspector at http://localhost:4040.
4. For two-party interop, each side runs its own tunnel and points `bpp_host_root` (or `bap_host_root`) at the *other side's* URL. Revert the host variables to `http://beckn-router:9000` to return to local-only.

Full recipe: devkit README — Over-the-internet notes.

8.6 Swap in your real identity

The walkthrough so far ran on the shipped sandbox identities. To join the IES networks, replace them with yours in `config/local-simple-bap.yaml` (and the matching `local-simple-bpp.yaml`):

```yaml
modules:
  - name: bapTxnReceiver
    handler:
      plugins:
        registry:
          config:
            allowedNetworkIDs: "indiaenergystack.in/test-ies-data-sharing-network"   # comma-separated string
        keyManager:
          config:
            networkParticipant: your.subscriber.id        # your DeDi subscriberId
            keyId: <recordId-from-DeDi>                   # the recordId DeDi gave you
            signingPrivateKey: <base64-ed25519-private>   # the half you keep
            signingPublicKey:  <base64-ed25519-public>    # the half you published in DeDi
```

DeDi value	ONIX YAML path
Your `subscriber_id`	`plugins.keyManager.config.networkParticipant`
Your `record_id`	`plugins.keyManager.config.keyId`
Your Ed25519 **private** key	`plugins.keyManager.config.signingPrivateKey` *(Base64 RFC 4648, 32-byte seed)*
Your Ed25519 **public** key	`plugins.keyManager.config.signingPublicKey` *(matches what you published in DeDi)*
Network you're joining	`plugins.registry.config.allowedNetworkIDs` *(comma-separated string)*

Where these values come from — DeDi namespace claim, subscriber record publication, the keypair you generate with `tools/sign`, and the IES NFO handoff — is covered in Identifiers — Appendix E and Registries — How to apply for an IES listing. Stay on the **test network** until certified; the recommended prod pattern is two ONIX deployments below.

After the config swap, re-import the same Postman collection — only the identity ONIX signs with and the network it claims change. Use the same network ID in every payload's `context.networkId`: ONIX rejects messages whose `networkId` isn't in its `allowedNetworkIDs`.

8.7 The IES networks

IES operates two parallel Beckn networks anchored at the `indiaenergystack.in` namespace:

Network	`networkId`	Purpose
Test	`indiaenergystack.in/test-ies-data-sharing-network`	Pre-production exchange — used during onboarding, certification, and ongoing test / staging after a participant goes live in prod.
Production	`indiaenergystack.in/ies-data-sharing-network`	Live exchange with real DISCOMs, regulators, AMISPs.

Each network is its own DeDi registry under the `indiaenergystack.in` namespace. Membership in one does **not** imply membership in the other — every participant must be referenced from each network's registry separately. ONIX enforces this on every inbound message via `allowedNetworkIDs`. The full inventory of registries under the namespace (including the inter-DISCOM data-sharing, P2P trading, and DER integration networks) is in Registries — IES networks today.

8.8 Pagination — large datasets across multiple `status` / `on_status` messages

When a dataset is too large to fit in a single `on_confirm` (e.g. a quarter of AMI interval reads for a 500-meter feeder), the BAP and BPP exchange the payload across multiple `on_status` messages — one **page** per message. Two complementary patterns are in active use in the devkit:

8.8.1 BAP-PULL — the BAP drives the cadence

1. **First page.** The BAP sends `status` **without** a `pageCursor` (or omits the `pagination` tag entirely). The BPP treats a missing cursor as page 0.
2. **Subsequent pages.** The BPP's `on_status` carries `pageInfo.nextCursor` (and `pageInfo.isLast: false`). The BAP issues a new `status` with that cursor.
3. **Done.** The BAP keeps pulling until `pageInfo.nextCursor` is null **and** `pageInfo.isLast` is `true`. It then assembles all `dataPayload` slices and runs downstream processing.

Where the cursor lives on the request. Beckn v2 `Contract` is a closed schema and cannot host tags directly; `context.tags` is reserved for routing / transaction state. The pagination cursor therefore rides on `message.tags`:

```
{
  "context": {"action": "status", "transactionId": "...", "messageId": "...", "timestamp": "..."},
  "message": {
    "tags": [{
      "descriptor": {"code": "pagination"},
      "list": [
        {"descriptor": {"code": "pageCursor"},   "value": "ami-meter-blr-zone-a-q1-p2"},
        {"descriptor": {"code": "collectionId"}, "value": "ami-meter-blr-zone-a-q1-2026"}
      ]
    }],
    "contract": { "id": "<contract-id>" }
  }
}
```

Where `pageInfo` lives on the response. The BPP's `on_status` carries the slice in `message.contract.performance[].performa` alongside a `pageInfo` block:

```
"performanceAttributes": {
  "@context":  "https://raw.githubusercontent.com/beckn/DDM/main/specification/schema/DatasetItem/v1/context.jsonl
  "@type":     "DatasetItem",
  "dataPayload": [ /* this page's slice of CustomerProfile / IntervalProfile entries */ ],
  "pageInfo": {
    "@type":        "PageInfo",
    "sequence":     1,
    "pageSize":     167,
    "total":        500,
    "isLast":       false,
    "cursor":       "ami-meter-blr-zone-a-q1-p2",
    "nextCursor":   "ami-meter-blr-zone-a-q1-p3",
    "collectionId": "ami-meter-blr-zone-a-q1-2026"
  }
}
```

The cursor is **opaque** to the BAP — it's the BPP's internal identifier for the next slice. Echo it back unchanged.

8.8.2 BPP-PUSH — the BPP drives the cadence

1. **The BPP fires several back-to-back `on_status` messages**, each with its own slice of `dataPayload` and a `pageInfo` carrying `sequence`, `isLast`, and `collectionId`. The BAP accumulates entries across all messages with the same `(transactionId, collectionId)`.
2. **Done.** The BAP only triggers downstream processing once it sees `pageInfo.isLast: true`.

`pageInfo.cursor` / `nextCursor` are unused in push mode (the BAP isn't requesting anything); `sequence` and `isLast` are what matter.

8.8.3 Worked examples in the devkit

The on-the-wire payloads (with sample interval reads and the full `pageInfo` block) live next to each use case:

- BAP-PULL: `uc1-meter-data/.../status-request-paged-pull.json` + matching `on-status-response-paged-pull.json`.
- BPP-PUSH: `on-status-response-paged-push-p1.json`, `…-p2.json`, `…-p3.json` (with `isLast: true` on the last one).

Both patterns share the same `transactionId` across every message in the exchange — see Appendix A — Context invariants.

8.9 Optional Beckn actions

Beyond the minimal `confirm` -> `on_confirm`, fire these only when your use case needs them:

Action	Sent by	When to use
`publish-catalog`	BPP	You're a provider listing datasets for discovery
`discover` / `on_discover`	BAP	The consumer doesn't yet know which BPP holds the dataset

Action	Sent by	When to use
`select` / `init` (+ `on_*`)	BAP	Terms (price, SLA, delivery mode) need agreeing before commitment
`status` / `on_status`	BAP / BPP	Payload prepared asynchronously *after* `on_confirm`; also the carrier for paged delivery (see Pagination)
`update` / `on_update`	BAP / BPP	Post-fulfilment changes — credential rotation, contract amendments

Each use-case page lists which actions it actually exercises.

8.10 Two registries, two ONIX deployments

The recommended production pattern — keep test and prod cleanly separated:

1. **Run two ONIX deployments — test and prod — on separate hostnames** (e.g. `test.example-np.com` and `beckn.example-np.com`). Each has its own TLS cert, its own keypair, its own DeDi subscriber record under your namespace, in two separate subscriber registries so they cannot be confused.
2. **`allowedNetworkIDs` is set narrowly on each side.** Test ONIX accepts only `indiaenergystack.in/test-ies-data-sharing-network`; prod ONIX accepts only `indiaenergystack.in/ies-data-sharing-network`. Cross-network traffic is rejected at the adapter — no stray test message can reach a prod counterparty.
3. **Phased onboarding:** start in test -> certification -> IES references your *prod* subscriber DeDi URL into the prod network -> from that moment your prod ONIX is allowed to transact with production NPs. Test stays useful for staging upgrades and certifying new use cases.

To run two devkit stacks side-by-side on the same host: copy `install/` to `install-prod/`, edit the published ports (`8081->8181`, `8082->8182`, `9000->9100`), adjust the `Caddyfile` listener, and use separate compose project names:

```
docker compose -p ies-test -f install/docker-compose.yml      up -d
docker compose -p ies-prod -f install-prod/docker-compose.yml up -d
```

In real production each stack would typically run on a separate host or VM with its own TLS termination. See devkit README — Hosting the site for the patterns.

8.11 Make the sandbox your own

When you outgrow the sandbox, your app connects to ONIX in exactly two places — `bapUri` / `bppUri` are *not* one of them (those always point at ONIX receivers):

- **Inbound** — after verifying a message, ONIX forwards it to the **webhook URL set in the routing config** (`config/local-simple-routing-BAPReceiver.yaml` / `…-BPPReceiver.yaml`). Take over that URL and you've replaced the sandbox.
- **Outbound** — your app POSTs messages to ONIX's `caller` endpoint.

BAP. Stand up a service with a webhook endpoint for the `on_*` callbacks your flow needs, reachable from `onix-bap`. Change `target.url` in `local-simple-routing-BAPReceiver.yaml` from `http://sandbox-bap:3001/api/bap-webhook` to your service, restart `onix-bap`, then stop `sandbox-bap`.

BPP. Your provider receives requests (`confirm`, `status`, …) on the webhook set in `local-simple-routing-BPPReceiver.yaml` (devkit default: `http://sandbox-bpp:3002/api/webhook`). Acknowledge each with the ACK envelope, then respond asynchronously by POSTing the matching callback to `http://onix-bpp:8082/bpp/caller/on_<action>` — copy the wire shape from `uc*/examples/on-*-response*.json` and swap in your real data.

You can also build the bundled beckn/sandbox image yourself, modify the canned fixtures and handlers in `src/`, and use `sandbox-2.0:local` as a stepping stone before swapping in your own service.

8.12 What you can exchange (schema families)

The wire envelope accepts arbitrary JSON inside `dataPayload`; **validation is opt-in per object**, driven by JSON-LD self-description (see Appendix C — Schema validation). The IES schema families maintained in this repo:

Family	What it carries	Use case
MeterData	Smart meter telemetry — `IntervalProfile` (15-minute reads), `DailyProfile`, `BillingProfile`, etc.	Smart Meter Data Exchange
MeterDataCredential	W3C Verifiable Credential wrapping `MeterData` for provenance attestation	Smart Meter Data Exchange (credentialed delivery)
ArrFiling	Aggregate Revenue Requirement line items by fiscal year	DISCOM Regulatory Filing
`IES_Policy` (+ `IES_Program`)	Machine-readable tariff rate structures (energy slabs, ToD surcharges)	Tariff Intelligence
MeterDataRequest / MeterDataRequestCredential	Query / authorisation shape and its credentialed wrapper	Meter-data request flows

Any JSON payload can be exchanged; the families above are simply the ones IES ships pre-built and validated.

8.13 Checklist

Devkit sandbox to production go-live, staged. Role: [ ] BAP (consumer) [ ] BPP (provider) [ ] Both.

8.13.1 Stage 0 — Scope and team

- [ ] Datasets identified (meter telemetry, ARR filings, tariff policies, …) -> What you can exchange
- [ ] Source system mapped per dataset (MDMS / billing / regulatory filing)
- [ ] IT/data SPOC and an authorised signatory nominated

8.13.2 Stage 1 — Devkit validation (local sandbox)

Prove the wiring and your payload handling — no real identity, no real network.

- [ ] Clone and start the stack; all 7 containers healthy
- [ ] Postman collection imported for your role + use case

- [ ] `confirm` sent (ACK) and `on_confirm` landed in `sandbox-bap`/`sandbox-bpp`
- [ ] (BAP) your app consumes the `dataPayload` (MeterData / ArrFiling / tariff); (BPP) your provider emits `on_confirm` (or `on_status`)

8.13.3 Stage 2 — Network identity (DeDi)

- [ ] DeDi namespace claimed and domain-verified -> Registries — Step-by-step
- [ ] Ed25519 signing keypair generated -> Identifiers — Step 2
- [ ] Subscriber record published (`subscriber_id`, `subscriber_url`, `type`, `signing_public_key`, `countries`) and lookup resolves
- [ ] Noted: `subscriber_id`, `record_id` (= ONIX `keyId`), keypair

8.13.4 Stage 3 — Test-network certification

- [ ] ONIX config swapped to your real identity -> Swap in your real identity
- [ ] `allowedNetworkIDs` = `indiaenergystack.in/test-ies-data-sharing-network`
- [ ] IES Secretariat emailed your DeDi lookup URL -> Registries — How to apply; subscriber confirmed under the test network
- [ ] End-to-end `confirm` -> `on_confirm` with another participant; pagination tested if applicable

8.13.5 Stage 4 — Production deployment

- [ ] Second ONIX on the production hostname (TLS, separate keypair + DeDi record) -> Two registries, two ONIX deployments
- [ ] Prod `allowedNetworkIDs` = `indiaenergystack.in/ies-data-sharing-network`; Secretariat re-applied for the prod reference
- [ ] First prod exchange completed; test ONIX kept for staging/certification

8.13.6 Stage 5 — Operational hardening

- [ ] Reverse proxy + TLS in front of ONIX (never expose `:8081`/`:8082`)
- [ ] Signing key in a secret manager; key-rotation runbook in place
- [ ] Schema validation enabled -> Appendix C; monitoring on ONIX health, signature failures, registry lookups
- [ ] On-call rota for BAP/BPP webhooks; your own app has replaced `sandbox-bap`/`sandbox-bpp` -> Make the sandbox your own

---

Chapter 9

Appendices

The sections below are theory and reference. Skip if you're mid-deployment.

9.1 Appendix A — Beckn protocol lifecycle

IES Data Exchange uses the Beckn Protocol v2.0 interaction model:

Phase	BAP calls	BPP responds	When you need it
Discovery	`discover`	`on_discover`	Consumer doesn't yet know which provider to contract with. Requires the BPP to have called `publish-catalog` earlier.
Negotiation	`select` / `init`	`on_select` / `on_init`	Terms (price, SLA, delivery mode) need to be agreed before commitment.
Commitment	`confirm`	`on_confirm`	**The minimal flow.** Payload can be delivered inline here.
Polled / paged delivery	`status`	`on_status`	Payload prepared asynchronously after `on_confirm`; also the carrier for pagination.
Post-fulfilment	`update`	`on_update`	Credential rotation, contract amendments.

Minimal viable exchange: `confirm` -> `on_confirm`, with the dataset embedded in the callback. Everything else is optional.

Beckn is **asynchronous**: every call is answered by an `ACK` — produced by the receiving application's webhook and cascaded back hop-by-hop as each HTTP response, ending as the synchronous reply the original caller sees — and the paired response (`on_*`) arrives later as a callback. ONIX manages this transparently.

9.1.1 Message structure

Every Beckn message shares the same envelope (v2.0 wire format, camelCase):

```
{
  "context": {
    "networkId": "indiaenergystack.in/test-ies-data-sharing-network",
    "version": "2.0.0",
    "action": "confirm",
    "bapId": "bap.example.com",
    "bapUri": "https://bap.example.com/bap/receiver",
    "bppId": "bpp.example.com",
    "bppUri": "https://bpp.example.com/bpp/receiver",
    "transactionId": "b2c3d4e5-f6a7-8901-bcde-f12345678901",
    "messageId": "1e2f3a4b-5c6d-7890-4567-890123456789",
    "timestamp": "2026-04-01T09:25:00Z",
    "schemaContext": ["https://raw.githubusercontent.com/beckn/DDM/main/specification/schema/DatasetItem/v1.1/conte
  },
  "message": { /* action-specific payload */ }
}
```

9.1.2 Context invariants

Two correlation rules from the Beckn v2.0 spec — every implementation honours them so participants and audit trails can stitch related messages together:

- **`transactionId` is constant** across every message in one exchange. From the first `discover` / `select` / `confirm` to the final `on_status` / `on_update`, the same UUID flows through.
- **`messageId` is shared** by a request and its paired `on_*` callback; each new request gets a new one. Treat the pair as one logical message with two hops.

Authoritative reference: beckn/protocol-specifications-v2 — `api/v2.0.0`.

9.1.3 `DatasetItem` and `accessMethod`

The resource a data exchange agrees on is a **dataset**. **`DatasetItem`** (from DDM, Beckn's Decentralized Data Marketplace schema family) is the per-dataset record shape that rides inside `message.contract.commitments[].resources[]`, qualified by `resourceAttributes`:

```
"resources": [{
  "id": "ds-ami-meter-data-blr-zone-a-q1-2026",
  "descriptor": { "name": "IntelliGrid AMI Meter Data — Bengaluru Zone A — Q1 2026" },
  "resourceAttributes": {
    "@context": "https://raw.githubusercontent.com/beckn/DDM/main/specification/schema/DatasetItem/v1.1/context.js
    "@type": "DatasetItem",
    "accessMethod": "INLINE",
    "dataPayload": { /* inline MeterData, ArrFiling, ... */ }
  }
}]
```

`accessMethod` values:

Mode	Description
`INLINE`	Embedded in the Beckn callback message (typically `on_confirm` or paged `on_status`)
`DOWNLOAD`	BPP provides a signed download URL + checksum
`MQTT` / `KAFKA` / `API` / `DATA_LAKE`	Streaming transports — connection credentials in `on_confirm.performance[].performanceAttributes`
`DATA_ENCLAVE`	Data accessible only within a secure compute environment

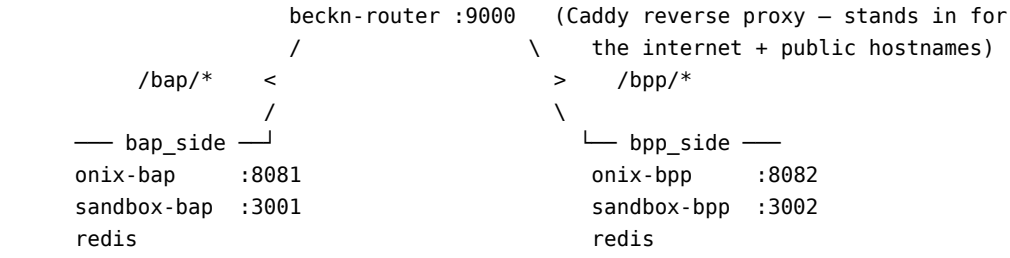
Mode	Description
`OFF_CHANNEL`	Delivery through an agreed out-of-band channel

IES Data Exchange today primarily uses `INLINE` (often paged via the pagination protocol) — data arrives as part of the protocol message, making the exchange end-to-end verifiable.

9.2 Appendix B — Architecture at a glance

Your application never speaks Beckn directly — it talks to **ONIX**, which exposes a `caller` endpoint (where your app hands it outbound messages to sign and dispatch) and a `receiver` endpoint (where the network delivers inbound messages, which ONIX verifies and forwards to your app's webhook). One caller / receiver pair per role — `/bap/caller`, `/bap/receiver`, `/bpp/caller`, `/bpp/receiver` — and one ONIX deployment can host any combination under one identity. **Roles are configuration, not separate software.**

The devkit simulates **two participants** (`bap.example.com` and `bpp.example.com`), so it runs two ONIX containers on mutually isolated docker networks:



Component	Role
`onix-bap` / `onix-bpp`	Beckn protocol adapter — signs, verifies, validates `dataPayload`, dispatches, forwards inbound to the app webhook
`sandbox-bap`	Consumer-side stand-in app — receives `on_*` callbacks on its webhook and logs them
`sandbox-bpp`	Provider-side stand-in app — receives requests on its webhook and auto-responds with example payloads via `/bpp/caller`
`beckn-router`	Plain Caddy reverse proxy — routes by path (`/bap/*` -> `onix-bap`, `/bpp/*` -> `onix-bpp`) and carries the dummy participant hostnames as docker aliases. **Not a Beckn entity** — deployment plumbing
`redis`	Per-side message cache used by ONIX

The hostnames in `bapUri` / `bppUri` are participant identities: dummy docker aliases locally, your real public URL (DeDi-published) in production.

9.2.1 The four ONIX endpoints

Endpoint	Role	Direction	Who calls it
`/bap/caller`	BAP	outbound	**Your consumer app** hands ONIX a request (`confirm`, `discover`, …) to sign and dispatch
`/bap/receiver`	BAP	inbound	**The network** (the counterparty's ONIX) delivers `on_*` callbacks here; ONIX verifies and forwards them to your app's webhook
`/bpp/caller`	BPP	outbound	**Your provider app** hands ONIX an `on_*` response to sign and dispatch
`/bpp/receiver`	BPP	inbound	**The network** delivers requests (`confirm`, `status`, …) here; ONIX verifies and forwards them to your provider's webhook

What happens after a `receiver` verifies a message is defined in the routing configs (`local-simple-routing-{BAPReceiver,BPPReceiver,BAPCaller,BPPCaller}.yaml`). The receiver routing config controls the webhook URL inbound messages are forwarded to — that's where you wire in your own app.

9.2.2 Identity resolution

When a message arrives, ONIX:

1. Reads the sender from the message context (`bapId` on a forward request, `bppId` on a callback).
2. Looks the sender up in the **DeDi registry**: `GET https://fabric.nfh.global/registry/dedi/lookup/<subscriber_id>/sub`
   The response carries the sender's callback URL, signing public key, and network memberships.
3. Cross-checks those memberships against the local `allowedNetworkIDs` config. A sender outside the boundary of trust is rejected.
4. Verifies the signature.

Two participants on different (or non-overlapping) networks cannot reach each other through their ONIX adapters. The DeDi -> ONIX field mapping (`subscriber_id` -> `networkParticipant`, `record_id` -> `keyId`, Ed25519 keypair) is in Swap in your real identity.

9.3 Appendix C — Schema validation

The wire envelope accepts arbitrary JSON inside `dataPayload`; validation is **opt-in per object**, driven by JSON-LD self-description:

- **Payload declares `@context` + `@type` -> ONIX validates it.** ONIX fetches the OpenAPI 3.x **`attributes.yaml` that sits next to the `context.jsonld`** named in `@context` (same URL, different filename — every IES / Beckn schema family publishes the pair side by side), and validates the object against the `components.schemas` entry whose name matches `@type` exactly (`DatasetItem`, `MeterData`, `ArrFiling`, …). A failed validation rejects the message.
- **Payload omits `@context` -> ONIX passes it through.** Useful for prototyping a new dataset shape.

ONIX only fetches `@context` URLs from **allow-listed hosts** (default: `raw.githubusercontent.com`, `schema.beckn.io`). To validate payloads from your own schema repo, add the host to `extendedSchema` in your ONIX config.

9.3.1 Failure modes

- `no schema found for @type: XYZ` — the `attributes.yaml` doesn't define a schema with that name. Either `@type` is wrong, or the schema file at the resolved URL doesn't include it.
- Schema validation error — the object is missing a required field, has the wrong type, or otherwise doesn't match the OpenAPI definition. ONIX includes the JSON path of the failure in the error.

9.3.2 Publishing your own schema for ONIX to validate

1. Author an OpenAPI 3.x `attributes.yaml` with your schemas in `components.schemas`.
2. Author a JSON-LD `context.jsonld` mapping your field names to URIs.
3. Host both at a URL ONIX is allowed to reach — they must sit at the same path (`<base>/context.jsonld` and `<base>/attributes.yaml`).
4. In your payload, set `@context` to the context URL and `@type` to the matching schema name.

No code change in ONIX — the dispatch is purely URL-driven. The families under schemas/ and beckn/DDM are working references for how to lay out the pair.

9.4 Appendix D — Hostnames sandbox vs production

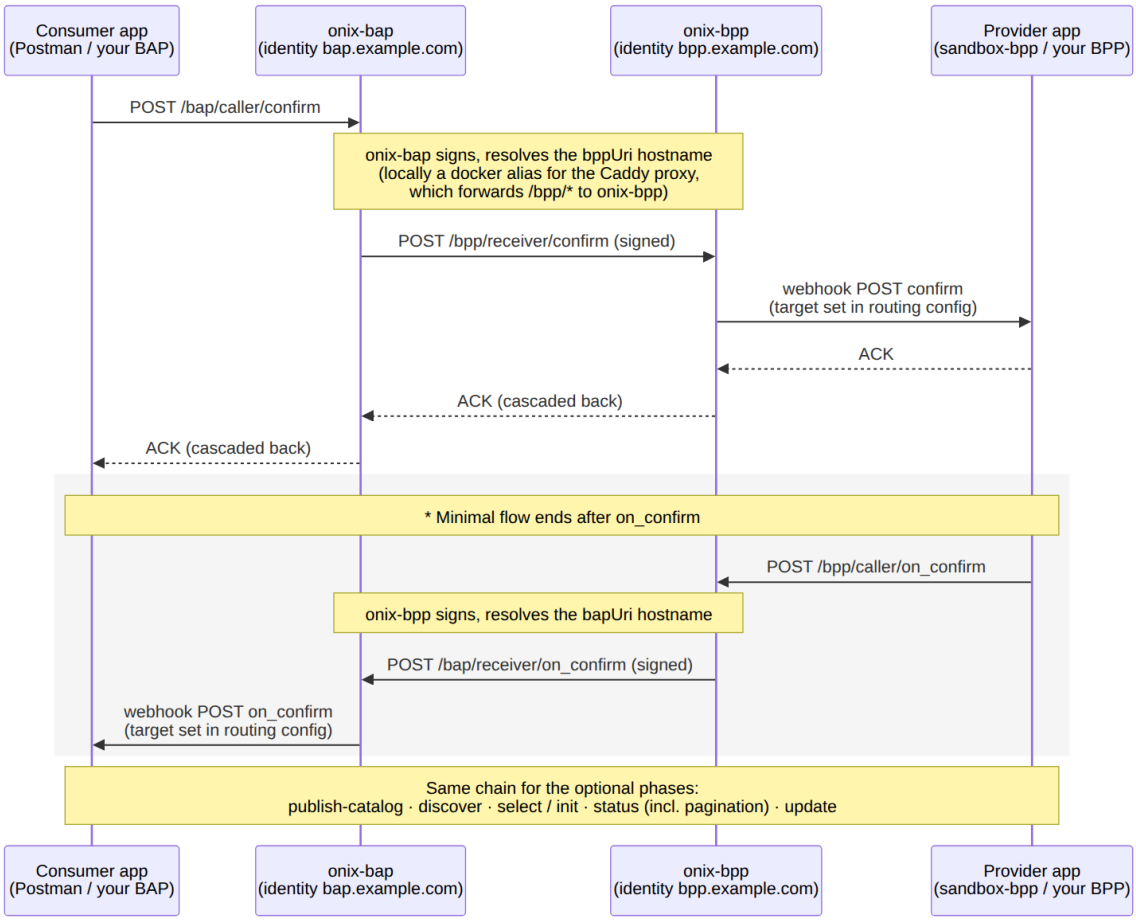
The hostnames in `bapUri` / `bppUri` represent **participant identities**. In production they are your real public URLs, published in your DeDi subscriber record. In the devkit they are **dummy aliases that only resolve inside the docker network**: compose attaches `bap.example.com` and `bpp.example.com` as aliases of `beckn-router`, so dispatch lands on the Caddy proxy, which path-routes to the right ONIX `receiver`.

That gives two kinds of URL at different layers:

Concern	Devkit sandbox	Real network
HTTP target — where your client (Postman, your app) POSTs `confirm`, `discover`, etc.	`http://localhost:8081/bap/caller` *(port-mapped to `onix-bap`)*	Your ONIX deployment URL behind TLS
HTTP target — where your provider POSTs `on_confirm` / `on_status`	`http://localhost:8082/bpp/caller` *(port-mapped to `onix-bpp`)*	Your ONIX deployment URL behind TLS
Payload hostnames — `bapUri` / `bppUri` in each message's `context`; resolved by the *sending* ONIX to reach the counterparty's `receiver`	`http://beckn-router:9000/{bap,bpp}/receiver` (or the docker aliases — both resolve to the proxy)	Your public callback URLs (matching what you published in your DeDi subscriber record)
`networkId`, `bapId` / `bppId`, `allowedNetworkIDs`	placeholder values shipped in `config/`	See Swap in your real identity

Your Postman client never connects to `beckn-router` — it POSTs to the port-mapped `caller` endpoints (`:8081` / `:8082`); the payload variables substitute the docker-resolvable hostnames into the message body for ONIX to use.

9.5 Appendix E — Generic Beckn flow (sequence diagram)



`beckn-router` is intentionally not a participant in this diagram — it's a path-only Caddy reverse proxy, not a Beckn entity. In production it's replaced by your own TLS-terminating reverse proxy.

9.6 References

- DEG Data Exchange devkit — the source for the walkthrough above (Postman collections, fixtures, configs)
- Beckn ONIX — the protocol adapter; `tools/sign` for the Ed25519 keypair
- Beckn Protocol v2.0.0 spec
- Identifiers and Addressing — `did:web`, subscriber records, holder binding
- Registries and Directories — DeDi namespace, network registries, IES Secretariat application
- Energy Credentials — credentialed payloads (MeterDataCredential, ArrFilingCredential, …)
- Use cases — end-to-end flows that exercise this protocol

Chapter 10

Use Cases

This section documents the **flagship end-to-end use cases** that the India Energy Stack (IES) building blocks come together to deliver. Each guide is self-contained: it names the actors, identifies which building blocks are used, and walks an implementer through the steps to stand the use case up — from identifier issuance to a working data or credential exchange.

If the Identifiers, Registries, Energy Credentials, and Data Exchange chapters describe **the building blocks**, this section describes **what you can build with them**.

10.1 The Six Use Cases

#	Use Case	What it does	Primary actors	Building blocks
1	Smart Meter Data Exchange (data model)	Standard, audit-trailed exchange of `MeterData` telemetry between AMISP, DISCOM, regulator, and consented third parties over IES Data Exchange.	AMISP, DISCOM, SERC	Identifiers · Registries · Data Exchange · Energy Credentials (optional consent)
2	Consumer Meter Digest (checklist)	Consumer pulls their own granular meter readings on demand as a signed credential and shares them with a third party of their choice.	Consumer, DISCOM, Third-party app	Identifiers · Registries · Energy Credentials · Data Exchange

#	Use Case	What it does	Primary actors	Building blocks
3	Consumer Energy Passport (checklist)	Wallet-held credential binding consumer identity to their connection, meter, sanctioned load, and DER/storage assets — issued by the DISCOM, verifiable everywhere.	Consumer, DISCOM, Verifier (bank, marketplace, society)	Identifiers · Registries · Energy Credentials
4	DISCOM Regulatory Filing (checklist)	Aggregate Revenue Requirement (ARR) and compliance filings delivered as structured, signed, machine-verifiable objects from DISCOM to SERC.	DISCOM, SERC	Identifiers · Registries · Data Exchange
5	Tariff Intelligence (checklist)	Tariff orders (slab billing, time-of-day, deviation penalties) and data-exchange rules published as policy-as-code that billing systems, apps, and meters can consume directly.	SERC, DISCOM, App developer	Identifiers · Registries · Energy Credentials · Data Exchange
6	Energy Trading (checklist) — **WIP**	Inter-discom prosumer-to-prosumer energy trade carried as a signed contract over IES Data Exchange. Variant of Data Exchange: same wire, the payload is `DEGContract` + `EnergyTradeDelivery` instead of `DatasetItem`. Network rules and revenue flows enforced by signed Rego bundles hosted on DeDi.	Prosumer, Trading Platform (BAP/BPP), Ledger Provider, DISCOM	Identifiers · Registries · Data Exchange · Energy Credentials

DER Visibility — tracking rooftop solar, batteries and EV chargers per feeder using IES identifiers — is a seventh flagship use case and is documented separately (out of scope for this section for now).

10.2 How to Read These Guides

Each use case page follows the same structure so you can navigate quickly:

1. **Scenario** — the real-world problem this solves
2. **Actors and roles** — who is the BAP / BPP / Issuer / Holder / Verifier
3. **Building blocks used** — which IES chapters you'll touch and why
4. **Schemas and payloads** — the data objects flowing through the use case
5. **Setup steps** — ordered checklist from cold start to working exchange
6. **Operate** — how to run, observe, and verify the use case end-to-end
7. **Open items** — what is still being finalised upstream
8. **References** — canonical schemas, devkit examples, ies-docs links

---

10.3 Maturity Snapshot (as of 2026-06)

Component	Status
Identifier patterns and DeDi registries	Stable
Energy credential schemas — `ElectricityCredential v1.2`	Stable
Energy credential variant — Consumer Energy Passport (issuance pattern over `ElectricityCredential/v1.2`)	See Consumer Energy Passport use case
Energy credential variant — Consumer Meter Digest (issuance pattern over `MeterDataCredential/v0.6`)	See Consumer Meter Digest use case
Data exchange protocol (Beckn + DDM envelope)	Stable; shipped in devkit
`MeterData v0.6` (meter telemetry) schema	Current canonical — supersedes the earlier `IES_Report` working name
`MeterDataRequest v0.6` + `MeterDataRequestCredential v0.1`	Current
`ArrFiling` schema	**Draft — being finalised**
Tariff policy schema	Stable
Tariff publication via data exchange	Shipped in devkit
Energy-trading schemas (`P2PTrade`, `DEGContract`, `EnergyTradeOffer`, `EnergyTradeDelivery`, `DiscomLedgerProvider`, `BecknTimeSeries`)	Stable in DEG `p2p-trading-ies-wave2` devkit
Energy-trading Rego bundle — network rules	Stable; carried by `policyUrl` on the contract
Energy-trading Rego bundle — settlement / revenue flows	**Draft — wheeling and penalty placeholders pending**

The open items (`ArrFiling`, the wheeling / penalty rules in the energy-trading settlement bundle, and the auto-`revenueflows` middleware in the wave-2 ONIX config) do not block implementation — the example payloads in the devkit are stable enough to integrate against, and the rego bundles can be updated independently of code via DeDi.

10.4 Smart Meter Data Exchange

A standard, audit-trailed way to exchange smart-meter telemetry between an AMISP, a DISCOM, a state regulator, and consented third parties — over IES Data Exchange, carrying the MeterData payload (currently `v0.6`).

You replace bespoke FTP drops, proprietary MDMS APIs, and CSV email attachments with one signed, schema-validated flow. The same surface area works AMISP->DISCOM, DISCOM->SERC, and DISCOM->consented-third-party.

> For consumer-pull of *their own* meter data (one-meter, on-demand, wallet-shareable), see Consumer Meter Digest. This guide is the bulk, scheduled, machine-to-machine flow.

10.4.1 Building blocks used

This use case is a thin composition over four existing building blocks. The setup steps below map one-to-one to them — read the building-block pages for the detail; come back here for what changes for the meter-data case.

What you need	Building block	What it gives you
A name and a signing key your counterparties can verify	Identifiers & Addressing, Registries	A DeDi namespace under your control and your current public key published into it. A formal `did:web` document is **not** required for this use case; the namespace + key are enough for Beckn signing and verification.
A trust boundary — "who can transact on this network"	Registries — IES networks today	Your subscriber record referenced in the IES network registry
The transport that carries discovery, contract, audit, and the payload	Data Exchange	The Beckn lifecycle + ONIX adapter; the same `accessMethod` covers inline payloads and signed-URL handoff
(Optional) Cryptographic proof that the requester is authorised	MeterDataRequestCredential	A signed credential the seeker attaches at `confirm`; provider verifies offline

---

10.4.2 The dataset — `MeterData/v0.6`

The payload is the MeterData compact-profile schema (currently `v0.6`). v0.6 standardises eight profile shapes covering every smart-meter cadence:

Profile	Carries
`CUSTOMER`	Slow-changing customer / service-point / meter installation metadata
`INTERVAL`	Block load survey at 15- or 30-minute resolution
`DAILY`	Daily accumulated load survey
`MONTHLY`	Monthly billing resets (cumulative kWh, ToU buckets, MD)
`BILL_DETAILS`	Utility-side billing computed details

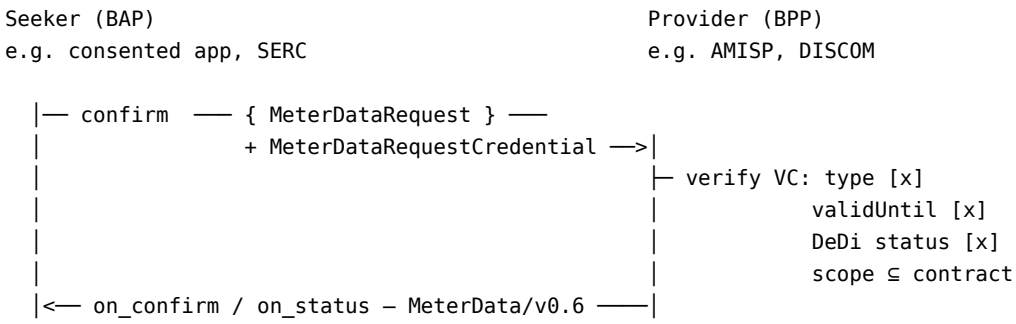
Profile	Carries
`INSTANTANEOUS`	Real-time snapshot of V / A / P / Q
`EVENT`	IS 15959 diagnostic / tamper events
`ALARM`	Real-time active alerts

For the Indian-terminology mapping (OBIS codes, IS 15959 event IDs, CIM master-data fields) see the companion page IES Meter Data Model.

> **MeterData v0.6 supersedes the earlier `IES_Report` working name.** Any older docs or devkits referencing `IES_Report` should be read as `MeterData/v0.6`.

10.4.3 Optional consent — `MeterDataRequestCredential`

When a third party (a consented app, a researcher, sometimes a regulator) is the BAP, the provider's offer policy can require an authorisation credential at `confirm` time. IES uses `MeterDataRequestCredential` — a W3C VC 2.0 that wraps a `MeterDataRequest` and identifies the requester, scopes the request (which meters, which time window, which profile), and is signed by the authoriser (typically the DISCOM, sometimes the consumer themselves via OpenCred).



The provider checks credential type, expiry, DeDi revocation status, and that the embedded request scope is within the contracted scope — no callback to the issuer.

10.4.4 Setup steps

The order below takes you from zero to a working AMISP -> DISCOM flow on the local devkit, then upgrades to the real IES network.

1. Decide scope

Meter population, geography, granularity (15-min / daily / monthly), counterparties (AMISP->DISCOM only, DISCOM->SERC, DISCOM->third-party). Phase 1 should be one cadence and one counterparty.

2. Get a network identity

Both BAP and BPP need a DeDi subscriber record with a published signing key. If you have one already (any other Beckn flow on this network), reuse it. If not, follow Data Exchange — Checklist -> Stage 2 (Network identity).

You do **not** need to rename anything in your CIS, MDMS, or asset registers. Your existing IDs are reused as the tail of the DID.

3. Stand up the Data Exchange adapters

Both sides run an ONIX adapter. The fastest start is the devkit sandbox:

```
git clone https://github.com/beckn/DEG.git
cd DEG/devkits/data-exchange/install
docker compose up -d
```

This brings up `sandbox-bap`, `sandbox-bpp`, and `beckn-router` pre-wired with placeholder identities. End-to-end network onboarding is the Data Exchange — Checklist; don't duplicate the steps here — read that page.

4. Publish your dataset catalogue (BPP)

Publish a Beckn catalogue entry for the `MeterData/v0.6` dataset you offer — `programID`, geographic scope, refresh cadence, `accessMethod` (`INLINE` for <=MB-scale chunks; `SIGNED_URL` for daily/monthly bulk), and any required credentials (point at `MeterDataRequestCredential` if you require one). Pre-agreed bilateral subscriptions can skip `discover` and go straight to `confirm`.

5. Exercise the flow

The minimal lifecycle is `confirm` -> `on_confirm` -> (`status` -> `on_status` if delivery is asynchronous):

Step	Sender	What happens
`confirm`	BAP	Requests data for a meter list + date range. Optionally attaches a `MeterDataRequestCredential`.
`on_confirm`	BPP	Acknowledges; declares whether delivery is inline or async.
`on_status`	BPP	Delivers `MeterData/v0.6` inline or returns a signed-URL pointer.

The payload sits inside `message.contract.commitments[].resources[].resourceAttributes` qualified with the DDM `DatasetItem/v1.1` context — see Data Exchange — Core Concepts.

6. Connect your real metering system

Replace the sandbox response with a live connection to your HES / MDMS. Verify the Indian-OBIS -> `MeterData/v0.6` mapping for every reading you ship — IES Meter Data Model is the reference.

7. (Optional, at the end) Adopt the `did:web` convention for meters and assets

There are no new IDs to allocate. Your existing meter SLNOs, DT codes, feeder codes, and substation codes are the IDs of record and stay exactly as they are. The IES convention is a `did:web` wrapper format that reuses those existing IDs so they can be referenced across the network and used as the subject of Verifiable Credentials:

```
did:web:<dedi-host>:<your-namespace>:meters:<existing-meter-serial>
did:web:<dedi-host>:<your-namespace>:dts:<existing-DT-code>
did:web:<dedi-host>:<your-namespace>:feeders:<existing-feeder-code>
did:web:<dedi-host>:<your-namespace>:substations:<existing-substation-code>
```

`<dedi-host>` is the host of any DeDi runtime that publishes the DID document (e.g. `dedi.global` or a self-hosted DeDi-compatible service). The DID method is always `did:web`; DeDi just acts as a discovery layer for the document — see Glossary -> DeDi and Identifiers and Addressing -> Appendix C. Once an asset has a `did:web` form, OpenCred can issue VCs about it (commissioning, inspection, ownership). This step is

nice to have, not a blocker for first deployment — initial flows can use bare IDs inside `MeterData` payloads and adopt the `did:web` form incrementally.

10.4.5 Checklist for your meter-data rollout

For the **underlying network onboarding** (DeDi subscriber, ONIX, signing keys, sandbox->test->prod cut-over), follow the canonical Data Exchange — Checklist. Don't duplicate those items here.

The items below are **meter-data-specific** additions on top of that checklist:

- [ ] Meter population, geography, granularity, and counterparty for Phase 1 agreed
- [ ] Source system mapped (HES / MDMS endpoint, owner team, SLA)
- [ ] Indian-OBIS -> `MeterData/v0.6` mapping verified for every reading and event you ship (see IES Meter Data Model)
- [ ] Data-quality flags (missing / estimated) handled per the v0.6 `validationStatus` enum
- [ ] Catalogue entry published (`programID`, geographic scope, refresh cadence, `accessMethod`, credential requirements)
- [ ] (If third-party access in scope) `MeterDataRequestCredential` verification wired into the BPP — type, validity, DeDi status, scope contract
- [ ] (Optional) `did:web` convention adopted for meters / DTs / feeders — wrapping (not replacing) existing internal numbers, so VCs can be issued about them
- [ ] IT/data SPOC nominated; Authorised Signatory nominated

10.4.6 Open items

- **Chunking convention** for bulk historical pulls (daily / monthly / quarterly) is being agreed across deployments.
- **Quality flags** — the v0.6 `validationStatus` enum is stable; the recommended profile of allowed values per cadence is being tightened.

10.4.7 References

- MeterData — schema and examples (current: `v0.6`)
- MeterDataRequest — request payload schema (current: `v0.6`)
- MeterDataRequestCredential — authorisation VC (current: `v0.1`)
- IES Meter Data Model — OBIS / IS 15959 / CIM -> MeterData v0.6 mapping
- Data Exchange chapter
- Data Exchange — Checklist
- Registries and Directories
- Identifiers and Addressing

10.4.8 IES Meter Data Model

The reference for how Indian smart-meter terminology — OBIS codes, IS 15959 profile buffers, IS 15959 event IDs, CIM master data — maps onto the `MeterData` schema (this page is pinned to the current `v0.6` shape) carried by IES Data Exchange.

If you are implementing Smart Meter Data Exchange, this is the page you keep open while wiring your HES / MDMS to v0.6 payloads.

> `MeterData/v0.6` is the current canonical schema. Earlier drafts of IES used the working name `IES_Report` — treat those references as superseded by `MeterData/v0.6`.

---

1. The shape of `MeterData/v0.6` in one glance

A `MeterData/v0.6` payload is a **JSON array** with two kinds of object:

Object	Role
`PayloadDescriptorProfile`	Dictionary — declares `payloadDescriptors[]` (one per quantity: OBIS, readingType, unit, flowDirection, reportedMode) and `compactSequences[]` (column layout for the dense `payloads` matrix).
One of the profile objects below	Carries actual readings or events, referencing a descriptor set by name.

The eight profile shapes:

Profile	Indian-side equivalent	When it ships
`CUSTOMER`	CIS / Asset Master / GIS master data	On enrolment, on change
`INTERVAL`	IS 15959 Load Profile (`1.0.99.1.0.255`)	15 / 30-min block load survey
`DAILY`	IS 15959 Daily Profile (`1.0.94.91.0.255`)	Once / day
`MONTHLY`	IS 15959 Billing Profile (`0.0.98.1.0.255`)	Billing reset (typically month-end)
`BILL_DETAILS`	Utility-side billing computed details	On bill finalisation
`INSTANTANEOUS`	OBIS `1.0.x.7.0.255` real-time registers	On poll / push
`EVENT`	IS 15959 Event Log Profiles (`0.0.99.98.x.255`)	On event + scheduled pull
`ALARM`	Real-time alarms (tamper / overload / low credit)	On occurrence

OBIS codes are first-class in v0.6 — they sit inside `payloadDescriptors[].obis` as a string verbatim from the meter. The schema does not translate them, so the mapping below is mostly about *which profile a given OBIS belongs in* and *which `readingType` / `unit` / `reportedMode` to advertise.*

The canonical, machine-readable OBIS catalogue ships as `IES codes.json` (GitHub Pages, served raw) under `schemas/MeterData/v0.6/` — each entry pins `obis`, `name`, `category`, `profiles[]`, `supportedModes[]`, `meterCategories[]`, and the IS 15959 source. **That file is the source of truth.** The tables on this page are the human-readable summary; if a mapping looks wrong, the JSON wins. Browse the source on GitHub.

2. IS 15959 profile buffer -> `MeterData/v0.6` profile

The Indian Load Profile / Daily Profile / Billing Profile buffers map one-to-one to v0.6 profile shapes. The `intervalPeriod.duration` distinguishes cadence.

IS 15959 profile (OBIS)	Typical period	`MeterData/v0.6` profile	`intervalPeriod.duration`
Load Profile (`1.0.99.1.0.255`)	15 / 30 min	`INTERVAL`	`PT15M` / `PT30M`
Daily Profile (`1.0.94.91.0.255`)	24 h	`DAILY`	`PT24H`
Billing Profile (`0.0.98.1.0.255`)	Monthly	`MONTHLY`	`P1M`
Voltage / Current event buffers (`0.0.99.98.0.255` / `.1.255`)	Event-driven	`EVENT`	`timePeriod` window
Power-failure / Tamper / Control buffers (`0.0.99.98.2.255` / `.4.255` / `.5.255`)	Event-driven	`EVENT` (POWER_FAIL etc.) / `ALARM` (tamper / control)	`timePeriod` window
Real-time registers (`1.0.x.7.0.255`)	Polled / pushed	`INSTANTANEOUS`	Point-in-time

3. OBIS register -> `payloadDescriptors[]`

For each profile, the OBIS register is carried in `payloadDescriptors[].obis` and qualified with `readingType`, `unit`, `flowDirection`, and `reportedMode` (`USAGE` / `READING` / `DEMAND`).

3.1 Energy registers — Load / Daily / Monthly

Quantity	OBIS (block incremental)	OBIS (cumulative)	Unit	flowDirection	reportedMode	Goes in profile
Active energy import	`1.0.1.29.0.255`	`1.0.1.8.0.255`	kWh	`IMPORT`	`USAGE`	`INTERVAL` / `DAILY` / `MONTHLY`
Active energy export	`1.0.2.29.0.255`	`1.0.2.8.0.255`	kWh	`EXPORT`	`USAGE`	`INTERVAL` / `DAILY` / `MONTHLY`
Apparent energy import	`1.0.9.29.0.255`	`1.0.9.8.0.255`	kVAh	`IMPORT`	`USAGE`	`INTERVAL` / `DAILY` / `MONTHLY`
Reactive energy (lag)	`1.0.5.29.0.255`	`1.0.5.8.0.255`	kVArh	`IMPORT`	`USAGE`	`INTERVAL` / `MONTHLY`
Reactive energy (lead)	`1.0.8.29.0.255`	`1.0.8.8.0.255`	kVArh	`EXPORT`	`USAGE`	`INTERVAL` / `MONTHLY`
ToU import (Zones 1..8)	—	`1.0.1.8.<z>.255`[1]	kWh	`IMPORT`	`USAGE`	`MONTHLY`

Quantity	OBIS (block incremental)	OBIS (cumulative)	Unit	flowDirection	reportedMode	Goes in profile
Maximum demand (kW)	—	1.0.1.6.0.255	kW	IMPORT	DEMAND	MONTHLY
Maximum demand (kVA)	—	1.0.9.6.0.255	kVA	IMPORT	DEMAND	MONTHLY

3.2 Instantaneous registers

Quantity	OBIS	Unit	reportedMode	Profile
Voltage (phase-neutral)	1.0.12.7.0.255	V	READING	INSTANTANEOUS
Phase current	1.0.11.7.0.255	A	READING	INSTANTANEOUS
Neutral current	1.0.91.7.0.255	A	READING	INSTANTANEOUS
Frequency	1.0.14.7.0.255	Hz	READING	INSTANTANEOUS
Signed power factor	1.0.13.7.0.255	—	READING	INSTANTANEOUS
Active power (import)	1.0.1.7.0.255	kW	DEMAND	INSTANTANEOUS
Apparent power (import)	1.0.9.7.0.255	kVA	DEMAND	INSTANTANEOUS

3.3 Identification registers — `CUSTOMER` profile

Field	OBIS	Goes in
Meter serial number	0.0.96.1.0.255	CUSTOMER
Manufacturer name	0.0.96.1.1.255	CUSTOMER
Device ID	0.0.96.1.2.255	CUSTOMER (D1 / D2 meter categories)
Firmware version	1.0.0.2.0.255	CUSTOMER
Clock (real-time)	0.0.1.0.0.255	Implicitly handled by intervalPeriod / timestamp

---

4. IS 15959 Event IDs -> `EVENT` / `ALARM`

Events carry `eventId` (numeric, IS 15959) and a human-readable `eventName` directly. Examples are in `schemas/MeterData/v0.6/examples/EventProfile.json`.

Event ID (occur)	Event ID (restore)	Description	Category	Profile
1	2	Phase-to-neutral under-voltage	Voltage	EVENT
3	4	Phase-to-neutral over-voltage	Voltage	EVENT
11	12	Phase missing	Voltage	EVENT
51	52	Phase current unbalance	Current	EVENT

Event ID (occur)	Event ID (restore)	Description	Category	Profile
61	62	CT reverse	Current	`EVENT`
69	70	Neutral disturbance	Current / others	`EVENT`
101	102	Power failure / power up	Power	`EVENT`
151	—	Parameters programming (config change)	Transaction	`EVENT`
201	202	Magnetic interference	Tamper	`ALARM` (active) / `EVENT` (logged)
205	—	Meter cover open (non-restorable)	Tamper	`ALARM`
301	302	Relay disconnect / connect (manual)	Control	`EVENT`
303	—	Relay disconnect (overload)	Control	`EVENT` / `ALARM`
304	—	Relay disconnect (low credit)	Control	`ALARM`
305	—	Relay disconnect (tamper auto-cutoff)	Control	`ALARM`
306	307	Token acceptance / rejection (prepaid)	Control	`EVENT`

`EVENT` profiles can carry the IS 15959 magnitude and duration directly (`magnitude`, `duration`); `ALARM` profiles are state-based (active / cleared) and intended for real-time pushes.

---

5. Data quality — `validationStatus`

v0.6 carries per-reading validation via the `overrides[]` block on a profile entry. Each override pins the `descriptorIndex` it qualifies, the `validationStatus`, the `source`, an optional `changeMethod`, and a `failCode`.

`validationStatus`	Meaning
`VALID`	Read direct from the meter; no estimation
`ESTIMATED`	Filled by VEE / interpolation
`MISSING`	No value available; the entry exists to preserve the interval index
`BAD`	Value present but flagged as out-of-range / suspect

Example fragment from `IntervalProfile.json`:

```
{ "id": 2, "payloads": [ 5.21, 0.12 ],
  "overrides": [{ "descriptorIndex": 0, "validationStatus": "ESTIMATED",
                  "source": "ESTIMATED", "changeMethod": "linear-interpolation",
                  "failCode": "VEE-MISSING-INTERVAL" }] }
```

---

6. CIM master data -> `CUSTOMER` profile

The CIM (IEC 61968-1 / 61968-9) master data the AMI deployment requires lands in the `CUSTOMER` profile. Use `[BaseProfile].meterRefs[]`, `serviceDeliveryPointRefs[]`, `customerRefs[]` for the relational keys, and `attributes` for the descriptive payload.

CIM area	Key fields	Source system	Reference in `CUSTOMER`
Asset master	Meter Serial, Make, Model, Hardware / Firmware, CT / PT ratios	Inventory / ERP	`meterRefs[]` + `attributes`
Consumer master	CA number, name, address, category (DS / NDS), sanctioned load	CIS / billing	`customerRefs[]` + linked ElectricityCredential v1.2 `customerProfile`
Network master	Substation, Feeder, DT, Pole, Phase	GIS	`attributes` (FEEDER_ID, DT_ID, SUBSTATION_ID, POLE_ID, PHASE)
Tariff master	ToU slots, slabs, fixed charges	Billing system	Out of band of MeterData; carried by `MeterServiceProfile` inside ElectricityCredential v1.2

When an OBIS reading arrives, the receiving MDM matches it via `meterRefs[]` -> asset -> consumer -> network -> tariff. The `serviceDeliveryPointRefs[]` value is the "service point" object that ties the three together (CIM IEC 61968-9 `UsagePoint`).

---

7. Worked example — 30-minute Load Survey

The full schema-validating example is at `schemas/MeterData/v0.6/examples/IntervalProfile.json`. The shape:

1. One `PayloadDescriptorProfile` declares `IntervalLoadSurveySet` with `kWh imp block` (OBIS `1.0.1.29.0.255`) and `kWh exp block` (OBIS `1.0.2.29.0.255`).
2. One `IntervalProfile` references the descriptor set, carries `intervalPeriod` (`start`, `duration: PT30M`), and packs each interval into a tight `payloads: [imp, exp]` array.
3. Quality overrides ride alongside, addressing the descriptor by index.

This is the minimal pattern. Daily / Monthly / Instantaneous / Event / Alarm follow the same descriptor-then-profile shape; their examples sit alongside in `schemas/MeterData/v0.6/examples/`.

---

8. References

- `MeterData` — schema, examples, user guide, reference guide (mappings on this page are pinned to `v0.6`)
- `IES codes.json` — canonical OBIS catalogue (machine-readable). Source on GitHub.
- Smart Meter Data Exchange — the use case that carries this payload
- Data Exchange — the wire that carries it
- `MeterDataRequest` — the matching request schema (current: `v0.6`)
- `MeterDataRequestCredential` — optional authorisation VC (current: `v0.1`)

- `ElectricityCredential` — sits alongside `MeterData` for the slow-changing customer / asset / service-connection data (current: `v1.2`)

Appendix — IS 15959 deep reference

This appendix is the full IS 15959 source-of-truth survey (formerly published as a standalone page). Use it to verify the mappings above against the underlying Indian standard.

A1. Profile buffers

Profile type	OBIS	Integration period	Mandatory capture sequence (typical)
Load Profile	`1.0.99.1.0.255`	15 / 30 min	Clock, kWh Imp, kVAh Imp, Avg Voltage, Avg Current
Billing Profile	`0.0.98.1.0.255`	Monthly	Billing Date, Total kWh, Total kVAh, MD kW, MD kVA, ToU registers
Daily Profile	`1.0.94.91.0.255`	Daily (24 h)	Clock (midnight), Cumulative kWh, Cumulative kVAh

A2. Event log buffers

Category	OBIS profile	Event types
Voltage	`0.0.99.98.0.255`	Under-voltage, over-voltage, phase missing
Current	`0.0.99.98.1.255`	Unbalance, CT reverse, neutral disturbance
Power failure	`0.0.99.98.2.255`	Power ON / OFF logs with timestamps
Transaction	`0.0.99.98.3.255`	Programming, date change, relay operation
Others / tamper	`0.0.99.98.4.255`	Magnetic interference, cover open
Control	`0.0.99.98.5.255`	Load-limit changes, relay triggers (prepaid)

A3. Communication flow **Polling (HES -> meter):** Association Request (AARQ) -> HLS authentication (AES-GCM-128, Security Suite 0) -> Selective Read of `profile_generic` by range -> meter responds with `compact-array` -> Release (RLRQ).

Push (meter -> HES -> MDM): Critical events (tamper / power-fail) are pushed via the PUSH Setup Object (`0.0.25.9.0.255`). Routine logs are fetched during the daily scheduled read. HES converts raw OBIS to an IEC 61968-9 message for the MDM.

Control (MDM -> HES -> meter): MDM issues `Disconnect` request via Northbound REST / SOAP -> HES authenticates with meter, sends `ACTION` to Relay Control Object (`0.0.96.3.10.255`) -> HES reads Relay Status to confirm -> Feedback relayed to MDM.

A4. Network topology assumed by AT&C-loss accounting

- **Substation / Feeder level** — high-accuracy meters (Class 0.2s / 0.5s) on the outgoing 11 kV / 33 kV lines.
- **DT (LV side) level** — meters at the DTR for energy accounting.
- **Consumer level** — 1-phase or 3-phase end-user meters.

GIS holds Feeder <-> DT and DT <-> Consumer mappings; MDM stores them. The `CUSTOMER` profile carries the relevant identifiers via its `attributes` block (FEEDER_ID, DT_ID, SUBSTATION_ID, POLE_ID, PHASE).

A5. IS 15959 annexure summary

Annexure	Subject	Detail
A	Data types	Mandates `double-long-unsigned` for energy to avoid overflow
B	OBIS mapping	Master list for 1-phase, 3-phase, CT / PT meters
C	Load profile	Standardises the sequence of capture objects
D	Event IDs	Canonical 8-bit list (1–255)
E	Billing logic	Auto-reset (midnight of 1st) and MD reset behaviour
F	Security	Security Suite 0 requirements for AMI
G	Smart features	Protocols for firmware image transfer and PUSH parameters

10.5 Consumer Meter Digest

In a hurry? Jump to the Checklist.

The Consumer Meter Digest is a MeterDataCredential v0.6 issued, on consumer demand, holder-bound to the consumer's wallet, carrying their own meter readings for a specified period. It is not a new credential type — it is *how* an existing MeterDataCredential is configured when a consumer (rather than another DISCOM or AMISP) is the audience.

The Digest gives a consumer a portable, signed, verifier-friendly bundle of their telemetry — for a loan application, a rooftop-solar quote, a tariff comparison, a marketplace listing, a housing-society compliance check — without those parties having to phone the DISCOM.

10.5.1 Why this use case exists

Consumers regularly need to share their actual electricity-consumption pattern. Today they print PDFs of monthly bills and email scans. Verifiers have no way to confirm the bill is real and unaltered, so most of them call the DISCOM anyway. The Digest replaces that loop with a credential the verifier can check offline against the DISCOM's published key.

10.5.2 How it differs from a B2B MeterDataCredential

Same schema, same issuance pipeline. Three issuance-time differences:

Concern	B2B MeterDataCredential v0.6	Consumer Meter Digest
Triggered by	Beckn `confirm` from a DISCOM consumer-pull	Consumer request (often via DigiLocker or a consented wallet app)
`credentialSubject.id`	absent — bearer; the receiving DISCOM is the implicit subject	**set** to the consumer's wallet DID
`validUntil`	days to weeks	**hours to days** — Digests are point-in-time snapshots

The schema body — the `meterReference`, the `period`, the `readings` (raw `INTERVAL`/`DAILY`/`MONTHLY` profiles) or `summary` (aggregates), `dataQuality`, the issuer block, the proof — is identical to any other MeterDataCredential v0.6.

The `type` array stays `["VerifiableCredential", "MeterDataCredential"]` — no new VC type is introduced.

10.5.3 Actors and flow

Role	Who	What they do
Holder	Consumer	Initiates the request from their wallet / DigiLocker; stores the returned Digest; presents it to verifiers of their choosing
Issuer	DISCOM	Pulls the relevant readings from its MDM, signs the Digest, delivers it back into the consumer's wallet

Role	Who	What they do
Verifier	Bank, marketplace, energy app, housing society, EV-charger installer	Reads the Digest; resolves the DISCOM's `did:web` and (if cited) the regulator's licensing pointer to validate; reads the readings or summary

Granularity options today: `RAW_15M` (15-minute interval data), `DAILY`, `MONTHLY`, plus derived summaries (`SUMMARY_TOTAL`, `SUMMARY_PEAK`, `SUMMARY_TOD`). Maximum period typically 24 months for `MONTHLY`, 90 days for `RAW_15M`.

10.5.4 Building blocks

Block	Used for
Identifiers and Addressing	DISCOM's `did:web`; consumer's wallet `did:key`; meter and connection DIDs that anchor the Digest
Energy Credentials	Issuance, signing, verification, revocation — including the Consumer Meter Digest variant under "Credential variants"
Data Exchange	If the consumer-pull endpoint is fronted by your BPP over Beckn, the BAP/BPP machinery is the same as Smart Meter Data Exchange — only the trigger (consumer, not DISCOM) and the `validUntil` differ
DigiLocker delivery	The dominant delivery channel into the consumer's wallet

10.5.5 What you actually do

1. The consumer must hold a credential proving the right to request data for this meter — typically a Consumer Energy Passport (the holder-bound ElectricityCredential v1.2) or a minimal customer credential.
2. Catalogue a "consumer-pull" endpoint on your BPP that accepts a request bearing that credential and returns a MeterDataCredential v0.6.
3. Issue the Digest via Energy Credentials — Issue your first credential, setting `credentialSubject.id` to the consumer's wallet DID, `schemaId` to `MeterDataCredential/v0.6`, and `validUntil` to a short window matching the use case (24h for loan portals, up to 7d for non-time-sensitive flows).
4. Deliver to the consumer's wallet — DigiLocker delivery, or directly to a known DID inbox if the wallet exposes one.
5. Revocation rarely matters in practice (Digests typically expire faster than they would need revocation), but the same DeDi-hash revocation flow is available if you need it.

10.5.6 Checklist

Use-case-specific items on top of base credential issuance.

Step 0 — base issuance in place. Complete Energy Credentials -> Checklist Phases 1–2.

Step 1 — MDM read path.

- [ ] Read access tested for the granularities you'll support (`RAW_15M`, `DAILY`, `MONTHLY`, derived summaries)
- [ ] Max-range policy decided (e.g. 24 months for `MONTHLY`, 90 days for `RAW_15M`)
- [ ] Latency budget understood — consumer flows need a Digest in seconds

Step 2 — consumer-pull endpoint.

- [ ] Beckn `DatasetItem` for the pull endpoint published on your BPP (`accessMethod: INLINE`)
- [ ] `requiredCredential` restricts callers to a valid Consumer Energy Passport (or minimal customer credential)
- [ ] `granularityOptions` / `maxRange` match Step 1's policy

Step 3 — issuance shape.

- [ ] `credentialSubject.id` = wallet DID; `schemaId` = `MeterDataCredential/v0.6`
- [ ] `validUntil` short — 24h for loan portals, up to 7d for less time-sensitive flows
- [ ] Schema validation passes against MeterDataCredential v0.6

Step 4 — wallet delivery.

- [ ] DigiLocker pull tested end-to-end -> DigiLocker delivery
- [ ] One direct DID-push path tested for non-DigiLocker wallets
- [ ] Consumer sees the Digest within the Step 1 latency budget

Step 5 — verifier interop.

- [ ] Verification rehearsed with one verifier (bank / marketplace / housing society / EV installer)
- [ ] If you emit derived `summary` outputs, share the schema + description with verifiers up front
- [ ] Revocation flow tested (even though Digests usually expire before needing it)

Team. [ ] IT SPOC (MDM read path + BPP catalogue) · [ ] Customer-ops SPOC (wallet support) · [ ] Governance / Compliance SPOC (data-disclosure policy)

10.5.7 References

- MeterDataCredential v0.6 schema — the schema this use case rides on
- Energy Credentials — Credential variants — where the Digest variant is documented
- Smart Meter Data Exchange use case — the B2B sibling of this consumer flow
- DigiLocker delivery

10.6 Consumer Energy Passport

In a hurry? Jump to the Checklist.

The Consumer Energy Passport is an ElectricityCredential v1.2 issued holder-bound to a consumer's wallet. It is not a new credential type — it is *how* the existing v1.2 credential is shaped, issued, and delivered when the consumer is the audience.

A DISCOM signs it. The consumer carries it in DigiLocker or a DID wallet. Banks, marketplaces, regulators, subsidy portals, and consented apps verify it offline using the DISCOM's published `did.json`.

10.6.1 Why this use case exists

Consumers carry paper attestations of their connection details — sanctioned load, tariff category, asset list — for KYC at banks, rooftop-solar marketplaces, EV-charger installers, housing societies, and subsidy portals. Every recipient calls or emails the DISCOM to confirm. The Passport replaces that loop with a credential the verifier can check in milliseconds, without ever contacting you.

10.6.2 How it differs from a bearer ElectricityCredential

Same schema, same issuance pipeline. Two issuance-time differences:

Concern	Bearer ElectricityCredential v1.2	Consumer Energy Passport
`credentialSubject.id`	absent — bearer; whoever holds the JSON is treated as the subject	**set** to the consumer's wallet DID (`did:key` or `did:jwk`)
`customerProfile.idRef`	optional	**populated** with a verifiable government-ID reference (e.g. masked Aadhaar, DigiLocker pull receipt)

Everything else (`customerProfile`, `customerDetails`, `energyResources[]`, `consumptionProfiles[]`, `issuer`, `proof`, revocation) is identical to any other ElectricityCredential v1.2.

The `type` array stays `["VerifiableCredential", "ElectricityCredential"]` — no new VC type is introduced.

10.6.3 Actors and flow

Role	Who	What they do
Issuer	DISCOM	Signs the Passport once the consumer has been identity-proofed
Holder	Consumer	Stores the Passport in DigiLocker / a DID wallet; chooses when and to whom to present
Verifier	Bank, marketplace, regulator, subsidy portal, housing society, EV-charger installer	Reads the Passport from the wallet; verifies the issuer's signature, revocation status, and the holder-binding proof

A typical issuance happens once at customer onboarding (and then is re-issued on material change — meter swap, sanctioned-load change, DER commissioning).

10.6.4 Building blocks

Block	Used for
Identifiers and Addressing	DISCOM's `did:web`; consumer's wallet `did:key`; asset / meter / connection DIDs that appear inside the credential
Energy Credentials	The single home for signing, verifying, and revoking — including the Consumer Energy Passport variant under "Credential variants"
Holder binding	Wallet-DID binding pattern; presentation-time challenge / VP proof
DigiLocker delivery	The bulk delivery channel for Indian consumers; for many use cases DigiLocker's Aadhaar pull also acts as the identity-binding step

10.6.5 What you actually do

1. Set up the DISCOM `did:web` and run OpenCred -> Identifiers — Step-by-step.
2. Decide your identity-proofing method (DigiLocker pull, offline-KYC XML, in-person KYC, record-match) and document it for privacy review.
3. Issue the Passport via Energy Credentials — Issue your first credential, setting `credentialSubject.id` to the wallet DID and populating `customerProfile.idRef` with the government-ID reference.
4. Deliver into the consumer's DigiLocker or wallet -> DigiLocker delivery.
5. Wire revocation into the same flow you use for any v1.2 credential.

10.6.6 Selective disclosure

The Passport is a regular VC, so any selective-disclosure profile your wallet supports (SD-JWT-VC is the typical choice in 2026) works. The DISCOM is not in the disclosure loop — the wallet chooses which fields to present to each verifier.

10.6.7 Checklist

Use-case-specific items on top of base credential issuance.

Step 0 — base issuance in place. Complete Energy Credentials -> Checklist Phases 1–2 (foundations, issue / verify / revoke).

Step 1 — identity proofing. The Passport sets `credentialSubject.id` to the wallet DID and `customerProfile.idRef` to a government-ID *reference*, both vouched for at issuance — see Identifiers — Identity-proofing at issuance.

- [ ] Method chosen (DigiLocker pull / AADHAAR_OFFLINE_KYC / in-person / DISCOM record match)
- [ ] Privacy review completed; procedure tested with one consumer

Step 2 — wallet delivery.

- [ ] DigiLocker Pull URI registered -> DigiLocker delivery
- [ ] Direct DID-push tested for one non-DigiLocker wallet (where relevant)
- [ ] A freshly issued Passport reaches a wallet end-to-end

Step 3 — issuance shape.

- [ ] `credentialSubject.id` = wallet DID; `customerProfile.idRef` = verified government-ID *reference* (not the raw number)

- [ ] `validUntil` set to a sensible horizon (re-issued on material change)
- [ ] Schema validation passes against ElectricityCredential v1.2

Step 4 — verifier interop.

- [ ] Selective-disclosure profile agreed with the first verifiers (SD-JWT-VC typical)
- [ ] Verification rehearsed with one verifier (bank / marketplace / subsidy portal)
- [ ] Revocation tested — revoking invalidates all presentations within minutes

Team. [ ] Customer-ops SPOC (identity proofing) · [ ] IT SPOC (wallet delivery) · [ ] Governance / Compliance SPOC (privacy review)

10.6.8 References

- ElectricityCredential v1.2 schema — the schema this use case rides on
- Energy Credentials — Credential variants — where the Passport variant is documented
- Identifiers — Holder binding patterns
- DigiLocker delivery

10.7 DISCOM Regulatory Filing

In a hurry? Jump to the Checklist.

A DISCOM submits its Aggregate Revenue Requirement (ARR), tariff petition, true-up, or other compliance filing to a State Electricity Regulatory Commission (SERC) as a structured, signed, machine-verifiable object.

10.7.1 Scenario

Every year, every DISCOM in India submits an ARR filing to its state regulator — a detailed projection of power-purchase, transmission, O&M, depreciation, and return-on-equity costs that justifies the tariff it wants the SERC to approve. The same DISCOMs file true-up petitions, FPPCA reconciliations, multi-year tariff petitions, and various compliance returns through the year.

Today these arrive as **PDFs and Excel sheets**. SERC staff manually re-key the numbers into their analysis spreadsheets. Stakeholders and consumer-rep parties can only object on the same PDFs. There is no canonical machine-readable form, so no cross-DISCOM comparison, no automated trend analysis, no straight-through validation.

This use case replaces the PDF round-trip with a structured `IES_ARR_Filing` payload delivered over the IES Data Exchange protocol — signed by the DISCOM, ingested directly by the SERC, archived with a non-repudiable audit trail.

10.7.2 Actors and Roles

Role	Organisation (example)	What they do
BPP — data provider	DISCOM (e.g. BESCOM)	Generates and submits the filing
BAP — data consumer	SERC (e.g. KERC, APERC)	Receives, validates, and archives the filing
Optional secondary consumers	Consumer-rep parties, research orgs, Forum of Regulators	Receive the published filing via the same protocol, gated by public-disclosure norms

Note: unlike Smart Meter Data Exchange, here the **DISCOM is the BPP** (data provider) and the **regulator is the BAP**. The Beckn protocol is symmetric — the same Docker stack and adapter run both directions.

10.7.3 Building Blocks Used

Block	Role in this use case
Identifiers	The DISCOM and SERC each have a `did:web` identity (published `did.json` on their domain). The filing object carries identifiers for the filing, the fiscal year, the regulatory order it responds to, and the cost line items. Because the filing rides over Beckn, both parties also have entries in the IES DISCOMs / Regulators reference registries — see Identifiers and Addressing -> Appendix E.
Registries	For Beckn message-signature verification: DISCOM and SERC subscriber records resolved via DeDi / `fabric.nfh.global`. The IES-curated DISCOM reference registry and Regulator reference registry provide the Beckn network trust boundary.
Data Exchange	The Beckn-based protocol carries the `IES_ARR_Filing` payload from BPP (DISCOM) to BAP (SERC), with the same `confirm` / `on_confirm` / `status` / `on_status` lifecycle as the meter-data flow.
Energy Credentials	*Not used for the filing itself* — the filing is signed by the Beckn message envelope. Credentials enter the picture only if a consumer-rep needs an attested DISCOM identity letter, which is the Consumer Energy Passport / DISCOM-side analogue, not part of the filing flow.

10.7.4 The Dataset — `IES_ARR_Filing`

The `IES_ARR_Filing` schema carries structured cost data for one or more fiscal years. Two equivalent representations are supported:

A. Native IES shape

The shape most DISCOMs will generate directly from their financial systems:

```
{
  "filingId": "BESCOM-ARR-2026-27",
  "licensee": "BESCOM",
  "state": "Karnataka",
  "regulatoryCommission": "KERC",
  "submissionDate": "2026-01-15",
  "fiscalYears": [{
    "year": "2026-27",
    "totalRevenueRequirementCrINR": 18432.5,
    "lineItems": [
      { "category": "Power Purchase Cost",  "subcategory": "Long-term PPAs", "amountCrINR": 12450.5, "notes": "120
      { "category": "Transmission Charges", "subcategory": "PGCIL",          "amountCrINR":  1823.2 },
      { "category": "O&M Expenses",         "subcategory": "Employee costs", "amountCrINR":   892.7 },
      { "category": "Depreciation",                                          "amountCrINR":   634.1 },
      { "category": "Return on Equity",                                      "amountCrINR":   632.0 }
    ]
  }]
}
```

The `category` values follow the standard ARR cost categories prescribed by most SERCs, which keeps the schema natural for DISCOM finance teams.

B. OpenADR 3 REPORT representation

For alignment with global grid-data tooling and the same OpenADR 3 envelope used by meter telemetry:

Custom concept	OpenADR 3 mapping	Why
Fiscal year	`intervalPeriod` (`P1Y` starting 1 Apr)	Standard interval semantics
ARR cost categories	`resourceName`	Each cost heading is a logical resource
Financial values	`payloadType: PRICE` (units `INR_CRORES`)	Standard type for cost data
Energy quantities	`payloadType: USAGE` (units `MU`)	Standard type for energy volumes
Projections	`readingType: FORECAST`	Next-year ARR
Historical actuals	`readingType: SUMMED`	True-up sections

```
{
  "objectType": "REPORT",
  "reportID": "BESCOM-ARR-2026-27",
  "programID": "regulatory-arr-filing",
  "reportPayloadDescriptors": [
    { "payloadType": "PRICE", "readingType": "FORECAST", "units": "INR_CRORES" },
    { "payloadType": "USAGE", "readingType": "FORECAST", "units": "MU" }
  ],
  "resources": [
    {
      "resourceName": "Power Purchase Cost",
      "intervalPeriod": { "start": "2026-04-01T00:00:00+05:30", "duration": "P1Y" },
      "intervals": [{ "id": 0, "payloads": [{ "type": "PRICE", "values": [12450.5] }] }]
    },
    {
      "resourceName": "Total Energy Requirement",
      "intervalPeriod": { "start": "2026-04-01T00:00:00+05:30", "duration": "P1Y" },
      "intervals": [{ "id": 0, "payloads": [{ "type": "USAGE", "values": [15200.0] }] }]
    }
  ]
}
```

Either shape can be exchanged; the BAP declares which it expects in the catalogue entry, and the BPP serialises accordingly. Both share the same `filingId` so downstream archives can cross-reference.

> **Status — `IES_ARR_Filing` schema is being finalised.** The category enumeration is the open item: SERCs use slightly different cost taxonomies and the IES schema is converging on a superset with mapping notes. Integrate against the devkit examples; expect small enum additions, not breaking renames.

10.7.5 Setup Steps

1. Register the DISCOM and the SERC

Both parties need DeDi entries:

- DISCOM in `ies-discoms-reference-registry` under `india-energy-stack:<discom-short-code>`.
- SERC in `ies-regulators-reference-registry` under `india-energy-stack:<serc-short-code>`.

Each registration pins a `did:web` and the public key for Beckn message signing. See Registries — step-by-step.

2. Identifier hygiene

Mint a single canonical `filingId` per submission (typical pattern: `<DISCOM>-ARR-<FY>` or `<DISCOM>-TRUEUP-<FY>`). The same `filingId` should appear on any subsequent re-submissions or revisions — versioning lives on the data-exchange envelope (`updatedAt`), not on the ID.

If the filing responds to a specific tariff order, include the `policyID` of that order (see Tariff Intelligence) so the SERC's archive can stitch order <-> petition <-> true-up automatically.

3. Stand up the data-exchange adapters

The DISCOM runs a BPP adapter; the SERC runs a BAP adapter. The local devkit gives both:

```
git clone https://github.com/beckn/DEG.git
cd DEG/devkits/data-exchange/install
docker compose up -d
```

For production, follow Registry Setup and point the adapter at the IES registry endpoint and your `did:web` keys.

4. Catalogue the filing as a published dataset

The DISCOM publishes the filing through its BPP catalogue with:

- `descriptor.name` — human-readable filing title (e.g. *"BESCOM ARR Filing 2026-27"*)
- `category` — `ARR`, `TRUE_UP`, `FPPCA`, `COMPLIANCE` (one of the agreed regulatory categories)
- `accessMethod` — typically `INLINE` (filings are small) or `SIGNED_URL` if supporting workbooks accompany it
- `policyContext` — references to the SERC orders the filing responds to

5. Exercise the flow

The flow is `confirm` -> `on_confirm` -> `status` -> `on_status`, with the regulator (BAP) initiating `confirm` against the catalogue entry:

```
"resources": [{
  "id": "ds-bescom-arr-filing-2026-27",
  "descriptor": { "name": "BESCOM ARR Filing 2026-27" },
  "resourceAttributes": {
    "@context": "https://raw.githubusercontent.com/beckn/DDM/main/specification/schema/DatasetItem/v1.1/context.js
    "@type": "DatasetItem",
    "accessMethod": "INLINE",
    "dataPayload": { /* IES_ARR_Filing — see snippet above */ }
  }
}]
```

Both messages are signed; the SERC's archive stores the original signed envelope as the non-repudiable record of submission.

6. (Optional) Open the filing for public consumption

If the SERC mandates public disclosure of filings, the same dataset is republished from the SERC's BPP under the public-disclosure catalogue. Consumer-rep parties and researchers become BAPs against that catalogue, with settlement value `0` per public-disclosure norms.

10.7.6 Operate

```
# Import the regulatory-filing BAP collection
#   DEG/devkits/data-exchange/uc2-regulatory-data/postman/
# Set bap_host_root + bpp_host_root to http://beckn-router:9000
# Fire `confirm`.

docker logs sandbox-bap 2>&1 | grep -E 'on_(confirm|status)' | tail -10
```

The signed envelope received by the SERC is what should be persisted — re-validating the signature against the DISCOM's registry-published key is the audit-trail check.

10.7.7 Why This Matters

ARR filings arrive today as PDFs/Excel sheets that regulators manually re-key. Delivering them as structured `IES_ARR_Filing` JSON — signed, timestamped, schema-validated — replaces re-keying with direct ingest, gives a non-repudiable audit trail, and lets every DISCOM submit in the same canonical shape. The OpenADR 3 representation additionally makes the same data legible to grid-management tooling.

10.7.8 Open Items

- **`IES_ARR_Filing` schema canonicalisation.** The schema currently lives on `beckn/DEG:ies-specs`; it will move to `India-Energy-Stack`. The cost-category enumeration is the active conversation — expect additions, not breaking renames.
- **Workbook attachments.** Many filings ship with supporting Excel models; the convention for attaching them (separate `DatasetItem` per workbook vs. embedded base64) is being agreed.
- **Cross-filing references.** A standard reference shape for *"this filing responds to SERC Order X"* is being aligned with the Tariff Intelligence `policyID` pattern.

10.7.9 Checklist

For the DISCOM (filing) and the SERC (receiving). Role: [ ] DISCOM [ ] SERC [ ] Both.

1. Decide which filings to start with — usually annual ARR first, then true-up, FPPCA, compliance returns.

- [ ] Filing types selected; source system identified for each (financial / regulatory-affairs)
- [ ] Native vs OpenADR-3 representation agreed with the counterparty

2. Register both parties on the IES network.

- [ ] DISCOM in the DISCOMs reference registry; SERC in the Regulators reference registry
- [ ] Signing keypairs generated and stored in a secrets manager

3. Agree the cost-category mapping — map your finance system's categories to the `IES_ARR_Filing` enumeration once.

- [ ] Categories mapped and signed off by regulatory affairs + finance
- [ ] Tariff-order references (`policyID`) tracked so each filing cites the order it answers

4. Generate a sample filing as structured data.

- [ ] One prior-year filing converted to `IES_ARR_Filing` and schema-validated
- [ ] Line-item parity confirmed against the source PDF
- [ ] Workbook-attachment policy decided (separate dataset / embedded / omitted)

5. Stand up the data-exchange adapters — DISCOM provider-side, SERC consumer-side; prove on the devkit first.

- [ ] Adapters running on the devkit sandbox
- [ ] Test `confirm` -> `on_status` with the sample filing succeeds; both sides verify signatures against the registry

6. Catalogue the filing and submit.

- [ ] Catalogue entry published per filing (`ARR` / `TRUE_UP` / `FPPCA` / `COMPLIANCE`)
- [ ] SERC archives the original signed envelope as the non-repudiation record
- [ ] Re-submission policy agreed (`updatedAt`, same `filingId`)

7. (Optional) Open the filing for public consumption.

- [ ] Public-disclosure catalogue entry published (settlement value `0`), if mandated
- [ ] Consumer-rep parties / researchers onboarded as BAPs

8. Production deployment and go-live.

- [ ] HTTPS live; signing keys in a secrets manager; monitoring / alerting active
- [ ] One real production submission completed and verified
- [ ] Operations runbook (key rotation, late-amendment handling)

9. Team. [ ] Regulatory / finance SPOC · [ ] IT SPOC · [ ] Authorised Signatory

For the underlying network onboarding, see the Data Exchange -> Checklist.

10.7.10 References

- `IES_ARR_Filing` schema (upstream)
- Example payloads (devkit)
- ies-docs ARR examples
- Data Exchange — Core Concepts
- Data Exchange — Quick Start

10.8 Tariff Intelligence

In a hurry? Jump to the Checklist.

Tariff orders, time-of-day surcharges, deviation penalties, and data-exchange rules published as machine-readable *policy-as-code* — consumable by billing systems, consumer apps, smart meters, and analytics agents directly.

10.8.1 Scenario

A SERC issues a tariff order today as a PDF. Every DISCOM in that state then manually transcribes the rate slabs, time-of-day surcharges, fixed charges, and category definitions into its billing system. Consumer-side apps (bill estimators, EV charging planners, rooftop-solar payback calculators) each interpret the same order independently. Drift, ambiguity, and bugs are inevitable.

The same problem applies more broadly than tariff orders. Any **policy** that the energy sector needs to interpret consistently — sanctioned-load deviation penalties, peak-hour incentives, DR program participation rules, data-quality SLAs for AMISP reporting, public-disclosure thresholds — benefits from being published once, as code, by the issuing authority.

The Tariff Intelligence use case packages this as `IES_Policy` artefacts. The SERC (or any policy issuer) publishes machine-readable policy alongside the regulatory order; DISCOMs, apps, and meters ingest it directly with no re-keying and no interpretation drift. Issuers may publish policy in the public data exchange, or hand it inline during a private data exchange to set the terms of that exchange.

10.8.2 Actors and Roles

Role	Organisation (example)	What they do
BPP — policy publisher	SERC (e.g. MERC), or any policy authority (utility, regulator)	Publishes the signed `IES_Policy` artefact
BAP — policy consumer	DISCOM billing system, consumer-app developer, smart meter firmware, analytics agent	Subscribes to and ingests the policy
Identity authority	IES registries	The publisher's `did:web` is the trust anchor for the signed policy

Publication is typically open under public-disclosure norms (settlement value `0` in the example payloads), so the policy consumer set is unrestricted — every billing implementation in the state can subscribe.

10.8.3 Building Blocks Used

Block	Role in this use case
Identifiers	The `policyID` (e.g. `MUM-RES-T1`) is an IES identifier; the policy is bound to the publisher's `did:web` and references the `programID` of the regulatory program (consumer category, tariff scheme) it belongs to.

Block	Role in this use case
Registries	The publishing SERC/utility is looked up in the IES Regulators or DISCOMs reference registry to obtain its public key for signature verification.
Data Exchange	Policies are distributed over the same Beckn-based protocol used for telemetry and filings. Policies may flow in the public data exchange, or be carried inline as part of a private data exchange to negotiate that exchange's terms.
Energy Credentials	*Not used to carry the policy itself.* Credentials enter when a downstream agent needs to prove its tariff category or sanctioned load to a billing-time policy engine — that's the Consumer Energy Passport flow.

10.8.4 The Dataset — `IES_Policy`

The `IES_Policy` schema captures policy as structured, signed JSON-LD. For tariffs, a single `IES_Policy` object carries:

- **Identity** — `id`, `policyID`, `policyName`, `policyType` (`TARIFF` or `DISPATCH_GUIDE`), `programID` linking to an `IES_Program`.
- **Validity** — `samplingInterval` as an ISO 8601 recurrence pattern (e.g. `R/2024-04-10T00:00:00Z/P1M` — re-evaluated monthly from April 10, 2024).
- **`energySlabs[]`** — progressive consumption tiers, each `{ id, start (kWh, inclusive), end (kWh, exclusive or null for open-ended), price }`.
- **`surchargeTariffs[]`** — time-of-day adjustments, each `{ id, recurrence, interval (ToD start + duration), value, unit (PERCENTorINR_PER_KWH) }`.

Example — Mumbai Residential Telescopic

```
{
  "@context": "https://raw.githubusercontent.com/beckn/DEG/ies-specs/specification/external/schema/ies/core/contex
  "@type": "IES_Policy",
  "id": "policy-mumbai-res-001",
  "objectType": "POLICY",
  "policyID": "MUM-RES-T1",
  "policyName": "Mumbai Residential Telescopic 2024",
  "policyType": "TARIFF",
  "programID": "program-merashehar-001",
  "samplingInterval": "R/2024-04-10T00:00:00Z/P1M",
  "energySlabs": [
    { "id": "slab-0-100",    "@type": "EnergySlab", "start": 0,   "end": 100,  "price": 4.5 },
    { "id": "slab-101-300",  "@type": "EnergySlab", "start": 101, "end": 300,  "price": 7.2 },
    { "id": "slab-301-plus", "@type": "EnergySlab", "start": 301, "end": null, "price": 9.8 }
  ],
  "surchargeTariffs": [
    {
      "id": "surcharge-evening-peak",
      "@type": "SurchargeTariff",
      "recurrence": "P1D",
      "interval": { "start": "T18:00:00Z", "duration": "PT4H" },
```

```
      "value": 20,
      "unit": "PERCENT"
    },
    {
      "id": "discount-night-offpeak",
      "@type": "SurchargeTariff",
      "recurrence": "P1D",
      "interval": { "start": "T23:00:00Z", "duration": "PT6H" },
      "value": -10,
      "unit": "PERCENT"
    }
  ]
}
```

The `@type: "IES_Policy"` matters — ONIX's Extended Schema validator dispatches by `@type` against the schema name in `attributes.yaml`.

Beyond tariffs

The same `IES_Policy` envelope, with a different `policyType` and additional payload fields, expresses:

- **Deviation penalties** — sanctioned-load overshoots, scheduling deviations
- **Data-quality SLAs** — minimum reporting frequency, allowable missing-read percentage for AMISPs
- **Program participation rules** — DR enrolment criteria, peak-time rebate thresholds
- **Public-disclosure thresholds** — which datasets must be open vs. consented

The pattern is the same: a signed, addressable, machine-verifiable rule that consumers ingest directly.

10.8.5 Setup Steps

1. Register the publisher

The SERC (or utility) publishing the policy is registered in the appropriate DeDi registry:

- SERC -> `ies-regulators-reference-registry`
- DISCOM publishing a sub-policy -> `ies-discoms-reference-registry`

The registry entry pins the `did:web` and public key used to sign the `IES_Policy` object.

2. Mint identifiers

- A canonical `policyID` per policy (e.g. `MUM-RES-T1`, `KA-LT2-IND-TOD-2026`). This is the **stable handle** every downstream system uses to refer to this policy.
- A `programID` per `IES_Program` (consumer category, regulatory scheme) that groups related policies.

See Identifiers and Addressing -> Appendix C.

3. Author the policy

Author the policy directly in `IES_Policy` JSON-LD. The canonical schemas are in `beckn/DEG ies-specs core/` — `IES_Policy`, `IES_Program`, `EnergySlab`, `SurchargeTariff`. Validate against the JSON Schema before signing.

The DISCOM-facing pattern is to keep an internal CI pipeline that converts an authored YAML or spreadsheet into the canonical JSON-LD, runs schema validation, and submits it for signing.

4. Sign and stage for publication

The publisher's BPP signs the policy with its `did:web` private key. The signed bundle becomes the artefact that downstream verifiers will hash-check against the registry-published public key.

5. Publish via Data Exchange

The publisher's BPP exposes the policy through its catalogue. Consumers (`discover`/`select` or pre-agreed `confirm`) receive it through the standard lifecycle:

```
"performance": [{
  "id": "perf-tariff-policy-delivery-001",
  "status": { "code": "DELIVERY_COMPLETE" },
  "commitmentIds": ["commitment-tariff-policy-001"],
  "performanceAttributes": {
    "@context": "https://raw.githubusercontent.com/beckn/DDM/main/specification/schema/DatasetItem/v1/context.jsonl
    "@type": "DatasetItem",
    "dataset:accessMethod": "INLINE",
    "dataPayload": { /* IES_Policy — see above */ }
  }
}]
```

Public-disclosure policies typically use `accessMethod: INLINE` and settlement value `0` — anyone can pull them without contract.

6. (Optional) Use policies inline during private exchange

A BPP may attach a policy alongside its catalogue offer to set the **terms of the data exchange itself** — e.g. *"to consume this AMISP feed, your refresh must not exceed 1× per 15 minutes, and you must commit to this retention SLA."* The consumer's `confirm` is then evaluated against the inline policy. The policy is the same `IES_Policy` schema; only the carriage differs.

10.8.6 Consuming a Tariff in a Billing System

```python
def apply_tariff(consumption_kwh: float, hour_of_day: int, policy: dict) -> float:
    """Compute the bill given an IES_Policy. Real billing iterates over actual
    usage intervals; this is the rate-lookup core."""

    # 1. Find the applicable energy slab. `end` is exclusive; `null` = open-ended.
    base_rate = 0.0
    for slab in policy["energySlabs"]:
        end = slab.get("end")
        if slab["start"] <= consumption_kwh and (end is None or consumption_kwh < end):
            base_rate = slab["price"]
            break

    # 2. Apply the first matching time-of-day surcharge.
    surcharge_value = 0.0
    surcharge_unit  = "PERCENT"
    for tod in policy.get("surchargeTariffs", []):
        start_h = int(tod["interval"]["start"][1:3])
        dur_h   = int(tod["interval"]["duration"].split("PT")[1].rstrip("H")) \
                  if tod["interval"]["duration"].endswith("H") else 0
        end_h   = (start_h + dur_h) % 24
        in_window = (start_h <= hour_of_day < end_h) if start_h < end_h \
```

```
                else (hour_of_day >= start_h or hour_of_day < end_h)
        if in_window:
            surcharge_value = tod["value"]
            surcharge_unit  = tod.get("unit", "PERCENT")
            break

    if surcharge_unit == "PERCENT":
        effective_rate = base_rate * (1 + surcharge_value / 100)
    else:
        effective_rate = base_rate + surcharge_value

    return consumption_kwh * effective_rate
```

This function is what replaces the hand-coded slab/ToD table that every DISCOM billing system carries today.

10.8.7 Operate

```
git clone https://github.com/beckn/DEG.git
cd DEG/devkits/data-exchange/install
docker compose up -d

# Import the tariff BAP collection
#   DEG/devkits/data-exchange/uc3-tariff-policy/postman/
# Set bap_host_root + bpp_host_root to http://beckn-router:9000
# Fire `confirm`.

docker logs sandbox-bap 2>&1 | grep -E 'on_(confirm|status)' | tail -10

# Or one-shot the whole lifecycle via the Arazzo runner:
cd DEG/devkits/data-exchange/uc3-tariff-policy/workflows
./run-arazzo.sh -w confirm-through-delivery -v
```

10.8.8 Open Items

- **Policy types beyond `TARIFF` and `DISPATCH_GUIDE`.** Schemas for deviation-penalty and data-quality-SLA policies are in draft; they reuse the `IES_Policy` envelope with new `policyType` values.
- **Versioning convention.** When a SERC issues a tariff amendment, the agreed pattern is a new `id` (URN) with the same `policyID` and an explicit `replaces` link to the prior version — being formalised.
- **Policy bundles.** A bundle envelope to ship a set of related policies (e.g. all categories for an FY) as one signed package is being discussed.

10.8.9 Checklist

For a SERC, DISCOM, or other policy authority. Role: [ ] Publisher (SERC / utility) [ ] Consumer (DISCOM billing / app / meter).

1. Decide which policies to publish — usually residential and LT-commercial tariff orders first; the same envelope later carries deviation penalties, DR rules, data-quality SLAs.

- [ ] Policy types selected (`TARIFF`, `DISPATCH_GUIDE`, …); source documents identified

2. Register on the IES network.

- [ ] Publisher registered in the appropriate reference registry (Regulators / DISCOMs)
- [ ] Signing keypair generated and stored in a secrets manager

3. Mint stable identifiers — a stable `policyID` (e.g. `MUM-RES-T1`, `KA-LT2-IND-TOD-2026`); related policies grouped under a `programID`.

- [ ] Policy-ID scheme adopted; program IDs defined

4. Author the policy as structured data — energy slabs, ToD surcharges, validity window, recurrence.

- [ ] Authored in `IES_Policy` JSON-LD; schema validation passes; parity with the order PDF confirmed

5. Sign with the publisher's key.

- [ ] Signing pipeline tested (key access, sign, verify round-trip)
- [ ] Versioning agreed (new `id` per amendment, same `policyID`, explicit `replaces` link)

6. Publish via the data exchange.

- [ ] Catalogue entry published; public-disclosure access policy documented (inline delivery, settlement `0`)

7. Sample-consume in a billing system — confirm a bill computes from the JSON-LD without re-keying.

- [ ] Sample consumer ingests the policy and computes a bill
- [ ] Edge cases tested (open-ended top slab, surcharge-window wrap-around, percent vs absolute adders)

8. Production deployment and go-live.

- [ ] HTTPS live; signing keys in a secrets manager; monitoring / alerting active
- [ ] First real tariff published alongside the SERC order; amendment / republication runbook in place

9. Team. [ ] Legal / regulatory SPOC · [ ] IT SPOC · [ ] Authorised Signatory

For the underlying network onboarding, see the Data Exchange -> Checklist.

10.8.10 References

- `IES_Policy`, `IES_Program`, `EnergySlab`, `SurchargeTariff` (upstream)
- ies-docs tariff specification example
- Example payloads (devkit)
- Data Exchange — Core Concepts
- Data Exchange — Quick Start
- ies-docs `08_Energy_policy_as_code.md`

10.9 Energy Trading (WIP)

In a hurry? Jump to the Checklist.

Status — work in progress.

Two prosumers on different discoms execute a direct, signed energy trade. Each discom is represented in the protocol by a regulated Ledger Provider. The same wire protocol that carries datasets in Data Exchange carries the trade — but the payload is a contract and its fulfillment, not a dataset.

Energy Trading is a **variant of the Data Exchange building block**. The Beckn lifecycle (`search` -> `select` -> `init` -> `confirm` -> `status`), the ONIX adapter, registry-resolution, signing, and audit are reused unchanged. Every leg carries its payload inline inside the Beckn `message.contract` block as a `DEGContract`; what differs by leg is the `BecknTimeSeries` **payloadType vocabulary** the contract carries:

- **TP <-> TP and TP <-> LP — trade-negotiation legs.** PayloadTypes describe price, quantity, and allocations: `PRICE_PER_KWH`, `AVAILABLE_QTY`, `REQUESTED_QTY`, `BUYER_DISCOM_ALLOC`, `SELLER_DISCOM_ALLOC`, `FINAL_ALLOC`.
- **DISCOM <-> LP — meter-data sub-transaction during reconciliation.** PayloadTypes describe actually injected / consumed quantities per interval, supplied by the DISCOM to its contracted LP as input to allocation. Same `message.contract` envelope, same `BecknTimeSeries` shape — just a different payloadType vocabulary. This is **not** Smart Meter Data Exchange and **does not use** the `MeterData` schema or the DDM `DatasetItem` envelope; the meter quantities ride inside the same contract block as everything else.

If you have Data Exchange working, you have the protocol; this page describes the payload, the four-actor topology, and the policy bundle on top.

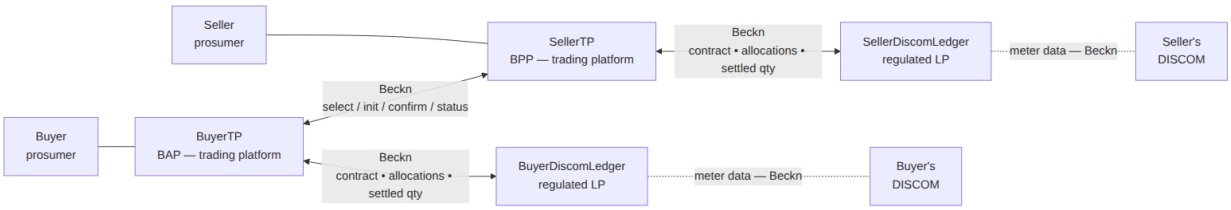
10.9.1 Actors and roles

Role	Who	participantId convention in devkit
Buyer	Prosumer importing energy	`buyer`
Seller	Prosumer exporting energy	`seller`
BuyerTP (BAP)	Buyer's trading platform	`example.bap.com`
SellerTP (BPP)	Seller's trading platform	`example.bpp.com`
BuyerDiscomLedger	Regulated Ledger Provider on behalf of the buyer's discom	`buyer-discom-ledger`
SellerDiscomLedger	Regulated Ledger Provider on behalf of the seller's discom	`seller-discom-ledger`

Network policy mandates that every discom operate through a regulated **Ledger Provider (LP)**. Each discom contracts exactly one LP; two discoms may share an LP or use different ones. When the buyer and seller share a discom the two LPs collapse into one — same protocol, fewer hops (covered in `devkits/p2p-trading-ies-wave2`).

Customer PII and meter data stay with the customer's own discom and TP. Price stays between the two TPs only. Each ledger only sees the allocation and settled quantity for its own side and its counterparty.

10.9.2 Block diagram



Two regulated ledger services in the protocol — one per discom — held by the discom's contracted Ledger Provider. No central exchange. The two ledgers never speak to each other directly; the two TPs are the only liaison between them. Each TP keeps its own internal book of trades, but that is application state, not a separate Beckn participant.

10.9.3 Building blocks used

Block	Role in this use case
Identifiers and Addressing	Every actor (`buyer`, `seller`, two TPs, two LPs) is a `did:web` participant. Meters / DTs / feeders referenced in the trade reuse the existing IDs wrapped in `did:web` form (see SMDX § Adopt the did:web convention).
Registries and Directories	The four actors are subscribers referenced in the IES network registries. Public keys resolve through any DeDi runtime. The LP<->discom mapping is recorded in `DiscomLedger-Provider.participantAttributes.utilityId` on each trade.
Data Exchange	The wire. The same Beckn + ONIX stack carries every leg — trade-negotiation and DISCOM<->LP meter-data alike — as `DEGContract` inside `message.contract`, with `BecknTimeSeries` payloads (`accessMethod: INLINE`). The DDM `DatasetItem` envelope is not used in this use case.
Energy Credentials	The seller's `ElectricityCredential` attests to the meter, sanctioned-load, and DER details that back the offer.

The "Energy Trading" half of the work is a payload schema family and a policy bundle that sit on top of Data Exchange.

10.9.4 The dataset is a contract — `P2PTrade`

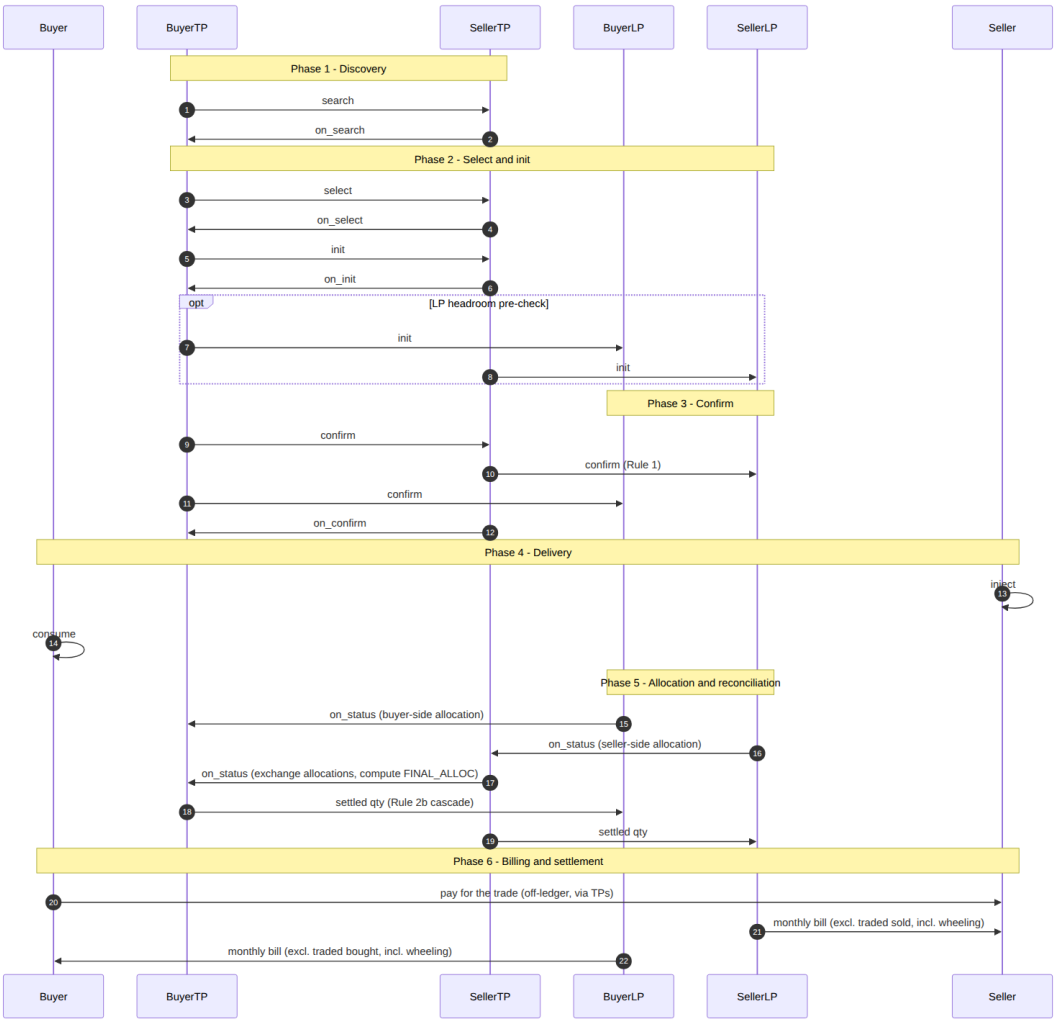
The Beckn message body wraps six DEG schemas published at `schema.beckn.io`:

Schema	What it carries	Source
`P2PTrade`	The contract `@type`: agreed quantity, price, delivery window, the four roles, the policy URL	DEG
`EnergyTradeOffer`	The seller's offer block: price per kWh, available quantity, validity window, source-type constraint (no GRID-sourced energy)	DEG
`EnergyTradeDelivery`	The performance block populated during reconciliation: per-interval `BUYER_DISCOM_ALLOC`, `SELLER_DISCOM_ALLOC`, and `FINAL_ALLOC`	DEG
`DEGContract`	The envelope: `roles[]` (buyer / seller / buyerDiscom / sellerDiscom -> participantIds), the rego policy URL, computed `revenueFlows`	DEG
`DiscomLedgerProvider`	The LP<->discom binding (`utilityId`, `ledgerUrl`)	DEG
`BecknTimeSeries`	The `commitmentAttributes` carrier — declares `payloadDescriptors` (`PRICE_PER_KWH` in INR, `AVAILABLE_QTY` / `REQUESTED_QTY` / `*_ALLOC` in kWh) and per-interval `payloads[]`	DEG

Trade contracts ride **inline** inside the Beckn `on_confirm` callback in `message.contract.commitments[].resources[].resourceAtt` qualified with the DEG `DEGContract` context. The DDM `DatasetItem` envelope is not used anywhere in this use case; the contract block carries every payload directly — trade-negotiation and DISCOM<->LP meter-quantity legs alike, both as `BecknTimeSeries` with the appropriate payloadType vocabulary.

10.9.5 The six phases

This is the inter-discom flow. Intra-discom (buyer and seller behind the same discom) collapses Phase 2's optional limit check and Phase 5 into a single ledger.



The cascade choreography in Phases 3 and 5 — `Rule 1` (seller-side ledger record on `/confirm`), `Rule 2a` (peer-TP forward), `Rule 2b` (own-discom cascade) — is implemented by the `degledgerrecorder` ONIX plugin. You configure it; you do not write it. Full design and the loop-free proof live in the DEG devkit README.

The allocation logic on each LP can be as simple as **pro-rata across the customer's trades in the delivery window**. The same Phase-5 message flow supports multiple rounds — e.g., a provisional allocation, a final allocation after meter-data finalisation, or a deviation true-up — by repeating the `/status` round-trip with a fresh `BecknTimeSeries` payload. Iteration is a payload concern, not a protocol concern.

10.9.6 Ledger interfaces

Each LP exposes the same Beckn endpoints any BPP exposes:

- `/bpp/receiver` — accepts `/confirm` (contract entry) and `/status` (meter-data sub-transactions).
- `/bap/receiver` — accepts `/on_confirm` and `/on_status` callbacks from the TPs.
- `/bap/caller` — emits `/on_status` callbacks (e.g. allocation updates) to the discom actor.

Authentication is the standard Beckn signing flow against the network registry. Every leg in the cascade rewrites `context.bapId` / `bappUri` / `bppId` / `bppUri` so each ledger leg is correctly identified end-to-end; nothing custom is required of the implementer beyond the ONIX config blocks the devkit ships.

10.9.7 Setup steps

Mirrors the Data Exchange Quick Start. Everything below assumes you've already followed Data Exchange — Checklist Stages 0–2 (devkit running, DeDi subscriber record, signing key).

1. Stand up the wave 2 devkit

```
git clone https://github.com/beckn/DEG.git
cd DEG/devkits/p2p-trading-ies-wave2/install
docker compose up -d
```

This starts the BAP-side (`onix-buyerapp`, `sandbox-buyerapp`, `onix-ledger-buyerdiscom`, `onix-buyerdiscom`) and BPP-side (`onix-sellerapp`, `sandbox-sellerapp`, `onix-ledger-sellerdiscom`, `onix-sellerdiscom`) stacks, bridged by one `beckn-router` (Caddy) on `:9000` resolving each actor by per-node hostname (e.g. `seller-discom-ledger.example.com`).

2. Drive the flow via Postman

Four collections sit under `uc1/postman/` — one per role (buyer TP, seller TP, buyer discom ledger, seller discom ledger). Import the role you are integrating, leave the defaults in place, and fire `/select` -> `/init` -> `/confirm` -> `/status` from there. The full Phase 1–5 lifecycle is covered by the role-specific requests in those collections.

3. Swap in your real identity

Same procedure as Data Exchange: register your TP (or LP) as a DeDi subscriber under your namespace, point the ONIX adapter's `allowedNetworkIDs`, `networkParticipant`, and `keyId` at your real identity, and replace the sandbox containers with your TP / LP application. The full path is Data Exchange -> Registry Setup.

4. Map your application logic to the four roles

If you are a …	You implement	Talks to
Trading-platform vendor (BAP or BPP)	Your matching engine + the ONIX BAP/BPP wiring	The peer TP (Beckn `/select`–`/status`) + your own LP (Beckn `/confirm`, `/status`)
LP for one or more discoms	Your ledger app behind a Beckn BPP+BAP	Both TPs you serve + the discom actor (meter-data sub-tx)
Discom (utility)	A thin Beckn BPP that emits meter data for your LPs	Your contracted LP only

The DEG devkit ships sandbox implementations of all four — replace one at a time as your real component matures.

10.9.8 Policy-as-code (Rego / OPA)

Every `DEGContract` carries a **`policyUrl`**. That URL points to a Rego policy bundle hosted on a DeDi runtime and digitally signed. **Two distinct rego bundles** apply, both enforceable offline by any participant.

The IES network mandates which policy bundles are in force on a given network and **publishes them as policy-as-code records on a DeDi runtime** (`dedi.global` or any DeDi-compatible host). DeDi serves

the same role for policy that it does for keys: a trusted, verifiable, version-controlled source. A participant fetches the bundle, evaluates locally with OPA, and the answer is cryptographically attributable to the published version. There is no central policy server to call out to at trade time.

Network policy

Enforces "is this a valid trade on this network at all?" Examples drawn from `p2p-trading-ies-wave2_network.rego`:

Rule	What it checks
Roles	The four required roles (`buyer`, `seller`, `buyerDiscom`, `sellerDiscom`) are present and each maps to a known `utilityId`; buyer's discom seller's discom for inter-discom trades.
No self-trade	Buyer and seller meter IDs are different.
Generation source	Offer's `sourceType` is not `GRID` (network admits only DER-sourced energy).
TimeSeries shape	`PRICE_PER_KWH` is denominated in INR; `AVAILABLE_QTY` / `REQUESTED_QTY` in kWh; every `payloadType` used in `payloads[]` is declared in `payloadDescriptors`.
Context alignment	`context.bppId` / `context.bapId` match the participantIds the payload claims.
Performance integrity	`FINAL_ALLOC <= min(BUYER_DISCOM_ALLOC, SELLER_DISCOM_ALLOC)` per interval.
TEST / PROD separation	Test-network identifiers carry the `TEST_` prefix consistently; production networks check approved `utilityId`s only.

A network operator can change the rule set without recompiling code — publish a new bundle on DeDi, bump the `policyUrl` on the next contract, every participant resolves the new bundle on first use.

Settlement / invoicing policy

A second rego bundle (`p2p_trading_ies_wave2_revenue.rego`) computes the **`revenueFlows`** entry on each contract from the final allocation:

- Buyer pays `FINAL_ALLOC × PRICE_PER_KWH` (signed negative).
- Seller receives the same amount.
- BuyerDiscom and SellerDiscom collect wheeling charges (and any deviation penalty) — currently `0` placeholders pending the tariff rule plug-in.

The result lands in `message.contract.consideration[0].considerationAttributes` as a `RevenueFlow` JSON-LD object signed by the policy author. Settlement reconciliation reads from there.

Mandating the settlement bundle the same way as the network bundle keeps every participant computing the same answer from the same inputs. The wheeling charge a discom collects is no longer a bilateral spreadsheet — it is the output of a signed rego function over a signed contract.

10.9.9 Checklist

Work in progress — plan against the stable parts. Items are additions on top of the Data Exchange -> Checklist (do that first). Role: [ ] Buyer TP (BAP) [ ] Seller TP (BPP) [ ] Ledger Provider [ ] DISCOM.

1. Network identity (shared with Data Exchange) — reuse an existing Beckn identity, or follow Data Exchange -> Stage 2.

- [ ] DeDi subscriber record under the right network namespace
- [ ] Signing key in a secrets manager, never in config

2. Devkit running end-to-end — prove the four-actor topology and the Phase-3 cascade before integrating real systems.

- [ ] `devkits/p2p-trading-ies-wave2/install` up and healthy
- [ ] Role Postman collection: `/select` -> `/init` -> `/confirm` -> `/status` completes

3. Schemas mapped to the DEG family.

- [ ] `P2PTrade` + `DEGContract` envelope (four roles, policy URL)
- [ ] `EnergyTradeOffer` for your catalogue; `BecknTimeSeries` descriptors per `payloadType`
- [ ] (LP) `DiscomLedgerProvider` entry registered

4. `degledgerrecorder` cascade configured — configure the plugin (you don't write the cascade); edit only the `participants` block.

- [ ] Plugin enabled (`ledgerUriSource: payload`, `ledgerApi: beckn`)
- [ ] Rules 1, 2a, 2b verified on the devkit; no loops (every 2a path terminates at a discom)

5. Policy bundles resolved — both published on DeDi by the NFO; fetch, verify, evaluate locally.

- [ ] Network and settlement bundle URLs resolve and signatures verify
- [ ] Local OPA eval rejects rule-violating payloads and computes revenue flows; bundle version pinned per contract

6. DISCOM <-> LP meter-data sub-transaction (Phase 5) — meter quantities flow as `BecknTimeSeries` (not `MeterData` / `DatasetItem`).

- [ ] (DISCOM) meter quantities published to the LP as `BecknTimeSeries`
- [ ] (LP) meter-quantity payloadTypes wired into the allocation function

7. Production cut-over (shared with Data Exchange).

- [ ] HTTPS live; keys in a secrets manager; monitoring / alerting / on-call for your role
- [ ] One real trade completed and reconciled; runbook (key rotation, bundle upgrades, disputes)

8. Team. [ ] IT / data SPOC · [ ] Commercial / settlement SPOC · [ ] Authorised Signatory

---

10.9.10 References

- DEG devkit — `p2p-trading-ies-wave2` — code, examples, Postman collections
- Inter-discom energy trading implementation guide
- Full inter-discom specification
- `degledgerrecorder` plugin — the cascade engine
- Schemas on `schema.beckn.io` — `P2PTrade`, `DEGContract`, `EnergyTradeOffer`, `EnergyTradeDelivery`, `DiscomLedgerProvider`, `BecknTimeSeries`
- Sample bill-calculation worksheet

Chapter 11

Schemas

This directory holds the canonical schema files referenced from the IES Accelerator documentation. Each subdirectory mirrors a schema family that the accelerator's deployable services (OpenCred, ONIX, etc.) are expected to issue against.

Repo root: https://github.com/India-Energy-Stack/ies-accelerator

11.1 Why this directory exists

The IES Accelerator docs (`/energy-credentials/...`, `/data-exchange/...`) need stable, repo-relative links to the schema files they describe. Pointing at external sources directly creates two problems:

1. External URLs change shape or move (e.g. branch renames, repo reorganisation upstream).
2. JSON-LD `@context` URLs inside example credentials need to **resolve at runtime** for verifiers to validate; relying on external hosts couples our deployments to their uptime.

Mirroring the schema files into this repo gives us:

- Stable repo-relative paths (`/schemas/<family>/<version>/`) for documentation links.
- Stable `india-energy-stack.github.io/ies-accelerator/...` URLs (served by GitHub Pages from this repo) for `@context` resolution and external JSON-LD consumers.
- Offline-installable copies — DISCOMs running in air-gapped envs can sync this directory once and not need outbound HTTPS to a third-party host every time a credential is verified.

11.2 Provenance and updates

Every mirrored family carries a `README.md` at its root naming the upstream source and version. The mirror is verbatim — we do **not** edit the schema files in place. If upstream changes, the workflow is:

1. Re-run the fetch step against the new upstream version.
2. Place the new files in a sibling version directory (e.g. `v1.1/`, not overwriting `v1.0/`).
3. Update the family-level `README.md` to mark the previous version's status.

If you find a discrepancy between a mirrored file and the upstream source, treat upstream as authoritative and open an issue so the mirror can be refreshed.

11.3 Schema Families

Family	Latest	All Versions	Description
ElectricityCredential	v1.2	v1.0 · v1.1 · v1.2	W3C VC issued per meter by DISCOMs — customer identity, DERs, tariff profiles
MeterData	v0.6	v0.5 · v0.6	Smart meter telemetry compact profiles (8 profile shapes)
MeterDataCredential	v0.6	v0.6	W3C VC wrapping MeterData for provenance attestation
MeterDataRequest	v0.6	v0.5 · v0.6	Query schema: capabilities, authorisation, request
MeterDataRequestCredential	v0.1	v0.1	W3C VC wrapping MeterDataRequest for seeker authorisation
ArrFiling	v0.5	v0.5	DISCOM regulatory ARR filing — ArrFiling, ArrFiscalYear, ArrLineItem
OutageNotification	v0.1	v0.1	**WIP** — planned/unplanned outage notices for web/outage-map publishing + push to consumers

11.4 External Schemas

Some schemas used across IES are defined and published outside this repository (canonically at schema.beckn.io). Their field references are **auto-generated from the schema sources** (by `scripts/generate_external_field_tables.py`) and collected on one page so the book — GitBook and PDF — carries a complete reference:

- **External Schemas** — Energy Trading (P2P): `P2PTrade`, `EnergyTradeOffer`, `EnergyCustomer`, `EnergyOrderItem`, `RevenueFlow`, `DEGContract`, `EnergyResource`; Demand Flexibility: `DemandFlexNeed`, `DemandFlexBuyOffer`, `DemandFlexPerformance`.

11.5 ElectricityCredential

> **Mirror.** This directory is a verbatim mirror of the Beckn DEG `ElectricityCredential` specification, vendored into this repo so the IES accelerator docs can reference schema files at stable repo-relative paths.
>
> Canonical upstream source: `beckn/DEG/specification/schema/ElectricityCredential`. If you find a discrepancy between this mirror and upstream, treat upstream as authoritative and open an issue on this repo so the mirror can be refreshed.

Unified W3C Verifiable Credential (VC Data Model 2.0) issued per meter by electricity distribution utilities. Combines customer identity with optional consumption, generation, and storage profiles in a single credential.

Namespace prefix: `deg:` -> `https://schema.beckn.io/deg/`

Tags: `energy` · `credential` · `verifiable-credential` · `electricity` · `customer` · `deg`

11.5.1 Versions

Version	Status	Notes
v1.2	**Current**	Composable EnergyResource kinds (7 typed discriminants); directional power fields; EV charger kind
v1.1	Previous	Unified `energyResources[]` (EnergyResource/v2.0), multi-meter topology support
v1.0	Deprecated	Separate `consumptionProfiles[]`, `generationProfiles[]`, `storageProfiles[]` arrays

11.5.2 Inheritance

```
beckn:Credential
  └── EnergyCredential
        └── ElectricityCredential  <- this schema
```

11.5.3 credentialSubject Properties (v1.1)

Property	Type	Required	Description
`id`	`string` (URI)		Optional DID of the customer/credential subject
`customerProfile`	object	[x]	Non-PII: customer number, all energy resources, tariff profiles
`customerProfile.customerNumber`	string	[x]	Utility CA number
`customerProfile.energyResources`	array `[]`	[x]	All physical assets — meters, DERs, storage (EnergyResource/v2.0)

Property	Type	Required	Description
customerProfile.consumptionProfiles[]	array		Tariff/load profiles, linked to meters via meterId
customerDetails	object		PII — full name, installation address, connection date

11.5.4 Linked Data (v1.1)

Term	IRI
ElectricityCredential	deg:ElectricityCredential
customerProfile	deg:customerProfile
customerDetails	deg:customerDetails
energyResources	deg:energyResources
consumptionProfiles	deg:consumptionProfiles
meterId	deg:meterId

11.5.5 v1.0 -> v1.1 Migration Summary

v1.0	v1.1
customerProfile.meterNumber	energyResources[METER].id
customerProfile.meterType	energyResources[METER].attributes.meterType
generationProfiles[].assetId	energyResources[DER].id
generationProfiles[].capacityKW	energyResources[DER].attributes.ratedPowerKw
generationProfiles[].manufacturer	energyResources[DER].attributes.make
storageProfiles[].storageCapacityKWh	energyResources[DER].attributes.energyCapacityKwh
storageProfiles[].storageType	energyResources[DER].attributes.storageType
fullName duplicated per profile entry	customerDetails.fullName (once only)

See v1.1/README.md for the full migration table.

11.5.6 Usage

Issued by electricity distribution utilities to consumers and prosumers. Used for: - P2P energy trading eligibility and identity verification - Demand response and virtual power plant enrollment - DER asset registration and grid service qualification - Program enrollment and tariff management

11.5.7 v1.2

Files — attributes.yaml · schema.json · context.jsonld · vocab.jsonld · examples/

Mirror. Files in this directory are mirrored verbatim from `beckn/DEG/specification/schema/ElectricityCredential/v1.`
Upstream is canonical.

11.5.8 ElectricityCredential v1.2

W3C Verifiable Credential (VC Data Model 2.0) issued per meter by electricity distribution utilities.

v1.2 introduces a **composable EnergyResource hierarchy**: each entry in `energyResources[]` is discriminated by `type` into one of seven typed kinds, each with a typed `attributes` bag. All power and capacity fields use **QuantitativeValue `{value, unit}`** with short unit aliases (`kW`, `kWh`, `kVA`, `kVAR`, `kV`, `MW`, `MWh`, `MVA`, `MVAR`, `V`, `W`) mapped to QUDT IRIs via JSON-LD context.

Structure

```
credentialSubject
├── id                          (optional — customer DID)
├── customerProfile             (required — non-PII)
│   ├── customerNumber          (required — CA number)
│   ├── idRef                   (optional — external identity reference)
│   ├── energyResources[]       (required — all physical assets, min 1)
│   │   ├── id                  (meter serial for METER; stable id for DERs)
│   │   ├── type                (discriminator — see kinds below)
│   │   ├── attributes          (kind-specific bag inheriting EnergyResourceCommonAttributes)
│   │   ├── subResources[]      (child resource ids or inline objects)
│   │   └── parentResources[]   (parent resource ids — e.g. the meter a DER sits behind)
│   └── consumptionProfiles[]   (optional — tariff/load per meter, linked via meterId)
└── customerDetails             (optional — PII)
    ├── fullName                (PII — only here)
    ├── installationAddress
    └── serviceConnectionDate
```

EnergyResource kinds

Kind	`type` values	CIM class (IEC 61970/61968)
EnergyResourceMeter	METER	`cim:Meter` / `cim:EndDevice` (IEC 61968-9)
EnergyResourceGenerator	SOLAR_PV, WIND, HYDRO, BIOGAS, CHP, FUEL_CELL	`cim:GeneratingUnit` subtypes (IEC 61970-302)
EnergyResourceBess	BESS	`cim:BatteryUnit` (IEC 61970-302)
EnergyResourceEvCharger	EV_CHARGER, EV_V2G	`cim:ElectricVehicleChargingStation` (CIM17+)
EnergyResourceInverter	INVERTER	`cim:PowerElectronicsConnection` (IEC 61970-302)
EnergyResourceLoad	SMART_HVAC, SMART_WATER_HEATER, CONTROLLABLE_LOAD	`cim:EnergyConsumer` / `cim:ConformLoad` (IEC 61970-301)
EnergyResourceDistBus	DT, BUS, FEEDER, MICROGRID	`cim:PowerTransformer`, `cim:BusbarSection`, `cim:Feeder`, `cim:Substation` (IEC 61970-301)

Deprecated type aliases (still valid for backward compatibility): `SOLAR` -> use `SOLAR_PV`; `BATTERY` -> use `BESS`.

v1.1 -> v1.2 migration

Change	v1.1 (scalar)	v1.2 (QuantitativeValue)
Power fields	`ratedPowerKw`, `maxExportKw`, `maxImportKw` (number)	`ratedPower`, `maxExport`, `maxImport` (`{value, unit: W|kW|MW}`)
Generator nameplate	`nominalPowerKw` (number)	`nominalPower` (`{value, unit: W|kW|MW}`)
Storage capacity	`storageCapacityKwh` (number)	`storageCapacity` (`{value, unit: kWh|MWh}`)
Apparent power	`ratedApparentPowerKva` (number)	`ratedApparentPower` (`{value, unit: kVA|MVA}`)
Reactive power	`maxReactivePowerKvar`, `minReactivePowerKvar` (number)	`maxReactivePower`, `minReactivePower` (`{value, unit: kVAR|MVAR}`)
Network voltage	`nominalVoltageKv` (number)	`nominalVoltage` (`{value, unit: V|kV}`)
Tariff load	`sanctionedLoadKw`, `contractMaxDemandKw` (number)	`sanctionedLoad`, `contractMaxDemand` (`{value, unit: W|kW|MW}`)

Files

File	Description
`attributes.yaml`	OpenAPI 3.1.1 schema with composable EnergyResource hierarchy
`schema.json`	Bundled JSON Schema (draft 2020-12) — self-contained
`context.jsonld`	JSON-LD context with unit aliases (kW->qudt:KiloW, kWh->qudt:KiloW-HR, etc.)
`vocab.jsonld`	RDF vocabulary with CIM class alignments
`examples/example.json`	Single meter + SOLAR_PV + WIND + 2× BESS
`examples/example-submetering.json`	Building main meter + 2 tenant sub-meters + rooftop solar
`examples/example-parallel-metering.json`	Import meter + export meter (solar FIT)

For full documentation see the upstream README.

Field reference

*Auto-generated from `schema.json`. A field name in **bold** with a trailing * is required; all others are optional. **Type** shows units for QuantitativeValue models. Where a field is derived from a standard, its description begins with **Based on** and the standard reference (from the field's `x-standard` annotation). One table per object.*

ElectricityCredential v1.2

Field	Type	Description
`id` *	uri	—
`type` *	list of text	—
`issuer` *	object	—
`validFrom` *	date-time	—
`validUntil`	date-time	—
`credentialStatus`	object	—
`credentialSubject` *	object	—
`proof`	object	—

CustomerDetails

Field	Type	Description
`fullName` *	text	**Based on** CIM (IEC 61968-1 Customer.name). Full name of the customer as per ID proof.
`careOf`	object	Care-of (c/o) reference person used to uniquely identify / disambiguate a customer who may share the same fullName with others in a locality (e.g. a village). Pairs a name with an optional, gender-neutral relationship.
`installationAddress` *	Location	**Based on** GeoJSON RFC 7946; schema.org PostalAddress; CIM (IEC 61968-1 ServiceLocation). Physical location of the metered installation. geo (GeoJSON Point, coordinates [longitude, latitude]) is required; address (schema.org PostalAddress fields) is optional.
`serviceConnectionDate` *	date-time	**Based on** CIM (IEC 61968-1 ServiceLocation activation date). Date and time the service connection was activated, with timezone offset (ISO 8601).

Location

Field	Type	Description
`geo` *	GeoJSONGeometry	—
`address`	Address	—

GeoJSONGeometry

Field	Type	Description
`bbox`	list of number	Optional bounding box `[west, south, east, north]` in degrees.
`coordinates`	list of —	Coordinates per RFC 7946 for all types **except** GeometryCollection. Order is **[lon, lat, (alt)]**. For Polygons, this is an array of linear rings; each ring is an array of positions.
`geometries`	list of GeoJSONGeometry	Member geometries when `type` is **GeometryCollection**.
`type` *	`Point` / `LineString` / `Polygon` / `MultiPoint` / `MultiLineString` / `MultiPolygon` / `GeometryCollection`	—

Address

Field	Type	Description
`addressCountry`	text	Country name or ISO-3166-1 alpha-2 code.
`addressLocality`	text	City/locality.
`addressRegion`	text	State/region/province.
`extendedAddress`	text	Address extension (apt/suite/floor, C/O).
`postalCode`	text	Postal/ZIP code.
`streetAddress`	text	Street address (building name/number and street).

CustomerProfile

Field	Type	Description
`customerNumber` *	text	Utility customer account (CA) number.
`idRef`	IdRef	—
`energyResources` *	list of EnergyResource	All physical energy assets for this account. Each entry is discriminated by 'type' into one of seven composable kinds.
`consumptionProfiles`	list of ConsumptionProfile	Tariff and load characteristics per meter connection. Each entry links to a METER via meterId.

IdRef

Field	Type	Description
`issuedBy` *	uri	DID or URI of the issuing authority.
`subjectId` *	text	Subject identifier in authority-domain:id-value format.

EnergyResourceMeter

Field	Type	Description
id *	text	Stable identifier for this resource. For METER resources the meter serial number is conventional.
type *	text	Asset-class discriminator. Must be “METER”. CIM cim:Meter (IEC 61968-9).
subResources	list of text or EnergyResource	Topology — child resources. Each item is EITHER a bare id string OR an inline EnergyResource object.
parentResources	list of text	Upward topology — ids of parent resources.
attributes	EnergyResourceMeterAttributes	—

EnergyResourceCommon

Field	Type	Description
id	text	Stable identifier for this resource. For METER resources the meter serial number is conventional.
type	text	Asset class discriminator. Constrained to a kind-specific enum or const in each typed kind schema.
subResources	list of text or EnergyResource	Topology — child resources. Each item is EITHER a bare id string OR an inline EnergyResource object.
parentResources	list of text	Upward topology — ids of parent resources.
attributes	EnergyResourceCommonAttributes	Attribute bag. Inherits EnergyResourceCommonAttributes via allOf plus kind-specific fields.

EnergyResourceCommonAttributes

Field	Type	Description
make	text	Manufacturer (free text).
model	text	Model (free text).
ratedPower	QVPower (W / kW / MW)	**Based on** CIM (IEC 61968-9 EndDeviceInfo.ratedPower; IEC 61970 GeneratingUnit.maxOperatingP). Manufacturer-rated peak power (nameplate value in principal direction). Kept for backward compatibility — prefer maxExport.
maxExport	QVPower (W / kW / MW)	**Based on** CIM (IEC 61970 GeneratingUnit.maxOperatingP; IEC 61970-302 PowerElectronicsConnection.maxP, injection). Maximum power this resource injects to the grid (generates/discharges). Always >=0. For bidirectional resources (BESS, V2G) this is the max discharge rate. Supersedes ratedPower.
maxImport	QVPower (W / kW / MW)	**Based on** CIM (IEC 61970-302 PowerElectronicsConnection.maxP, absorption). Maximum power this resource draws from the grid (absorbs/charges). Always >=0. For bidirectional resources (BESS, V2G) this is the max charge rate.
telemetryProvider	text	Vendor API / data source identifier for telemetry.
commissioningDate	date-time	ISO 8601 date-time the asset was commissioned.
location	Location	Physical location of this asset.
serialNumber	text	**Based on** CIM (IEC 61968-9 EndDeviceInfo.serialNumber). Manufacturer-assigned device serial number from the equipment nameplate. Distinct from id (which is the network-issued DID).
inspection	object	**Based on** IEEE 1547-2018 Cl. 11 (commissioning); CEA Connectivity Regs 2013 (amended 2018). Commissioning / safety inspection record for the asset. Captured by the distribution licensee at energisation and on re-certification events.
aggregator	object	**Based on** IEEE 2030.5; IEC 61850-7-420 (DER control roles). Third-party flexibility / demand-response enrolment for this asset. Present when an aggregator is authorised to dispatch or observe the resource. Controllability flag is asset-level; the asset may still be observable even when controllable is false.

QVPower

Field	Type	Description
value *	number	—
unit *	W / kW / MW	—

EnergyResourceMeterAttributes

Field	Type	Description
make	text	Manufacturer (free text).
model	text	Model (free text).
ratedPower	QVPower (W / kW / MW)	**Based on** CIM (IEC 61968-9 EndDeviceInfo.ratedPower; IEC 61970 GeneratingUnit.maxOperatingP). Manufacturer-rated peak power (nameplate value in principal direction). Kept for backward compatibility — prefer maxExport.
maxExport	QVPower (W / kW / MW)	**Based on** CIM (IEC 61970 GeneratingUnit.maxOperatingP; IEC 61970-302 PowerElectronicsConnection.maxP, injection). Maximum power this resource injects to the grid (generates/discharges). Always >=0. For bidirectional resources (BESS, V2G) this is the max discharge rate. Supersedes ratedPower.
maxImport	QVPower (W / kW / MW)	**Based on** CIM (IEC 61970-302 PowerElectronicsConnection.maxP, absorption). Maximum power this resource draws from the grid (absorbs/charges). Always >=0. For bidirectional resources (BESS, V2G) this is the max charge rate.
telemetryProvider	text	Vendor API / data source identifier for telemetry.
commissioningDate	date-time	ISO 8601 date-time the asset was commissioned.
location	Location	Physical location of this asset.
serialNumber	text	**Based on** CIM (IEC 61968-9 EndDeviceInfo.serialNumber). Manufacturer-assigned device serial number from the equipment nameplate. Distinct from id (which is the network-issued DID).
inspection	object	**Based on** IEEE 1547-2018 Cl. 11 (commissioning); CEA Connectivity Regs 2013 (amended 2018). Commissioning / safety inspection record for the asset. Captured by the distribution licensee at energisation and on re-certification events.
aggregator	object	**Based on** IEEE 2030.5; IEC 61850-7-420 (DER control roles). Third-party flexibility / demand-response enrolment for this asset. Present when an aggregator is authorised to dispatch or observe the resource. Controllability flag is asset-level; the asset may still be observable even when controllable is false.
meterCapability	Electromechanical / CMRI / AMR / AMI	**Based on** CIM (IEC 61968-9 AmiBillingReadyKind). Communication/automation generation. Electromechanical: induction-disc. CMRI: manual optical-port (India legacy). AMR: one-way automated read. AMI: two-way smart meter.
energyDirection	Forward / Reverse / Bidirectional / Net	**Based on** CIM (FlowDirectionKind); ESPI NAESB REQ.21. Energy flow direction metered at this point.
functions	list of ToU / NetMetering / MaxDemand / LoadControl / TamperDetection / PowerQuality / EventLogging	**Based on** CIM (IEC 61968-9 EndDeviceFunction). Active meter capabilities.
feeder	text	Feeder identifier this meter is supplied from.
bus	text	Busbar identifier at the meter's connection point.
communicationTechnology	PLC / RF_Mesh / GPRS / NB-IoT / LoRa / ZigBee / Other	Last-mile physical-layer communication technology.
applicationProtocol	DLMS_COSEM / ANSI_C12_18 / IEC_61850 / Modbus / Other	Application-layer protocol for meter data. DLMS_COSEM: IEC 62056, mandatory for India AMI per BIS IS 16444. ANSI_C12_18: North American. Orthogonal to communicationTechnology (physical layer).

EnergyResourceGenerator

Field	Type	Description
id *	text	Stable identifier for this resource. For METER resources the meter serial number is conventional.
type *	SOLAR_PV / SOLAR / WIND / HYDRO / BIOGAS / CHP / FUEL_CELL	Asset-class discriminator. SOLAR is deprecated — use SOLAR_PV.
subResources	list of text or EnergyResource	Topology — child resources. Each item is EITHER a bare id string OR an inline EnergyResource object.
parentResources	list of text	Upward topology — ids of parent resources.
attributes	EnergyResourceGeneratorAttributes	—

EnergyResourceGeneratorAttributes

Field	Type	Description
make	text	Manufacturer (free text).
model	text	Model (free text).
ratedPower	QVPower (W / kW / MW)	**Based on** CIM (IEC 61968-9 EndDeviceInfo.ratedPower; IEC 61970 GeneratingUnit.maxOperatingP). Manufacturer-rated peak power (nameplate value in principal direction). Kept for backward compatibility — prefer maxExport.

Field	Type	Description
maxExport	QVPower (W / kW / MW)	**Based on** CIM (IEC 61970 GeneratingUnit.maxOperatingP; IEC 61970-302 PowerElectronicsConnection.maxP, injection). Maximum power this resource injects to the grid (generates/discharges). Always >=0. For bidirectional resources (BESS, V2G) this is the max discharge rate. Supersedes ratedPower.
maxImport	QVPower (W / kW / MW)	**Based on** CIM (IEC 61970-302 PowerElectronicsConnection.maxP, absorption). Maximum power this resource draws from the grid (absorbs/charges). Always >=0. For bidirectional resources (BESS, V2G) this is the max charge rate.
telemetryProvider	text	Vendor API / data source identifier for telemetry.
commissioningDate	date-time	ISO 8601 date-time the asset was commissioned.
location	Location	Physical location of this asset.
serialNumber	text	**Based on** CIM (IEC 61968-9 EndDeviceInfo.serialNumber). Manufacturer-assigned device serial number from the equipment nameplate. Distinct from id (which is the network-issued DID).
inspection	object	**Based on** IEEE 1547-2018 Cl. 11 (commissioning); CEA Connectivity Regs 2013 (amended 2018). Commissioning / safety inspection record for the asset. Captured by the distribution licensee at energisation and on re-certification events.
aggregator	object	**Based on** IEEE 2030.5; IEC 61850-7-420 (DER control roles). Third-party flexibility / demand-response enrolment for this asset. Present when an aggregator is authorised to dispatch or observe the resource. Controllability flag is asset-level; the asset may still be observable even when controllable is false.
nominalPower	QVPower (W / kW / MW)	**Based on** CIM (IEC 61970 GeneratingUnit.nominalP). Nominal (nameplate) power output. Use when distinct from maxExport (peak).
efficiency	number	Conversion efficiency as a percentage (0–100). Most relevant for FUEL_CELL and CHP resources.
dcArrayCapacity	QVPower (W / kW / MW)	**Based on** IS 16221 (PV module qualification); IEC 61727 (PV grid interface). DC-side nameplate capacity of a photovoltaic array at Standard Test Conditions (industry term: "kWp"). For PV systems this is typically larger than the AC-side maxExport because of inverter clipping and DC-to-AC ratios. Relevant for SOLAR_PV resources. The unit is the standard QUDT power alias kW — the STC/peak semantic is documented here, not encoded in the unit string.

EnergyResourceStorage

Field	Type	Description
id *	text	Stable identifier for this resource. For METER resources the meter serial number is conventional.
type *	BESS / BATTERY	Asset-class discriminator. BATTERY is deprecated — use BESS. CIM: BatteryUnit (IEC 61970-302).
subResources	list of text or EnergyResource	Topology — child resources. Each item is EITHER a bare id string OR an inline EnergyResource object.
parentResources	list of text	Upward topology — ids of parent resources.
attributes	EnergyResourceStorageAttributes	—

EnergyResourceStorageAttributes

Field	Type	Description
make	text	Manufacturer (free text).
model	text	Model (free text).
ratedPower	QVPower (W / kW / MW)	**Based on** CIM (IEC 61968-9 EndDeviceInfo.ratedPower; IEC 61970 GeneratingUnit.maxOperatingP). Manufacturer-rated peak power (nameplate value in principal direction). Kept for backward compatibility — prefer maxExport.
maxExport	QVPower (W / kW / MW)	**Based on** CIM (IEC 61970 GeneratingUnit.maxOperatingP; IEC 61970-302 PowerElectronicsConnection.maxP, injection). Maximum power this resource injects to the grid (generates/discharges). Always >=0. For bidirectional resources (BESS, V2G) this is the max discharge rate. Supersedes ratedPower.
maxImport	QVPower (W / kW / MW)	**Based on** CIM (IEC 61970-302 PowerElectronicsConnection.maxP, absorption). Maximum power this resource draws from the grid (absorbs/charges). Always >=0. For bidirectional resources (BESS, V2G) this is the max charge rate.
telemetryProvider	text	Vendor API / data source identifier for telemetry.
commissioningDate	date-time	ISO 8601 date-time the asset was commissioned.
location	Location	Physical location of this asset.
serialNumber	text	**Based on** CIM (IEC 61968-9 EndDeviceInfo.serialNumber). Manufacturer-assigned device serial number from the equipment nameplate. Distinct from id (which is the network-issued DID).

Field	Type	Description
`inspection`	object	**Based on** IEEE 1547-2018 Cl. 11 (commissioning); CEA Connectivity Regs 2013 (amended 2018). Commissioning / safety inspection record for the asset. Captured by the distribution licensee at energisation and on re-certification events.
`aggregator`	object	**Based on** IEEE 2030.5; IEC 61850-7-420 (DER control roles). Third-party flexibility / demand-response enrolment for this asset. Present when an aggregator is authorised to dispatch or observe the resource. Controllability flag is asset-level; the asset may still be observable even when controllable is false.
`storageCapacity`	QVEnergy (`kWh` / `MWh`)	**Based on** CIM (IEC 61970-302 BatteryUnit.ratedE). Rated stored-energy capacity. Replaces storageCapacityKwh from v1.0.
`storageType`	`LithiumIon` / `LeadAcid` / `FlowBattery` / `NaS` / `NiCd` / `Flywheel` / `Other`	Battery storage technology type.
`stateOfHealthPct`	number	Battery state-of-health as a percentage (0–100).
`roundTripEfficiencyPct`	number	**Based on** IEC 62933-2-1 (performance test method). AC-to-AC round-trip efficiency as a percentage (0–100): the fraction of energy returned to the grid relative to energy drawn during a full charge/discharge cycle. Distinct from stateOfHealthPct (cumulative life indicator) and from inverter conversion efficiency.

QVEnergy

Field	Type	Description
`value` *	number	—
`unit` *	`kWh` / `MWh`	—

EnergyResourceEVCharger

Field	Type	Description
`id` *	text	Stable identifier for this resource. For METER resources the meter serial number is conventional.
`type` *	`EV_CHARGER` / `EV_V2G`	Asset-class discriminator. EV_CHARGER: EVSE hardware. EV_V2G: Vehicle-to-Grid capable EVSE with ISO 15118-20 / OCPP 2.1 BPT.
`subResources`	list of text or EnergyResource	Topology — child resources. Each item is EITHER a bare id string OR an inline EnergyResource object.
`parentResources`	list of text	Upward topology — ids of parent resources.
`attributes`	EnergyResourceEVChargerAttributes	

EnergyResourceEVChargerAttributes

Field	Type	Description
`make`	text	Manufacturer (free text).
`model`	text	Model (free text).
`ratedPower`	QVPower (`W` / `kW` / `MW`)	**Based on** CIM (IEC 61968-9 EndDeviceInfo.ratedPower; IEC 61970 GeneratingUnit.maxOperatingP). Manufacturer-rated peak power (nameplate value in principal direction). Kept for backward compatibility — prefer maxExport.
`maxExport`	QVPower (`W` / `kW` / `MW`)	**Based on** CIM (IEC 61970 GeneratingUnit.maxOperatingP; IEC 61970-302 PowerElectronicsConnection.maxP, injection). Maximum power this resource injects to the grid (generates/discharges). Always >=0. For bidirectional resources (BESS, V2G) this is the max discharge rate. Supersedes ratedPower.
`maxImport`	QVPower (`W` / `kW` / `MW`)	**Based on** CIM (IEC 61970-302 PowerElectronicsConnection.maxP, absorption). Maximum power this resource draws from the grid (absorbs/charges). Always >=0. For bidirectional resources (BESS, V2G) this is the max charge rate.
`telemetryProvider`	text	Vendor API / data source identifier for telemetry.
`commissioningDate`	date-time	ISO 8601 date-time the asset was commissioned.
`location`	Location	Physical location of this asset.
`serialNumber`	text	**Based on** CIM (IEC 61968-9 EndDeviceInfo.serialNumber). Manufacturer-assigned device serial number from the equipment nameplate. Distinct from id (which is the network-issued DID).
`inspection`	object	**Based on** IEEE 1547-2018 Cl. 11 (commissioning); CEA Connectivity Regs 2013 (amended 2018). Commissioning / safety inspection record for the asset. Captured by the distribution licensee at energisation and on re-certification events.

Field	Type	Description
aggregator	object	**Based on** IEEE 2030.5; IEC 61850-7-420 (DER control roles). Third-party flexibility / demand-response enrolment for this asset. Present when an aggregator is authorised to dispatch or observe the resource. Controllability flag is asset-level; the asset may still be observable even when controllable is false.
connectorType	Type1 / Type2 / CCS1 / CCS2 / CHAdeMO / GB_T / NACS / Other	Physical connector standard. Type1: IEC 62196-2 Type 1 (J1772). Type2: IEC 62196-2 Type 2 (Mennekes). CCS1/CCS2: Combined Charging System DC fast charge. CHAdeMO: CHAdeMO DC fast charge. GB_T: GB/T 20234. NACS: SAE J3400.
controlProtocol	OCPP_1.6 / OCPP_2.0.1 / OCPP_2.1 / ISO_15118_2 / ISO_15118_20 / Other	EVSE control and smart-charging protocol.
v2xProtocol	CHAdeMO_V2G / CCS_BPT / ISO_15118_20_AC_BPT / ISO_15118_20_DC_BPT / Other	Vehicle-to-Grid / V2X protocol. Present only for EV_V2G resources.

EnergyResourceInverter

Field	Type	Description
id *	text	Stable identifier for this resource. For METER resources the meter serial number is conventional.
type *	text	Asset-class discriminator. Must be “INVERTER”. CIM: PowerElectronicsConnection (IEC 61970-302).
subResources	list of text or EnergyResource	Topology — child resources. Each item is EITHER a bare id string OR an inline EnergyResource object.
parentResources	list of text	Upward topology — ids of parent resources.
attributes	EnergyResourceInverterAttributes	—

EnergyResourceInverterAttributes

Field	Type	Description
make	text	Manufacturer (free text).
model	text	Model (free text).
ratedPower	QVPower (W / kW / MW)	**Based on** CIM (IEC 61968-9 EndDeviceInfo.ratedPower; IEC 61970 GeneratingUnit.maxOperatingP). Manufacturer-rated peak power (nameplate value in principal direction). Kept for backward compatibility — prefer maxExport.
maxExport	QVPower (W / kW / MW)	**Based on** CIM (IEC 61970 GeneratingUnit.maxOperatingP; IEC 61970-302 PowerElectronicsConnection.maxP, injection). Maximum power this resource injects to the grid (generates/discharges). Always >=0. For bidirectional resources (BESS, V2G) this is the max discharge rate. Supersedes ratedPower.
maxImport	QVPower (W / kW / MW)	**Based on** CIM (IEC 61970-302 PowerElectronicsConnection.maxP, absorption). Maximum power this resource draws from the grid (absorbs/charges). Always >=0. For bidirectional resources (BESS, V2G) this is the max charge rate.
telemetryProvider	text	Vendor API / data source identifier for telemetry.
commissioningDate	date-time	ISO 8601 date-time the asset was commissioned.
location	Location	Physical location of this asset.
serialNumber	text	**Based on** CIM (IEC 61968-9 EndDeviceInfo.serialNumber). Manufacturer-assigned device serial number from the equipment nameplate. Distinct from id (which is the network-issued DID).
inspection	object	**Based on** IEEE 1547-2018 Cl. 11 (commissioning); CEA Connectivity Regs 2013 (amended 2018). Commissioning / safety inspection record for the asset. Captured by the distribution licensee at energisation and on re-certification events.
aggregator	object	**Based on** IEEE 2030.5; IEC 61850-7-420 (DER control roles). Third-party flexibility / demand-response enrolment for this asset. Present when an aggregator is authorised to dispatch or observe the resource. Controllability flag is asset-level; the asset may still be observable even when controllable is false.
ratedApparentPower	QVApparentPower (kVA / MVA)	**Based on** SunSpec DER Model 702 (maxVA); CIM (IEC 61970-302 PowerElectronicsConnection.ratedS). Rated apparent power. Replaces ratedApparentPowerKva from v1.0.
maxReactivePower	QVReactivePower (kVAR / MVAR)	**Based on** SunSpec DER Model 702 (maxVar); CIM (IEC 61970-302 PowerElectronicsConnection.maxQ). Maximum reactive power injection (leading / over-excited). Replaces maxReactivePowerKvar from v1.0.
minReactivePower	QVReactivePower (kVAR / MVAR)	**Based on** SunSpec DER Model 702 (maxVarNeg); CIM (IEC 61970-302 PowerElectronicsConnection.minQ). Maximum reactive power absorption (lagging / under-excited). Value is typically negative. Replaces minReactivePowerKvar from v1.0.

Field	Type	Description
rideThroughCategory	CategoryI / CategoryII / CategoryIII	**Based on** IEEE 1547-2018 (ride-through category). Abnormal operating performance category. CategoryI: basic. CategoryII: enhanced for distribution-connected DER. CategoryIII: advanced for large/transmission-connected DER.
operatingMode	GridFollowing / GridForming / Standby	**Based on** CIM (IEC 61970-302 PowerElectronicsConnection.inverterMode). Inverter grid-interaction mode. GridFollowing: PLL-based sync. GridForming: own V/f reference (microgrid islanding, black-start). Standby: energised but not injecting.
voltVarEnabled	yes / no	**Based on** IEEE 2030.5 (opModVoltVar); SunSpec DER Model 705. Volt-VAr curve active.
freqDroopEnabled	yes / no	**Based on** IEEE 1547-2018; SunSpec DER Model 711. Frequency-Watt droop active.
enterServiceRampTimeSec	number	**Based on** SunSpec DER Model 703 (ESRmpTms). Seconds to ramp from 0 to rated power after reconnection.

QVApparentPower

Field	Type	Description
value *	number	—
unit *	kVA / MVA	—

QVReactivePower

Field	Type	Description
value *	number	—
unit *	kVAR / MVAR	—

EnergyResourceLoad

Field	Type	Description
id *	text	Stable identifier for this resource. For METER resources the meter serial number is conventional.
type *	SMART_HVAC / SMART_WATER_HEATER / CONTROLLABLE_LOAD	Asset-class discriminator. CIM: EnergyConsumer / ConformLoad (IEC 61970-301).
subResources	list of text or EnergyResource	Topology — child resources. Each item is EITHER a bare id string OR an inline EnergyResource object.
parentResources	list of text	Upward topology — ids of parent resources.
attributes	EnergyResourceLoadAttributes	—

EnergyResourceLoadAttributes

Field	Type	Description
make	text	Manufacturer (free text).
model	text	Model (free text).
ratedPower	QVPower (W / kW / MW)	**Based on** CIM (IEC 61968-9 EndDeviceInfo.ratedPower; IEC 61970 GeneratingUnit.maxOperatingP). Manufacturer-rated peak power (nameplate value in principal direction). Kept for backward compatibility — prefer maxExport.
maxExport	QVPower (W / kW / MW)	**Based on** CIM (IEC 61970 GeneratingUnit.maxOperatingP; IEC 61970-302 PowerElectronicsConnection.maxP, injection). Maximum power this resource injects to the grid (generates/discharges). Always >=0. For bidirectional resources (BESS, V2G) this is the max discharge rate. Supersedes ratedPower.
maxImport	QVPower (W / kW / MW)	**Based on** CIM (IEC 61970-302 PowerElectronicsConnection.maxP, absorption). Maximum power this resource draws from the grid (absorbs/charges). Always >=0. For bidirectional resources (BESS, V2G) this is the max charge rate.
telemetryProvider	text	Vendor API / data source identifier for telemetry.
commissioningDate	date-time	ISO 8601 date-time the asset was commissioned.
location	Location	Physical location of this asset.
serialNumber	text	**Based on** CIM (IEC 61968-9 EndDeviceInfo.serialNumber). Manufacturer-assigned device serial number from the equipment nameplate. Distinct from id (which is the network-issued DID).

Field	Type	Description
inspection	object	**Based on** IEEE 1547-2018 Cl. 11 (commissioning); CEA Connectivity Regs 2013 (amended 2018). Commissioning / safety inspection record for the asset. Captured by the distribution licensee at energisation and on re-certification events.
aggregator	object	**Based on** IEEE 2030.5; IEC 61850-7-420 (DER control roles). Third-party flexibility / demand-response enrolment for this asset. Present when an aggregator is authorised to dispatch or observe the resource. Controllability flag is asset-level; the asset may still be observable even when controllable is false.
controlProtocol	OpenADR_2.0b / OCPP_2.0.1 / SunSpec_Modbus / EEBus / Modbus / Other	Demand-response / control protocol supported by this load device.
loadCategory	Heating / Cooling / WaterHeating / Lighting / EV / Industrial / Other	**Based on** CIM (IEC 61970-301 ConformLoad classification). Functional category of this load.

EnergyResourceNetwork

Field	Type	Description
id *	text	Stable identifier for this resource. For METER resources the meter serial number is conventional.
type *	DT / BUS / FEEDER / MICROGRID	Asset-class discriminator. DT -> PowerTransformer, BUS -> BusbarSection, FEEDER -> Feeder (EquipmentContainer), MICROGRID -> Substation / microgrid container (IEC 61970-301).
subResources	list of text or EnergyResource	Topology — child resources. Each item is EITHER a bare id string OR an inline EnergyResource object.
parentResources	list of text	Upward topology — ids of parent resources.
attributes	EnergyResourceNetworkAttributes	—

EnergyResourceNetworkAttributes

Field	Type	Description
make	text	Manufacturer (free text).
model	text	Model (free text).
ratedPower	QVPower (W / kW / MW)	**Based on** CIM (IEC 61968-9 EndDeviceInfo.ratedPower; IEC 61970 GeneratingUnit.maxOperatingP). Manufacturer-rated peak power (nameplate value in principal direction). Kept for backward compatibility — prefer maxExport.
maxExport	QVPower (W / kW / MW)	**Based on** CIM (IEC 61970 GeneratingUnit.maxOperatingP; IEC 61970-302 PowerElectronicsConnection.maxP, injection). Maximum power this resource injects to the grid (generates/discharges). Always >=0. For bidirectional resources (BESS, V2G) this is the max discharge rate. Supersedes ratedPower.
maxImport	QVPower (W / kW / MW)	**Based on** CIM (IEC 61970-302 PowerElectronicsConnection.maxP, absorption). Maximum power this resource draws from the grid (absorbs/charges). Always >=0. For bidirectional resources (BESS, V2G) this is the max charge rate.
telemetryProvider	text	Vendor API / data source identifier for telemetry.
commissioningDate	date-time	ISO 8601 date-time the asset was commissioned.
location	Location	Physical location of this asset.
serialNumber	text	**Based on** CIM (IEC 61968-9 EndDeviceInfo.serialNumber). Manufacturer-assigned device serial number from the equipment nameplate. Distinct from id (which is the network-issued DID).
inspection	object	**Based on** IEEE 1547-2018 Cl. 11 (commissioning); CEA Connectivity Regs 2013 (amended 2018). Commissioning / safety inspection record for the asset. Captured by the distribution licensee at energisation and on re-certification events.
aggregator	object	**Based on** IEEE 2030.5; IEC 61850-7-420 (DER control roles). Third-party flexibility / demand-response enrolment for this asset. Present when an aggregator is authorised to dispatch or observe the resource. Controllability flag is asset-level; the asset may still be observable even when controllable is false.
nominalVoltage	QVVoltage (V / kV)	**Based on** CIM (IEC 61970-301 BaseVoltage.nominalVoltage). Nominal operating voltage. Replaces nominalVoltageKv from v1.0.
zone	text	Operating zone or region identifier used by the utility.
substationId	text	Parent substation identifier per utility records.
feederCode	text	Feeder code per utility records. Relevant for FEEDER and DT resources.

QVVoltage

Field	Type	Description
value *	number	—
unit *	V / kV	—

ConsumptionProfile

Field	Type	Description
meterId *	text	Matches the id of a METER entry in customerProfile.energyResources[].
sanctionedLoad *	QVPower (W / kW / MW)	**Based on** CIM (IEC 61968-9 UsagePoint). Sanctioned/approved import load.
sanctionedExportLoad	QVPower (W / kW / MW)	Sanctioned/approved grid export limit.
billingCycleDay	integer	Day of month on which the billing cycle resets.
contractMaxDemand	QVPower (W / kW / MW)	Maximum demand contracted with the utility for this connection.
tariffCategoryCode *	text	**Based on** CIM (IEC 61968-9 UsagePoint.serviceCategory). Billing/tariff category code assigned by the utility.
premisesType	Residential / Commercial / Industrial / Agricultural	Type of premises at the metering point.
connectionType	Single-phase / Three-phase	**Based on** CIM (IEC 61968-9 UsagePoint.phaseCode). Electrical connection type.
paymentMode	POSTPAID / PREPAID	**Based on** CIM (IEC 61968-9 AmiBillingReadyKind, ESPI). Billing/payment modality. POSTPAID: consume now, pay later. PREPAID: pay-before-use.
serviceStatus	active / suspended / closed	**Based on** CIM (IEC 61968-9 UsagePoint.status). Lifecycle state of the service connection (the UsagePoint), not of the meter device itself. 'active' = currently energised and billable; 'suspended' = temporarily disconnected (non-payment, inspection, fault) with the contract still on record; 'closed' = permanently terminated. Distinct from the meter device's operational state.

11.6 MeterData

Smart meter telemetry payload schema for the India Energy Stack. Defines compact profile shapes for meter readings, billing data, and events — designed for high-efficiency data-only exchange without cryptographic wrapping.

Namespace prefix: `ies:` -> `https://india-energy-stack.github.io/ies-accelerator/schemas/ies#`

Tags: `metering` · `telemetry` · `smart-meter` · `ies`

11.6.1 Versions

Version	Status	Notes
v0.6	**Current**	Adds ALARM profile, anonymised topology, reading mode, split billing
v0.5	Previous	Six profiles: CUSTOMER, INTERVAL, DAILY, BILLING, INSTANTANEOUS, EVENT

11.6.2 Profile shapes

Profile	Description
`CUSTOMER`	Slow-changing customer metadata, service points, meter installations
`INTERVAL`	Block load survey at high-resolution intervals (15 or 30 min)
`DAILY`	Daily accumulated load survey profiles
`MONTHLY`	Monthly billing resets — ToU buckets, MD, cumulative registers
`BILL_DETAILS`	Computed billing details — amount, due date, payment status
`INSTANTANEOUS`	Real-time snapshot of voltages, currents, powers
`EVENT`	IS 15959 diagnostic codes and tamper events
`ALARM`	Active alerts — tamper, low prepayment credit, voltage sag, overload *(v0.6+)*

11.6.3 Usage

Used in Beckn `on_status` flows by data providers (AMISP, MDM, DISCOM) to deliver smart meter telemetry. When delivered with provenance attestation, wrap in MeterDataCredential.

11.6.4 v0.6

Files — attributes.yaml · schema.json · context.jsonld · vocab.jsonld · examples/

11.6.5 Meter Data Compact Profiles (v0.6)

This directory contains the **Meter Data Compact Profiles** (v0.6) schema definitions and semantic models for telemetry data exchange. It is tailored for high-efficiency, data-only transmissions of smart meter readings and events, omitting heavy cryptographic wrapping when only raw telemetry payload is being exchanged.

The `MeterData` schema standardizes telemetry exchanges into **eight compact profile shapes** representing different read cadences and data structures from smart meters:

1. **`CUSTOMER`** — Slow-changing customer metadata, service points, and meter installations.
2. **`INTERVAL`** — Block load survey profile at high-resolution intervals (e.g., 15-minute or 30-minute blocks).
3. **`DAILY`** — Daily accumulated load survey profiles.
4. **`MONTHLY`** — Monthly billing resets (billing history cumulative total energy registers, ToU buckets, and Maximum Demand).
5. **`BILL_DETAILS`** — Utility billing computed details (billing amount, bill number, due dates, currency, prepaid balance, and payment status).
6. **`INSTANTANEOUS`** — Real-time snapshots of electrical quantities (voltages, currents, active/reactive powers) at a specific moment.
7. **`EVENT`** — Event logs matching standard IS 15959 diagnostic codes and tamper events.
8. **`ALARM`** — Real-time active alerts representing immediate state conditions (tamper, low prepayment credit, voltage sag, overload) from the meter.

Directory Structure and Files

File	Description
`attributes.yaml`	Canonical OpenAPI schema source of truth, describing all attributes, models, and types.
`schema.json`	Compiled Draft 2020-12 JSON Schema for standard validation of data payloads.
`context.jsonld`	Compiled JSON-LD context mapping telemetry attributes to semantic terms under `ies:` or `schema:` namespaces.
`vocab.jsonld`	Compiled JSON-LD vocabulary (ontology graph) defining all classes and properties.
`IES codes.json`	Canonical registry of OBIS and Short Codes mapping physical quantities to units, categories, and categories of meters.
`examples/`	Standard, JSON-LD augmented telemetry payload example files for all profile types.

Documentation Guides

- User Guide: Detailed guidelines on HES, MDM, and Billing system interactions, advertising system capabilities via request mechanisms, and verification scenarios.
- Reference Guide: Top-down reference documentation detailing different profile roles, centralised codes, and telemetry vs. non-telemetry handling.

Data Descriptor Engine & The Array Envelope

In v0.6, the telemetry architecture uses an optimized, strict inheritance structure powered by the **Data Descriptor Engine**.

Instead of transmitting bulky metadata inline with every physical reading, telemetry can be wrapped in a **JSON Array**.

Centralized `PayloadDescriptorProfile` A JSON array payload can contain one or more `PayloadDescriptorProfile` objects. This serves as the dictionary for all compact profiles inside the payload array: * **`payloadDescriptors`**: Defines the metadata for specific registers (e.g., `readingType`, `unit`, `flowDirection`, `category`, and `multiplier`). * **`compactSequences`**: Defines the physical column layout of the dense numerical payload matrices.

Dynamic `SequenceItem` Columns (`attribute`) A sequence defines what each column in the `payloads` array represents using the **`attribute`** property: * `value`: The actual meter reading number. * `occurredAt`: The timestamp string representing exactly when a specific demand or reading occurred. * `openingValue` / `closingValue`: To provide mathematical proofs for `USAGE` deltas. * `validationStatus`: Standard validation flags (e.g. `ESTIMATED`, `VALID`).

Because `payloads` allows mixed arrays of `number` and `string`, metadata is mapped densely alongside values, while sparse overrides are used for quality indications (such as validation status or fail codes in specific intervals).

Strict Derived Profiles Inside the array, individual profiles inherit strictly from `BaseProfile`. `BaseProfile` only contains identity (`meterRefs`, `customerRefs`, `serviceDeliveryPointRefs`). Derived profiles (like `IntervalProfile`) strictly allow **only** their relevant properties (`intervals`, `payloadDescriptorSetRef`, `intervalPeriod`).

> [!NOTE] ### Rationale for MonthlyProfile not using the compact matrix The compact form is prevented natively by the schema for `MonthlyProfile`. It strictly requires `readings` and `touBuckets`, and does not permit `intervals`. The compact matrix is designed for dense, highly symmetrical time-series data. A `MonthlyProfile` typically captures a single sparse snapshot of total registers, ToU buckets, and Maximum Demand peaks at the end of the month. Encoding sparse, highly variable metadata (e.g., MD timestamps) into a flat numerical array offers negligible compression and becomes extremely brittle when dealing with ad-hoc billing resets or meter swaps mid-cycle.
>
> *See MonthlyProfile_MultipleResets.json and Billing_MeterChange.json for examples of how the elaborated representation cleanly handles these edge cases.*

Identifier Flexibility (OBIS vs. Short Codes)

The schema relies on `IES codes.json` as the canonical registry for interpreting physical quantities. The exact mapping of OBIS codes to physical units, phases, and categories is defined in this file.

When transmitting telemetry, you specify a simple **`readingType`** string. This can be: * **Using OBIS Codes**: `"1.0.1.8.0.255"` * **Using Short Names**: `"kWh imp"` * **Using Canonical Links**: Payload descriptors can optionally include an `obis` string parameter to provide a direct physical mapping when a short name is used as the `readingType`.

See the MultiMeterBulkDataset.json (OBIS Codes) and MultiMeterBulkDatasetShortCodes.json (Short Codes) for bulk examples of both representations.

TelemetryMode (READING vs USAGE)

The schema natively distinguishes between raw register readings and calculated usage: - **`READING`**: Represents the physical register value at a point in time. For cumulative registers, this strictly increases dynamically. - **`USAGE`**: Represents the delta or consumed amount over a period. Demand registers inherently use `USAGE` mode. For energy, this represents the exact block consumption.

Modes are indicated inside the payload descriptor only via the `reportedMode` property.

Schema Generation and Compilation

The `schema.json`, `context.jsonld`, and `vocab.jsonld` files are compiled automatically from the core `attributes.yaml`.

To Compile Schemas To recompile schemas following modifications in `attributes.yaml`, run the following python scripts from the repository root:

```
python scripts/generate_schema_permissive.py schemas/MeterData/v0.6
```

Examples

These examples cover a wide variety of scenarios, organized by the system they typically originate from:

MDM (Meter Data Management) Examples MDM datasets are meter-centric and intentionally omit `customerRefs` to decouple the technical metering domain from the commercial billing domain. * `IntervalProfile.json`: Block load survey. * `DailyProfile.json`: Daily midnight snapshots. * `MDM_MonthlyProfile.json`: End of billing cycle snapshots across multiple meters. * `InstantaneousProfile.json`: Snapshot of voltage, current, power factor, etc. * `EventProfile.json`: Meter tampers, diagnostics, power outages. * `AlarmProfile.json`: Critical thresholds being crossed. * `MultiMeterBulkDataset.json`: Large scale transmission of intervals across many meters.

CIS / Billing Examples Billing and CIS examples enrich technical meter data with `customerRefs`, Tou buckets, and monetary calculations. * `Billing_MonthlyProfile.json`: Usage-centric monthly consumption linked to consumers. * `CustomerProfile.json`: Linking meters, service delivery points, and customers. * `BillDetails.json`: Computed monetary bill with due dates and calculated consumption. * `CustomerBillingSummary.json`: Historic billing trends for a consumer dashboard. * `CustomerMapping.json`: Mapping of meter serial numbers to commercial customer accounts.

Highlighted Examples

- **Meter Swap Mid-Cycle**: `Billing_MeterChange.json` demonstrates how to handle a meter replacement in the middle of a billing period by chaining multiple monthly profiles under different meter serials.
- **Multiple Monthly Resets**: `MonthlyProfile_MultipleResets.json` demonstrates how to represent multiple billing resets triggered in the same calendar month.
- **Identifier Representation Comparison**: Compare `MultiMeterBulkDataset.json` (using raw OBIS codes) with `MultiMeterBulkDatasetShortCodes.json` (using Short Names) to see how the payload descriptor engine abstracts identifier models.

Telemetry Verification & Validation

The validator is not limited to example files; it can be used to validate any `MeterData` payload. In this repository, all example JSON files are validated for structural compliance against `schema.json` and semantic compliance against `IES codes.json` using the v0.6 validator.

To Validate Payloads Run the following command from the `validation/` directory:

```
python validator.py <path_to_json_file_or_directory>
```

The validator dynamically parses JSON files. If they are arrays containing a `PayloadDescriptorProfile`, it matches `payloadDescriptorSetRef` against the array's descriptors, and asserts both matrix arity and strict column types based on the `attribute` enum.

Overlaps and Differences with ElectricityCredential

- **`meterType` vs `meterCategory`**:
  - `meterType` describes the physical communication/technology capability of the meter (e.g. `AMI`, `AMR`, `Prepaid`, `NetMeter`).
  - `meterCategory` describes the regulatory standards category under IS 15959 (e.g. `A`, `B`, `C`, `D1`–`D4`).
  - These represent distinct aspects of the physical/standard classification, and both are kept.
- **`premisesType` vs `consumerCategory`**:
  - `premisesType` (Residential, Commercial, etc. defined in `ElectricityCredential/v1.1`) classifies the physical type of the premises.
  - `consumerCategory` (LT2A, NDS, HT, etc. defined in `MeterData/v0.6`) represents the utility-specific tariff category.
  - To prevent duplication and maintain backwards compatibility, the pre-existing `consumerCategory` property was kept, and `premisesType` was excluded. This overlap is marked here for future reconciliation.
- **`energyResources` vs `meters/associations` (Export & Storage modeling)**:
  - `ElectricityCredential v1.1` models all physical assets (meters, solar PV arrays, battery storage) using a unified `energyResources` list containing `EnergyResource/v2.0` objects. This allows rich modeling of child DER/storage assets (`SOLAR`, `BATTERY`, `BESS`) directly detailing attributes like `ratedPowerKw` and `energyCapacityKwh` linked via topology references.
  - `MeterData v0.6` retains a lightweight representation focusing strictly on billing/metering associations, lacking top-level asset representations. To bridge this gap for grid planners, three lightweight properties were introduced in `v0.6`:
    - **`sanctionedExportLoadKw`** (on the `Customer` schema) to explicitly specify approved net-metering solar export limits.
    - **`generationCapacityKw`** (on the `Association` schema) to denote the rated solar/generation inverter capacity connected.
    - **`storageCapacityKw`** (on the `Association` schema) to denote the connected battery storage energy capacity in kWh.
  - This structural divergence is documented here for future alignment in subsequent major version releases.

Field reference

*Auto-generated from `schema.json`. A field name in **bold** with a trailing * is required; all others are optional. **Type** shows units for QuantitativeValue models. Where a field is derived from a standard, its description begins with **Based on** and the standard reference (from the field's `x-standard` annotation). One table per object.*

CustomerProfile

Field	Type	Description
profileType *	text	—
customerRefs	IdentifierList	—
timeZone	text	IANA time-zone, e.g. Asia/Kolkata.
customerDetails	CustomerDetails	—
customer *	Customer	—
serviceDeliveryPoints *	list of ServiceDeliveryPoint	—
meters *	list of Meter	—
associations *	list of Association	—

BaseProfile

Field	Type	Description
profileType *	text	—
customerRefs	IdentifierList	—
meterRefs *	IdentifierList	—
serviceDeliveryPointRefs	IdentifierList	—
payloadDescriptorSetRef	text	Reference ID matching a previously exchanged PayloadDescriptorProfile's payloadDescriptorSet.

IntervalProfile

Field	Type	Description
profileType *	text	—
customerRefs	IdentifierList	—
meterRefs *	IdentifierList	—
serviceDeliveryPointRefs	IdentifierList	—
payloadDescriptorSetRef	text	Reference ID matching a previously exchanged PayloadDescriptorProfile's payloadDescriptorSet.
compactSequenceRef	text	Name of the compact sequence to use from the payloadDescriptorSets.
intervalPeriod *	IntervalPeriod	—
intervals	list of Interval	—
readings	list of Reading	—

DailyProfile

Field	Type	Description
profileType *	text	—
customerRefs	IdentifierList	—
meterRefs *	IdentifierList	—
serviceDeliveryPointRefs	IdentifierList	—
payloadDescriptorSetRef	text	Reference ID matching a previously exchanged PayloadDescriptorProfile's payloadDescriptorSet.
compactSequenceRef	text	Name of the compact sequence to use from the payloadDescriptorSets.
intervalPeriod *	IntervalPeriod	—
intervals	list of Interval	—
readings	list of Reading	—

MonthlyProfile

Field	Type	Description
profileType *	text	—
customerRefs	IdentifierList	—
meterRefs *	IdentifierList	—
serviceDeliveryPointRefs	IdentifierList	—
payloadDescriptorSetRef	text	Reference ID matching a previously exchanged PayloadDescriptorProfile's payloadDescriptorSet.
timePeriod *	TimePeriod	—
readings *	list of Reading	—
touBuckets	list of TouBucket	—

BillDetails

Field	Type	Description
profileType *	text	—
customerRefs	IdentifierList	—
meterRefs *	IdentifierList	—
serviceDeliveryPointRefs	IdentifierList	—
payloadDescriptorSetRef	text	Reference ID matching a previously exchanged PayloadDescriptorProfile's payloadDescriptorSet.
timePeriod *	TimePeriod	—
readings	list of Reading	—
touBuckets	list of TouBucket	—
billNumber	text	—
billDate	date	—
dueDate	date	—
currency	text	ISO 4217 (INR, USD, …).
amountDue *	number	—
energyCharges	number	Charges for active/reactive energy consumption.
fixedCharges	number	Fixed charges/demand charges.
otherCharges	number	Other taxes, duties, surcharges, or adjustments.
prepaidBalance	number	Prepaid remaining balance/credit amount on the account/meter, if applicable.
paymentStatus	text	Status of the payment, e.g. PAID, UNPAID, PARTIAL.

InstantaneousProfile

Field	Type	Description
profileType *	text	—
customerRefs	IdentifierList	—
meterRefs *	IdentifierList	—
serviceDeliveryPointRefs	IdentifierList	—
payloadDescriptorSetRef	text	Reference ID matching a previously exchanged PayloadDescriptorProfile's payloadDescriptorSet.
timestamp *	date-time	—
readings *	list of Reading	—

AlarmProfile

Field	Type	Description
profileType *	text	—
customerRefs	IdentifierList	—
meterRefs *	IdentifierList	—
serviceDeliveryPointRefs	IdentifierList	—
payloadDescriptorSetRef	text	Reference ID matching a previously exchanged PayloadDescriptorProfile's payloadDescriptorSet.
timestamp *	date-time	—
alarms *	list of MeterAlarm	—

EventProfile

Field	Type	Description
profileType *	text	—
customerRefs	IdentifierList	—
meterRefs *	IdentifierList	—
serviceDeliveryPointRefs	IdentifierList	—
payloadDescriptorSetRef	text	Reference ID matching a previously exchanged PayloadDescriptorProfile's payloadDescriptorSet.
timePeriod *	TimePeriod	—
events *	list of MeterEvent	—

Identifier

Field	Type	Description
scheme *	`METER_SERIAL` / `METER_BADGE` / `MRID` / `OBIS` / `SHORT_CODE` / `CONSUMER_NUMBER` / `SERVICE_DELIVERY_POINT` / `DID` / `ORG` / `OTHER`	—
value *	text	—

Field	Type	Description
namespace	text	—

TimePeriod

Field	Type	Description
start *	date-time	—
duration *	duration	—

ReadingDefinition

Field	Type	Description
readingType *	text	—
supportedModes *	list of READING / USAGE	—
defaultMode *	READING / USAGE	—
name	text	—
shortLabel	text	—
unit	kWh / kVAh / kvarh / kW / kvar / kVA / V / A / Hz / PF / NONE / INR / USD	—
phase	NONE / R / Y / B / ABC	—
flowDirection	IMPORT / EXPORT / NONE	—
accumulationBehaviour	CUMULATIVE / DELTA / INSTANTANEOUS / SUMMATION / INDICATING	—
touZone	integer	—

PayloadDescriptorSet

Field	Type	Description
name *	text	—
payloadDescriptors *	list of PayloadDescriptor	—
compactSequences	list of CompactSequence	—

CompactSequence

Field	Type	Description
name *	text	—
sequenceItems *	list of SequenceItem	—

SequenceItem

Field	Type	Description
readingType *	text	—
attribute	value / occurredAt / openingValue / closingValue / validationStatus	—

PayloadDescriptor

Field	Type	Description
readingType *	text	—
obis	text	Optional canonical OBIS code when readingType uses a short code.
name	text	—
unit	kWh / kVAh / kvarh / kW / kvar / kVA / V / A / Hz / PF / NONE / INR / USD	—
flowDirection	IMPORT / EXPORT / NONE	—
category	text	—
reportedMode	READING / USAGE	—
touZone	integer	—
multiplier	number	Decimal scaling factor (e.g. 0.001 for milli, 1000 for kilo). Default value is 1.

Field	Type	Description
accuracy	number	Accuracy class or precision value, applied after the multiplier.

IntervalPeriod

Field	Type	Description
start *	date-time	—
duration *	duration	—

Interval

Field	Type	Description
id *	integer	—
intervalPeriod	IntervalPeriod	—
payloads	list of number / string / boolean	—
readings	list of Reading	—
overrides	list of Override	—

Override

Field	Type	Description
descriptorIndex *	integer	—
occurredAt	date-time	—
zone	integer	—
validationStatus	VALID / ESTIMATED / MANUAL / SUSPECT / REJECTED	—
source	METER / HES / ESTIMATED / MANUAL / IMPORT / MDM_COMPUTED / CIS_COMPUTED	—
changeMethod	text	—
failCode	text	—

Reading

Field	Type	Description
readingType *	text	—
value *	number	—
openingValue	number	MUST ONLY be provided when the associated payload descriptor specifies reportedMode=USAGE.
closingValue	number	MUST ONLY be provided when the associated payload descriptor specifies reportedMode=USAGE.
integrationPeriod	duration	Demand integration period, e.g. PT30M or PT15M.
timePeriod	TimePeriod	—
occurredAt	date-time	—
validationStatus	VALID / ESTIMATED / MANUAL / SUSPECT / REJECTED	—
source	METER / HES / ESTIMATED / MANUAL / IMPORT / MDM_COMPUTED / CIS_COMPUTED	—
changeMethod	text	—
failCode	text	—

TouBucket

Field	Type	Description
zone *	integer	—
readings *	list of Reading	—

Customer

Field	Type	Description
id *	Identifier	—
name	text	—
consumerCategory	text	Utility tariff category (e.g., LT2A, NDS, HT).
sanctionedLoadKw	number	—
billingCycleDay	integer	—
paymentMode	PREPAID / POSTPAID	Indicates if the connection is prepaid or postpaid.
connectionType	Single-phase / Three-phase	Type of connection (Single-phase or Three-phase).
contractMaxDemandKw	number	Maximum demand contracted with the utility for this connection, in kW.
tariffCategoryCode	text	Billing/tariff category code assigned by the utility.
sanctionedExportLoadKw	number	Sanctioned/approved grid export limit in kW.

ServiceDeliveryPoint

Field	Type	Description
id *	Identifier	—
address	Address	—
geo	GeoJSONGeometry	—

Meter

Field	Type	Description
id *	Identifier	—
make	text	Make of the meter.
model	text	Model of the meter.
meterType	AMR / AMI / Electromechanical / Forward / Reverse / Bidirectional / Prepaid / NetMeter / Other	—
meterCategory	A / B / C / D1 / D2 / D3 / D4	—
serviceKind	ELECTRICITY / GAS / WATER / HEAT	—

Association

Field	Type	Description
serviceDeliveryPointRefs *	IdentifierList	—
meterRefs *	IdentifierList	—
parentResources	list of Identifier	List of parent resources (such as feeders or DTs) for this association, replacing feederId and dtId.
telemetryProvider	text	Telemetry provider for this meter-service association.
commissioningDate	date-time	Commissioning date of the meter association.
generationCapacityKw	number	Rated power generation capacity (e.g. solar PV inverter capacity) in kW.
storageCapacityKw	number	Rated energy storage capacity in kWh.

MeterEvent

Field	Type	Description
timestamp *	date-time	—
eventId *	integer	—
eventName	text	—
phase	NONE / R / Y / B / ABC	—
sequence	integer	—
magnitude	number	—
duration	duration	—

MeterAlarm

Field	Type	Description
timestamp *	date-time	—
alarmId *	integer	—
alarmName	text	—
status *	ACTIVE / CLEARED	—
severity	CRITICAL / WARNING / INFO	—

PayloadDescriptorProfile

Field	Type	Description
`profileType` *	text	—
`id` *	text	Unique identifier for this specific configuration state.
`payloadDescriptorSets` *	list of PayloadDescriptorSet	—

CustomerDetails

Field	Type	Description
`name`	text	Full name of the customer as per ID proof.

11.7 MeterDataCredential

W3C Verifiable Credential (VC Data Model 2.0) that wraps a MeterData payload to attest the authenticity and provenance of delivered smart meter telemetry. Issued by data providers (AMISP, MDM, DISCOM).

Namespace prefix: `ies:` -> `https://india-energy-stack.github.io/ies-accelerator/schemas/ies#`

Tags: `credential` · `verifiable-credential` · `metering` · `ies`

11.7.1 Versions

Version	Status	Notes
v0.6	**Current**	Wraps MeterData v0.6; BitstringStatusListEntry revocation

---

11.7.2 Inheritance

```
beckn:Credential
  └── EnergyCredential
        └── MeterDataCredential  <- this schema
```

11.7.3 Usage

Attached to Beckn `on_status` responses. Cryptographically binds the delivered MeterData payload to the issuing provider's DID, enabling downstream verifiers to confirm identity, provenance, and tamper-evidence.

11.7.4 v0.6

Files — attributes.yaml · schema.json · context.jsonld · vocab.jsonld · examples/

11.7.5 MeterDataCredential v0.6

A **W3C Verifiable Credential (VC Data Model 2.0)** that wraps a `MeterData v0.6` payload to attest the authenticity and provenance of delivered smart meter telemetry.

Purpose

In a Beckn data-exchange flow, a **provider** (e.g., AMISP, MDM system) responds to a confirmed `MeterDataRequest` with an `on_status` message. The provider wraps the telemetry payload in a `MeterDataCredential` so that:

1. **Identity is bound** — the data is cryptographically tied to the issuing provider's DID.
2. **Provenance is verifiable** — any downstream system can verify who delivered the data and when.
3. **Tamper-evidence** — the payload cannot be altered without invalidating the signature.
4. **Revocation is supported** — the DeDi registry allows the provider to invalidate a credential if data is recalled (e.g., due to meter fault or privacy request).

The receiver checks: credential type is `MeterDataCredential`, `validUntil` has not passed, status is not revoked, and the embedded `meterData` profiles are within the contracted scope of the originating `MeterDataRequest`.

Inheritance

`MeterDataCredential` is a **subclass of `EnergyCredential v2.0`** (defined in the DEG specification):

```
beckn:Credential
  └── EnergyCredential (https://schema.beckn.io/EnergyCredential/v2.0)
        └── MeterDataCredential
```

The common VC envelope is **inherited, not duplicated**:

Inherited from EnergyCredential	Defined here
`issuer` (id, name, `licenseNumber`)	`credentialSubject.meterData`
`validFrom` / `validUntil`	`MeterDataCredential` type
`credentialStatus` (DeDi registry)	
`proof` (Ed25519Signature2020)	

In `attributes.yaml` this is expressed as `allOf: - $ref: https://schema.beckn.io/EnergyCredential/v2.0`, matching the same pattern used by `MeterDataRequestCredential`, `ConsumptionProfileCredential`, and other DEG energy credentials.

The `@context` of a credential instance must include: 1. `https://www.w3.org/ns/credentials/v2` 2. `https://schema.beckn.io/EnergyCredential/v2.0/context.jsonld` 3. `https://india-energy-stack.github.io/ies-accelerator/schemas/MeterDataCredential/v0.6/context.jsonld`

Schema Files

File	Purpose
`attributes.yaml`	OpenAPI 3.1.1 source of truth with `x-jsonld` annotations
`schema.json`	JSON Schema Draft 2020-12 for validation
`context.jsonld`	JSON-LD context (semantic term mappings)
`vocab.jsonld`	RDF vocabulary / ontology definitions
`examples/example-customer-profile.json`	VC wrapping a single CustomerProfile
`examples/example-interval-profile.json`	VC wrapping a PayloadDescriptorProfile + IntervalProfile array
`examples/example-monthly-profile.json`	VC wrapping a MonthlyProfile with ToU buckets

Credential Structure

```
MeterDataCredential (W3C VC 2.0)
├── @context        – W3C credentials/v2 + EnergyCredential + MeterDataCredential context
├── id              – urn:uuid:...
├── type            – ["VerifiableCredential", "MeterDataCredential"]
├── issuer
│   ├── id          – DID of the data provider (AMISP, MDM, DISCOM)
│   ├── name        – Human-readable name
│   └── licenseNumber – Regulatory licence (e.g., KERC-AMISP-2025-007)
├── validFrom       – Issuance datetime
├── validUntil      – Expiry (typically 1 year)
├── credentialStatus – DeDi revocation registry reference
├── credentialSubject
│   ├── id          – DID of the consumer or asset entity
│   └── meterData   – MeterData v0.6 payload
│       ├── (single profile)  – one of CUSTOMER / INTERVAL / DAILY / MONTHLY /
│       │                         BILL_DETAILS / INSTANTANEOUS / EVENT / ALARM / DESCRIPTOR
│       └── (array of profiles) – mixed profile types, typically starting with DESCRIPTOR
└── proof           – Ed25519Signature2020 (or equivalent)
```

---

meterData payload shapes

The `meterData` value follows the `MeterData v0.6` schema — either a **single** `EnergyData` profile object or an **array** of profile objects. All standard profile types are supported:

`profileType`	Description
`CUSTOMER`	Slow-changing customer + service-point + meter-list summary
`INTERVAL`	Block Load Survey — PT15M / PT30M cadence interval blocks
`DAILY`	Daily Load Profile — P1D interval blocks
`MONTHLY`	Monthly billing readings (with optional ToU buckets)
`BILL_DETAILS`	Computed billing amount, bill number, due dates
`INSTANTANEOUS`	Snapshot of electrical quantities at one moment
`EVENT`	IS 15959 event log over a time period
`ALARM`	Real-time active alarm indicators

profileType	Description
DESCRIPTOR	PayloadDescriptor sets (exchanged before or alongside compact interval data)

Arrays typically start with a DESCRIPTOR profile followed by one or more data profiles that reference it via payloadDescriptorSetRef.

Relationship to MeterDataRequestCredential

MeterDataCredential is the **response** counterpart to MeterDataRequestCredential:

	MeterDataRequestCredential	MeterDataCredential
Issued by	Requester (DISCOM)	Provider (AMISP / MDM)
Wraps	MeterDataRequest	MeterData payload
Used in	confirm — proves authorisation to request	on_status — attests delivered data
Subject	Requesting entity DID	Consumer / asset DID

Usage in Beckn Data Exchange (UC1)

Provider side — on_status The provider delivers the telemetry as a MeterDataCredential in dataPayload. The embedded meterData must cover the profile types and time window originally requested in the MeterDataRequest.

Receiver side — verification

1. Resolve the issuer DID and retrieve the public key at verificationMethod.
2. Verify the proof signature over the canonical serialisation of the credential.
3. Check validUntil has not passed and credentialStatus is not revoked via the DeDi registry.
4. Validate meterData against the MeterData v0.6 schema.

Examples

File	Description
example-customer-profile.json	BESCOM AMISP attests CustomerProfile for consumer RR-1234
example-interval-profile.json	BESCOM AMISP attests PT15M interval data (with PayloadDescriptor)
example-monthly-profile.json	BESCOM AMISP attests January 2026 monthly readings with ToU buckets

---

Changelog

Version	Date	Notes
v0.6	2026-06-06	Initial draft — aligns with MeterData v0.6

Field reference

*Auto-generated from `schema.json`. A field name in **bold** with a trailing * is required; all others are optional. **Type** shows units for QuantitativeValue models. Where a field is derived from a standard, its description begins with **Based on** and the standard reference (from the field's `x-standard` annotation). One table per object.*

MeterDataCredential

Field	Type	Description
`credentialSubject`	MeterDataCredentialSubject	The subject of the credential: the consumer or asset entity whose meter data is attested, plus the MeterData payload.

MeterDataCredentialSubject

Field	Type	Description
`id`	uri	DID of the consumer or asset entity whose meter data is being delivered.
`meterData` *	schema.json	The attested meter data payload. May be a single EnergyData profile or an array of profiles per MeterData v0.6.

11.8 MeterDataRequest

Schema for querying smart meter telemetry. Specifies which meters, what data, and for how long — sent by a data consumer (BAP/seeker) to a data provider (BPP) in Beckn `select`/`init` flows.

Namespace prefix: `ies:` -> `https://india-energy-stack.github.io/ies-accelerator/schemas/ies#`

Tags: `metering` · `request` · `telemetry` · `ies`

11.8.1 Versions

Version	Status	Notes
v0.6	**Current**	Split into three composable models: MeterDataCapabilities, MeterDataAuthorisation, MeterDataRequest
v0.5	Previous	Single unified request schema

11.8.2 Models (v0.6)

Model	Purpose
`MeterDataCapabilities`	Provider declares what profiles, register keys, and query boundaries are supported
`MeterDataAuthorisation`	Consumer proves consent scope — meters/feeders, time window, profile types
`MeterDataRequest`	Active query binding capabilities to a specific authorisation

---

11.8.3 Usage

Consumer attaches a MeterDataRequestCredential at `confirm` time to prove authorisation. Provider validates before delivering MeterData wrapped in a MeterDataCredential.

11.8.4 v0.6

Files — attributes.yaml · schema.json · context.jsonld · vocab.jsonld · examples/

11.8.5 MeterDataRequest v0.6

This directory contains the schemas, context mapping, and vocabularies for querying smart meter telemetry and profiles. In version 0.6, the request layer has been split into three distinct, composable models: 1. **MeterDataCapabilities** 2. **MeterDataAuthorisation** 3. **MeterDataRequest**

---

1. MeterDataCapabilities

Represents the data sharing capabilities supported by a meter data provider (e.g. DISCOM). It acts as the blueprint of what data points are available, specifying what profile types, register keys, telemetry modes, and query boundaries are supported.

- **Main Schema**: `schema.json#/$defs/MeterDataCapabilities`
- **JSON-LD Type**: `ies:MeterDataCapabilities`
- **Link to Example**: `Capabilities_Example.json`

---

2. MeterDataAuthorisation

Represents the cryptographic or digital grant/token allowing an entity (such as a Technical Service Provider) to access a specific subset of data capabilities. It defines: - **`grantor`**: The authorizing entity (e.g. Utility/DISCOM). - **`grantee`**: The authorized entity (e.g. TSP). - **`purpose`**: The explicit reason for accessing the data (e.g. ESG reporting). - **`validFrom` / `validUntil`**: The validity period. - **`capabilities`**: The allowed `MeterDataCapabilities` object.

- **Main Schema**: `schema.json#/$defs/MeterDataAuthorisation`
- **JSON-LD Type**: `ies:MeterDataAuthorisation`
- **Link to Example**: `Authorisation_Example.json`

---

3. MeterDataRequest

Represents the actual data query payload sent by the authorized entity to pull telemetry. It contains query criteria and proves authorization and user consent: - **`consumers`** (optional): List of target consumer DIDs/URIs. - **`resources`** (optional): List of target meter serials or service point URIs. - **`scope`**: Hierarchy of query (`ResourceOnly`, `ResourceAndChildren`, `ChildrenOnly`). - **`from` / `duration`**: The start and duration of the query time window. - **`consumerConsent`** (optional): Array of consumer consent DIDs or receipt hashes. - **`authorisation`**: Inline `MeterDataAuthorisation` object or URL pointing to the grant. - **`capabilitiesRequested`**: The requested `MeterDataCapabilities` subset.

- **Main Schema**: `schema.json` (root schema)
- **JSON-LD Type**: `ies:MeterDataRequest`
- **Link to Example**: `Request_Example.json`

Field reference

*Auto-generated from `schema.json`. A field name in **bold** with a trailing * is required; all others are optional. **Type** shows units for QuantitativeValue models. Where a field is derived from a standard, its description begins with **Based on** and the standard reference (from the field's `x-standard` annotation). One table per object.*

MeterDataRequest

Field	Type	Description
consumers	list of uri	List of consumer URIs/DIDs for whom the data is requested (optional).
resources	list of uri	List of resource URIs/DIDs (e.g. meter DIDs, service point IDs) to query (optional).
scope	ResourceOnly / ResourceAndChildren / ChildrenOnly	—
from *	date-time	ISO 8601 UTC date-time indicating the start time of the requested data window.
duration *	duration	ISO 8601 duration string representing the length of the requested data window (e.g., PT15M, P1D, P30D).
consumerConsent	list of text	Consent references/receipts proving authorization (optional).
authorisation	MeterDataAuthorisation or uri	Inline authorization object or URI reference to it.
capabilitiesRequested *	MeterDataCapabilities	—
maxRecordsShared	integer	Maximum number of records that should be shared or returned in a single batch/page.

MeterDataAuthorisation

Field	Type	Description
grantor *	uri	URI/DID of the entity authorizing access.
grantee *	uri	URI/DID of the authorized entity.
purpose *	text	Reason/purpose for data access.
validFrom *	date-time	ISO 8601 UTC date-time indicating when the authorization becomes valid.
validUntil *	date-time	ISO 8601 UTC date-time indicating when the authorization expires.
capabilities *	MeterDataCapabilities	—

MeterDataCapabilities

Field	Type	Description
profiles *	list of ProfileCapability	List of profiles and their specific values/modes.
supportedScopes	list of ResourceOnly / ResourceAndChildren / ChildrenOnly	Hierarchical scopes supported by the query processor.
maxHistoryDuration	duration	Maximum length of historical data window supported by the provider.

ProfileCapability

Field	Type	Description
profileType *	CustomerProfile / IntervalProfile / DailyProfile / MonthlyProfile / BillDetails / InstantaneousProfile / EventProfile / AlarmProfile	—
readings	list of ValueCapability	Granular capabilities for specific registers and metrics. If omitted, all readings under this profile are supported.

ValueCapability

Field	Type	Description
value *	text	The OBIS code (e.g. 1.0.1.8.0.255) or short code (e.g. kWh imp) representing the register.
mode	READING / USAGE	The telemetry mode (READING, USAGE) supported or requested for this register.
multiplier	number	Optional decimal multiplier scaling factor (e.g., 0.001 or 1000).
accuracy	number	Optional accuracy class or precision value.

11.9 MeterDataRequestCredential

W3C Verifiable Credential (VC Data Model 2.0) that wraps a MeterDataRequest to prove a data requester's authorisation for accessing smart meter telemetry. Presented by the seeker at `confirm` time.

Namespace prefix: `ies: -> https://india-energy-stack.github.io/ies-accelerator/schemas/ies#`

Tags: `credential · verifiable-credential · metering · authorisation · ies`

11.9.1 Versions

Version	Status	Notes
v0.1	**Current**	Initial release; wraps MeterDataRequest v0.6

11.9.2 Inheritance

```
beckn:Credential
  └── EnergyCredential
        └── MeterDataRequestCredential  <- this schema
```

11.9.3 Usage

The provider's offer policy requires the seeker to present this credential at `confirm`. It identifies the requesting entity (DID + licence number), scopes the request, and is cryptographically signed — enabling the provider to verify without a round-trip to the issuer.

11.9.4 v0.1

Files — attributes.yaml · schema.json · context.jsonld · vocab.jsonld · examples/

11.9.5 MeterDataRequestCredential v0.1

A **W3C Verifiable Credential (VC Data Model 2.0)** that wraps a `MeterDataRequest` to prove a data requester's authorisation for accessing smart meter telemetry.

Purpose

In a Beckn data-exchange flow, a **seeker** (e.g., DISCOM) discovers an AMI dataset offered by a **provider** (e.g., AMISP). The provider's offer policy requires the seeker to attach a `MeterDataRequestCredential` at `confirm` time. This credential:

1. **Identifies** the requesting entity (DID + regulatory licence number).
2. **Scopes** the request — which meters/feeders, what hierarchy, what time window, which profile types.
3. **Is cryptographically signed**, so the provider can verify it without a round-trip to the issuer.
4. **Can be revoked** via the DeDi registry if authorisation is withdrawn.

The provider checks: credential type is `MeterDataRequestCredential`, `validUntil` has not passed, status is not revoked, and the embedded `MeterDataRequest.resources` are within the contracted scope.

Inheritance

`MeterDataRequestCredential` is a **subclass of `EnergyCredential v2.0`** (defined in the DEG specification):

```
beckn:Credential
  └── EnergyCredential (https://schema.beckn.io/EnergyCredential/v2.0)
        └── MeterDataRequestCredential
```

The common VC envelope is **inherited, not duplicated**:

Inherited from EnergyCredential	Defined here
`issuer` (id, name, `licenseNumber`)	`credentialSubject.meterDataRequest`
`validFrom` / `validUntil`	`MeterDataRequestCredential` type
`credentialStatus` (DeDi registry)	
`proof` (Ed25519Signature2020)	

In `attributes.yaml` this is expressed as `allOf: - $ref: https://schema.beckn.io/EnergyCredential/v2.0`, matching the same pattern used by `ConsumptionProfileCredential`, `GenerationProfileCredential`, and other DEG energy credentials.

The `@context` of a credential instance must include: 1. `https://www.w3.org/ns/credentials/v2` 2. `https://schema.beckn.io/EnergyCredential/v2.0/context.jsonld` 3. `https://india-energy-stack.github.io/ies-accelerator/schemas/MeterDataRequestCredential/v0.1/context.jsonld`

Schema Files

File	Purpose
`attributes.yaml`	OpenAPI 3.1.1 source of truth with `x-jsonld` annotations
`schema.json`	JSON Schema Draft 2020-12 for validation
`context.jsonld`	JSON-LD context (semantic term mappings)
`vocab.jsonld`	RDF vocabulary / ontology definitions
`examples/example.json`	Complete W3C VC 2.0 example

Credential Structure

```
MeterDataRequestCredential (W3C VC 2.0)
├── @context        – W3C credentials/v2 + IES MeterDataRequestCredential context
├── id              – urn:uuid:...
├── type            – ["VerifiableCredential", "MeterDataRequestCredential"]
├── issuer
│   ├── id          – DID of the requesting entity (e.g., DISCOM)
│   ├── name        – Human-readable name
│   └── licenseNumber – Regulatory licence (e.g., KERC-DISCOM-2025-001)
├── validFrom       – Issuance datetime
├── validUntil      – Expiry (short window, e.g., 30 days)
├── credentialStatus – DeDi revocation registry reference
├── credentialSubject
│   ├── id          – DID of the requesting entity
│   └── meterDataRequest (MeterDataRequest v0.6)
│       ├── consumers      – Optional list of consumer DIDs
│       ├── resources      – Optional list of meter/feeder DIDs to query
│       ├── scope          – ResourceOnly | ResourceAndChildren | ChildrenOnly
│       ├── from           – Start of data window (ISO 8601 UTC)
│       ├── duration       – Length of data window (ISO 8601 duration)
│       ├── consumerConsent – Optional list of consumer consent DIDs
│       ├── authorisation  – Reference or inline authorisation grant
│       ├── capabilitiesRequested – The requested capabilities (MeterDataCapabilities)
│       └── maxRecordsShared – Optional cap on returned records
└── proof           – Ed25519Signature2020 (or equivalent)
```

Usage in Beckn Data Exchange (UC1)

Provider side — `catalog/publish` The provider advertises: 1. **`ies:dataSchema`** in `resourceAttributes` — URL to the MeterData v0.6 schema used for response payloads. 2. **`ies:capabilityProfile`** in `resourceAttributes` — a `MeterDataRequest` object describing the maximum scope the provider can serve (think of it as the provider's data-access envelope). 3. **`offerAttributes.policy.requiredCredentials`** — declares that a valid `MeterDataRequestCredential` must be attached at `confirm`.

Seeker side — `confirm` The seeker includes the signed credential in `commitmentAttributes.ies:meterDataRequest`. The embedded `MeterDataRequest` must be a subset of the provider's `capabilityProfile`.

Provider side — `on_status` The response `dataPayload` contains **IES MeterData v0.6** profile objects (e.g., `IntervalProfile`, `CustomerProfile`) — the schema advertised in `ies:dataSchema`.

Example

See `examples/example.json` for a complete W3C VC 2.0 instance where BESCOM (DISCOM) requests Q1 2026 interval + daily + monthly data for all meters under feeder `IN-KA-BLR-ZONE-A`.

Changelog

Version	Date	Notes
v0.1	2026-05-26	Initial draft

Field reference

*Auto-generated from `schema.json`. A field name in **bold** with a trailing ***** is required; all others are optional. **Type** shows units for QuantitativeValue models. Where a field is derived from a standard, its description begins with **Based on** and the standard reference (from the field's `x-standard` annotation). One table per object.*

MeterDataRequestCredential

Field	Type	Description
`credentialSubject`	MeterDataRequestCredentialSubject	The subject of the credential: the requesting entity and the specific MeterDataRequest they are authorised to make.

MeterDataRequestCredentialSubject

Field	Type	Description
`id`	uri	DID of the requesting entity (e.g., the DISCOM).
`meterDataRequest` *	schema.json	The scoped, time-bounded data request. Specifies the meter resources, hierarchical scope, time window, and profile types being requested. The provider validates this against its capability profile before fulfilling delivery.

11.10 ArrFiling

Schema for Aggregate Revenue Requirement (ARR) filings made by electricity distribution licensees (DISCOMs) to State Electricity Regulatory Commissions (SERCs) in India. Standardises diverse state-level regulatory filing formats into a unified structure.

Namespace prefix: `ies:` -> `https://india-energy-stack.github.io/ies-accelerator/schemas/ies#`

Tags: `regulatory · arr · discom · serc · ies`

11.10.1 Versions

Version	Status	Notes
v0.5	**Current**	Three relational models: ArrFiling, ArrFiscalYear, ArrLineItem

---

11.10.2 Models

Model	Purpose
`ArrFiling`	Root metadata — filing ID, DISCOM details, SERC, control period, currency, unit scale
`ArrFiscalYear`	Fiscal year entry — baseline/control-period classification, amount basis (actuals/approved/proposed)
`ArrLineItem`	Single line item — Power Purchase Cost, O&M Expenses, Net ARR, true-up adjustments

---

11.10.3 Usage

Used in the Regulatory Data Exchange flow. DISCOMs publish ARR filings as structured Beckn catalog items; SERCs and downstream systems consume them via standard discovery and retrieval flows.

11.10.4 v0.5

Files — attributes.yaml · schema.json · context.jsonld · vocab.jsonld · examples/

11.10.5 Aggregate Revenue Requirement (ARR) Filing Schema (v0.5)

This directory contains the **Aggregate Revenue Requirement (ARR) Filing** (v0.5) schema definitions and semantic models for utility regulatory data exchange. It is designed to standardize the structure of ARR filings made by distribution licensees (DISCOMs) to State Electricity Regulatory Commissions (SERCs) in India.

Overview

The `ArrFiling` schema unifies diverse state-level regulatory filing formats by structuring them into three relational classes:

1. **`ArrFiling` (Root)** — Describes global metadata about a filing, including filing ID, licensee/DISCOM details, receiving regulatory commission, control period, currency, and unit scale.
2. **`ArrFiscalYear`** — Connects a specific fiscal year to its baseline/control-period classification and amount basis (e.g., Audited actuals, SERC-Approved values, or proposed projections).
3. **`ArrLineItem`** — Represents a single cost, revenue, subtotal, or true-up adjustment line item matching the regulatory sub-forms (e.g. Power Purchase Cost, O&M Expenses, Net ARR).

Directory Structure and Files

File	Description
`attributes.yaml`	Canonical OpenAPI 3.1.0 schema source of truth, describing all attributes, models, and types.
`schema.json`	Compiled Draft 2020-12 JSON Schema for standard validation of data payloads.
`context.jsonld`	Compiled JSON-LD context mapping ARR attributes to semantic terms under the `ies:` prefix.
`vocab.jsonld`	Compiled JSON-LD vocabulary ontology graph defining classes and properties.
`examples/`	JSON-LD absolute-linked regulatory example files.

Schema Generation and Compilation

The JSON Schema and JSON-LD context/vocabulary are compiled automatically from the core `attributes.yaml` using our generic Python build script.

To Compile Schemas To recompile schemas following modifications in `attributes.yaml`, run the following command from the repository root:

```
python scripts/generate_schema.py schemas/ArrFiling/v0.5
```

Payload Verification & Validation

Example JSON files are validated for structural compliance against `schema.json` using our dedicated validation script.

To Validate Example Payloads Run the following command from the repository root:

```
python scripts/validate_schema.py schemas/ArrFiling/v0.5/schema.json schemas/ArrFiling/v0.5/examples
```

JSON-LD Integration

All example payloads in the `examples/` directory have been augmented to support standard Linked Data semantic resolution: 1. **`@context`** — Points to the canonical absolute URL `https://india-energy-stack.github.io/ies-accelerator/schemas/ArrFiling/v0.5/context.jsonld` to resolve terms. 2. **`@type`** — Indicates the profile class shape (e.g., `ArrFiling`), aligning the regulatory datasets to the broader India Energy Stack ecosystem.

Field reference

*Auto-generated from `schema.json`. A field name in **bold** with a trailing ***** is required; all others are optional. **Type** shows units for QuantitativeValue models. Where a field is derived from a standard, its description begins with **Based on** and the standard reference (from the field's `x-standard` annotation). One table per object.*

ArrFiling

Field	Type	Description
`objectType` *	`ARR_FILING`	—
`id`	text	Unique identifier for this filing.
`filingId` *	text	Regulatory filing reference number.
`filingDate`	date	—
`filingType`	`MYT` / `ANNUAL` / `TRUE_UP` / `REVISED`	MYT - Multi-Year Tariff control period filing (multiple years, mix of actual/proposed) ANNUAL - single year or historical year-by-year approved data TRUE_UP - reconciliation of actuals vs previously approved amounts REVISED - amended filing with corrections
`licensee` *	text	Full name of the distribution licensee.
`licenseeCode`	text	Short code for the licensee.
`stateProvince`	text	—
`regulatoryCommission` *	text	SERC or Joint ERC that receives the filing.
`controlPeriodStart`	text	Start fiscal year of MYT control period (only for MYT filings).
`controlPeriodEnd`	text	End fiscal year of MYT control period.
`currency` *	`INR`	—
`unitScale` *	`CRORE` / `LAKH` / `ABSOLUTE`	Scale of all amounts in the filing.
`status`	`DRAFT` / `SUBMITTED` / `UNDER_REVIEW` / `APPROVED` / `REJECTED`	—
`formReference`	text	Regulatory form identifier.
`notes`	list of text	Footnotes, regulatory order references, and explanatory notes.
`fiscalYears` *	list of ArrFiscalYear	—

ArrFiscalYear

Field	Type	Description
`fiscalYear` *	text	Fiscal year label.
`yearType`	`BASE_YEAR` / `CONTROL_PERIOD` / `HISTORICAL`	BASE_YEAR - reference year in an MYT filing (typically the year before the control period) CONTROL_PERIOD - a year within the MYT control period being filed for HISTORICAL - a past year with finalized data (used in annual/historical filings)
`amountBasis` *	`AUDITED` / `APPROVED` / `PROPOSED` / `TRUED_UP` / `NOT_FILED`	AUDITED - actual costs verified by auditors APPROVED - amounts approved by the SERC in a tariff order PROPOSED - amounts requested by the DISCOM, pending SERC approval TRUED_UP - reconciled amounts after comparing actuals to approved NOT_FILED - placeholder year in a control period, no data yet
`lineItems` *	list of ArrLineItem	—

ArrLineItem

Field	Type	Description
lineItemId *	text	Stable identifier for this line item across years. Use kebab-case, e.g., “power-purchase-cost”, “interest-working-cap”.
serialNumber	integer	Display order as shown in the regulatory form.
category *	VARIABLE / FIXED / INCOME / SUB_TOTAL / ARR / ADJUSTMENT	VARIABLE - costs varying with energy volume (power purchase) FIXED - costs independent of volume (O&M, depreciation, interest, return) INCOME - revenue credits that reduce the ARR (negative amounts) SUB_TOTAL - computed aggregation of other line items ARR - the final net/aggregate revenue requirement ADJUSTMENT - true-up corrections, pass-throughs, FPPCA adjustments
subCategory	POWER_PURCHASE / NETWORK_COST / O_AND_M / DEPRECIATION / INTEREST / RETURN_ON_EQUITY / PROVISIONAL / OTHER / NON_TARIFF_INCOME / REVENUE_CREDIT / TOTAL / NET_ARR	Functional sub-classification for analysis and comparison across DISCOMs.
head *	text	Short heading as used in the regulatory form. Varies by DISCOM — use the original text from the filing.
particulars	text	Detailed description or the “Particulars” column value. Useful when head alone is ambiguous (e.g., “Others” head with “Incentive/Disincentive on achievement of norms” as particulars).
amount *	number / null	Amount in the filing’s currency and unitScale. Null means the line item exists in the form but was not filed / not applicable. Negative values represent credits, deductions, or adjustments that reduce ARR.
formReference	text	Reference to the supporting sub-form or schedule.
componentOf	text	lineItemId of the parent subtotal this item contributes to.
formula	text	Human-readable computation formula expressed as references to other lineItemIds. Only present on SUB_TOTAL and ARR items.

11.11 OutageNotification

[warn] **WORK IN PROGRESS — subject to change.** Early draft family (v0.1) for discussion; not yet stable.

Schema for a distribution licensee (DISCOM) to publish **planned and unplanned electricity outage** details — as a website / outage-map feed (pull) and as push notifications to affected or subscribed consumers (push). GIS-ready (GeoJSON, WGS84) and future-proof (additive enums, namespaced extensions).

Namespace prefix: `ies:` -> `https://india-energy-stack.github.io/ies-accelerator/schemas/ies#`

Tags: `outage` · `feeder` · `discom` · `oms` · `cap` · `gis` · `ies`

11.11.1 Versions

Version	Status	Notes
v0.1	**Draft (WIP)**	Initial model: OutageNotification + sub-models; provenance links to MeterData AlarmProfile

---

11.11.2 Models

Model	Purpose
`OutageNotification`	Root — class (planned/breakdown/roster/load-shedding), status, cause, affected assets, area, timing, impact, response, public info, provenance
`OutageAsset`	Affected feeder/DT/substation with smart-meter ref and optional GeoJSON geometry
`OutageProvenance`	Source, AMISP code, raw feeder-status signal, and alarm refs into MeterData

---

11.11.3 Design note

The full design rationale — standards survey, mapping tables (OMS/PVVNL/CAP/CIM), GIS/outage-map readiness, enum provenance, and the feeder-status ingest (detection) layer — lives alongside the schema:

- v0.1/OutageNotification_Design.md — design note
- v0.1/FeederStatusIngest.openapi.yaml — feeder-status ingest API (detection layer)

11.11.4 Usage

DISCOMs publish outage notices as a feed (`/outages`, `/outages.geojson`), a CAP feed for alerting, and webhooks to subscribers. Auto-detected outages carry provenance back to the smart-meter signal and the MeterData `AlarmProfile`.

11.11.5 v0.1

Files — attributes.yaml · schema.json · context.jsonld · vocab.jsonld · examples/

11.11.6 Outage Notification Schema (v0.1)

[warn] **WORK IN PROGRESS — subject to change.** This is an early draft (v0.1) published for discussion. Models, field names, enums, and the SD/BD/RS interpretation are not final and may change without notice before a stable release. Do not depend on it for production integrations yet.

Machine-readable schema for a distribution licensee (DISCOM) to publish **planned and unplanned electricity outage** details. One canonical object that is publishable as a **website / outage-map feed** (pull) and **pushable to affected or subscribed consumers** (push). Designed to be **GIS-ready** and **future-proof**.

Overview

`OutageNotification` describes a single outage (planned shutdown, breakdown, roster/load-shedding) with its cause, affected assets, area, timing, impact, field response, and public-facing text. Where an outage is auto-detected, `provenance` links back to the originating smart-meter signal and the IES `MeterData AlarmProfile`.

It composes proven standards by layer:

- **IEC 61968-3 (CIM)** — outage/UsagePoint data semantics, planned vs unplanned.
- **ODIN** (US DOE/ORNL) — publication pattern; coded + free-text cause.
- **OASIS CAP v1.2** (ITU-T X.1303) — push-alert envelope (`msgType`, `references`, `severity`, area + polygon, headline/instruction).
- **MultiSpeak** — AMI/MDMS -> OMS detection flow and provenance.
- **GeoJSON (RFC 7946)** — GIS geometry, WGS84, for outage maps.
- **IEEE 1366 / RDSS** — restoration time + customer counts for SAIDI/SAIFI.

Design rationale, the full standards survey, mapping tables (OMS/PVVNL/CAP/CIM), the GIS/outage-map readiness section, and the feeder-status ingest (detection) layer are in the design note alongside this schema: OutageNotification_Design.md (and the ingest stub FeederStatusIngest.openapi.yaml).

Enum provenance (borrowed vs local)

Each enum declares its origin in its `description` (and a machine-readable `$comment`); enums borrowed wholesale from a standard are also semantically anchored in `context.jsonld` via their term IRI.

Enum	Origin
`msgType`	**Borrowed** — OASIS CAP v1.2 `alert/msgType` (anchored `urn:oasis:names:tc:emergency:cap:1.2#msgType`)
`severity`	**Borrowed** — OASIS CAP v1.2 `info/severity` (anchored `urn:oasis:names:tc:emergency:cap:1.2#severity`)
`cause.category` / `cause.subcategory`	**Aligned** — IEEE 1782-2022 §4.4 (ten cause categories) and §4.5 (subcategories), used with IEEE 1366
`Identifier.scheme`	**Mixed** — `MRID` from CIM (IEC 61968/61970), `DID` from W3C DID Core; remaining values local
`feederStatus`	**Local** normalization; energized/de-energized align with CIM UsagePoint semantics
`outageClass`	**Local** — UPPCL OMS "Down Info" value set
`status`, `category`, `assetLevel`, `consumerCategory`, `source`	**Local** — not from a published standard

Note: `category` (outage category) is **not** CAP's `info/category` — it is a local value set with a deliberately different membership.

Models

Model	Purpose
`OutageNotification`	Root — class, status, cause, assets, area, timing, impact, response, public info, provenance
`OutageCause`	IEEE 1782 cause category + subcategory, vendor code (verbatim), free-text reason
`OutageAsset`	Affected asset (feeder/DT/substation/line) with meter ref + optional geometry
`OutageNetworkContext`	DISCOM org hierarchy (Discom > Zone > Circle > Division > Subdivision > Substation)
`OutageAffectedArea`	CAP area — text + admin areas + GeoJSON geometry
`OutageImpact`	Customers affected + de-/energized UsagePoints (CIM)
`OutageTiming`	Window (TimePeriod), ETR, actual restoration, SLA target
`OutageResponse`	Back-feed, resource status, complaint count
`OutagePublicInfo`	CAP info block for web/push
`OutageProvenance`	Source, AMISP code, raw signal, alarm refs into MeterData

---

Directory Structure and Files

File	Description
`attributes.yaml`	Canonical OpenAPI 3.1.0 source of truth describing all models, attributes, and types.
`schema.json`	Compiled Draft 2020-12 JSON Schema for payload validation.
`context.jsonld`	Compiled JSON-LD context mapping attributes to `ies:` terms.
`vocab.jsonld`	Compiled JSON-LD vocabulary (classes and properties).
`examples/`	JSON-LD example payloads (planned multi-feeder shutdown; unplanned breakdown with provenance; restoration UPDATE).
`OutageNotification_Design.md`	Design note — standards survey, mapping tables, GIS readiness, enum provenance.
`FeederStatusIngest.openapi.yaml`	Real-time feeder-status ingest API (detection layer).
`fault_reason_crosswalk.json`	UPPCL FAULT_REASON master (FORM_ID 8312) -> `cause.category/cause.subcategory` (IEEE 1782 §4.4/§4.5) crosswalk.
`OutageNotification_Implementation.md`	Step-by-step guide to build a production pull/push outage-notification system.
`tools/`	PVVNL planned-shutdown CSV + transformer script (CSV -> OutageNotification JSON).

Schema Generation and Validation

`schema.json`, `context.jsonld`, and `vocab.jsonld` are **compiled** from `attributes.yaml` — do not edit them by hand.

```
# from this directory
make            # generate + validate
# or, from the repo root:
python3 scripts/generate_schema_permissive.py schemas/OutageNotification/v0.1
python3 scripts/validate_schema.py schemas/OutageNotification/v0.1/schema.json schemas/OutageNotification/v0.1/exa
```

Field reference

*Auto-generated from `schema.json`. A field name in **bold** with a trailing ***** is required; all others are optional. **Type** shows units for QuantitativeValue models. Where a field is derived from a standard, its description begins with **Based on** and the standard reference (from the field's `x-standard` annotation). One table per object.*

OutageNotification Payload

Field	Type	Description
`objectType` *	`OUTAGE_NOTIFICATION`	—
`id` *	Identifier	Stable notice identity; reused across UPDATE/CANCEL messages.
`outageClass` *	`PLANNED` / `BREAKDOWN` / `SCHEDULED_ROSTERING` / `EMERGENCY_ROSTERING`	Outage class, from the UPPCL OMS "Down Info" set (local, additive enum). Rostering = rotational load-shedding (scheduled vs emergency). Restoration is a status transition (status=RESTORED), not a class.
`status` *	`SCHEDULED` / `ACTIVE` / `PARTIALLY_RESTORED` / `RESTORED` / `CANCELLED`	Outage lifecycle state. LOCAL enum (not from a published standard); loosely aligned with the CIM Outage lifecycle.
`msgType`	`ALERT` / `UPDATE` / `CANCEL`	**Based on** OASIS CAP v1.2 (alert/msgType). Message type; UPDATE/CANCEL refer to a prior notice via `references`.
`references`	list of Identifier	**Based on** OASIS CAP v1.2 (alert/references). Prior notice ids this message updates or cancels (CAP `references`).
`severity`	`EXTREME` / `SEVERE` / `MODERATE` / `MINOR` / `UNKNOWN`	**Based on** OASIS CAP v1.2 (info/severity). Severity of the outage.
`category`	`MAINTENANCE` / `UPGRADE` / `FAULT` / `LOAD_SHEDDING` / `WEATHER` / `SAFETY` / `OTHER`	Coarse descriptor for display/filtering (local enum; not CAP `info/category`). For analytics, prefer the standardized `cause.category`.
`cause`	OutageCause	—
`forceMajeure`	yes / no	OMS "Force Majeure" flag.
`issuedBy`	Party	—
`issuedAt`	date-time	—
`detectedAt`	date-time	When MDMS/SCADA first detected the outage (unplanned).
`lastUpdatedAt`	date-time	—
`network`	OutageNetworkContext	—
`affectedAssets` *	list of OutageAsset	—
`affectedArea`	OutageAffectedArea	—
`impact`	OutageImpact	—
`timing` *	OutageTiming	—
`response`	OutageResponse	—
`publicInfo`	OutagePublicInfo	—
`provenance`	OutageProvenance	—
`extensions`	object	Namespaced DISCOM-specific fields, e.g. { "uppcl": { "breakdownId": "6357257", "downType": "FEEDER" } }.

OutageCause

Field	Type	Description
`category`	`EQUIPMENT` / `LIGHTNING` / `PLANNED` / `POWER_SUPPLY` / `PUBLIC` / `VEGETATION` / `WEATHER` / `WILDLIFE` / `UNKNOWN` / `OTHER`	**Based on** IEEE 1782-2022 §4.4 (with IEEE 1366). Standardized interruption cause category, for reliability benchmarking. Load-shedding/rostering has no standard category — map scheduled rostering to PLANNED, emergency to OTHER, and carry the nature in `outageClass`.

Field	Type	Description
subcategory	DEGRADATION / EQUIPMENT_ERROR / ENVIRONMENTAL / DIRECT_STRIKE / INDIRECT_STRIKE / NEW_CONSTRUCTION / MAINTENANCE / CUSTOMER_REQUEST / OTHER_UTILITY_REQUEST / GENERATION / TRANSMISSION / SUBTRANSMISSION / DISTRIBUTION / DISTRIBUTED_GENERATION_STORAGE / OTHER_UTILITY_SUPPLY / DIG_IN / FOREIGN_CONTACT / FIRE_POLICE / VEHICLE / WITHIN_CLEARANCE_ZONE / OUTSIDE_CLEARANCE_ZONE / PRECIPITATION / ICE / WIND / EXTREME_TEMPERATURE / MAMMAL / BIRD / REPTILE_AMPHIBIAN / NO_SPECIFIC_CAUSE_FOUND / UTILITY_ERROR / OTHER_UTILITY_INITIATED / OTHER	**Based on** IEEE 1782-2022 §4.5. Standardized cause subcategory; must be valid for the chosen category (see $comment for the mapping). Leave unset if the field finding is inconclusive.
faultType	text	Vendor asset/voltage level of the fault, e.g. 33KV, 11KV, DT, LT.
code	text	Vendor/OMS fault-reason code, carried verbatim (e.g. UPPCL FAULT_REASON) — not standardized; map to category/subcategory for interoperability (see fault_reason_crosswalk.json). Set codeNamespace where the code's authority matters.
codeNamespace	text	Authority/vendor that defines code (e.g. the OMS vendor or DISCOM).
text	text	Free-text reason; may be localized (e.g. Hindi).

OutageAsset

Field	Type	Description
id *	Identifier	Asset id/name; ideally a stable GIS feature id or MRID for map join.
assetLevel *	SUBSTATION / FEEDER / DT / LINE_SEGMENT / SERVICE_POINT	Network level of the affected asset (local enum; aligns with CIM Equipment/UsagePoint).
voltageLevel	text	e.g. 33kV, 11kV, LT, DT.
consumerCategory	URBAN / RURAL / AGRICULTURE / INDUSTRIAL / MIXED / OTHER	Consumer mix on the asset (local enum; PVVNL "Feeder Type").
meterRef	Identifier	Feeder/substation smart-meter number — join key to MDMS/MeterData.
parentRef	Identifier	Parent asset (e.g. a feeder's substation, a DT's feeder).
geo	GeoJSONGeometry	**Based on** GeoJSON (RFC 7946), WGS84. Optional inline geometry (WGS84): Point (substation/DT), LineString (feeder route), Polygon (service area).

OutageNetworkContext

Field	Type	Description
discom	Identifier	—
zone	text	—
circle	text	—
division	text	—
subdivision	text	—
substation	Identifier	—
district	text	—

OutageAffectedArea

Field	Type	Description
text	text	Free-text localities (CAP areaDesc), e.g. "SEC-14, 9, 11 Rajnagar".
adminAreas	list of text	Named localities / wards / villages.
geo	GeoJSONGeometry	**Based on** GeoJSON (RFC 7946); CAP area. Polygon/MultiPolygon service area, or Point/circle (CAP). WGS84.

OutageImpact

Field	Type	Description
customersAffected	integer	—

Field	Type	Description
deEnergized	list of Identifier	**Based on** CIM (IEC 61968-3 UsagePoint). Optional SDP/UsagePoint refs (authenticated tier).
energized	list of Identifier	**Based on** CIM (IEC 61968-3 UsagePoint). Optional points confirmed still energized.

OutageTiming

Field	Type	Description
period *	TimePeriod	Outage window (Down From + duration).
estimatedRestoration	date-time	ETR (OMS "Estimated Time").
actualRestoration	date-time	Set when status=RESTORED; feeds IEEE 1366 indices.
slaTargetMinutes	integer	SLA target, e.g. 240 (4 hrs).

OutageResponse

Field	Type	Description
backFeedGiven	yes / no	Partial restoration via alternate feed (OMS "Back-Feed given").
resourceStatus	text	e.g. PENDING, RESOURCE_ALLOCATED, IN_PROGRESS.
complaintCount	integer	Linked consumer complaints (OMS "No. of Complaints").

OutagePublicInfo

Field	Type	Description
language	text	BCP-47, e.g. en, hi.
headline	text	—
description	text	—
instruction	text	What the consumer should do.

OutageProvenance

Field	Type	Description
source	MDMS / OMS / SCADA / RTDAS / AMI / MANUAL	System that raised this notice (local enum; RTDAS = Real-Time Data Acquisition System).
amispCode	text	AMI Service Provider that supplied the detection signal (feeder-status ingest API).
detectionRef	Identifier	Reference to the real-time detection record that raised the outage (e.g. UPPCL RTDAS_DATA_ID). Absence or a "0" sentinel indicates a manually entered outage (source=MANUAL).
signal	OutageSignal	—
alarmRefs	list of OutageAlarmRef	References into MeterData AlarmProfile records.

OutageSignal

Field	Type	Description
eventId	text	Idempotency key of the originating feeder-status event.
feederStatus	ENERGIZED / DE_ENERGIZED / UNKNOWN	Normalized feeder status (local enum); raw vendor code in rawCode. Energized/de-energized align with CIM UsagePoint semantics.
diPort	text	Digital-input port on the substation meter that carried the signal.
rawCode	text	Vendor-native status code as received (e.g. FEEDER_STATUS=102), for traceability.

OutageAlarmRef

Field	Type	Description
meterRef *	Identifier	—
alarmId	integer	—
timestamp	date-time	—

Party

Field	Type	Description
`id`	Identifier	—
`name`	text	—
`contact`	text	Phone/email/URL for queries (e.g. 1912).

Identifier

Field	Type	Description
`scheme` *	`METER_SERIAL` / `MRID` / `GIS_FEATURE_ID` / `SERVICE_DELIVERY_POINT` / `FEEDER` / `SUBSTATION` / `DT` / `CONSUMER_NUMBER` / `ORG` / `DID` / `OTHER`	**Based on** MRID: CIM (IEC 61968/61970); DID: W3C DID Core. Identifier scheme (mixed provenance). MRID and DID are borrowed from their standards; the rest are local vendor/DISCOM master keys.
`value` *	text	—
`namespace`	text	—

TimePeriod

Field	Type	Description
`start` *	date-time	—
`duration` *	duration	ISO-8601, e.g. PT2H.

11.12 External Schemas

Some schemas used across IES are **defined and published outside this repository** — served canonically at schema.beckn.io — and documented here so the book carries a complete field reference. Each schema links to its canonical definition; each domain links to its use-case guide and deployable devkit.

These tables are **auto-generated** from the schema sources by `scripts/generate_external_field_tables.py` (version `v2.0`). Don't edit by hand — re-run the generator to refresh. Where a field derives from a recognised standard, its description begins with **Based on** and the standard reference (from the source's `x-standard` annotation).

11.12.1 Energy Trading (P2P)

Use case · Devkit

A peer-to-peer trade is modelled as a `P2PTrade` contract (a beckn `Contract` subclass) whose energy-specific attributes are carried by the offer, customer, order-item, settlement and resource schemas below. `EnergyResource`, `DEGContract`, `RevenueFlow` and `BecknTimeSeries` are shared primitives — Demand Flexibility reuses them.

P2PTrade

Defined at `P2PTrade/v2.0` — P2PTrade.

*Inherits all fields from **EnergyContract**.*

EnergyTradeOffer

Defined at `EnergyTradeOffer/v2.0` — EnergyTradeOffer — Offer Attributes (v2.0).

Field	Type	Description
`validityWindow`	TimePeriod	Time window during which this offer can be selected/accepted. Typically set to expire before the earliest delivery starts. Present in catalog publish; may be omitted in downstream messages.
`contractAttributes`	object	JSON-LD container for contract terms attached at catalog publish time. Only roles with a non-null participantId are included (unknown parties are omitted at publish time). MUST carry @context and @type (typically DEGContract). Present in catalog publish; omitted in downstream messages where Contract.contractAttributes carries the full party list.
`commitmentAttributes`	object	Seller's BecknTimeSeries published at catalog time. Declares the full schema of the contract table via payloadDescriptors, each annotated with `insertedBy` (the role responsible for populating that column). Carries the seller's initial interval data (PRICE_PER_KWH + AVAILABLE_QTY per slot). Immutable after publish — repeated verbatim in all downstream messages as the authoritative offer definition. MUST carry @context (BecknTimeSeries/v1.0/context.jsonld) and @type: TimeSeries. Standard payloadDescriptors and their insertedBy: PRICE_PER_KWH (EVENT) — insertedBy: seller AVAILABLE_QTY (EVENT) — insertedBy: seller REQUESTED_QTY (EVENT) — insertedBy: buyer BUYER_DISCOM_ALLOC (REPORT) — insertedBy: buyerDiscom BUYER_DISCOM_STATUS (REPORT) — insertedBy: buyerDiscom SELLER_DISCOM_ALLOC (REPORT) — insertedBy: sellerDiscom SELLER_DISCOM_STATUS (REPORT) — insertedBy: sellerDiscom FINAL_ALLOC (REPORT) — insertedBy: sellerDiscom

Object: `EnergyTradeOffer.contractAttributes`

Field	Type	Description
`@type` *	text	—

Object: `EnergyTradeOffer.commitmentAttributes`

Field	Type	Description
`@type` *	`TimeSeries`	—

EnergyCustomer

Defined at `EnergyCustomer/v2.0` — EnergyCustomer — Customer Attributes (v2.0).

Field	Type	Description
`meterId` *	text	Meter identifier in DER address format (der://meter/{id}). Used for customer identification and energy delivery tracking in P2P trading flows.
`sanctionedLoad`	number	Sanctioned load capacity in kilowatts (kW). Optional field representing the approved electrical load capacity for the customer's connection. Used for load management and regulatory compliance.
`utilityCustomerId`	text	Customer's account identifier with the utility company (e.g., PG&E customer number). Used for billing, service identification, and utility system integration.
`utilityId`	text	Utility/DISCOM identifier for the customer's service territory (e.g., "TPDDL-DL", "BESCOM-KA"). Used in inter-utility / inter-DISCOM P2P trading to identify which utility serves this customer.
`platformUrl`	uri	Base URL (scheme + host[:port]) of the Beckn trading platform that represents this customer on the network. The platform exposes the standard Beckn paths under this base: `/bap/caller`, `/bap/receiver`, `/bpp/caller`, `/bpp/receiver`. Used by downstream nodes to address this customer's platform for cascade sub-transactions — e.g. a seller-side adapter cascading a /status query into its own discom ledger and needing the discom's response routed back to its own /bap/receiver. The path appended depends on the action's BAP<->BPP direction (/bap/receiver for on_*, /bpp/receiver for request actions).

EnergyOrderItem

Defined at `EnergyOrderItem/v2.0` — EnergyOrderItem.

Field	Type	Description
`providerAttributes` *	object	Provider/customer information for this order item, including meter ID and utility account.
`fulfillmentAttributes`	object	Optional fulfillment tracking for this order item. Only populated in on_status and on_update responses (not in init/confirm flows). Contains delivery status, meter readings, and energy allocation data.

Object: `EnergyOrderItem.providerAttributes`

Field	Type	Description
`@type`	`EnergyCustomer`	Type identifier for provider attributes

Object: `EnergyOrderItem.fulfillmentAttributes`

Field	Type	Description
`@type`	`EnergyTradeDelivery`	Type identifier for fulfillment attributes

RevenueFlow

Defined at `RevenueFlow/v2.0` — RevenueFlow — Consideration Attributes (v2.0).

Field	Type	Description
`revenueFlows` *	list of object	Per-role signed revenue-flow entries computed by the contract policy after settlement. Sum MUST be zero across the array.

Object: `RevenueFlow.revenueFlows[]`

Field	Type	Description
`role` *	text	—
`value` *	number	Signed amount. Positive = role receives; negative = role pays.
`currency` *	text	**Based on** ISO 4217. ISO-4217 currency code.
`description`	text	Human-readable rationale for the flow.

DEGContract

Defined at `DEGContract/v2.0` — DEG — Contract (v2.0).

Field	Type	Description
`roles` *	list of object	Contract roles. The participantId key is required on every role entry, but its value MAY be null in the catalog/offer (buyer and buyerDiscom unknown until select/init) and is bound to a concrete participant id at select/init. Identity attributes (meter, utility, utilityCustomerId) for each bound role live at Contract.participants[*].participantAttributes.
`policy` *	object	OPA/Rego policy governing this contract.
`revenueFlows`	list of object	Optional. Settlement-time per-role flows written by the `revenueflows` plugin when it is configured with outputMode: raw and outputPath ending at this property (the legacy demand-flex shape). Wave2 P2P trading instead uses Contract.consideration[*].considerationAttributes (RevenueFlow JSON-LD) and leaves this array unset.

Object: `DEGContract.roles[]`

Field	Type	Description
`role` *	text	—
`participantId` *	text (nullable)	Concrete participant id once bound (buyerPlatform/sellerPlatform use the utilityId-meter compound id, e.g. “TPDDL-DL-seller-001”; discom roles use the discom-ledger participant id). Null at catalog publish for sides not yet bound.

Object: `DEGContract.policy`

Field	Type	Description
`url` *	uri	—
`queryPath` *	text	—

Object: `DEGContract.revenueFlows[]`

Field	Type	Description
`role` *	text	—
`value` *	number	—
`currency` *	text	**Based on** ISO 4217.
`description`	text	—

EnergyResource

Defined at `EnergyResource/v2.0` — EnergyResource — Resource Attributes (v2.0).

Discriminated union of all typed EnergyResource kinds. Dispatched by the ‘type’ field. Each kind lives in its own schema at schema.beckn.io//v1.0 and is referenced above. EnergyResourceCommon (id, type, subResources, parentResources, attributes) and EnergyResourceCommonAttributes (make, model, maxExportKw, maxImportKw, ratedPowerKw, telemetryProvider, commissioningDate, location) are defined in EnergyResourceCommon/v1.0 and inherited by all kinds via allOf external $ref. For P2P-trading: minimal usage is {id, type}. For demand-flex and credentials: add attributes fields as needed. For targeted validation: $ref the specific kind directly.

Discriminated union (`oneOf`) of: `EnergyResourceMeter`, `EnergyResourceGenerator`, `EnergyResourceStorage`, `EnergyResourceEVCharger`, `EnergyResourceInverter`, `EnergyResourceLoad`, `EnergyResourceNetwork`.

BecknTimeSeries

Defined at `BecknTimeSeries/v1.0` — BecknTimeSeries — Time-Series Payload (v1.0).

*Based on **OpenADR 3.1.0**.*

A time-series payload. `intervalPeriod` sets the default temporal bounds; each `interval` carries one or more typed `payloads` (valuesMap rows). `payloadDescriptors` declare units/currency/ reading-type for each `payloadType` referenced in the rows; every type used in `intervals` SHOULD be declared in `payloadDescriptors`. `payloadType` follows OpenADR’s open-string convention — consumer profiles (e.g. `DemandFlexPerformance`) MAY restrict it to a domain-specific set and add cross-field membership checks.

Field	Type	Description
`@type`	`TimeSeries`	Optional JSON-LD type discriminator. Present when the embedder wants to surface the type in the payload; the parent schema's `$ref` to BecknTimeSeries is what drives validation.
`intervalPeriod` *	intervalPeriod	Default temporal bounds of the series — ISO 8601 `start` and `duration`. An individual interval may override these.
`payloadDescriptors` *	list of eventPayloadDescriptor or reportPayloadDescriptor	Sidecar describing each `payloadType` carried by the rows (units, currency, reading type, accuracy/confidence). Every `intervals[*].payloads[*].type` SHOULD appear here; consumer profiles enforce that as a cross-field constraint. Each descriptor must be either an OpenADR3 eventPayloadDescriptor (objectType: EVENT_PAYLOAD_DESCRIPTOR — for offer/bid signals) or a reportPayloadDescriptor (objectType: REPORT_PAYLOAD_DESCRIPTOR — for telemetry/meter readings). The objectType field is the discriminator.
`intervals` *	list of interval	Series rows. Each row carries an integer `id` and a list of typed `payloads` (valuesMap). Per-row `intervalPeriod` is optional and overrides the default.
`resourceName`	text	Identifies the single resource these readings are about — the data source. Mirrors OpenADR 3.1.0's `Report.resources[].resourceName`. Use when the series captures telemetry from a specific physical device or asset; in energy domains this is typically the meter id. Omit (or set to `"0"`, following OpenADR convention) for aggregate / unattributed series. Optional, so existing payloads continue to validate without change.
`clientName`	text	Identifies the reporting client — the party authoring this time-series and pushing it onto the wire. Mirrors OpenADR 3.1.0's `Report.clientName` (VEN name). In energy domains this is typically the utility / DISCOM / aggregator subscriber id. Optional.

11.12.2 Demand Flexibility

Guide · Devkit

A flexibility programme advertises a `DemandFlexNeed`, contracts via a `DemandFlexBuyOffer`, and reports measurement & verification with `DemandFlexPerformance` (per-meter telemetry as a `BecknTimeSeries`). It reuses the shared `EnergyResource`, `DEGContract`, `RevenueFlow` and `BecknTimeSeries` schemas listed under Energy Trading.

DemandFlexNeed

Defined at `DemandFlexNeed/v2.0` — Demand Flex — Need Attributes (v2.0).

Resource attributes describing a demand-flex need. Attached to Resource.resourceAttributes to describe what the utility needs from the network. Captures the direction (increase/reduce), event timing, capacity, and location.

Field	Type	Description
`direction` *	`INCREASE` / `REDUCE`	Whether the utility needs demand increase or demand reduction. REDUCE = curtailment (typical DR). INCREASE = load shifting to off-peak.
`eventWindow` *	object	The time window during which the flex event occurs. All times MUST be in UTC (ISO 8601 with Z suffix).
`capacityType`	`CURTAILMENT` / `SHIFT` / `GENERATION`	Type of flex capacity. CURTAILMENT = reduce consumption. SHIFT = move consumption to different time. GENERATION = increase local generation.
`maxCapacityKw` *	number	Maximum flex capacity needed in kW.
`location`	object	**Based on** GeoJSON (RFC 7946). Geographic area where flex is needed (GeoJSON).

Object: `DemandFlexNeed.eventWindow`

Field	Type	Description
`startDate` *	date-time	**Based on** ISO 8601 (UTC, RFC 3339 profile). Event start time in UTC.
`endDate` *	date-time	**Based on** ISO 8601 (UTC, RFC 3339 profile). Event end time in UTC.

Object: `DemandFlexNeed.location`

Field	Type	Description
`type`	`Point` / `Polygon`	—
`coordinates`	list of any	—

DemandFlexBuyOffer

Defined at `DemandFlexBuyOffer/v2.0` *— Demand Flex — Buy Offer Attributes (v2.0).*

DemandFlexBuyOffer

Field	Type	Description
`contractAttributes`	object	Portable DEGContract template. Present only in catalog/discover. Promoted to Contract.contractAttributes at select/init.
`inputs` *	list of DemandFlexRoleInput	One entry per role. participantId is null until the role is bound (e.g. seller is null at catalog, filled at init). inputs object is optional when participantId is null, required when participantId is set.

DemandFlexRoleInput

Field	Type	Description
`role` *	`buyer` / `seller`	—
`participantId` *	text (nullable)	Participant bound to this role. Null until bound.
`inputs`	object	Role-specific inputs. Structure varies by role and contract type. For buyer: incentivePerKwh, currency, penaltyRate, baselineMethodology, etc. For seller: plannedDemandChange (beckn:Quantity), participatingMeters (or participatingMetersRef when the cohort is bulk), energyResources (or energyResourcesRef) — EnergyResource objects each linked to a participatingMeters[*] entry via meterId — and reportDescriptors (see below). Bulk delivery: the offer is bound at confirm and cannot span Beckn messages, so when the seller's cohort would stretch a single confirm payload beyond a reasonable wire budget (e.g. > 10k meters) the *Ref siblings replace the inline arrays. participatingMetersRef / energyResourcesRef each carry a BecknResourceRef pointing at a content- addressed BPP-hosted bundle. participatingMetersDigest is an on-protocol tamper-evidence anchor that survives even if the ref URL later goes 404 — Beckn signing of the confirm payload commits the contract to that exact cohort.

Object: `DemandFlexRoleInput.inputs`

Field	Type	Description
`reportDescriptors`	BecknReportDescriptors (nullable)	Seller-side declaration of what telemetry will be delivered for this contract. Set to `null` when the contract requires no telemetry. When non-null, the seller commits to delivering a BecknTimeSeries (per device, in performance.meters[]. telemetry on the on_status reply) whose payloadDescriptors cover every entry in this array. See BecknReportDescriptors for the canonical payloadType / units / cardinality table (USAGE, POWER, SOC_END, GPS_LAT, GPS_LON, …).
`participatingMetersRef`	BecknResourceRef	Optional. Off-protocol delivery of the participatingMeters cohort when it's too large to inline. Mutually exclusive with `participatingMeters`. Consumers MUST treat the fetched body as the authoritative meter list once the receiver has verified `contentHash` and `count` against the body's canonicalized form.
`energyResources`	list of EnergyResource	Optional. EnergyResources enrolled in this contract. Each entry is an EnergyResource object whose `meterId` field references one of the meter URIs in `participatingMeters[*]` (many resources MAY share a meter). Identity and rated dimensioning live here; per-event state lives in BecknTimeSeries telemetry on the performance record's meters[]. EnergyResource is the canonical, technology-neutral asset class — EV chargers, batteries, solar PV, smart HVAC, etc. — shared with P2P-trading and any future DEG domain.
`energyResourcesRef`	BecknResourceRef	Optional. Off-protocol delivery of the energyResources cohort when it's too large to inline. Mutually exclusive with `energyResources`. Same verification rules as `participatingMetersRef`.
`participatingMetersDigest`	object	Optional on-protocol tamper-evidence anchor for the meter cohort, regardless of whether it was delivered inline (`participatingMeters`) or by reference (`participatingMetersRef`). Computed as SHA-256 of the ASCII-sorted, newline-joined meter IDs. Beckn-signing of the confirm payload then commits the seller to this exact cohort permanently — useful for downstream audit long after any off-protocol bundle URL has expired.

Object: `DemandFlexRoleInput.inputs.participatingMetersDigest`

Field	Type	Description
`count` *	integer	Total meter count in the cohort.

Field	Type	Description
`sha256OfSortedIds` *	text	**Based on** SHA-256 (FIPS 180-4). `sha256:<64-hex>` of the cohort's meter IDs after they have been ASCII-sorted and joined with a single `\n` separator (no trailing newline).

DemandFlexPerformance

Defined at `DemandFlexPerformance/v2.0` — Demand Flex — Performance Attributes (v2.0).

Performance attributes for demand-flex M&V (Measurement & Verification). Attached to Performance.performanceAttributes in on_status callbacks.

Field	Type	Description
`eventId`	text	Identifier of the flex event being measured.
`methodology`	text	Baseline methodology used across all meters (e.g., "5of10" means average of 5 highest-consumption days out of last 10).
`meters`	list of object	Per-meter M&V data. Each entry binds a meter ID to its time-series telemetry for the event window. When the cohort fits in a single message the BPP SHOULD inline the full array and omit `pageInfo` entirely. For larger cohorts the BPP either ships paged slices (sibling `pageInfo` present) or substitutes `metersRef` and omits `meters[]` from this message. Settlement rego runs only on the assembled view — receivers MUST NOT fire settlement-dependent actions until `pageInfo.isLast == true` or the `metersRef` body has been fetched and verified.
`pageInfo`	BecknPageInfo	Optional. Present only when the `meters[]` collection has been split across multiple on_status messages (paged inline delivery). Receivers assemble all pages of the same (transactionId, performance.id) by `pageInfo.sequence` and defer settlement until `pageInfo.isLast == true`. Omit entirely when the whole cohort fits in one message — absence of `pageInfo` is the signal that this message is self-contained.
`metersRef`	BecknResourceRef	Optional. Off-protocol delivery of the per-meter telemetry bundle when even paged-inline (`pageInfo`) is too heavy. Mutually exclusive with `meters[]` and `pageInfo` on the same message — the BPP substitutes this reference for the entire collection. Receivers fetch `uri`, verify against `contentHash` and `count`, then run settlement against the fetched body.

Object: `DemandFlexPerformance.meters[]`

Field	Type	Description
`meterId` *	text	Meter identifier.
`telemetry` *	BecknTimeSeries	BecknTimeSeries carrying BASELINE (always) and — once the event has completed — USAGE payloads, aligned to the same interval grid. Validated inline via `$ref`, so the embedded payload only needs `@type` (optional) — `@context` is declared once at the envelope level via `context.schemaContext[]`.